

Thesis

Jason Leff

2025-09-19

Setup

```
rm(list=ls())
data <- read.csv('data_with_pre_data.csv')
set.seed(67)
```

Initializing Functions

```
heatmap <- function(ConfMatrix, color, title) { #Unused heatmap function
  cm_table <- as.data.frame(ConfMatrix$table)
  ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
    geom_tile(color = "white") +
    geom_text(aes(label = Freq), color = "black", size = 4) +
    scale_fill_gradient(low = "white", high = color) +
    theme_minimal() +
    labs(title = paste0("Confusion Matrix Heatmap (", title, ")"),
         x = "Actual Class", y = "Predicted Class")
}
```

```
within_one_class_pct <- function(ConfusionMatrix) { #Unused ordinal statistic calculator
  classes <- rownames(ConfusionMatrix)
  n_class <- length(classes)
  idx <- seq_len(n_class)
  total <- sum(ConfusionMatrix)
  count_valid <- 0
  for (i in idx) {
    valid_pred <- i
    if (i == 1) {valid_pred <- c(valid_pred, 2)}
    else if (i == n_class) {valid_pred <- c(valid_pred, n_class - 1)}
    else {valid_pred <- c(valid_pred, i - 1, i + 1)}
    count_valid <- count_valid + sum(ConfusionMatrix[i, valid_pred])
  }
  count_valid / total
}
```

```
macro_auc <- function(y_true, prob_matrix) { #Used to calculate AUC for each model
  classes <- levels(y_true)
  aucs <- numeric(length(classes))
  for (i in seq_along(classes)) {
    cls <- classes[i]
    y_binary <- as.numeric(y_true == cls)
    p <- prob_matrix[, cls]

    roc_obj <- try(roc(y_binary, p, quiet = TRUE), silent = TRUE)

    if (inherits(roc_obj, "try-error") || is.null(roc_obj$auc)) {
      aucs[i] <- NA_real_
    } else {
      aucs[i] <- as.numeric(roc_obj$auc)
    }
  }
  mean(aucs, na.rm = TRUE)
}
```

Normalize to Per Minute Data

```
normalize_stats <- function(df) {
  nba_start <- which(names(df) == "NBA_MIN") #split the data
  nba_end <- which(names(df) == "PRE_G")

  for (i in (nba_start+1):(nba_end-1)) {
    colname <- names(df)[i]
    if (!grepl("PCT", colname, ignore.case = TRUE) && is.numeric(df[[colname]])) { #divide total column
      df[[colname]] <- round(as.numeric(df[[colname]]) / as.numeric(df[["NBA_MIN"]]),3)
    }
  }
  pre_start <- which(names(df) == "PRE_MP")
  for (i in (pre_start+1):ncol(df)) { #repeat for college
    colname <- names(df)[i]
    if (!grepl("PCT", colname, ignore.case = TRUE) && is.numeric(df[[colname]])) {
      df[[colname]] <- round(as.numeric(df[[colname]]) / as.numeric(df[["PRE_MP"]]),3)
    }
  }
  return(df)
}
per_min_data <- normalize_stats(data)
```

PRE-NBA FT% and 3P% vs NBA 3P% filtered for 25 attempts

```
data_filtered2 <- data %>%
  filter(PRE_3PA >= 25 & PRE_FTA >= 25 & NBA_FG3A >= 25)

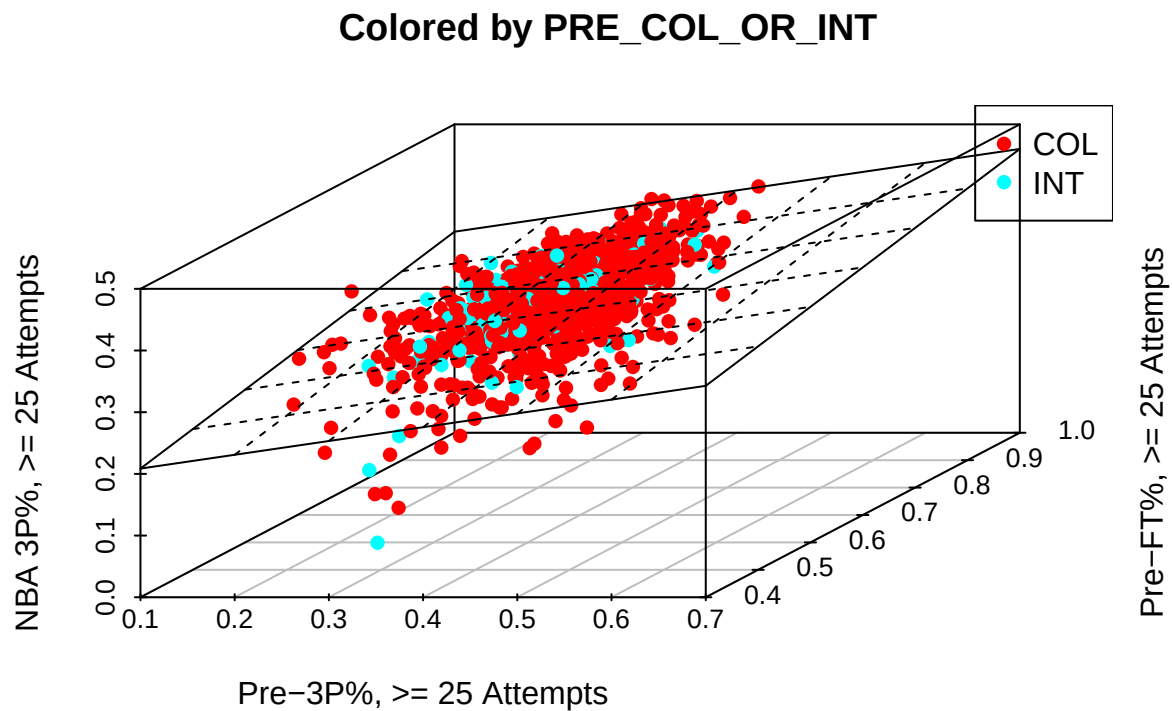
clean_data <- na.omit(data_filtered2[c("PLAYER", "NBA_FG3_PCT", "PRE_3P_PCT", "PRE_FT_PCT", "PRE_COL_or_"]
```

```
model <- lm(NBA_FG3_PCT ~ PRE_3P_PCT + PRE_FT_PCT, data = clean_data)
summary(model)
```

```
##
## Call:
## lm(formula = NBA_FG3_PCT ~ PRE_3P_PCT + PRE_FT_PCT, data = clean_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.252344 -0.019491  0.008559  0.033728  0.095787
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.10796    0.01583   6.822 1.49e-11 ***
## PRE_3P_PCT   0.22357    0.03315   6.743 2.51e-11 ***
## PRE_FT_PCT   0.19565    0.02277   8.593 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.04865 on 1085 degrees of freedom
## Multiple R-squared:  0.1633, Adjusted R-squared:  0.1618
## F-statistic: 105.9 on 2 and 1085 DF,  p-value: < 2.2e-16
```

```
color_factor <- as.factor(clean_data$PRE_COL_or_INT)
colors <- rainbow(length(levels(color_factor)))[color_factor]
s3d <- scatterplot3d(clean_data$PRE_3P_PCT, clean_data$PRE_FT_PCT, clean_data$NBA_FG3_PCT,
                     color = colors, pch = 16,
                     xlab = "Pre-3P%, >= 25 Attempts", ylab = "Pre-FT%, >= 25 Attempts", zlab = "NBA 3P%",
                     main = "Colored by PRE_COL_OR_INT")
s3d$plane3d(model, lty.box = "solid")

legend("topright", legend = levels(color_factor),
       col = rainbow(length(levels(color_factor))), pch = 16)
```



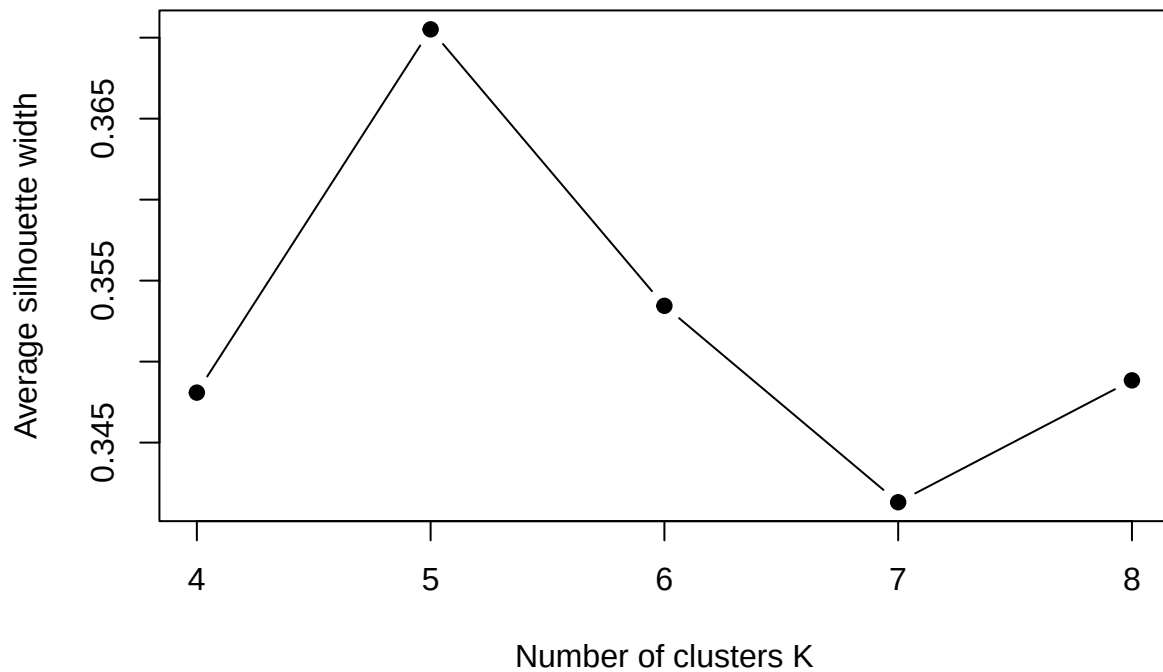
Clustering to Create Classes

```
indices <- which(data$NBA_FG3A >= 10 & data$NBA_FG3_PCT > 0.05) #Use a bit less strict filtering
clust_data <- na.omit(per_min_data[indices, c("NBA_FG3A", "NBA_FG3_PCT")])
clust_data_scaled <- scale(clust_data)

sil_width <- numeric(8)
for (k in 4:8) { #use silhouette method
  set.seed(123)
  km <- kmeans(clust_data_scaled, centers = k, nstart = 25)
  ss <- silhouette(km$cluster, dist(clust_data_scaled))
  sil_width[k] <- mean(ss[, 3])
}

plot(4:8, sil_width[4:8], type = "b", pch = 19, #plot results
     xlab = "Number of clusters K",
     ylab = "Average silhouette width",
     main = "Silhouette Method for Optimal K")
```

Silhouette Method for Optimal K



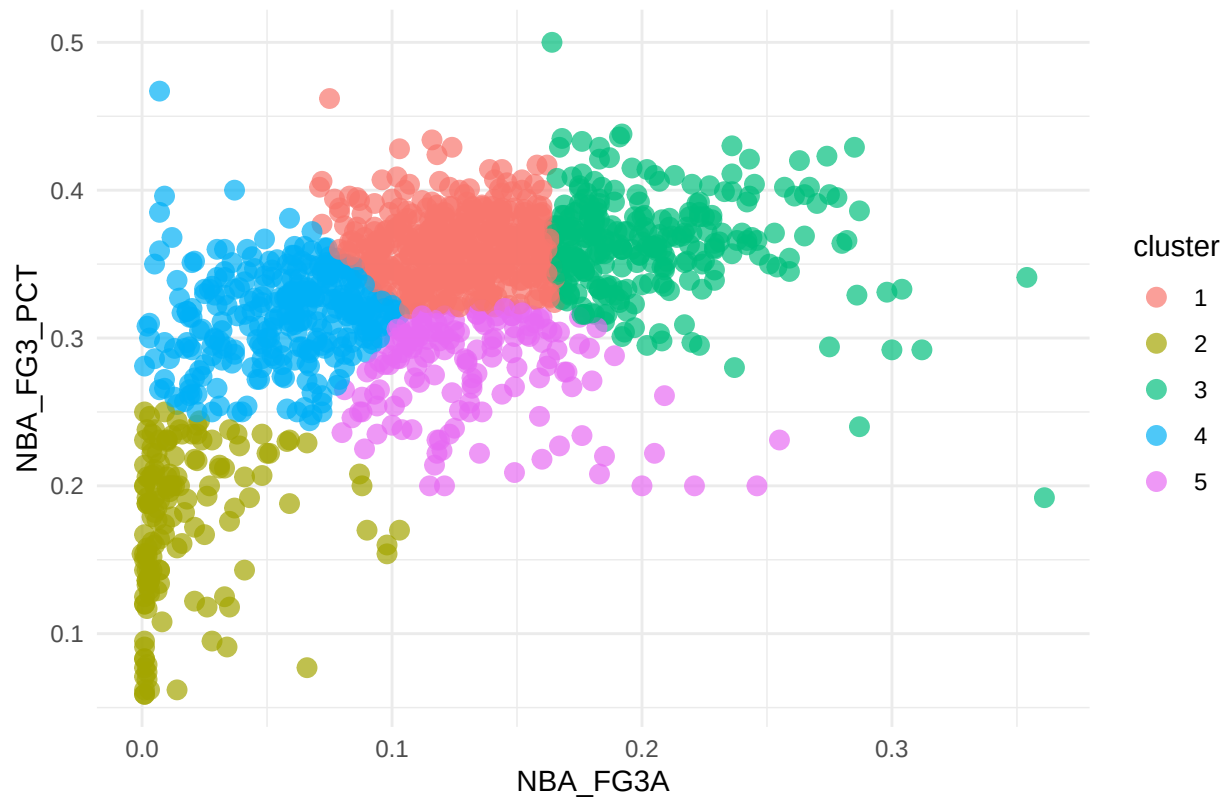
```
best_k <- which.max(sil_width) #set best k

set.seed(123)
km_best <- kmeans(clust_data_scaled, centers = best_k, nstart = 25) #graph the best k clusters

clust_plot <- data.frame(clust_data, cluster = factor(km_best$cluster))

ggplot(clust_plot, aes(x = NBA_FG3A, y = NBA_FG3_PCT, color = cluster)) +
  geom_point(size = 3, alpha = 0.7) +
  labs(title = paste("K-means Clustering with K =", best_k,
    " (Silhouette Method)")) +
  theme_minimal()
```

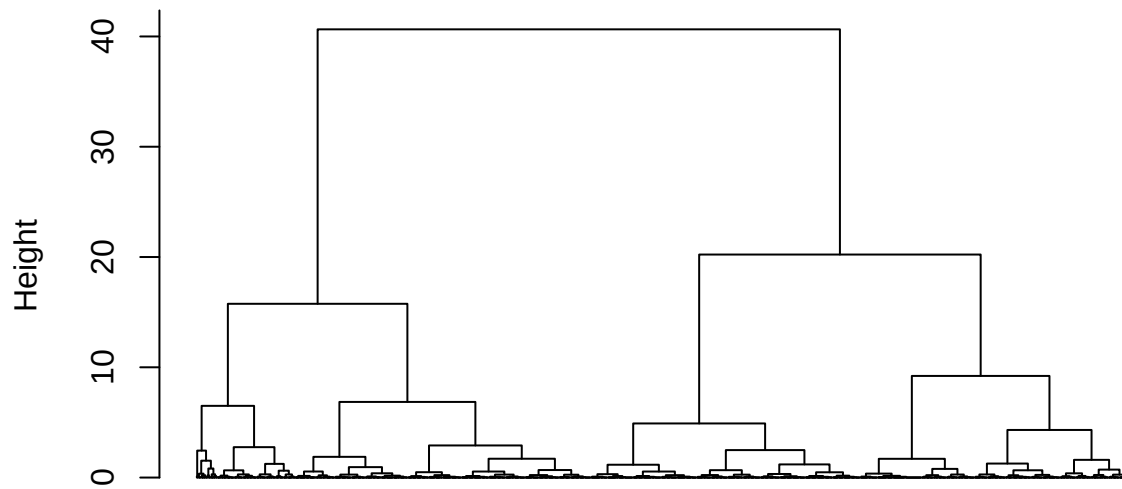
K-means Clustering with K = 5 (Silhouette Method)



Individual Clustering

```
y <- per_min_data$NBA_FG3A[indices]
sel <- indices[ which(!is.na(per_min_data$NBA_FG3A[indices])) ]
x <- na.omit(per_min_data$NBA_FG3A[indices])
x_scaled <- scale(x)
d <- dist(x_scaled)
hc <- hclust(d, method = "ward.D2")
plot(hc, labels = FALSE, hang = -1,
     main = "Hierarchical Clustering Dendrogram (NBA_FG3A)")
```

Hierarchical Clustering Dendrogram (NBA_FG3A)



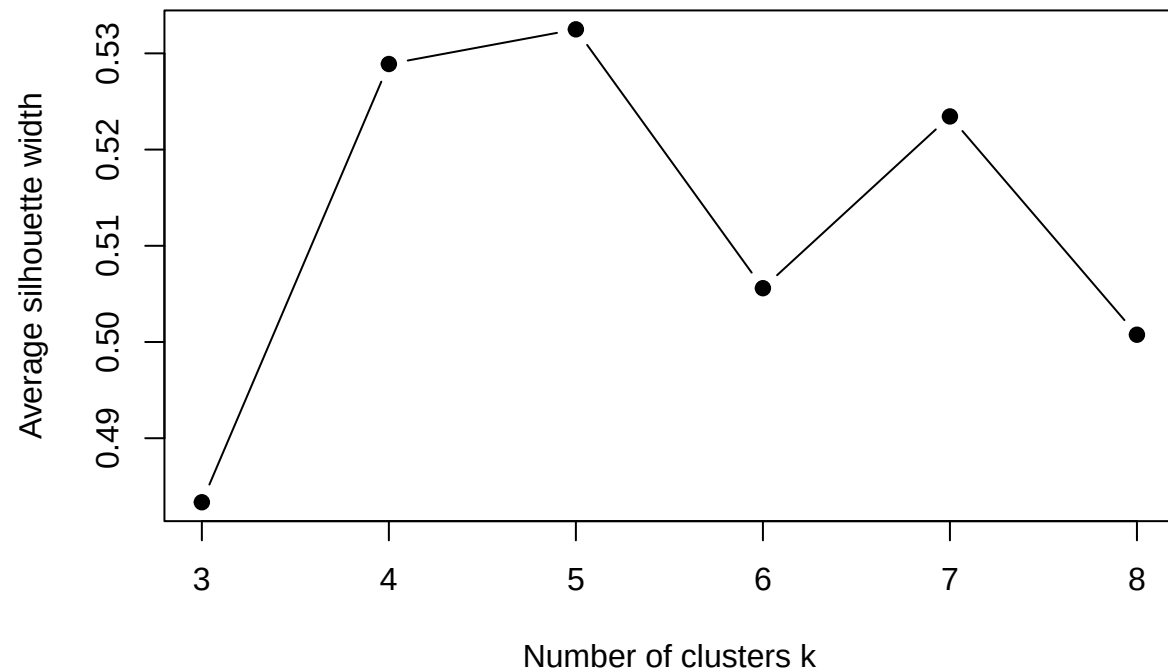
d
hclust (*, "ward.D2")

```
sil_width <- numeric(20)

for (k in 3:8) {
  clusters <- cutree(hc, k)
  ss <- silhouette(clusters, d)
  sil_width[k] <- mean(ss[, 3])
}

plot(3:8, sil_width[3:8], type = "b", pch = 19,
     xlab = "Number of clusters k",
     ylab = "Average silhouette width",
     main = "Silhouette Method (NBA_FG3A)")
```

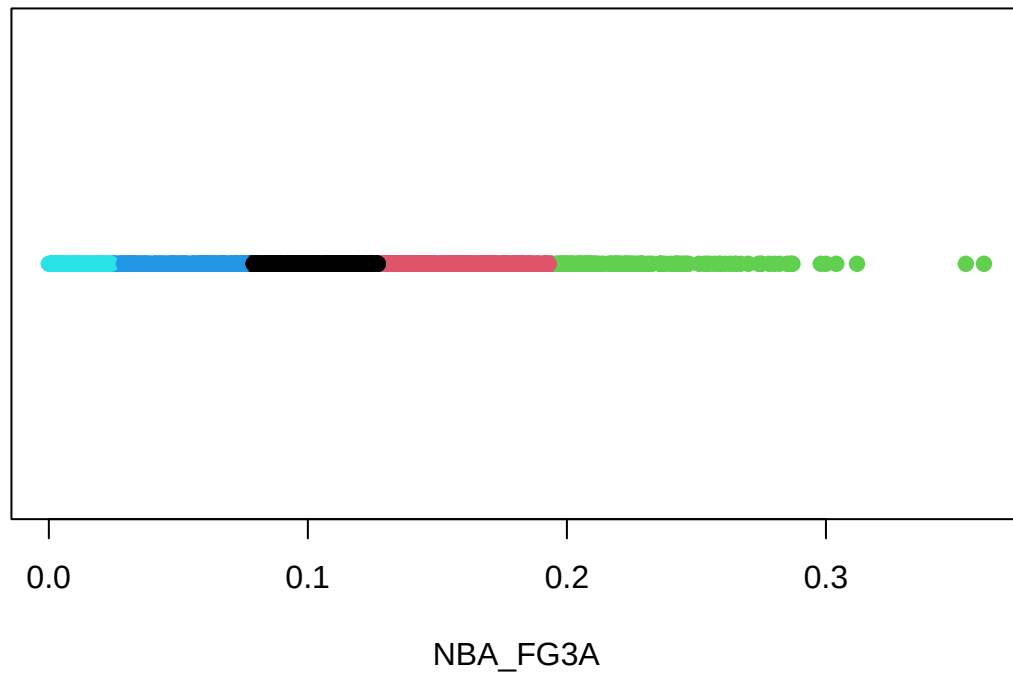
Silhouette Method (NBA_FG3A)



```
best_k <- which.max(sil_width)
clusters_best <- cutree(hc, k = best_k)

plot(x, rep(0, length(x)), col = clusters_best, pch = 19,
     xlab = "NBA_FG3A", ylab = "",
     yaxt = "n", main = paste("Hierarchical Clustering on NBA_FG3A (k =", best_k, ")"))
```


Hierarchical Clustering on NBA_FG3A (k = 5)



```
cluster_medians <- tapply(x, clusters_best, median)
ordered_clusters <- names(sort(cluster_medians, decreasing = FALSE))
new_label_map <- setNames(seq_along(ordered_clusters), ordered_clusters)
clusters_ordered <- as.integer(new_label_map[as.character(clusters_best)])
per_min_data$Class_3PA <- NA_integer_
per_min_data$Class_3PA[sel] <- clusters_ordered
per_min_data$Class_3PA[is.na(per_min_data$Class_3PA)] <- 1
per_min_data$Class_3PA <- factor(per_min_data$Class_3PA,
                                levels = as.character(1:best_k),
                                ordered = TRUE)
round(tapply(x, per_min_data$Class_3PA[sel], mean), 3)
```

```
##      1      2      3      4      5
## 0.009 0.054 0.104 0.158 0.230
```

```
table(per_min_data$Class_3PA, useNA = "ifany")
```

```
##
##      1      2      3      4      5
## 344 218 372 409 136
```

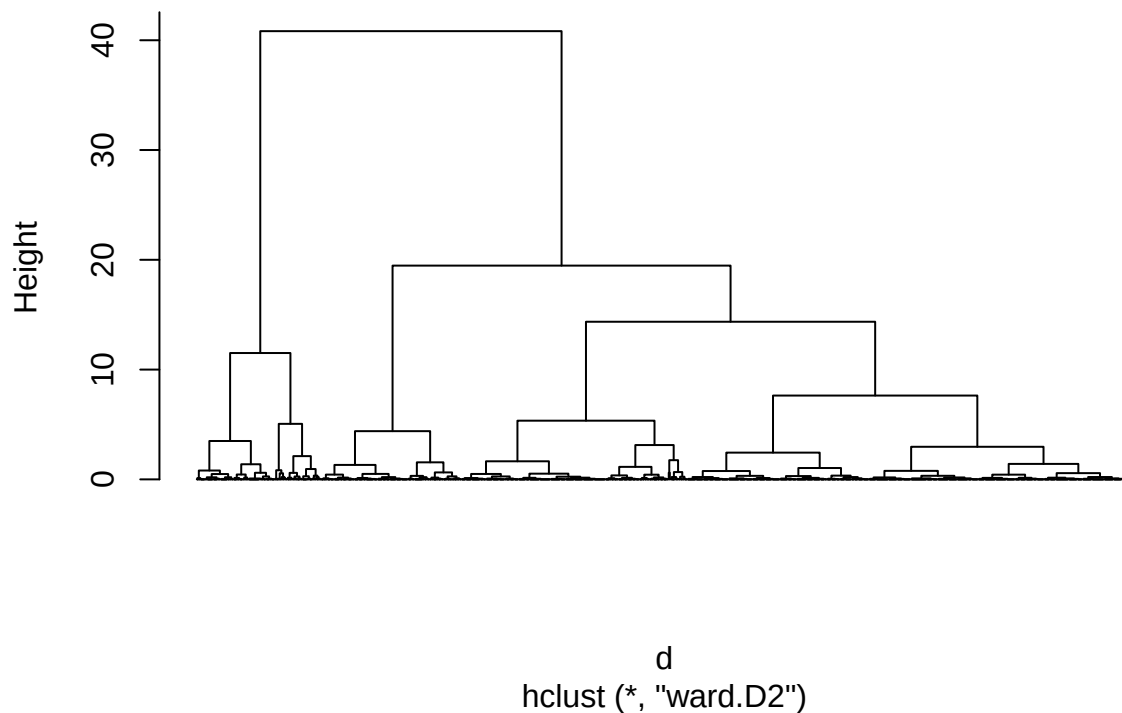
```
#head(per_min_data)
```

```

y <- per_min_data$NBA_FG3_PCT[indices]
not_na_idx <- which(!is.na(y))
sel <- indices[ which(!is.na(per_min_data$NBA_FG3_PCT[indices])) ]
x <- na.omit(data$NBA_FG3_PCT[indices])
x_scaled <- scale(x)
d <- dist(x_scaled)
hc <- hclust(d, method = "ward.D2")
plot(hc, labels = FALSE, hang = -1,
     main = "Hierarchical Clustering Dendrogram (NBA_FG3_PCT)")

```

Hierarchical Clustering Dendrogram (NBA_FG3_PCT)



```

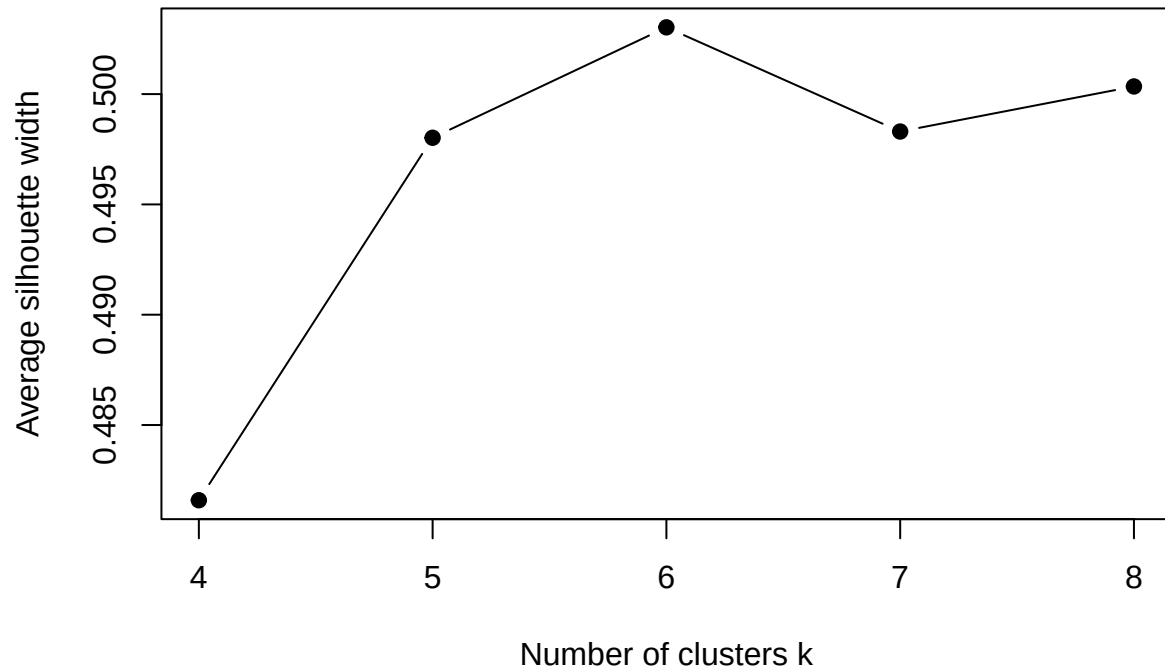
sil_width <- numeric(20)

for (k in 4:8) {
  clusters <- cutree(hc, k)
  ss <- silhouette(clusters, d)
  sil_width[k] <- mean(ss[, 3])
}

plot(4:8, sil_width[4:8], type = "b", pch = 19,
     xlab = "Number of clusters k",
     ylab = "Average silhouette width",
     main = "Silhouette Method (NBA_FG3_PCT)")

```

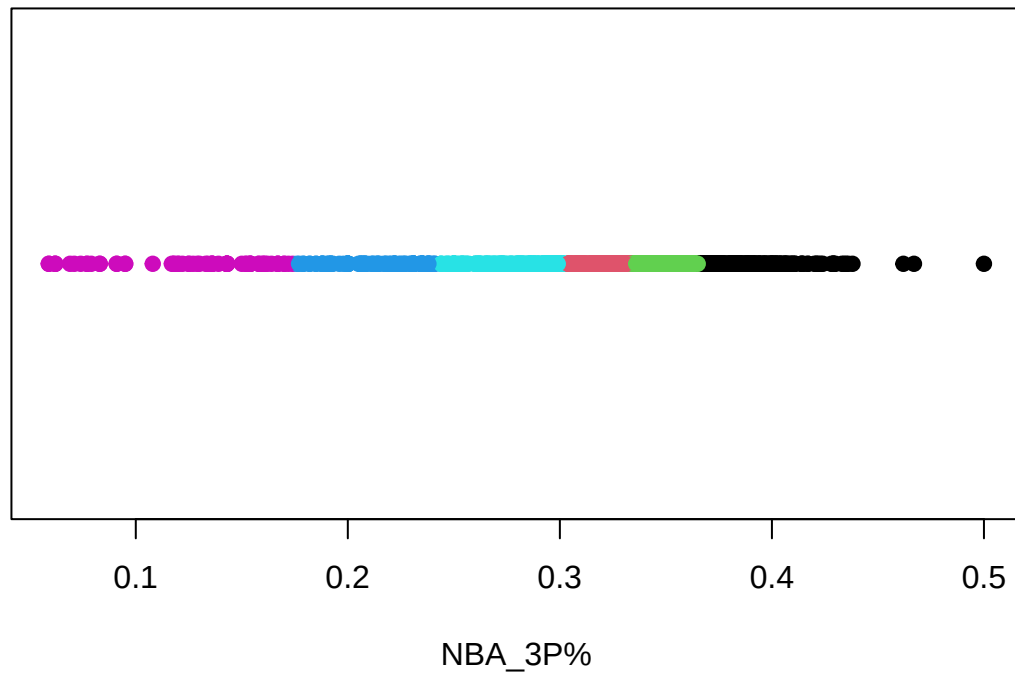
Silhouette Method (NBA_FG3_PCT)



```
best_k <- which.max(sil_width)
clusters_best <- cutree(hc, k = best_k)

plot(x, rep(0, length(x)), col = clusters_best, pch = 19,
     xlab = "NBA_3P%", ylab = "",
     yaxt = "n", main = paste("Hierarchical Clustering on NBA_FG3_PCT (k =", best_k, ")"))
```

Hierarchical Clustering on NBA_FG3_PCT (k = 6)



```
cluster_medians <- tapply(x, clusters_best, median)
ordered_clusters <- names(sort(cluster_medians, decreasing = FALSE))
new_label_map <- setNames(seq_along(ordered_clusters), ordered_clusters)
clusters_ordered <- as.integer(new_label_map[as.character(clusters_best)])
per_min_data$Class_3P_PCT <- NA_integer_
per_min_data$Class_3P_PCT[sel] <- clusters_ordered
per_min_data$Class_3P_PCT[is.na(per_min_data$Class_3P_PCT)] <- 1
per_min_data$Class_3P_PCT <- factor(per_min_data$Class_3P_PCT,
                                   levels = as.character(1:best_k),
                                   ordered = TRUE)
round(tapply(x, per_min_data$Class_3P_PCT[sel], median), 3)
```

```
##      1      2      3      4      5      6
## 0.136 0.214 0.286 0.321 0.350 0.382
```

```
table(per_min_data$Class_3P_PCT, useNA = "ifany")
```

```
##
##      1      2      3      4      5      6
## 250 108 194 251 364 312
```

```
#head(per_min_data)
```

Assign Clusters to the dataset

```
# Assign the cluster numbers
per_min_data$Cluster3Shooting <- NA_integer_
valid_rows <- indices[complete.cases(per_min_data[indices, c("NBA_FG3A", "NBA_FG3_PCT")])]
per_min_data$Cluster3Shooting[valid_rows] <- km_best$cluster
table(per_min_data$Cluster3Shooting)
```

```
##
##   1   2   3   4   5
## 425 148 267 304 149
```

```
# Rearrange for any functions that may still use ordinal
per_min_data$Cluster3Shooting_reordered <- dplyr::case_when(
  per_min_data$Cluster3Shooting == 2 ~ 1L,
  per_min_data$Cluster3Shooting == 5 ~ 2L,
  per_min_data$Cluster3Shooting == 4 ~ 3L,
  per_min_data$Cluster3Shooting == 1 ~ 4L,
  per_min_data$Cluster3Shooting == 3 ~ 5L,
  TRUE ~ 1L
)

per_min_data$Cluster3Shooting_reordered <- dplyr::case_when(
  per_min_data$NBA_FG3_PCT < 0.3 & per_min_data$Cluster3Shooting_reordered == 5L ~ 2L,
  TRUE ~ per_min_data$Cluster3Shooting_reordered
)
```

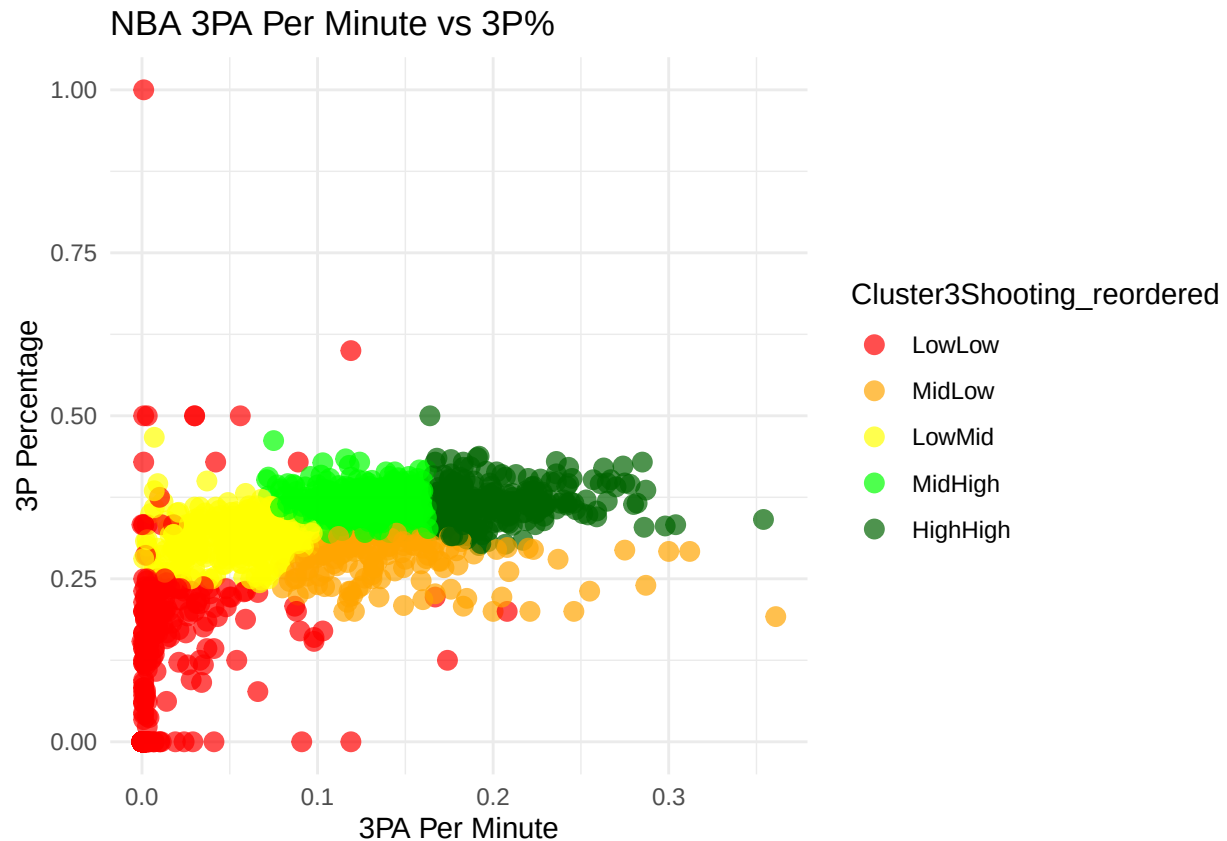
```
# Check summary
table(per_min_data$Cluster3Shooting_reordered)
```

```
##
##   1   2   3   4   5
## 334 159 304 425 257
```

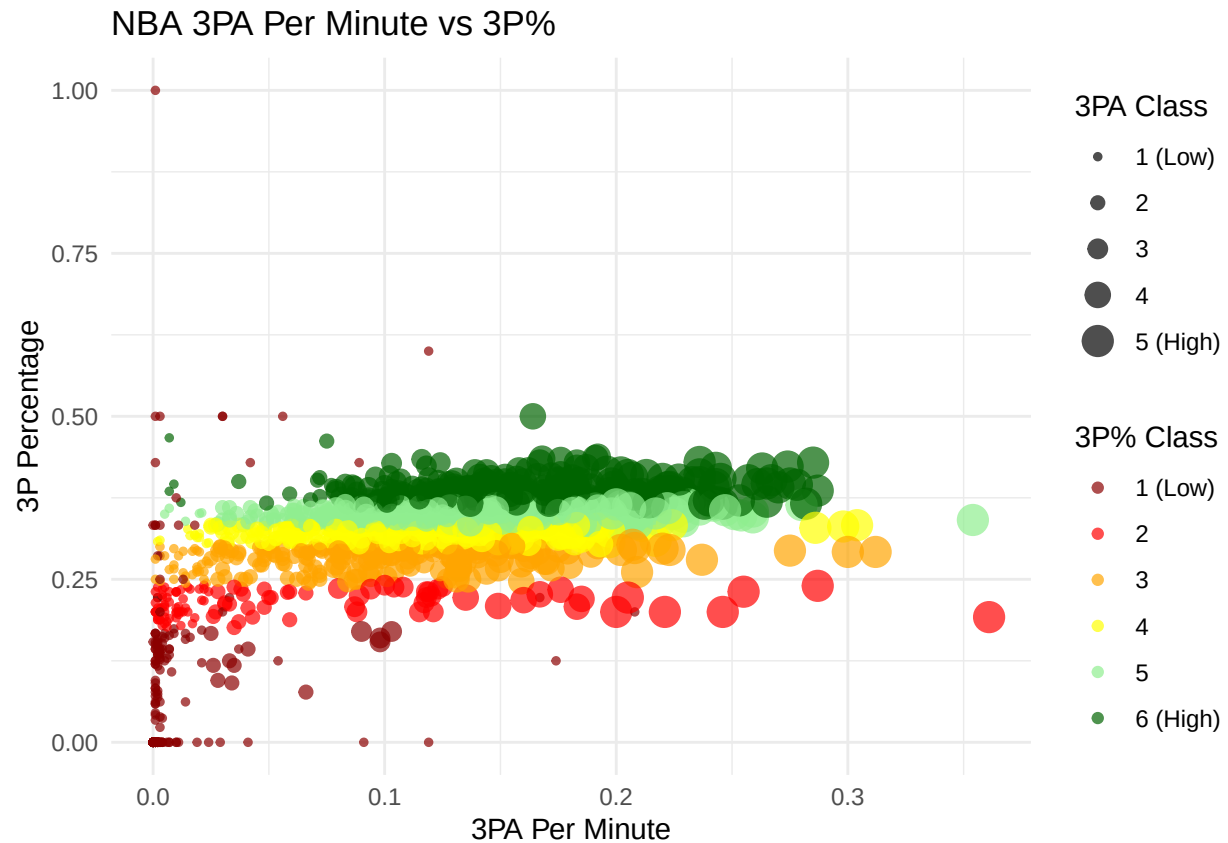
```
# Set labels
class_levels <- c("LowLow", "MidLow", "LowMid", "MidHigh", "HighHigh")
per_min_data$Cluster3Shooting_reordered <- factor(per_min_data$Cluster3Shooting_reordered,
  levels = 1:5,
  labels = class_levels,
  ordered = TRUE)
```

```
# Graph clusters with better colors
ggplot(per_min_data, aes(x = NBA_FG3A, y = NBA_FG3_PCT, color = Cluster3Shooting_reordered)) +
  geom_point(size = 3, alpha = 0.7) +
  scale_color_manual(
    values = c("LowLow" = "red",
              "MidLow" = "orange",
              "LowMid" = "yellow",
              "MidHigh" = "green",
              "HighHigh" = "darkgreen")
  ) +
```

```
labs(
  title = "NBA 3PA Per Minute vs 3P%",
  x = "3PA Per Minute",
  y = "3P Percentage"
) +
theme_minimal()
```



```
ggplot(per_min_data, aes(x = NBA_FG3A, y = NBA_FG3_PCT)) +
  geom_point(aes(color = factor(Class_3P_PCT), size = factor(Class_3PA)), alpha = 0.7) +
  scale_color_manual(name = "3P% Class", values = c("1" = "darkred", "2" = "red", "3" = "orange",
    "4" = "yellow", "5" = "lightgreen", "6" = "darkgreen"),
    labels = c("1 (Low)", "2", "3", "4", "5", "6 (High)")) +
  scale_size_manual(name = "3PA Class", values = c("1" = 1, "2" = 2, "3" = 3, "4" = 4,
    "5" = 5),
    labels = c("1 (Low)", "2", "3", "4", "5 (High)")) +
  labs(title = "NBA 3PA Per Minute vs 3P%", x = "3PA Per Minute", y = "3P Percentage") +
  theme_minimal()
```



Initialize Train / Test Split

```
# Specific set of predictors mentioned in report
X <- per_min_data[, c(8,13,38:40,42,43,45,46,48:50, 52:55, 56:66)]
X[is.na(X)] <- 0
y <- factor(per_min_data[, ncol(per_min_data)], levels = class_levels)

df <- data.frame(X, response = y)
df_scaled <- data.frame(scale(df[, -ncol(df)]), response = df$response)

# Split train and test
set.seed(67)
train_index <- createDataPartition(df_scaled$response, p = 0.8, list = FALSE)
train_data <- df_scaled[train_index, ]
test_data <- df_scaled[-train_index, ]
```

Backwards Stepwise Selection

Check for collinearity

```
vif(step_model)
```

```
##      y>=MidLow  y>=LowMid  y>=MidHigh y>=HighHigh    PRE_3PA  PRE_3P_PCT
##      1.978648   2.304373   1.549787   1.331524   1.769551   1.339271
## PRE_FT_PCT    PRE_TOV
##      1.523065   1.026334
```

```
predictors <- df_scaled[, c("PRE_FGA", "PRE_FG_PCT", "PRE_3PA",
                             "PRE_FTA", "PRE_DRB", "PRE_BLK", "PRE_TOV", "PRE_PTS")]
cor_matrix <- cor(predictors)
print(round(cor_matrix, 3))
```

```
##          PRE_FGA PRE_FG_PCT PRE_3PA PRE_FTA PRE_DRB PRE_BLK PRE_TOV PRE_PTS
## PRE_FGA      1.000   -0.230   0.434   0.337  -0.039  -0.190   0.279   0.853
## PRE_FG_PCT  -0.230    1.000  -0.699   0.323   0.368   0.609  -0.035   0.151
## PRE_3PA      0.434   -0.699   1.000  -0.350  -0.304  -0.579  -0.083   0.206
## PRE_FTA      0.337   0.323  -0.350   1.000   0.185   0.245   0.408   0.615
## PRE_DRB     -0.039   0.368  -0.304   0.185   1.000   0.373   0.005   0.079
## PRE_BLK     -0.190   0.609  -0.579   0.245   0.373   1.000  -0.016   0.002
## PRE_TOV      0.279  -0.035  -0.083   0.408   0.005  -0.016   1.000   0.278
## PRE_PTS      0.853   0.151   0.206   0.615   0.079   0.002   0.278   1.000
```

Our main issue is between PPG and FGA, check both.

```
noPTS <- polr(response ~ PRE_FGA+PRE_FG_PCT+PRE_3PA+PRE_FTA+PRE_DRB+PRE_BLK+PRE_TOV, data = train_data,
noFGA <- polr(response ~ PRE_FG_PCT+PRE_3PA+PRE_FTA+PRE_DRB+PRE_BLK+PRE_TOV+PRE_PTS, data = train_data,

AIC(noPTS);AIC(noFGA)
```

```
## [1] 3092.504
```

```
## [1] 3084.665
```

Because the no FGA model has the lower AIC, it is the better model because we chose AIC as our criterion.

```
summary(noFGA)
```

```
## Call:
## polr(formula = response ~ PRE_FG_PCT + PRE_3PA + PRE_FTA + PRE_DRB +
##      PRE_BLK + PRE_TOV + PRE_PTS, data = train_data, Hess = TRUE)
##
## Coefficients:
##              Value Std. Error t value
## PRE_FG_PCT -0.1289    0.10210  -1.262
```



```
## PRE_3PA      1.0900    0.12490    8.727
## PRE_FTA     -0.3250    0.09995   -3.252
## PRE_DRB      0.1751    0.06435    2.721
## PRE_BLK     -0.4367    0.08509   -5.132
## PRE_TOV     -0.2594    0.06238   -4.158
## PRE_PTS      0.2969    0.10379    2.860
##
## Intercepts:
##              Value      Std. Error t value
## LowLow|MidLow   -1.7440    0.0876   -19.9160
## MidLow|LowMid   -0.9783    0.0754   -12.9824
## LowMid|MidHigh    0.1876    0.0693    2.7093
## MidHigh|HighHigh  2.1728    0.0937   23.1858
##
## Residual Deviance: 3062.665
## AIC: 3084.665
```

```
vif(noFGA)
```

```
##          PRE_FG_PCT      PRE_3PA      PRE_FTA      PRE_DRB
##          3.090255      4.184378      3.083794      1.320215
##          PRE_BLK      PRE_TOV      PRE_PTS      LowLow|MidLow
##          1.808852      1.302522      3.477962      2.097783
##          MidLow|LowMid  LowMid|MidHigh  MidHigh|HighHigh
##          2.458664      1.606307      1.334068
```

```
predictors <- df_scaled[, c("PRE_FG_PCT", "PRE_3PA",
  "PRE_FTA", "PRE_DRB", "PRE_BLK", "PRE_TOV", "PRE_PTS")]
cor_matrix <- cor(predictors)
print(round(cor_matrix, 3))
```

```
##          PRE_FG_PCT PRE_3PA PRE_FTA PRE_DRB PRE_BLK PRE_TOV PRE_PTS
## PRE_FG_PCT      1.000 -0.699  0.323  0.368  0.609 -0.035  0.151
## PRE_3PA         -0.699  1.000 -0.350 -0.304 -0.579 -0.083  0.206
## PRE_FTA          0.323 -0.350  1.000  0.185  0.245  0.408  0.615
## PRE_DRB          0.368 -0.304  0.185  1.000  0.373  0.005  0.079
## PRE_BLK          0.609 -0.579  0.245  0.373  1.000 -0.016  0.002
## PRE_TOV         -0.035 -0.083  0.408  0.005 -0.016  1.000  0.278
## PRE_PTS          0.151  0.206  0.615  0.079  0.002  0.278  1.000
```

Initializing Storing Results

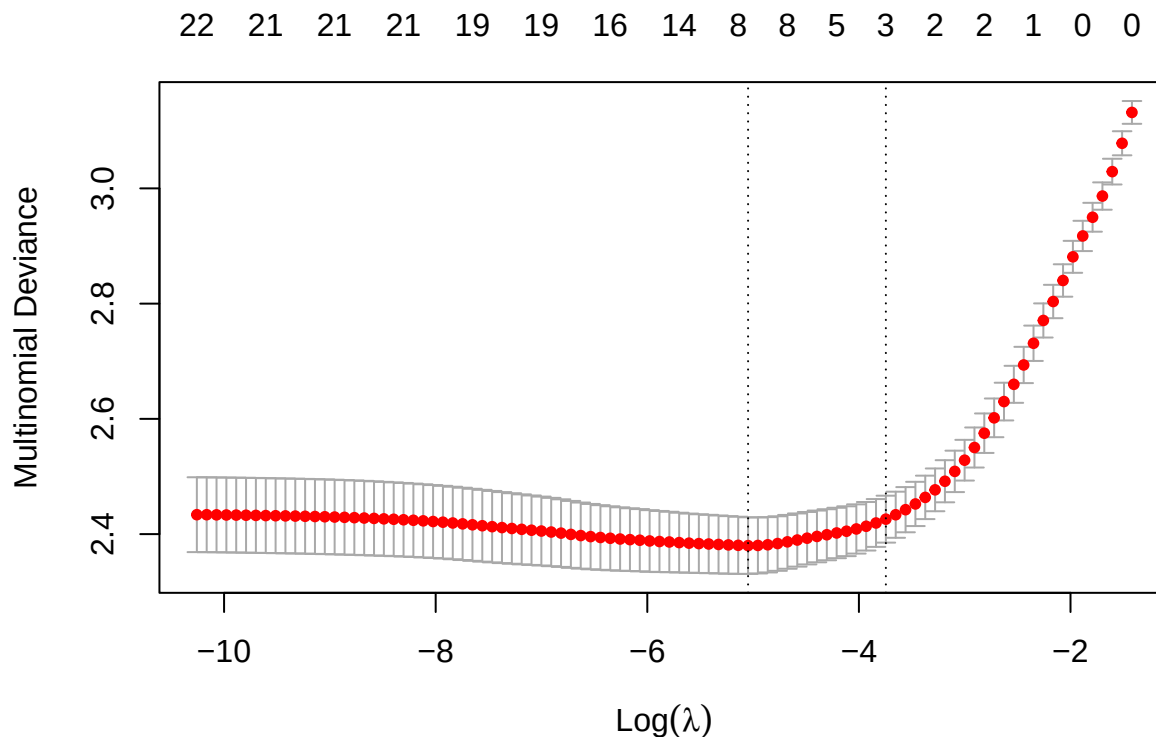
```
model_results <- data.frame(
  Model = character(),
  Accuracy = numeric(),
  Precision = numeric(),
  Recall = numeric(),
  F1 = numeric(),
  AUC = numeric(),
  stringsAsFactors = FALSE
)
```

LASSO with CV for best lambda

```
set.seed(67)
#initialize data in matrices
x_train <- as.matrix(train_data[, -ncol(train_data)])
y_train <- train_data$response
x_test  <- as.matrix(test_data[, -ncol(test_data)])
y_test  <- test_data$response

#run CV model
cv_lasso <- cv.glmnet(x_train, y_train,
                      family = "multinomial",
                      alpha = 1,
                      nfolds = 10)

plot(cv_lasso)
```



```
# Check optimal values of lambda
cat("Optimal lambda (min):", cv_lasso$lambda.min, "\n")
```

```
## Optimal lambda (min): 0.006424854
```

```
cat("Optimal lambda (1se):", cv_lasso$lambda.1se, "\n")
```

```
## Optimal lambda (1se): 0.02363305
```

```
coef(cv_lasso, s = "lambda.min")
```

```
## $LowLow
## 28 x 1 sparse Matrix of class "dgCMatrix"
##              1
## (Intercept)  -0.52297855
## WEIGHT       .
## HEIGHT_INCHES .
## PRE_G        .
## PRE_GS       .
## PRE_MP       0.18467284
## PRE_FGA      -0.02784222
## PRE_FG_PCT   0.05910089
## PRE_3PA      -2.04544408
## PRE_3P_PCT   -0.16914489
## PRE_2PA      .
## PRE_2P_PCT   .
## PRE_eFG_PCT  .
## PRE_FTA      0.01578232
## PRE_FT_PCT   -0.20201991
## PRE_ORB      .
## PRE_DRB      .
## PRE_TRB      .
## PRE_AST      -0.08650347
## PRE_STL      .
## PRE_BLK      0.31615267
## PRE_TOV      0.21704931
## PRE_PF       .
## PRE_PTS      .
## PRE_FG_pct_inc .
## PRE_3P_pct_inc -0.01882526
## PRE_2P_pct_inc 0.21306929
## PRE_FT_pct_inc 0.01023089
##
## $MidLow
## 28 x 1 sparse Matrix of class "dgCMatrix"
##              1
## (Intercept)  -0.25602980
## WEIGHT       .
## HEIGHT_INCHES .
## PRE_G        .
## PRE_GS       .
## PRE_MP       .
## PRE_FGA      .
## PRE_FG_PCT   -0.25953912
## PRE_3PA      0.04494135
## PRE_3P_PCT   .
## PRE_2PA      .
## PRE_2P_PCT   .
## PRE_eFG_PCT  .
## PRE_FTA      .
## PRE_FT_PCT   .
## PRE_ORB      .
```

```

## PRE_DRB      .
## PRE_TRB      .
## PRE_AST      .
## PRE_STL      .
## PRE_BLK      .
## PRE_TOV      -0.07777835
## PRE_PF       .
## PRE_PTS      .
## PRE_FG_pct_inc .
## PRE_3P_pct_inc .
## PRE_2P_pct_inc -0.07039046
## PRE_FT_pct_inc 0.10931940
##
## $LowMid
## 28 x 1 sparse Matrix of class "dgCMatrix"
##           1
## (Intercept) 0.442708286
## WEIGHT      .
## HEIGHT_INCHES .
## PRE_G       .
## PRE_GS      .
## PRE_MP      .
## PRE_FGA     0.102919747
## PRE_FG_PCT  .
## PRE_3PA     -1.072860097
## PRE_3P_PCT  .
## PRE_2PA     .
## PRE_2P_PCT  -0.009818944
## PRE_eFG_PCT .
## PRE_FTA     0.088996526
## PRE_FT_PCT  .
## PRE_ORB     0.079663585
## PRE_DRB     .
## PRE_TRB     .
## PRE_AST     .
## PRE_STL     .
## PRE_BLK     .
## PRE_TOV     0.131784086
## PRE_PF      .
## PRE_PTS     .
## PRE_FG_pct_inc .
## PRE_3P_pct_inc 0.031249863
## PRE_2P_pct_inc .
## PRE_FT_pct_inc -0.060626965
##
## $MidHigh
## 28 x 1 sparse Matrix of class "dgCMatrix"
##           1
## (Intercept) 0.805897141
## WEIGHT      -0.036151118
## HEIGHT_INCHES .
## PRE_G       -0.082033996
## PRE_GS      .
## PRE_MP      .

```

```

## PRE_FGA      .
## PRE_FG_PCT   .
## PRE_3PA      .
## PRE_3P_PCT   0.084010361
## PRE_2PA      .
## PRE_2P_PCT   .
## PRE_eFG_PCT  .
## PRE_FTA      -0.066255771
## PRE_FT_PCT   0.098382260
## PRE_ORB      0.131880020
## PRE_DRB      .
## PRE_TRB      .
## PRE_AST      .
## PRE_STL      .
## PRE_BLK      .
## PRE_TOV      .
## PRE_PF       -0.007737816
## PRE_PTS      .
## PRE_FG_pct_inc .
## PRE_3P_pct_inc .
## PRE_2P_pct_inc 0.064541103
## PRE_FT_pct_inc .
##
## $HighHigh
## 28 x 1 sparse Matrix of class "dgCMatrix"
##              1
## (Intercept)  -0.46959707
## WEIGHT       0.01410073
## HEIGHT_INCHES .
## PRE_G        .
## PRE_GS       .
## PRE_MP       .
## PRE_FGA      .
## PRE_FG_PCT   .
## PRE_3PA      0.79901229
## PRE_3P_PCT   .
## PRE_2PA      .
## PRE_2P_PCT   .
## PRE_eFG_PCT  0.14678369
## PRE_FTA      -0.21619745
## PRE_FT_PCT   0.41246760
## PRE_ORB      -0.29654640
## PRE_DRB      0.16196171
## PRE_TRB      .
## PRE_AST      .
## PRE_STL      .
## PRE_BLK      .
## PRE_TOV      -0.37048881
## PRE_PF       0.15730980
## PRE_PTS      .
## PRE_FG_pct_inc .
## PRE_3P_pct_inc -0.07105415
## PRE_2P_pct_inc .
## PRE_FT_pct_inc .

```

```
# Calculate Test Accuracy
preds_min <- predict(cv_lasso, newx = x_test, s = "lambda.min", type = "class")
accuracy_min <- mean(as.character(preds_min) == as.character(y_test))
cat("Test accuracy (lambda.min):", accuracy_min, "\n")
```

```
## Test accuracy (lambda.min): 0.4778157
```

```
# Calculate CM
preds_min <- factor(preds_min, levels = levels(y_test))
conf_mat_min <- confusionMatrix(preds_min, y_test, mode = "prec_recall")
print(conf_mat_min)
```

```
## Confusion Matrix and Statistics
```

```
##
##           Reference
## Prediction LowLow MidLow LowMid MidHigh HighHigh
##   LowLow      48      2      16      10      2
##   MidLow       0      0       0       0      0
##   LowMid      10      1      13       6      0
##   MidHigh      8     20      31      50     20
##   HighHigh     0      8       0      19     29
```

```
## Overall Statistics
```

```
##
##           Accuracy : 0.4778
##           95% CI : (0.4194, 0.5367)
##   No Information Rate : 0.2901
##   P-Value [Acc > NIR] : 9.686e-12
##
##           Kappa : 0.3112
```

```
## McNemar's Test P-Value : NA
```

```
##
```

```
## Statistics by Class:
```

```
##
```

```
##           Class: LowLow Class: MidLow Class: LowMid Class: MidHigh
## Precision           0.6154           NA           0.43333           0.3876
## Recall              0.7273           0.0000           0.21667           0.5882
## F1                  0.6667           NA           0.28889           0.4673
## Prevalence          0.2253           0.1058           0.20478           0.2901
## Detection Rate      0.1638           0.0000           0.04437           0.1706
## Detection Prevalence 0.2662           0.0000           0.10239           0.4403
## Balanced Accuracy    0.7976           0.5000           0.57185           0.6042
```

```
##           Class: HighHigh
```

```
## Precision           0.51786
## Recall              0.56863
## F1                  0.54206
## Prevalence          0.17406
## Detection Rate      0.09898
## Detection Prevalence 0.19113
## Balanced Accuracy    0.72853
```

```

# Calculate Class Error
class_error_min <- 1 - diag(conf_mat_min$table) / colSums(conf_mat_min$table)
print(class_error_min)

##      LowLow      MidLow      LowMid      MidHigh      HighHigh
## 0.2727273 1.0000000 0.7833333 0.4117647 0.4313725

# AUC Vectors
probs_min <- predict(cv_lasso, newx = x_test, s = "lambda.min", type = "response")
probs_min <- probs_min[,1]
colnames(probs_min) <- levels(y_test)

# Repeat
preds_1se <- predict(cv_lasso, newx = x_test, s = "lambda.1se", type = "class")
accuracy_1se <- mean(as.character(preds_1se) == as.character(y_test))
cat("Test accuracy (lambda.1se):", accuracy_1se, "\n")

## Test accuracy (lambda.1se): 0.4812287

preds_1se <- factor(preds_1se, levels = levels(y_test))
conf_mat_1se <- confusionMatrix(preds_1se, y_test, mode = "prec_recall")
print(conf_mat_1se)

## Confusion Matrix and Statistics
##
##              Reference
## Prediction LowLow MidLow LowMid MidHigh HighHigh
##   LowLow      51      2      17      11      2
##   MidLow       0      0       0       0      0
##   LowMid       5      1      10       6      1
##   MidHigh     10     23     33     55     23
##   HighHigh     0      5       0     13     25
##
## Overall Statistics
##
##              Accuracy : 0.4812
##              95% CI : (0.4228, 0.5401)
##   No Information Rate : 0.2901
##   P-Value [Acc > NIR] : 4.251e-12
##
##              Kappa : 0.3101
##
##   McNemar's Test P-Value : 1.472e-09
##
## Statistics by Class:
##
##              Class: LowLow Class: MidLow Class: LowMid Class: MidHigh
## Precision              0.6145              NA              0.43478              0.3819
## Recall                  0.7727              0.0000              0.16667              0.6471
## F1                      0.6846              NA              0.24096              0.4803
## Prevalence              0.2253              0.1058              0.20478              0.2901
## Detection Rate          0.1741              0.0000              0.03413              0.1877

```

```
## Detection Prevalence      0.2833      0.0000      0.07850      0.4915
## Balanced Accuracy        0.8159      0.5000      0.55544      0.6096
##                           Class: HighHigh
## Precision                0.58140
## Recall                   0.49020
## F1                       0.53191
## Prevalence               0.17406
## Detection Rate           0.08532
## Detection Prevalence     0.14676
## Balanced Accuracy        0.70791
```

```
class_error_1se <- 1 - diag(conf_mat_1se$table) / colSums(conf_mat_1se$table)
print(class_error_1se)
```

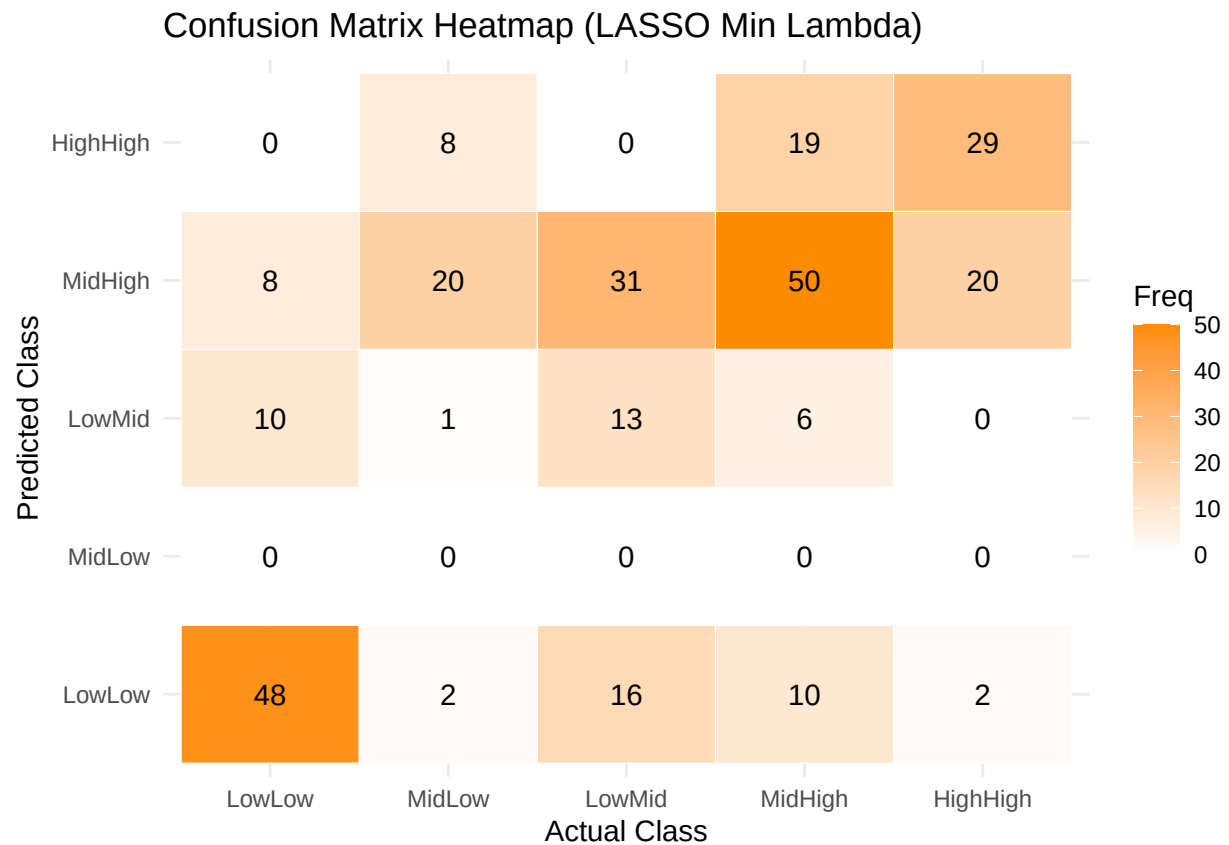
```
##      LowLow      MidLow      LowMid      MidHigh      HighHigh
## 0.2272727 1.0000000 0.8333333 0.3529412 0.5098039
```

```
probs_1se <- predict(cv_lasso, newx = x_test, s = "lambda.1se", type = "response")
probs_1se <- probs_1se[,1]
colnames(probs_1se) <- levels(y_test)
```

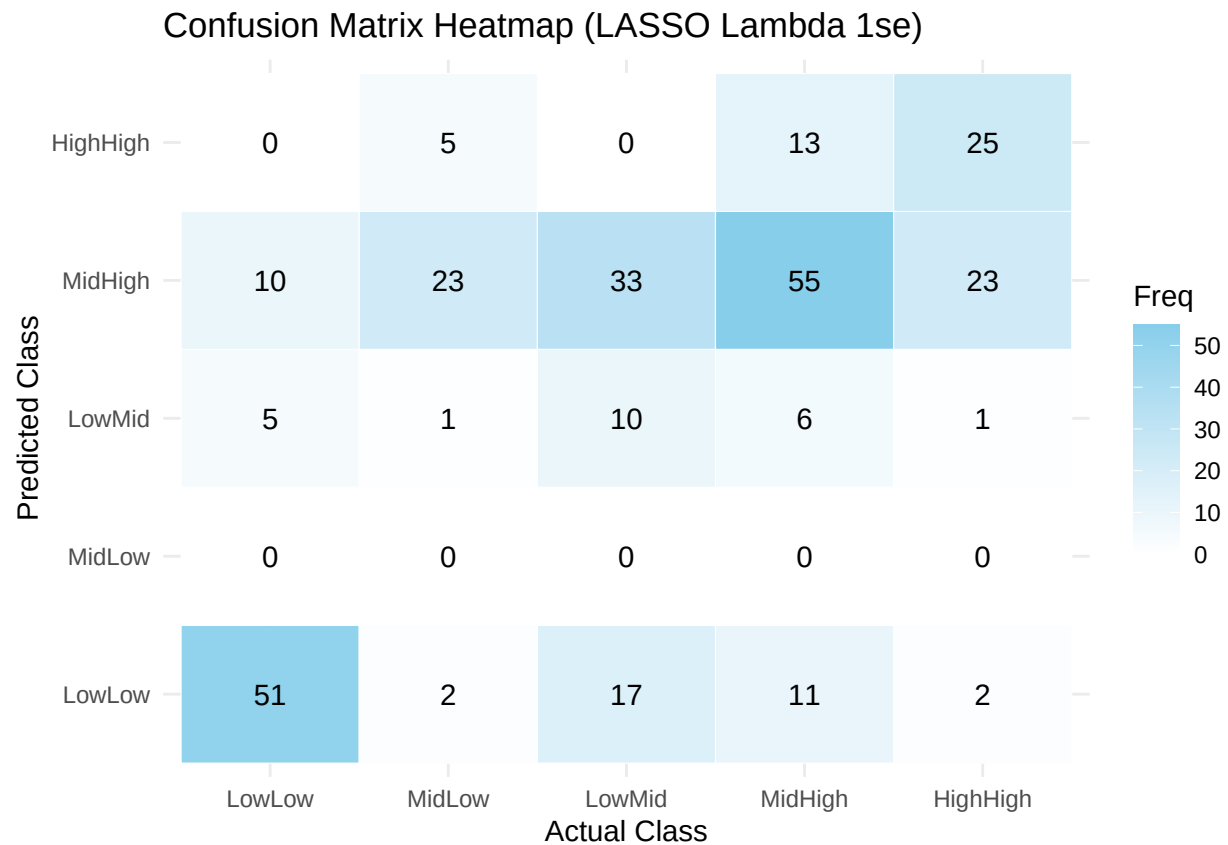
```
# Load model results (done for every model)
model_results <- rbind(model_results, data.frame(
  Model = "LASSO Min Lambda",
  Accuracy = accuracy_min,
  Precision = mean(conf_mat_min$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(conf_mat_min$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(conf_mat_min$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(y_test, probs_min)
))

model_results <- rbind(model_results, data.frame(
  Model = "LASSO 1se Lambda",
  Accuracy = accuracy_1se,
  Precision = mean(conf_mat_1se$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(conf_mat_1se$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(conf_mat_1se$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(y_test, probs_1se)
))
```

```
# Create confusion matrix (done for every model)
cm_table <- as.data.frame(conf_mat_min$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "darkorange") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (LASSO Min Lambda)",
       x = "Actual Class", y = "Predicted Class")
```

```
cm_table <- as.data.frame(conf_mat_1se$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "skyblue") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (LASSO Lambda 1se)",
       x = "Actual Class", y = "Predicted Class")
```



LDA

```
set.seed(22222)
# Run cross validation
train_control <- trainControl(method = "cv", number = 10,
                              summaryFunction = multiClassSummary)

lda_cv <- train(response ~ PRE_FG_PCT + PRE_3PA + PRE_FTA + PRE_DRB +
                 PRE_BLK + PRE_TOV + PRE_PTS, data = train_data,
                 method = "lda",
                 trControl = train_control)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.
```

```
print(lda_cv)
```

```
## Linear Discriminant Analysis
##
## 1186 samples
## 7 predictor
## 5 classes: 'LowLow', 'MidLow', 'LowMid', 'MidHigh', 'HighHigh'
```

```
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1069, 1068, 1067, 1067, 1067, 1066, ...
## Resampling results:
##
##   Accuracy   Kappa     Mean_F1  Mean_Sensitivity  Mean_Specificity
##   0.5002687  0.3445918  NaN      0.4398053       0.8680822
##   Mean_Pos_Pred_Value  Mean_Neg_Pred_Value  Mean_Precision  Mean_Recall
##   NaN                0.87167                NaN        0.4398053
##   Mean_Detection_Rate  Mean_Balanced_Accuracy
##   0.1000537           0.6539438

cat("Cross-validated Accuracy (training set):", max(lda_cv$results$Accuracy), "\n")
```

```
## Cross-validated Accuracy (training set): 0.5002687
```

```
# Set fit model
lda_fit <- lda(response ~ PRE_FG_PCT + PRE_3PA + PRE_FTA + PRE_DRB +
               PRE_BLK + PRE_TOV + PRE_PTS,
               data = train_data)

pred_obj <- predict(lda_fit, test_data)

all_pred  <- factor(pred_obj$class, levels = class_levels)
all_actual <- factor(test_data$response, levels = class_levels)
all_probs <- pred_obj$posterior

# Find CM
conf_mat_lda <- confusionMatrix(all_pred, all_actual, mode = "prec_recall")
print(conf_mat_lda)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction LowLow MidLow LowMid MidHigh HighHigh
##   LowLow      44      2      13      10      1
##   MidLow       0      0       0       0      0
##   LowMid      13      1      21      12      3
##   MidHigh      9     22      26      41     18
##   HighHigh     0      6       0      22     29
##
## Overall Statistics
##
##           Accuracy : 0.4608
##           95% CI : (0.4026, 0.5197)
##   No Information Rate : 0.2901
##   P-Value [Acc > NIR] : 4.867e-10
##
##           Kappa : 0.2928
##
##   McNemar's Test P-Value : 1.322e-05
##
```

```
## Statistics by Class:
##
##
##           Class: LowLow Class: MidLow Class: LowMid Class: MidHigh
## Precision           0.6286           NA           0.42000           0.3534
## Recall              0.6667           0.0000           0.35000           0.4824
## F1                  0.6471           NA           0.38182           0.4080
## Prevalence          0.2253           0.1058           0.20478           0.2901
## Detection Rate      0.1502           0.0000           0.07167           0.1399
## Detection Prevalence 0.2389           0.0000           0.17065           0.3959
## Balanced Accuracy    0.7761           0.5000           0.61277           0.5609
##
##           Class: HighHigh
## Precision           0.50877
## Recall              0.56863
## F1                  0.53704
## Prevalence          0.17406
## Detection Rate      0.09898
## Detection Prevalence 0.19454
## Balanced Accuracy    0.72646
```

```
# Calculate Accuracy and AUC
```

```
LDA_acc <- mean(as.character(all_pred) == as.character(all_actual))
cat("Test Accuracy (LDA):", LDA_acc, "\n")
```

```
## Test Accuracy (LDA): 0.4607509
```

```
overall_multiclass <- multiclass.roc(all_actual, all_probs)
cat("Overall multiclass AUC (test set):", overall_multiclass$auc, "\n")
```

```
## Overall multiclass AUC (test set): 0.7513292
```

```
# Graph AUC / Calculate Macro AUC
```

```
fpr_grid <- seq(0, 1, length.out = 200)
tpr_matrix <- matrix(NA, nrow = length(fpr_grid), ncol = length(class_levels))
```

```
for (i in seq_along(class_levels)) {
  class <- class_levels[i]
  binary_actual <- factor(ifelse(all_actual == class, class, "Other"),
                           levels = c("Other", class))
  class_probs <- all_probs[, class]
```

```
roc_obj <- roc(response = binary_actual,
               predictor = class_probs,
               levels = c("Other", class),
               positive = class,
               quiet = TRUE)
```

```
cat("AUC of", class, ":", auc(roc_obj), "\n")
```

```
fpr <- 1 - roc_obj$specificities
tpr <- roc_obj$sensitivities
```

```
interp_tpr <- approx(fpr, tpr, xout = fpr_grid, ties = mean, rule = 2)$y
```

```
tpr_matrix[, i] <- interp_tpr
}
```

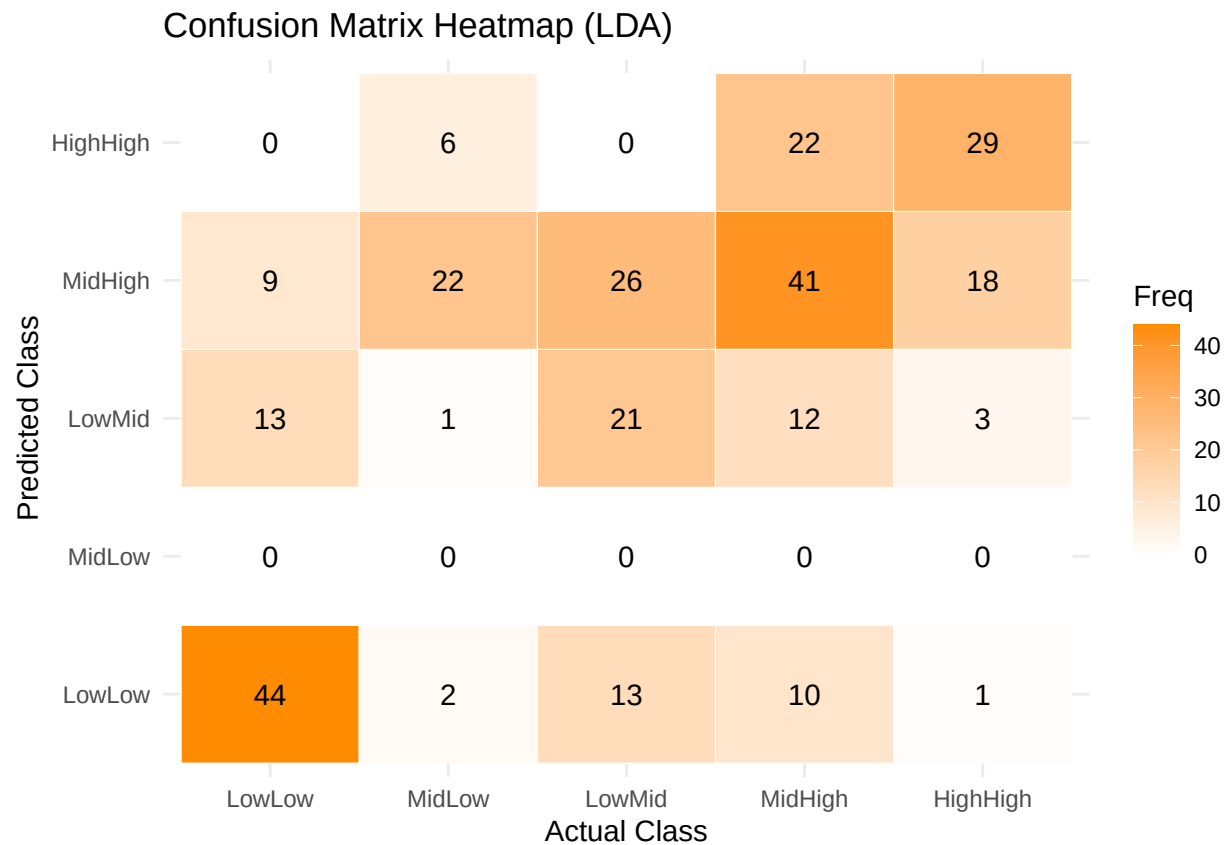
```
## AUC of LowLow : 0.8869977
## AUC of MidLow : 0.7089387
## AUC of LowMid : 0.7186695
## AUC of MidHigh : 0.6300905
## AUC of HighHigh : 0.8317939
```

```
macro_tpr_lda <- rowMeans(tpr_matrix, na.rm = TRUE)
macro_auc_lda <- trapz(fpr_grid, macro_tpr_lda)
cat("Macro AUC (LDA test set):", macro_auc_lda, "\n")
```

```
## Macro AUC (LDA test set): 0.7552616
```

```
model_results <- rbind(model_results, data.frame(
  Model = "LDA",
  Accuracy = LDA_acc,
  Precision = mean(conf_mat_lda$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(conf_mat_lda$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(conf_mat_lda$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc_lda
))
```

```
cm_table <- as.data.frame(conf_mat_lda$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "darkorange") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (LDA)",
       x = "Actual Class", y = "Predicted Class")
```



QDA

```
# Same steps as LDA
set.seed(22222)
train_control <- trainControl(method = "cv", number = 10,
                              summaryFunction = multiClassSummary)

qda_cv <- train(response ~ PRE_FG_PCT + PRE_3PA + PRE_FTA + PRE_DRB +
                 PRE_BLK + PRE_TOV + PRE_PTS,
                 data = train_data,
                 method = "qda",
                 trControl = train_control)

## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.

print(qda_cv)

## Quadratic Discriminant Analysis
##
## 1186 samples
##    7 predictor
```

```
## 5 classes: 'LowLow', 'MidLow', 'LowMid', 'MidHigh', 'HighHigh'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1069, 1068, 1067, 1067, 1067, 1066, ...
## Resampling results:
##
## Accuracy Kappa Mean_F1 Mean_Sensitivity Mean_Specificity
## 0.4486229 0.278488 NaN 0.3958338 0.8548818
## Mean_Pos_Pred_Value Mean_Neg_Pred_Value Mean_Precision Mean_Recall
## 0.3557292 0.8584005 0.3557292 0.3958338
## Mean_Detection_Rate Mean_Balanced_Accuracy
## 0.08972457 0.6253578

cat("Cross-validated Accuracy (training set):", max(qda_cv$results$Accuracy), "\n")
```

```
## Cross-validated Accuracy (training set): 0.4486229
```

```
qda_fit <- qda(response ~ PRE_FG_PCT + PRE_3PA + PRE_FTA + PRE_DRB +
               PRE_BLK + PRE_TOV + PRE_PTS,
              data = train_data)

pred_obj <- predict(qda_fit, test_data)

all_pred <- factor(pred_obj$class, levels = class_levels)
all_actual <- factor(test_data$response, levels = class_levels)
all_probs <- pred_obj$posterior

conf_mat_qda <- confusionMatrix(all_pred, all_actual, mode = "prec_recall")
print(conf_mat_qda)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction LowLow MidLow LowMid MidHigh HighHigh
## LowLow      44      4      13      11      1
## MidLow       0      1       1       2      0
## LowMid      10      1      12       4      4
## MidHigh     11     18     32     52     16
## HighHigh     1      7       2     16     30
##
## Overall Statistics
##
##           Accuracy : 0.4744
##           95% CI : (0.416, 0.5333)
##           No Information Rate : 0.2901
##           P-Value [Acc > NIR] : 2.178e-11
##
##           Kappa : 0.3082
##
## Mcnemar's Test P-Value : 1.100e-06
##
```

```
## Statistics by Class:
##
##           Class: LowLow Class: MidLow Class: LowMid Class: MidHigh
## Precision           0.6027      0.250000      0.38710      0.4031
## Recall              0.6667      0.032258      0.20000      0.6118
## F1                  0.6331      0.057143      0.26374      0.4860
## Prevalence          0.2253      0.105802      0.20478      0.2901
## Detection Rate       0.1502      0.003413      0.04096      0.1775
## Detection Prevalence 0.2491      0.013652      0.10580      0.4403
## Balanced Accuracy    0.7695      0.510404      0.55923      0.6208
##
##           Class: HighHigh
## Precision           0.5357
## Recall              0.5882
## F1                  0.5607
## Prevalence          0.1741
## Detection Rate       0.1024
## Detection Prevalence 0.1911
## Balanced Accuracy    0.7404
```

```
QDA_acc <- mean(as.character(all_pred) == as.character(all_actual))
cat("Test Accuracy (QDA):", QDA_acc, "\n")
```

```
## Test Accuracy (QDA): 0.4744027
```

```
overall_multiclass <- multiclass.roc(all_actual, all_probs)
cat("Overall multiclass AUC (QDA test set):", overall_multiclass$auc, "\n")
```

```
## Overall multiclass AUC (QDA test set): 0.7344629
```

```
fpr_grid <- seq(0, 1, length.out = 200)
tpr_matrix <- matrix(NA, nrow = length(fpr_grid), ncol = length(class_levels))

for (i in seq_along(class_levels)) {
  class <- class_levels[i]
  binary_actual <- factor(ifelse(all_actual == class, class, "Other"),
                           levels = c("Other", class))
  class_probs <- all_probs[, class]

  roc_obj <- roc(response = binary_actual,
                 predictor = class_probs,
                 levels = c("Other", class),
                 positive = class,
                 quiet = TRUE)

  cat("AUC of", class, ":", auc(roc_obj), "\n")

  fpr <- 1 - roc_obj$specificities
  tpr <- roc_obj$sensitivities

  interp_tpr <- approx(fpr, tpr, xout = fpr_grid, ties = mean, rule = 2)$y
  tpr_matrix[, i] <- interp_tpr
}
```



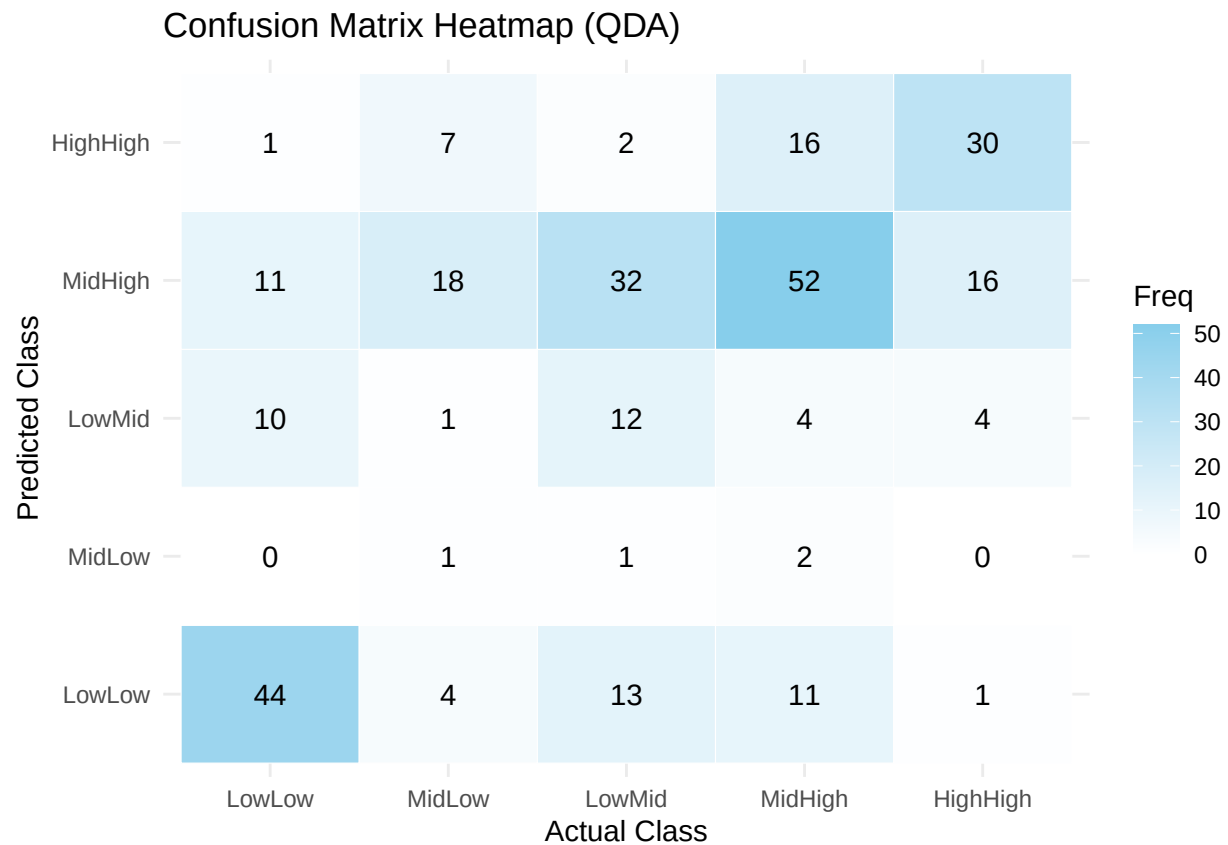
```
## AUC of LowLow : 0.8817247
## AUC of MidLow : 0.6381433
## AUC of LowMid : 0.6808298
## AUC of MidHigh : 0.6595023
## AUC of HighHigh : 0.8197213
```

```
macro_tpr_qda <- rowMeans(tpr_matrix, na.rm = TRUE)
macro_auc_qda <- trapz(fpr_grid, macro_tpr_qda)
cat("Macro AUC (QDA test set):", macro_auc_qda, "\n")
```

```
## Macro AUC (QDA test set): 0.7359076
```

```
model_results <- rbind(model_results, data.frame(
  Model = "QDA",
  Accuracy = QDA_acc,
  Precision = mean(conf_mat_qda$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(conf_mat_qda$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(conf_mat_qda$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc_qda
))
```

```
cm_table <- as.data.frame(conf_mat_qda$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "skyblue") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (QDA)",
       x = "Actual Class", y = "Predicted Class")
```



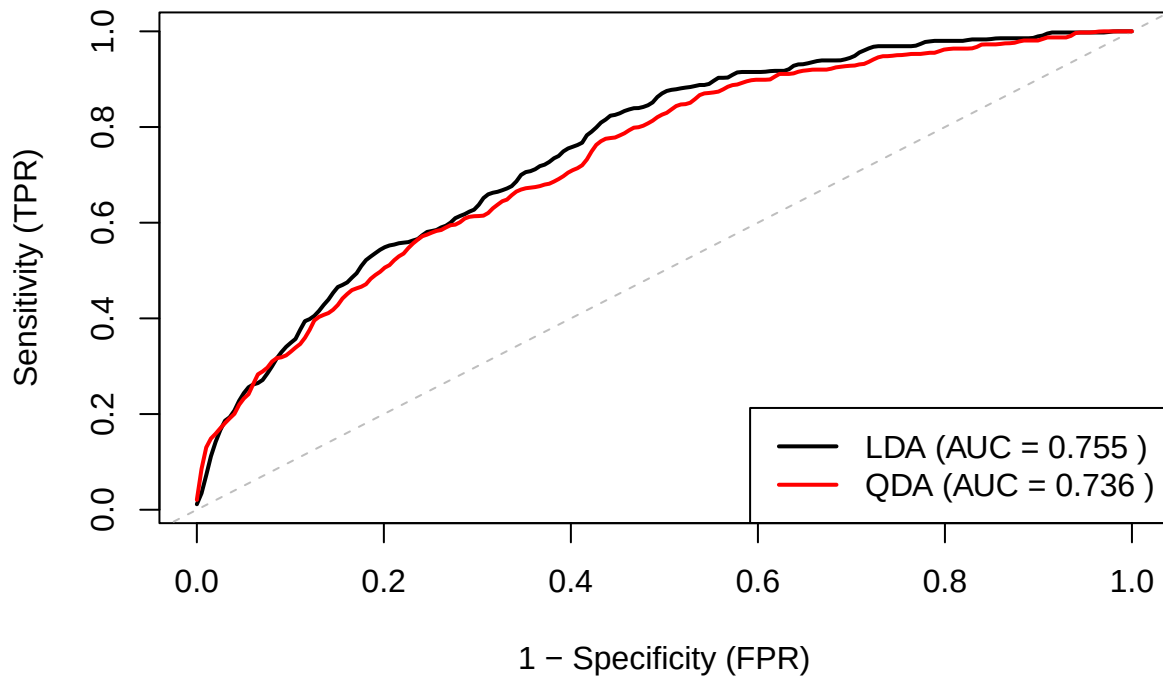
Comparison of LDA vs QDA

```
LDA_acc - QDA_acc
```

```
## [1] -0.01365188
```

```
# Graph AUC curves
plot(fpr_grid, macro_tpr_lda, type = "l", lwd = 2, col = "black",
     xlab = "1 - Specificity (FPR)", ylab = "Sensitivity (TPR)",
     main = paste("Macro-average ROC Curves of LDA and QDA"))
abline(a = 0, b = 1, lty = 2, col = "gray")
lines(fpr_grid, macro_tpr_qda, lwd = 2, col = "red")
legend("bottomright",
      legend = c(paste("LDA (AUC =", round(macro_auc_lda, 3), ")"),
                 paste("QDA (AUC =", round(macro_auc_qda, 3), ")")),
      col = c("black", "red"), lwd = 2)
```

Macro-average ROC Curves of LDA and QDA



PCA

```
# Set PCA statistics
pca_result <- prcomp(df_scaled[, -ncol(df_scaled)], center = TRUE, scale. = TRUE)
explained_var <- summary(pca_result)$importance[2, ]
num_pcs <- which(cumsum(explained_var) >= 0.9)[1]

# Set PCA dataframe
df_pca <- data.frame(pca_result$x[, 1:num_pcs], response = df_scaled$response)
train_data_pca <- df_pca[train_index, ]
test_data_pca <- df_pca[-train_index, ]

# PCA with CV
train_control <- trainControl(method = "cv", number = 10,
                             summaryFunction = multiClassSummary)

model_pca_cv <- train(response ~ ., data = train_data_pca,
                      method = "multinom",
                      trControl = train_control,
                      trace = FALSE)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.
```

```
print(model_pca_cv)
```

```
## Penalized Multinomial Regression
```

```
##
## 1186 samples
## 13 predictor
## 5 classes: 'LowLow', 'MidLow', 'LowMid', 'MidHigh', 'HighHigh'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1066, 1068, 1066, 1068, 1068, 1068, ...
## Resampling results across tuning parameters:
##
## decay Accuracy Kappa Mean_F1 Mean_Sensitivity Mean_Specificity
## 0e+00 0.4916458 0.3317066 NaN 0.4300239 0.8653756
## 1e-04 0.4916458 0.3317066 NaN 0.4300239 0.8653756
## 1e-01 0.4916458 0.3318102 NaN 0.4301905 0.8654077
## Mean_Pos_Pred_Value Mean_Neg_Pred_Value Mean_Precision Mean_Recall
## 0.3890302 0.8701681 0.3890302 0.4300239
## 0.3890302 0.8701681 0.3890302 0.4300239
## 0.3891351 0.8702773 0.3891351 0.4301905
## Mean_Detection_Rate Mean_Balanced_Accuracy
## 0.09832916 0.6476997
## 0.09832916 0.6476997
## 0.09832916 0.6477991
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was decay = 0.1.
```

```
cat("Cross-validated Accuracy (training set):", max(model_pca_cv$results$Accuracy), "\n")
```

```
## Cross-validated Accuracy (training set): 0.4916458
```

```
# Set our fit model
model_pca <- multinom(response ~ ., data = train_data_pca, trace = FALSE)

preds_pca <- predict(model_pca, newdata = test_data_pca)
preds_pca <- factor(preds_pca, levels = levels(test_data_pca$response))

# Calc CM
cm_pca <- confusionMatrix(preds_pca, test_data_pca$response, mode = "prec_recall")
print(cm_pca)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction LowLow MidLow LowMid MidHigh HighHigh
## LowLow      46      2      11      9      0
## MidLow       0      0       0      0      0
## LowMid      10      2      15      7      3
## MidHigh     10     20      34     44     20
## HighHigh     0      7       0     25     28
##
## Overall Statistics
##
##           Accuracy : 0.4539
```

```
##                      95% CI : (0.3959, 0.5129)
##      No Information Rate : 0.2901
##      P-Value [Acc > NIR] : 2.123e-09
##
##                      Kappa : 0.281
##
##      McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: LowLow Class: MidLow Class: LowMid Class: MidHigh
## Precision                0.6765                NA                0.40541                0.3438
## Recall                   0.6970                0.0000                0.25000                0.5176
## F1                       0.6866                NA                0.30928                0.4131
## Prevalence               0.2253                0.1058                0.20478                0.2901
## Detection Rate           0.1570                0.0000                0.05119                0.1502
## Detection Prevalence     0.2321                0.0000                0.12628                0.4369
## Balanced Accuracy        0.8000                0.5000                0.57779                0.5569
##
##                      Class: HighHigh
## Precision                0.46667
## Recall                   0.54902
## F1                       0.50450
## Prevalence               0.17406
## Detection Rate           0.09556
## Detection Prevalence     0.20478
## Balanced Accuracy        0.70839
```

```
test_acc <- mean(as.character(preds_pca) == as.character(test_data_pca$response))
cat("Test Accuracy (PCA-based model):", test_acc, "\n")
```

```
## Test Accuracy (PCA-based model): 0.4539249
```

```
loadings <- pca_result$rotation[, 1:num_pcs]
print("PCA Loadings (contribution of original variables):")
```

```
## [1] "PCA Loadings (contribution of original variables):"
```

```
print(loadings)
```

```
##                      PC1          PC2          PC3          PC4          PC5
## WEIGHT                0.310576735 -0.11600745  0.022405694 -0.090180825  0.01486769
## HEIGHT_INCHES         0.306018471 -0.12869752 -0.071442881 -0.070894339  0.04853623
## PRE_G                 -0.091069205 -0.39616697  0.179522649 -0.150512294 -0.04018886
## PRE_GS                -0.108814951 -0.33153667  0.243856055 -0.150788708 -0.11745689
## PRE_MP                -0.131801927 -0.34671668  0.252903108 -0.147007736 -0.06628751
## PRE_FGA               -0.060335081  0.20465217  0.167377182 -0.448716734  0.07429732
## PRE_FG_PCT            0.297628706 -0.08839393  0.066116020 -0.009784959 -0.09895303
## PRE_3PA               -0.304268051  0.10539397 -0.006859503 -0.100076805  0.11678585
## PRE_3P_PCT            -0.209899910  0.05912609  0.069077397 -0.027605119 -0.03071896
## PRE_2PA               0.163609176  0.25793448  0.356691231 -0.006040583 -0.13122173
## PRE_2P_PCT            0.092818662  0.24657497  0.352240719  0.283858437 -0.18429705
## PRE_eFG_PCT           0.065374880  0.25458067  0.359438727  0.286058254 -0.16684636
```

## PRE_FTA	0.170662156	0.11379092	0.164509523	-0.343449549	-0.08557226
## PRE_FT_PCT	-0.247252477	0.07428605	0.092619611	-0.120717187	0.02573812
## PRE_ORB	0.252306293	-0.01153781	0.173584896	-0.046970034	0.35464067
## PRE_DRB	0.193106037	-0.03361403	-0.099911399	-0.107603026	-0.37179356
## PRE_TRB	0.236534587	-0.06157029	-0.102453639	-0.088923308	-0.45323070
## PRE_AST	-0.235526532	0.03941034	-0.091674798	-0.001542908	-0.37897486
## PRE_STL	-0.150644666	0.00788266	-0.186349012	-0.042236917	-0.47896250
## PRE_BLK	0.306173178	-0.04844280	-0.014637065	0.030408065	-0.02030777
## PRE_TOV	0.016687285	0.09544624	0.009094554	-0.275116011	-0.08817865
## PRE_PF	0.217643408	-0.16667332	-0.183079463	-0.046787190	0.04078897
## PRE_PTS	0.038980952	0.20570127	0.223216572	-0.460695696	0.01450406
## PRE_FG_pct_inc	0.004719971	-0.32262204	0.195555697	0.068318283	0.01528496
## PRE_3P_pct_inc	-0.185918557	-0.15406746	0.155208206	-0.022473690	-0.07223330
## PRE_2P_pct_inc	-0.029798898	-0.10309029	0.331152399	0.300150507	-0.04396151
## PRE_FT_pct_inc	-0.012801073	-0.28517770	0.189787389	0.053510073	-0.02871863
##	PC6	PC7	PC8	PC9	PC10
## WEIGHT	-0.102440463	0.01945161	-0.10761587	0.0441645755	0.071696264
## HEIGHT_INCHES	-0.197861654	0.03426717	-0.07151755	0.0003343841	0.104053219
## PRE_G	-0.003096375	-0.22717250	-0.03676991	-0.0148742508	-0.046949635
## PRE_GS	0.016534577	-0.33539249	-0.03253507	0.0190600895	-0.111168430
## PRE_MP	0.015534053	-0.28855828	-0.01783077	-0.0040057105	-0.074098536
## PRE_FGA	-0.100342569	0.14375073	0.12366593	0.3050992422	0.010059538
## PRE_FG_PCT	-0.004409603	-0.13322309	-0.23111057	-0.0781484906	0.275050775
## PRE_3PA	-0.254619740	0.10951615	0.04972839	0.1597272981	-0.063145956
## PRE_3P_PCT	-0.296272720	0.12555504	-0.25505483	-0.6921348003	0.238149213
## PRE_2PA	0.100909258	-0.04779157	0.12626039	0.0630074925	0.022293237
## PRE_2P_PCT	-0.025008678	-0.09948989	0.03965274	-0.0408271040	0.035244571
## PRE_eFG_PCT	-0.049976061	-0.09752382	0.01237576	-0.0373396158	0.032764705
## PRE_FTA	0.205143540	0.04317515	-0.18480518	-0.1013142886	0.019922747
## PRE_FT_PCT	-0.261178994	0.02336108	-0.33828357	-0.1089984569	-0.003053189
## PRE_ORB	0.124199677	-0.04014109	0.07343547	-0.1625242423	0.025815599
## PRE_DRB	-0.345438973	0.08949877	0.25515900	-0.1880550171	-0.431033802
## PRE_TRB	-0.277619833	0.07917485	0.06957106	0.0551909921	-0.111570151
## PRE_AST	0.310881381	-0.06803405	-0.13439517	-0.0227785634	0.035482104
## PRE_STL	0.158545817	0.08979451	-0.15790259	0.2435688666	0.381844051
## PRE_BLK	-0.036114530	-0.04979654	-0.06310273	0.0026893567	0.123028939
## PRE_TOV	0.531394851	0.18724542	0.12860582	-0.3867088692	-0.291267960
## PRE_PF	0.152259628	0.28034979	-0.09726541	-0.0614783636	0.162613277
## PRE_PTS	-0.109213441	0.11347422	-0.11912295	0.1513782274	0.160113006
## PRE_FG_pct_inc	0.044722428	0.45395118	-0.24968559	0.1021009545	-0.201073762
## PRE_3P_pct_inc	-0.075108836	0.27114166	0.49431413	-0.1656599680	0.269992950
## PRE_2P_pct_inc	0.037758904	0.43277509	-0.26247403	0.1596732804	-0.230500277
## PRE_FT_pct_inc	0.045372377	0.19061518	0.37416558	0.0154122996	0.399782204
##	PC11	PC12	PC13		
## WEIGHT	0.058519996	0.09333076	-0.028184110		
## HEIGHT_INCHES	-0.011694787	0.17386310	0.076222008		
## PRE_G	0.002715547	0.13765013	0.051480749		
## PRE_GS	0.027568844	0.14322979	0.024691520		
## PRE_MP	0.028870044	0.11442966	0.009178278		
## PRE_FGA	-0.076756280	0.25203787	-0.104767828		
## PRE_FG_PCT	0.056708376	-0.27041638	-0.010116838		
## PRE_3PA	-0.118367878	0.23071748	0.168132304		
## PRE_3P_PCT	-0.132992750	0.28816877	-0.346557121		
## PRE_2PA	0.028751024	0.15501850	-0.076811133		

```
## PRE_2P_PCT      0.016604797  0.09011226  0.147592764
## PRE_eFG_PCT    -0.009846565  0.09309186  0.167533579
## PRE_FTA        0.079548232 -0.39511675 -0.123032661
## PRE_FT_PCT     -0.005951696 -0.32729026  0.607281583
## PRE_ORB        0.095369951  0.06374772 -0.145015816
## PRE_DRB        -0.075628303 -0.09555228 -0.006762200
## PRE_TRB        -0.025164286 -0.02048343 -0.052993184
## PRE_AST        -0.058816769 -0.03062988  0.009038546
## PRE_STL        0.101423145  0.20489575 -0.103729875
## PRE_BLK        -0.055082361  0.25445469  0.332816936
## PRE_TOV        -0.140123639  0.15500691  0.244921273
## PRE_PF         0.066511609  0.31256945  0.375978211
## PRE_PTS        -0.013609522 -0.08498533  0.001161631
## PRE_FG_pct_inc  0.078072671 -0.08208679 -0.156071581
## PRE_3P_pct_inc  0.646933312 -0.10277365  0.122327463
## PRE_2P_pct_inc -0.024377950  0.07019345 -0.046466246
## PRE_FT_pct_inc -0.681751633 -0.24304954  0.051015208
```

```
# Run it
train_control <- trainControl(method = "cv", number = 10,
                             summaryFunction = multiClassSummary)

model_pca_cv <- train(response ~ ., data = train_data_pca,
                     method = "multinom",
                     trControl = train_control,
                     trace = FALSE)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.
```

```
cat("Cross-validated accuracy (PCA-based model, training set):",
    max(model_pca_cv$results$Accuracy), "\n")
```

```
## Cross-validated accuracy (PCA-based model, training set): 0.4966108
```

```
model_pca <- multinom(response ~ ., data = train_data_pca, trace = FALSE)

preds_pca <- predict(model_pca, newdata = test_data_pca)
preds_pca <- factor(preds_pca, levels = levels(test_data_pca$response))

cm_pca <- confusionMatrix(preds_pca, test_data_pca$response, mode = "prec_recall")
print(cm_pca)
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction LowLow MidLow LowMid MidHigh HighHigh
##   LowLow      46      2     11      9      0
##   MidLow       0      0      0      0      0
##   LowMid      10      2     15      7      3
##   MidHigh     10     20     34     44     20
##   HighHigh     0      7      0     25     28
```

```
##
## Overall Statistics
##
##           Accuracy : 0.4539
##           95% CI : (0.3959, 0.5129)
##       No Information Rate : 0.2901
##       P-Value [Acc > NIR] : 2.123e-09
##
##           Kappa : 0.281
##
## Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##           Class: LowLow Class: MidLow Class: LowMid Class: MidHigh
## Precision           0.6765           NA           0.40541           0.3438
## Recall              0.6970           0.0000           0.25000           0.5176
## F1                  0.6866           NA           0.30928           0.4131
## Prevalence          0.2253           0.1058           0.20478           0.2901
## Detection Rate      0.1570           0.0000           0.05119           0.1502
## Detection Prevalence 0.2321           0.0000           0.12628           0.4369
## Balanced Accuracy    0.8000           0.5000           0.57779           0.5569
##
##           Class: HighHigh
## Precision           0.46667
## Recall              0.54902
## F1                  0.50450
## Prevalence          0.17406
## Detection Rate      0.09556
## Detection Prevalence 0.20478
## Balanced Accuracy    0.70839
```

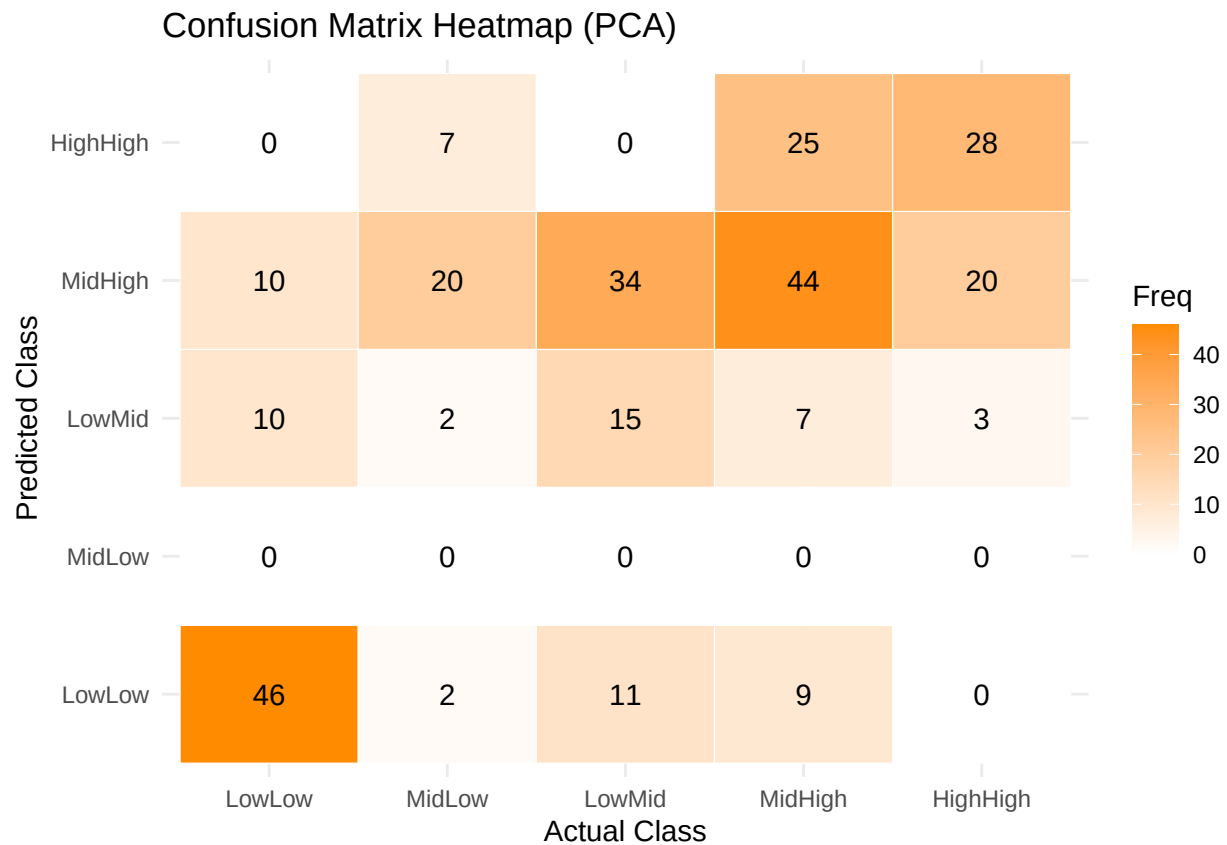
```
test_acc_pca <- mean(as.character(preds_pca) == as.character(test_data_pca$response))
cat("Test Accuracy (PCA-based model):", test_acc_pca, "\n")
```

```
## Test Accuracy (PCA-based model): 0.4539249
```

```
probs_pca <- predict(model_pca, newdata = test_data_pca, type = "probs")
probs_pca <- as.matrix(probs_pca)

colnames(probs_pca) <- colnames(probs_pca)
probs_pca <- probs_pca[, levels(test_data_pca$response), drop = FALSE]
```

```
cm_table <- as.data.frame(cm_pca$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "darkorange") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (PCA)",
       x = "Actual Class", y = "Predicted Class")
```

```
model_results <- rbind(model_results, data.frame(
  Model = "PCA",
  Accuracy = test_acc_pca,
  Precision = mean(cm_pca$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_pca$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_pca$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data_pca$response, probs_pca)
))
```

SVM Radial and Linear

```
set.seed(67)
# CV SVM
svm_linear <- train(
  response ~ .,
  data = train_data,
  method = "svmLinear2",
  trControl = trainControl(
    method = "cv",
    number = 10,
    classProbs = TRUE,
    summaryFunction = multiClassSummary
  ),
```

```
preProcess = c("center", "scale", "pca"),
tuneGrid = expand.grid(cost = c(0.01, 0.1, 1, 10)),
probability = TRUE
)

## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.
```

```
print(svm_linear)
```

```
## Support Vector Machines with Linear Kernel
##
## 1186 samples
## 27 predictor
## 5 classes: 'LowLow', 'MidLow', 'LowMid', 'MidHigh', 'HighHigh'
##
## Pre-processing: centered (27), scaled (27), principal component
## signal extraction (27)
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1067, 1067, 1068, 1068, 1067, 1066, ...
## Resampling results across tuning parameters:
##
## cost logLoss AUC prAUC Accuracy Kappa Mean_F1
## 0.01 1.220661 0.7559764 0.4257302 0.4645095 0.2767078 NaN
## 0.10 1.209106 0.7598262 0.4375248 0.4932324 0.3312741 NaN
## 1.00 1.210306 0.7616912 0.4392217 0.4941157 0.3337833 NaN
## 10.00 1.210145 0.7627623 0.4417608 0.4949484 0.3351066 NaN
## Mean_Sensitivity Mean_Specificity Mean_Pos_Pred_Value Mean_Neg_Pred_Value
## 0.3780854 0.8537640 NaN 0.8692518
## 0.4284745 0.8650533 NaN 0.8703919
## 0.4312712 0.8656352 NaN 0.8704900
## 0.4325163 0.8658924 NaN 0.8706191
## Mean_Precision Mean_Recall Mean_Detection_Rate Mean_Balanced_Accuracy
## NaN 0.3780854 0.09290189 0.6159247
## NaN 0.4284745 0.09864648 0.6467639
## NaN 0.4312712 0.09882314 0.6484532
## NaN 0.4325163 0.09898969 0.6492043
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was cost = 10.
```

```
cat("Best cost (C):", svm_linear$bestTune$C, "\n")
```

```
## Best cost (C):
```

```
cat("Cross-validated accuracy (training set):", max(svm_linear$results$Accuracy), "\n")
```

```
## Cross-validated accuracy (training set): 0.4949484
```

```

preds_svm_linear <- predict(svm_linear, newdata = test_data)
preds_svm_linear <- factor(preds_svm_linear, levels = levels(test_data$response))

cm_svm_linear <- confusionMatrix(preds_svm_linear, test_data$response, mode = "prec_recall")
print(cm_svm_linear)

```

```

## Confusion Matrix and Statistics
##
##              Reference
## Prediction LowLow MidLow LowMid MidHigh HighHigh
##   LowLow      52      2      10       9       0
##   MidLow       0      0       0       0       0
##   LowMid       4      2      23      10       4
##   MidHigh      10     23      27      47      16
##   HighHigh     0      4       0      19      31
##
## Overall Statistics
##
##              Accuracy : 0.5222
##              95% CI : (0.4633, 0.5806)
##   No Information Rate : 0.2901
##   P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.3713
##
## Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##              Class: LowLow Class: MidLow Class: LowMid Class: MidHigh
## Precision              0.7123              NA              0.5349              0.3821
## Recall                  0.7879              0.0000              0.3833              0.5529
## F1                      0.7482              NA              0.4466              0.4519
## Prevalence              0.2253              0.1058              0.2048              0.2901
## Detection Rate          0.1775              0.0000              0.0785              0.1604
## Detection Prevalence    0.2491              0.0000              0.1468              0.4198
## Balanced Accuracy        0.8477              0.5000              0.6487              0.5938
##
##              Class: HighHigh
## Precision              0.5741
## Recall                  0.6078
## F1                      0.5905
## Prevalence              0.1741
## Detection Rate          0.1058
## Detection Prevalence    0.1843
## Balanced Accuracy        0.7564

```

```

test_acc_linear <- mean(as.character(preds_svm_linear) == as.character(test_data$response))
cat("Test Accuracy (SVM Linear):", test_acc_linear, "\n")

```

```

## Test Accuracy (SVM Linear): 0.5221843

```

```

probs_svm_linear <- predict(svm_linear, newdata = test_data, type = "prob")
probs_svm_linear <- as.matrix(probs_svm_linear)

colnames(probs_svm_linear) <- colnames(probs_svm_linear)
probs_svm_linear <- probs_svm_linear[, levels(test_data_pca$response), drop = FALSE]

model_results <- rbind(model_results, data.frame(
  Model = "SVM Linear",
  Accuracy = test_acc_linear,
  Precision = mean(cm_svm_linear$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_svm_linear$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_svm_linear$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data$response, probs_svm_linear)
))

```

```

# Repeat
svm_radial <- train(
  response ~ .,
  data = train_data,
  method = "svmRadial",
  trControl = trainControl(
    method = "cv",
    number = 10,
    classProbs = TRUE,
    summaryFunction = multiClassSummary
  ),
  tuneGrid = expand.grid(
    C = c(0.1, 1, 10),
    sigma = c(0.01, 0.05, 0.1)
  ),
  prob.model = TRUE
)

```

```

## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.

```

```
print(svm_radial)
```

```

## Support Vector Machines with Radial Basis Function Kernel
##
## 1186 samples
## 27 predictor
## 5 classes: 'LowLow', 'MidLow', 'LowMid', 'MidHigh', 'HighHigh'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1067, 1068, 1066, 1067, 1068, 1069, ...
## Resampling results across tuning parameters:
##
##  C      sigma  logLoss  AUC      prAUC      Accuracy  Kappa      Mean_F1
##  0.1  0.01   1.265810  0.7381375  0.4196191  0.4646203  0.3104321    NaN
##  0.1  0.05   1.287806  0.7375152  0.4068036  0.4654448  0.3112115  0.4663253

```

```
##      0.1  0.10  1.310718  0.7366377  0.3994815  0.4603675  0.3005422  0.4198721
##      1.0  0.01  1.222721  0.7590793  0.4385806  0.4974802  0.3507443      NaN
##      1.0  0.05  1.232600  0.7623569  0.4287441  0.4924020  0.3434745  0.4387023
##      1.0  0.10  1.266850  0.7505242  0.4112051  0.4788633  0.3241935      NaN
##     10.0  0.01  1.208031  0.7733615  0.4504943  0.5007709  0.3551704  0.4183001
##     10.0  0.05  1.283665  0.7411509  0.4163312  0.4662572  0.3042913  0.4154865
##     10.0  0.10  1.298536  0.7345393  0.4062216  0.4746602  0.3139262  0.4108938
## Mean_Sensitivity Mean_Specificity Mean_Pos_Pred_Value Mean_Neg_Pred_Value
##      0.4281330      0.8616718      0.3433795      0.8661599
##      0.4285888      0.8618326      0.4133188      0.8666540
##      0.4172059      0.8595602      0.3881860      0.8645382
##      0.4538179      0.8697807      0.3881635      0.8737308
##      0.4487395      0.8682413      0.4534697      0.8716830
##      0.4339768      0.8642137      0.3834531      0.8682752
##      0.4574163      0.8707319      0.4601085      0.8739659
##      0.4171408      0.8601026      0.4592905      0.8641455
##      0.4226362      0.8620263      0.4846190      0.8669470
## Mean_Precision Mean_Recall Mean_Detection_Rate Mean_Balanced_Accuracy
##      0.3433795      0.4281330      0.09292406      0.6449024
##      0.4133188      0.4285888      0.09308897      0.6452107
##      0.3881860      0.4172059      0.09207349      0.6383830
##      0.3881635      0.4538179      0.09949604      0.6617993
##      0.4534697      0.4487395      0.09848039      0.6584904
##      0.3834531      0.4339768      0.09577266      0.6490952
##      0.4601085      0.4574163      0.10015418      0.6640741
##      0.4592905      0.4171408      0.09325143      0.6386217
##      0.4846190      0.4226362      0.09493203      0.6423313
##
## Accuracy was used to select the optimal model using the largest value.
## The final values used for the model were sigma = 0.01 and C = 10.
```

```
cat("Best parameters: C =", svm_radial$bestTune$C,
    " sigma =", svm_radial$bestTune$sigma, "\n")
```

```
## Best parameters: C = 10 sigma = 0.01
```

```
cat("Cross-validated accuracy (training set):", max(svm_radial$results$Accuracy), "\n")
```

```
## Cross-validated accuracy (training set): 0.5007709
```

```
preds_svm_radial <- predict(svm_radial, newdata = test_data)
preds_svm_radial <- factor(preds_svm_radial, levels = levels(test_data$response))

cm_svm_radial <- confusionMatrix(preds_svm_radial, test_data$response, mode = "prec_recall")
print(cm_svm_radial)
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction LowLow MidLow LowMid MidHigh HighHigh
##      LowLow      50      2      17      8      1
##      MidLow      0      1      0      0      0
```

```
## LowMid      11      3      26      16      2
## MidHigh     5      15      16      40      16
## HighHigh    0      10       1      21      32
##
## Overall Statistics
##
## Accuracy : 0.5085
## 95% CI : (0.4497, 0.5671)
## No Information Rate : 0.2901
## P-Value [Acc > NIR] : 3.612e-15
##
## Kappa : 0.3618
##
## McNemar's Test P-Value : 0.0001856
##
## Statistics by Class:
##
## Class: LowLow Class: MidLow Class: LowMid Class: MidHigh
## Precision      0.6410      1.000000      0.44828      0.4348
## Recall         0.7576      0.032258      0.43333      0.4706
## F1             0.6944      0.062500      0.44068      0.4520
## Prevalence     0.2253      0.105802      0.20478      0.2901
## Detection Rate 0.1706      0.003413      0.08874      0.1365
## Detection Prevalence 0.2662      0.003413      0.19795      0.3140
## Balanced Accuracy 0.8171      0.516129      0.64800      0.6103
##
## Class: HighHigh
## Precision      0.5000
## Recall         0.6275
## F1             0.5565
## Prevalence     0.1741
## Detection Rate 0.1092
## Detection Prevalence 0.2184
## Balanced Accuracy 0.7476
```

```
test_acc_radial <- mean(as.character(preds_svm_radial) == as.character(test_data$response))
cat("Test Accuracy (SVM Radial):", test_acc_radial, "\n")
```

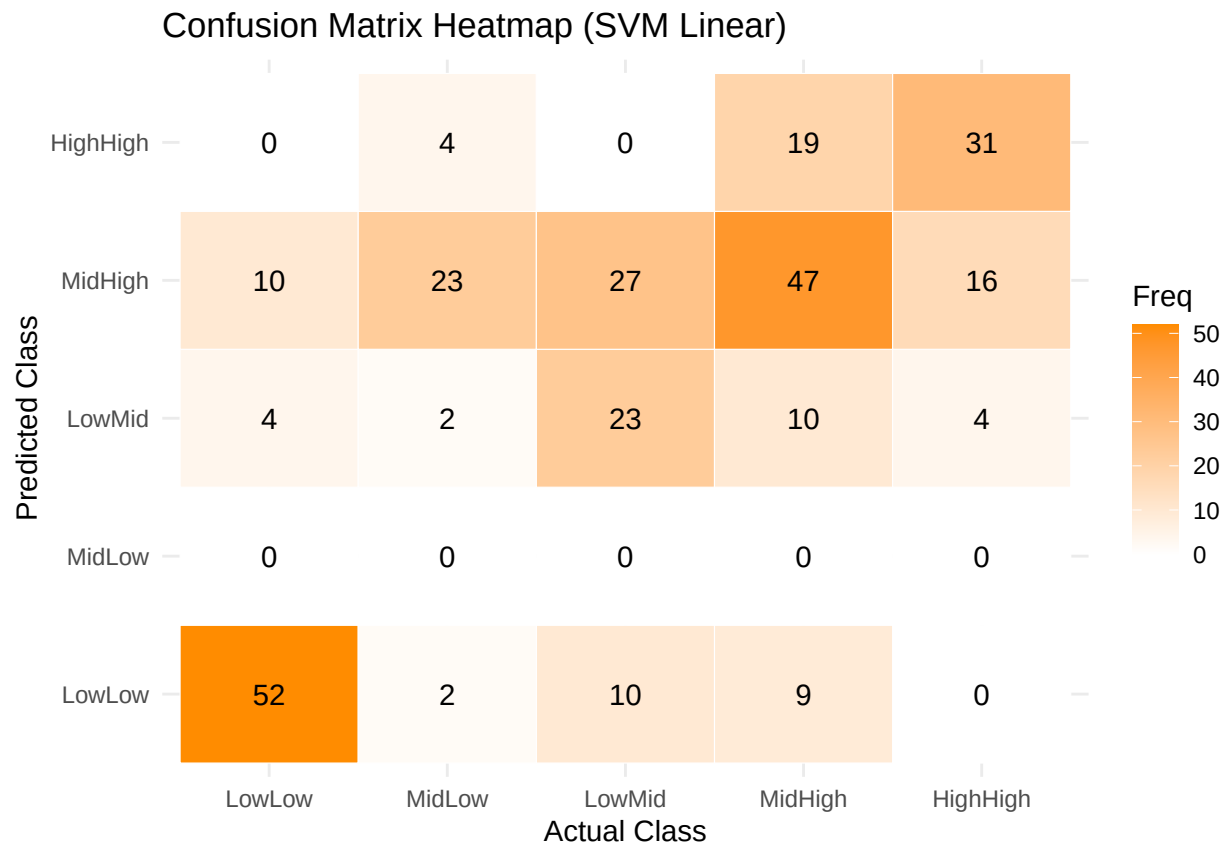
```
## Test Accuracy (SVM Radial): 0.5085324
```

```
probs_svm_radial <- predict(svm_radial, newdata = test_data, type = "prob")
probs_svm_radial <- as.matrix(probs_svm_radial)

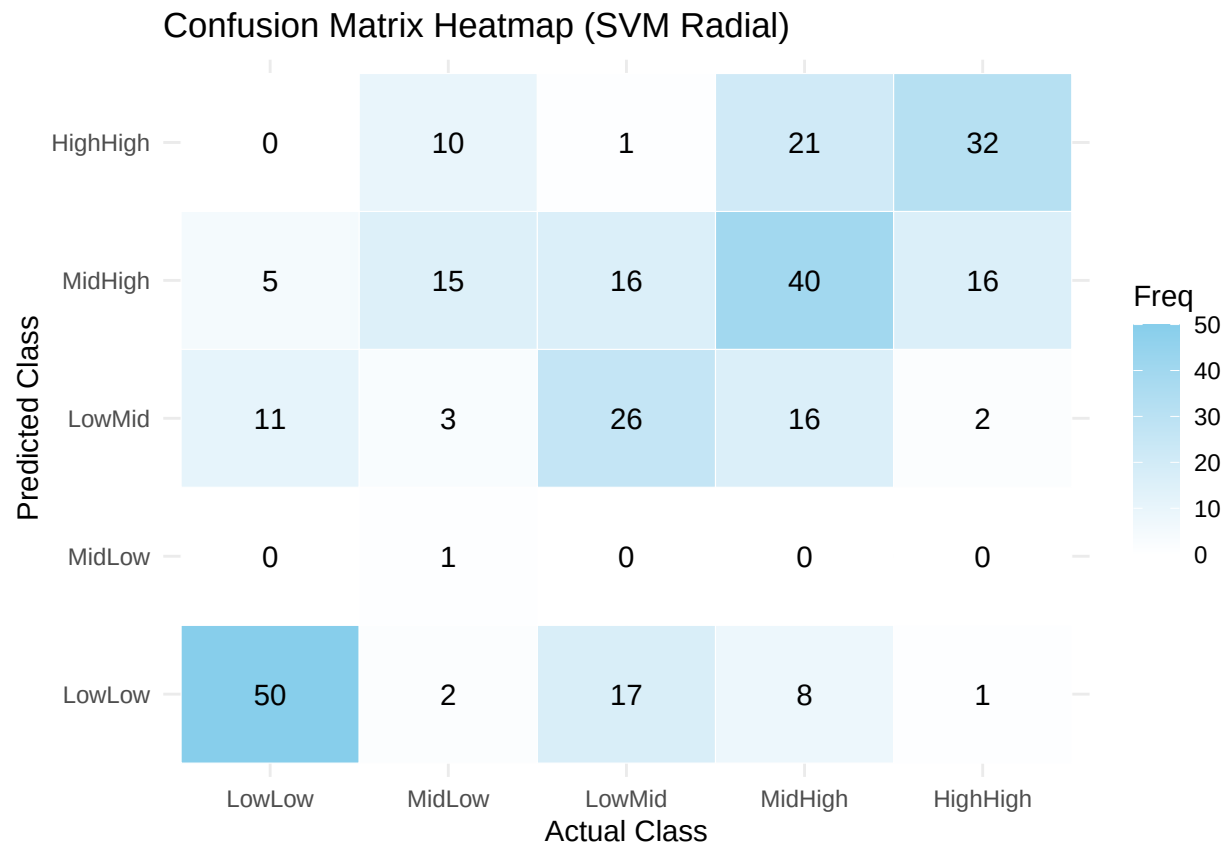
colnames(probs_svm_radial) <- colnames(probs_svm_radial)
probs_svm_radial <- probs_svm_radial[, levels(test_data$response), drop = FALSE]

model_results <- rbind(model_results, data.frame(
  Model = "SVM Radial Kernel",
  Accuracy = test_acc_radial,
  Precision = mean(cm_svm_radial$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_svm_radial$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_svm_radial$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data$response, probs_svm_radial)
))
```

```
cm_table <- as.data.frame(cm_svm_linear$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "darkorange") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (SVM Linear)",
       x = "Actual Class", y = "Predicted Class")
```



```
cm_table <- as.data.frame(cm_svm_radial$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "skyblue") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (SVM Radial)",
       x = "Actual Class", y = "Predicted Class")
```



Random Forest

```
set.seed(6767)
# CV for random forest
train_control <- trainControl(method = "cv", number = 10)

tune_grid <- expand.grid(mtry = c(2, 5, 7, 10, 15, 20))

results <- list()
ntree_values <- c(100, 250, 500)

for (nt in ntree_values) {
  rf_model <- train(response ~ ., data = train_data,
                    method = "rf",
                    trControl = train_control,
                    tuneGrid = tune_grid,
                    ntree = nt)

  results[[paste0("ntree_", nt)]] <- rf_model

  preds <- predict(rf_model, newdata = test_data)
  cm <- confusionMatrix(preds, test_data$response, mode = "prec_recall")
}
```



```

# Print results
cat("\nResults for ntree =", nt, "\n")
print(rf_model$results)
cat("Best mtry:", rf_model$bestTune$mtry, "\n")
cat("CV Accuracy:", max(rf_model$results$Accuracy), "\n")
}

```

```

##
## Results for ntree = 100 :
##   mtry Accuracy      Kappa AccuracySD      KappaSD
## 1    2 0.5008332 0.3425324 0.02843731 0.03839297
## 2    5 0.5219200 0.3735218 0.01630944 0.02144609
## 3    7 0.5109885 0.3594257 0.03069739 0.03900863
## 4   10 0.5042088 0.3507255 0.01904903 0.02541127
## 5   15 0.5051061 0.3540154 0.03039054 0.04008641
## 6   20 0.5084746 0.3579841 0.03373367 0.04509921
## Best mtry: 5
## CV Accuracy: 0.52192
##
## Results for ntree = 250 :
##   mtry Accuracy      Kappa AccuracySD      KappaSD
## 1    2 0.5195076 0.3653180 0.03084034 0.04098944
## 2    5 0.5260879 0.3779517 0.02865717 0.03988260
## 3    7 0.5093013 0.3554796 0.04112772 0.05559015
## 4   10 0.5100489 0.3587426 0.04014311 0.05433210
## 5   15 0.5151122 0.3651382 0.03645686 0.04837478
## 6   20 0.5084316 0.3566273 0.03134842 0.04258721
## Best mtry: 5
## CV Accuracy: 0.5260879
##
## Results for ntree = 500 :
##   mtry Accuracy      Kappa AccuracySD      KappaSD
## 1    2 0.5033184 0.3447466 0.03382129 0.04658683
## 2    5 0.5151614 0.3626811 0.03877724 0.05294920
## 3    7 0.5100848 0.3565511 0.02817233 0.03797243
## 4   10 0.5092515 0.3571203 0.03749754 0.04985950
## 5   15 0.5076065 0.3548243 0.02844090 0.03737150
## 6   20 0.5059189 0.3543597 0.04449500 0.05827652
## Best mtry: 5
## CV Accuracy: 0.5151614

```

```

# Use best model as fit
rf_model <- train(response ~ ., data = train_data,
                  method = "rf",
                  trControl = trainControl(method = "cv", number = 10,
                                           summaryFunction = multiClassSummary),
                  tuneGrid = data.frame(mtry = results[["ntree_500"]]$bestTune$mtry),
                  ntree = 500)

```

```

## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.

```

```

preds_rf <- predict(rf_model, newdata = test_data)
cm_rf <- confusionMatrix(preds_rf, test_data$response, mode = "prec_recall")

probs_rf <- predict(rf_model, newdata = test_data, type = "prob")
probs_rf <- as.matrix(probs_rf)

colnames(probs_rf) <- colnames(probs_rf)
probs_rf <- probs_rf[, levels(test_data$response), drop = FALSE]

model_results <- rbind(model_results, data.frame(
  Model = "Base Random Forest",
  Accuracy = mean(preds_rf == test_data$response),
  Precision = mean(cm_rf$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_rf$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_rf$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data$response, probs_rf)
))

print(rf_model)

```

```

## Random Forest
##
## 1186 samples
## 27 predictor
## 5 classes: 'LowLow', 'MidLow', 'LowMid', 'MidHigh', 'HighHigh'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1068, 1067, 1068, 1067, 1067, 1066, ...
## Resampling results:
##
## Accuracy Kappa Mean_F1 Mean_Sensitivity Mean_Specificity
## 0.5201402 0.3681794 NaN 0.4527455 0.8726608
## Mean_Pos_Pred_Value Mean_Neg_Pred_Value Mean_Precision Mean_Recall
## 0.440002 0.8777627 0.440002 0.4527455
## Mean_Detection_Rate Mean_Balanced_Accuracy
## 0.104028 0.6627032
##
## Tuning parameter 'mtry' was held constant at a value of 5

```

```
print(cm_rf)
```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction LowLow MidLow LowMid MidHigh HighHigh
## LowLow      46      2     11      8      1
## MidLow       0      0      0      0      0
## LowMid      12      2     20     10      1
## MidHigh       7     21     29     50     23
## HighHigh      1      6      0     17     26
##

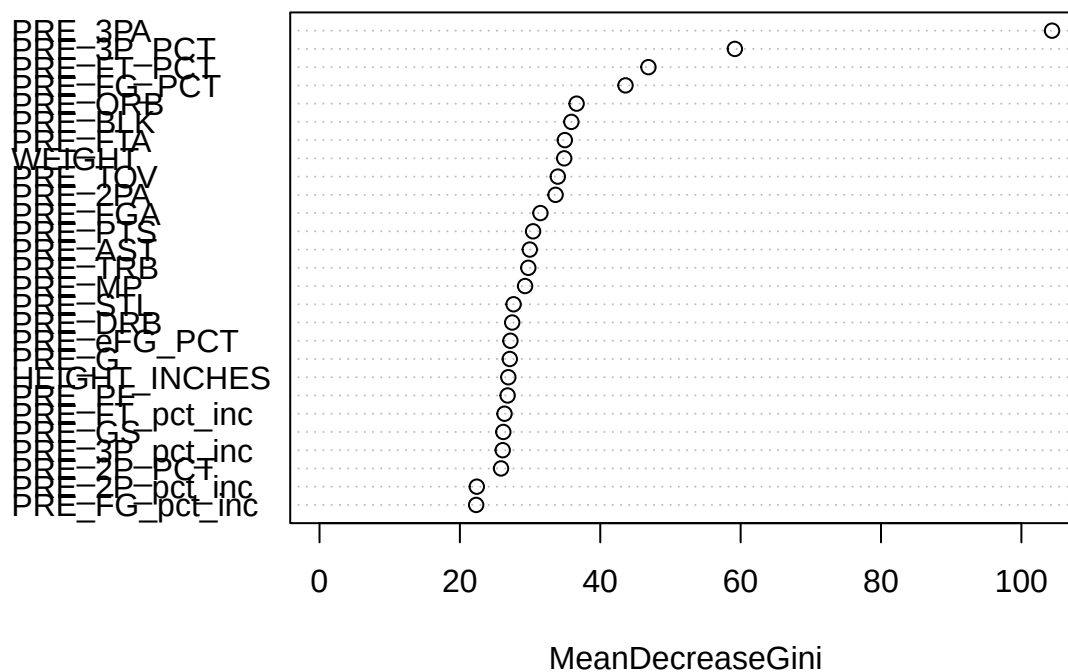
```

```

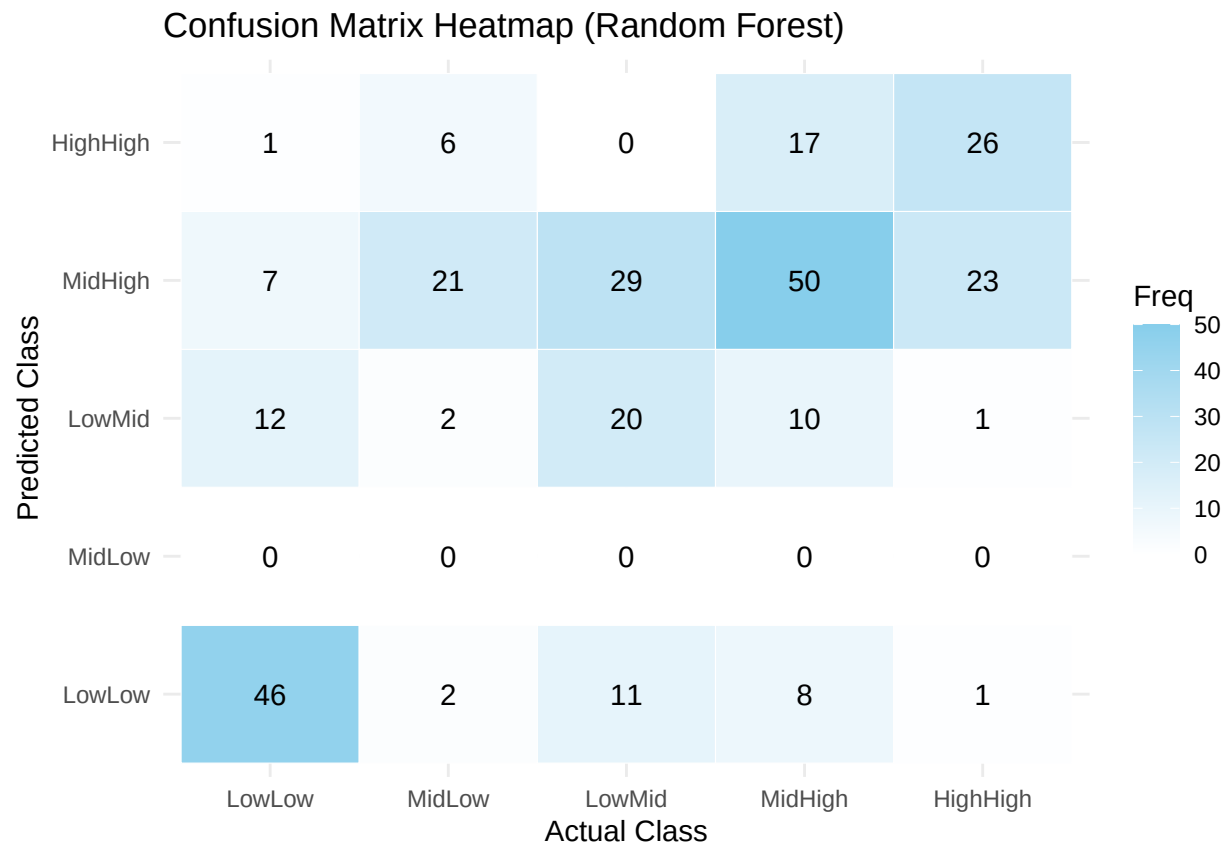
## Overall Statistics
##
##           Accuracy : 0.4846
##           95% CI   : (0.4261, 0.5435)
##           No Information Rate : 0.2901
##           P-Value [Acc > NIR] : 1.841e-12
##
##           Kappa : 0.32
##
## Mcnemar's Test P-Value : 6.722e-06
##
## Statistics by Class:
##
##           Class: LowLow Class: MidLow Class: LowMid Class: MidHigh
## Precision           0.6765           NA           0.44444           0.3846
## Recall              0.6970           0.0000           0.33333           0.5882
## F1                  0.6866           NA           0.38095           0.4651
## Prevalence          0.2253           0.1058           0.20478           0.2901
## Detection Rate      0.1570           0.0000           0.06826           0.1706
## Detection Prevalence 0.2321           0.0000           0.15358           0.4437
## Balanced Accuracy    0.8000           0.5000           0.61302           0.6018
##
##           Class: HighHigh
## Precision           0.52000
## Recall              0.50980
## F1                  0.51485
## Prevalence          0.17406
## Detection Rate      0.08874
## Detection Prevalence 0.17065
## Balanced Accuracy    0.70532

# Print Variable Importance Plot
varImpPlot(rf_model$finalModel)

```

rf_model\$finalModel

```
cm_table <- as.data.frame(cm_rf$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "skyblue") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (Random Forest)",
       x = "Actual Class", y = "Predicted Class")
```



Upsampling

```
set.seed(67)

# Cross validation for upsampling
train_control <- trainControl(method = "cv", number = 10,
                              sampling = "up",
                              summaryFunction = multiClassSummary)

rf_model_balanced <- train(response ~ ., data = train_data,
                           method = "rf",
                           trControl = train_control,
                           tuneGrid = data.frame(mtry = c(2, 5, 10, 12, 15, 20)),
                           ntree = 500)

## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.

print(rf_model_balanced)

## Random Forest
##
```

```
## 1186 samples
## 27 predictor
## 5 classes: 'LowLow', 'MidLow', 'LowMid', 'MidHigh', 'HighHigh'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1067, 1067, 1068, 1068, 1067, 1066, ...
## Additional sampling using up-sampling
##
## Resampling results across tuning parameters:
##
## mtry Accuracy Kappa Mean_F1 Mean_Sensitivity Mean_Specificity
## 2 0.5066280 0.3579724 0.4626606 0.4558197 0.8709739
## 5 0.5175538 0.3735908 0.4814094 0.4674086 0.8743334
## 10 0.5091861 0.3652468 0.4794498 0.4674164 0.8726567
## 12 0.5075767 0.3643431 0.4653746 0.4637465 0.8728001
## 15 0.5058824 0.3625377 0.4623991 0.4626567 0.8725236
## 20 0.5042363 0.3607163 0.4618516 0.4623397 0.8721591
## Mean_Pos_Pred_Value Mean_Neg_Pred_Value Mean_Precision Mean_Recall
## 0.4575902 0.8739556 0.4575902 0.4558197
## 0.4656626 0.8767613 0.4656626 0.4674086
## 0.4661615 0.8743503 0.4661615 0.4674164
## 0.4610279 0.8743401 0.4610279 0.4637465
## 0.4596569 0.8741495 0.4596569 0.4626567
## 0.4660612 0.8737114 0.4660612 0.4623397
## Mean_Detection_Rate Mean_Balanced_Accuracy
## 0.1013256 0.6633968
## 0.1035108 0.6708710
## 0.1018372 0.6700366
## 0.1015153 0.6682733
## 0.1011765 0.6675901
## 0.1008473 0.6672494
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was mtry = 5.
```

```
cat("Best mtry:", rf_model_balanced$bestTune$mtry, "\n")
```

```
## Best mtry: 5
```

```
cat("Cross-validated accuracy:", max(rf_model_balanced$results$Accuracy), "\n")
```

```
## Cross-validated accuracy: 0.5175538
```

```
preds_rf_up <- predict(rf_model_balanced, newdata = test_data)
cm_rf_up <- confusionMatrix(preds_rf_up, test_data$response, mode = "prec_recall")

probs_rf_up <- predict(rf_model_balanced, newdata = test_data, type = "prob")
probs_rf_up <- as.matrix(probs_rf_up)

colnames(probs_rf_up) <- colnames(probs_rf_up)
probs_rf_up <- probs_rf_up[, levels(test_data$response), drop = FALSE]
```

```

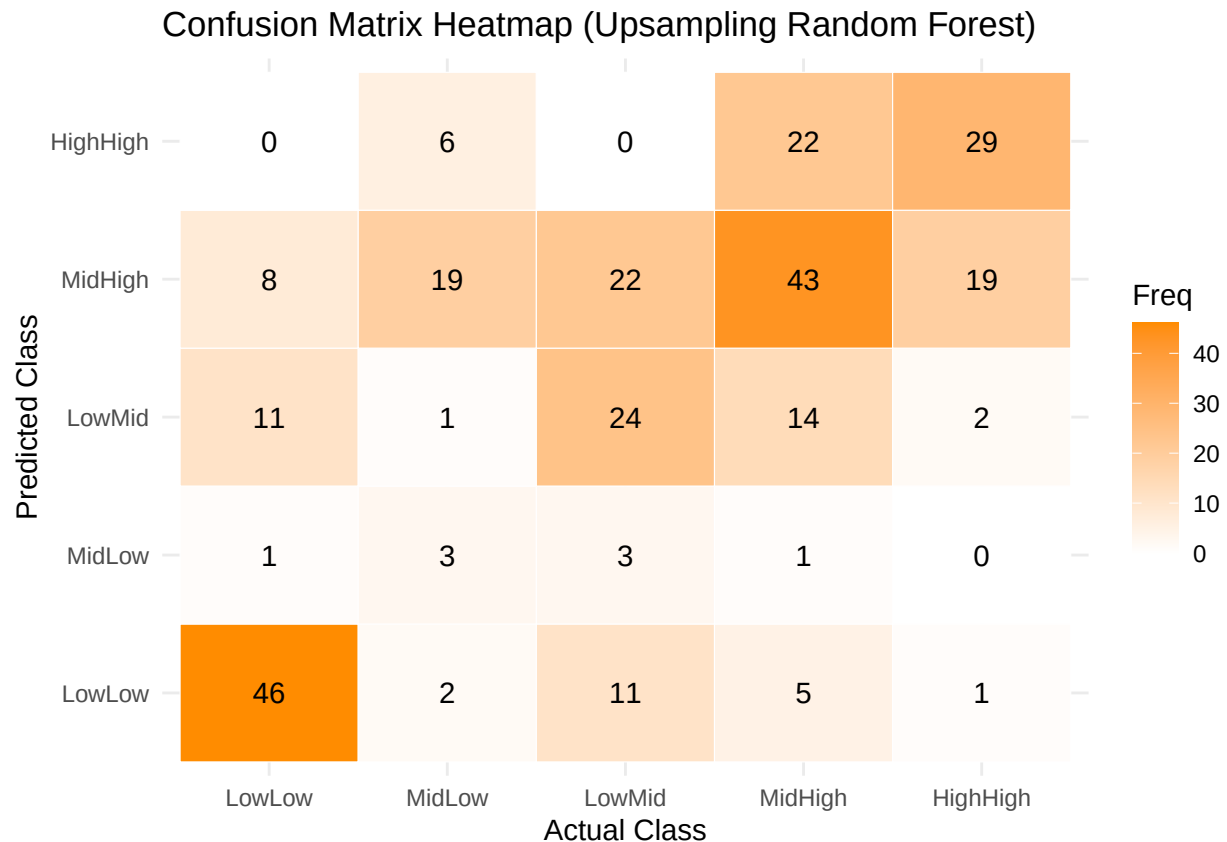
model_results <- rbind(model_results, data.frame(
  Model = "Upsampling Random Forest",
  Accuracy = mean(preds_rf_up == test_data$response),
  Precision = mean(cm_rf_up$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_rf_up$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_rf_up$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data$response, probs_rf_up)
))

```

```

cm_table <- as.data.frame(cm_rf_up$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "darkorange") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (Upsampling Random Forest)",
       x = "Actual Class", y = "Predicted Class")

```



Class Weights

```

set.seed(12)
# Set class weights

```

```

class_counts <- table(train_data$response)
class_weights <- 1 / class_counts
class_weights <- class_weights / sum(class_weights)

# CV for class weights
rf_weighted <- randomForest(response ~ ., data = train_data,
                             ntree = 500,
                             mtry = 7,
                             importance = TRUE,
                             classwt = class_weights)

preds_rf_weighted <- predict(rf_weighted, newdata = test_data)

cm_rf_weighted <- confusionMatrix(preds_rf_weighted, test_data$response, mode = "prec_recall")
accuracy_rf_weighted <- mean(as.character(preds_rf_weighted) == as.character(test_data$response))
cat("Test Accuracy:", accuracy_rf_weighted, "\n")

```

```
## Test Accuracy: 0.4709898
```

```
print(cm_rf_weighted)
```

```

## Confusion Matrix and Statistics
##
##              Reference
## Prediction LowLow MidLow LowMid MidHigh HighHigh
##   LowLow      44      1      10       6       2
##   MidLow       0      1       4       2       0
##   LowMid      14      2      20      11       1
##   MidHigh       8     19      25      50      25
##   HighHigh      0      8       1      16      23
##
## Overall Statistics
##
##              Accuracy : 0.471
##              95% CI : (0.4127, 0.5299)
##   No Information Rate : 0.2901
##   P-Value [Acc > NIR] : 4.831e-11
##
##              Kappa : 0.3051
##
##   McNemar's Test P-Value : 0.0001996
##
## Statistics by Class:
##
##              Class: LowLow Class: MidLow Class: LowMid Class: MidHigh
## Precision              0.6984      0.142857      0.41667      0.3937
## Recall                  0.6667      0.032258      0.33333      0.5882
## F1                      0.6822      0.052632      0.37037      0.4717
## Prevalence              0.2253      0.105802      0.20478      0.2901
## Detection Rate          0.1502      0.003413      0.06826      0.1706
## Detection Prevalence    0.2150      0.023891      0.16382      0.4334
## Balanced Accuracy       0.7915      0.504679      0.60658      0.6090

```



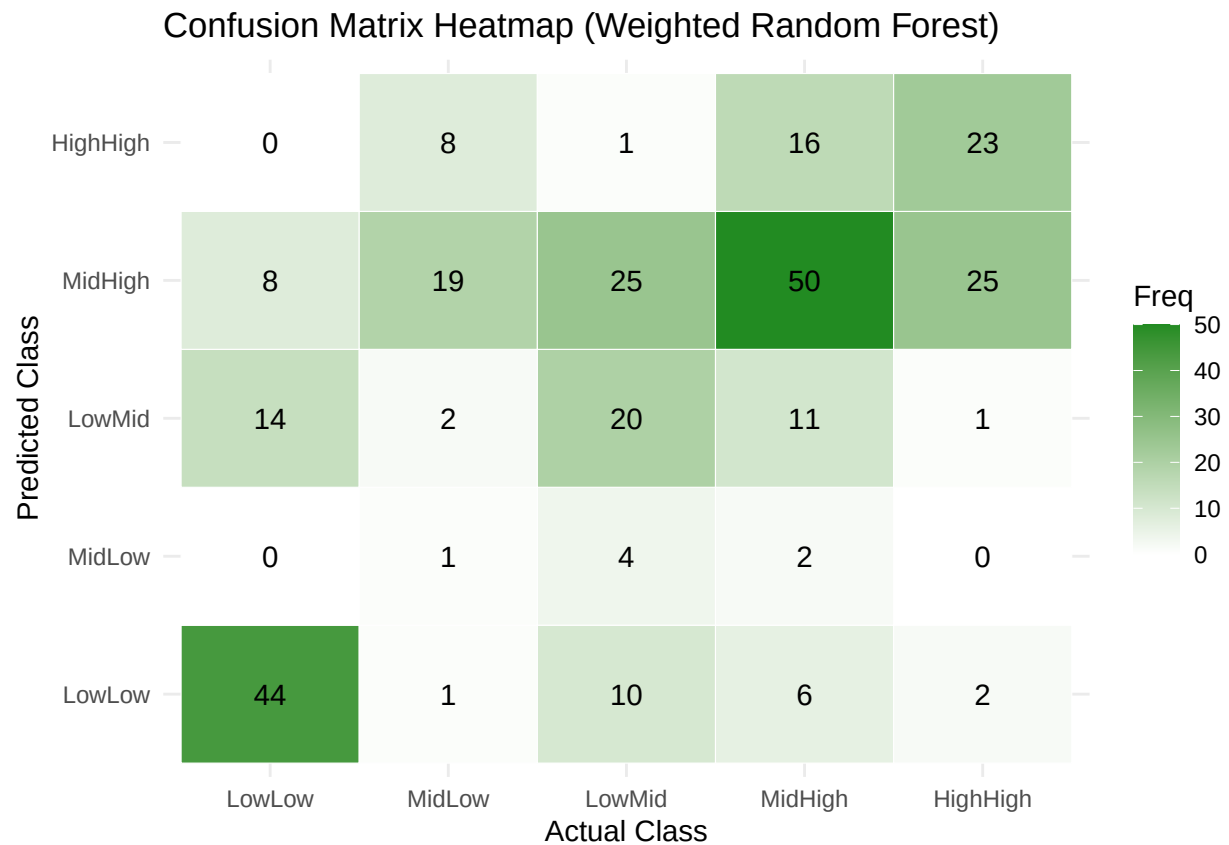
```
##                      Class: HighHigh
## Precision            0.4792
## Recall               0.4510
## F1                   0.4646
## Prevalence           0.1741
## Detection Rate       0.0785
## Detection Prevalence 0.1638
## Balanced Accuracy    0.6738
```

```
probs_rf_weighted <- predict(rf_weighted, newdata = test_data, type = "prob")
probs_rf_weighted <- as.matrix(probs_rf_weighted)

colnames(probs_rf_weighted) <- colnames(probs_rf_weighted)
probs_rf_weighted <- probs_rf_weighted[, levels(test_data$response), drop = FALSE]

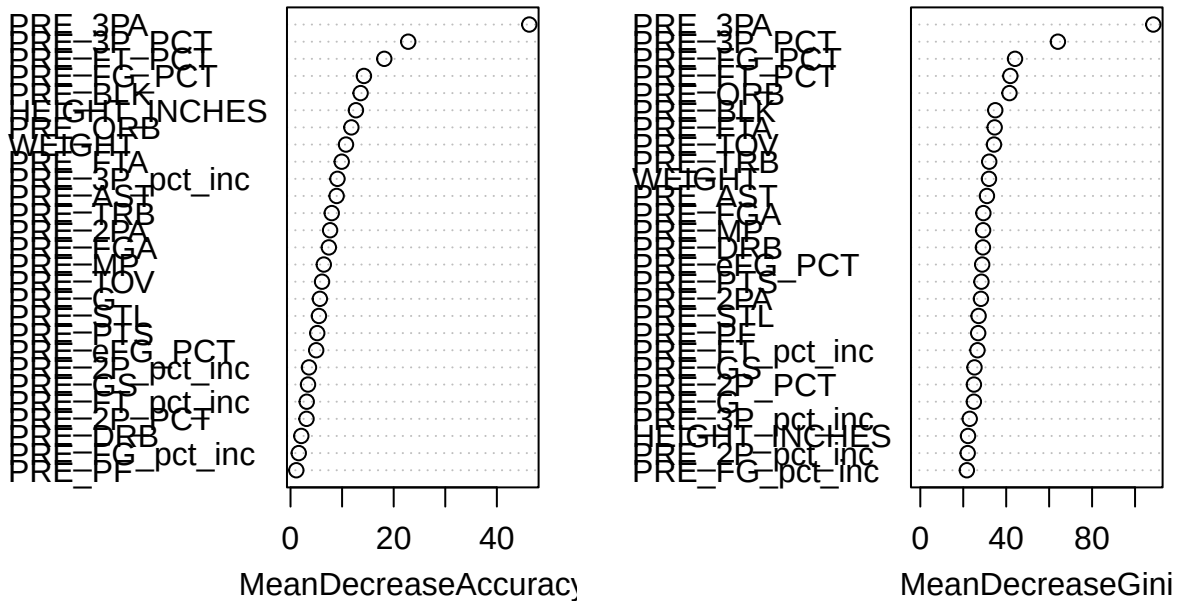
model_results <- rbind(model_results, data.frame(
  Model = "Weighted Random Forest",
  Accuracy = accuracy_rf_weighted,
  Precision = mean(cm_rf_weighted$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_rf_weighted$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_rf_weighted$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data$response, probs_rf_weighted)
))
```

```
cm_table <- as.data.frame(cm_rf_weighted$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "forestgreen") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (Weighted Random Forest)",
       x = "Actual Class", y = "Predicted Class")
```



```
# Original best model, so we check the VarImpPlot
varImpPlot(rf_weighted, main="RF Weighted Variable Importance")
```

RF Weighted Variable Importance



```
# Original best model, so we print a pretty tree
single_tree <- randomForest::getTree(rf_weighted, k = 1, labelVar = TRUE)
```

```
library(rpart)
library(rpart.plot)
```

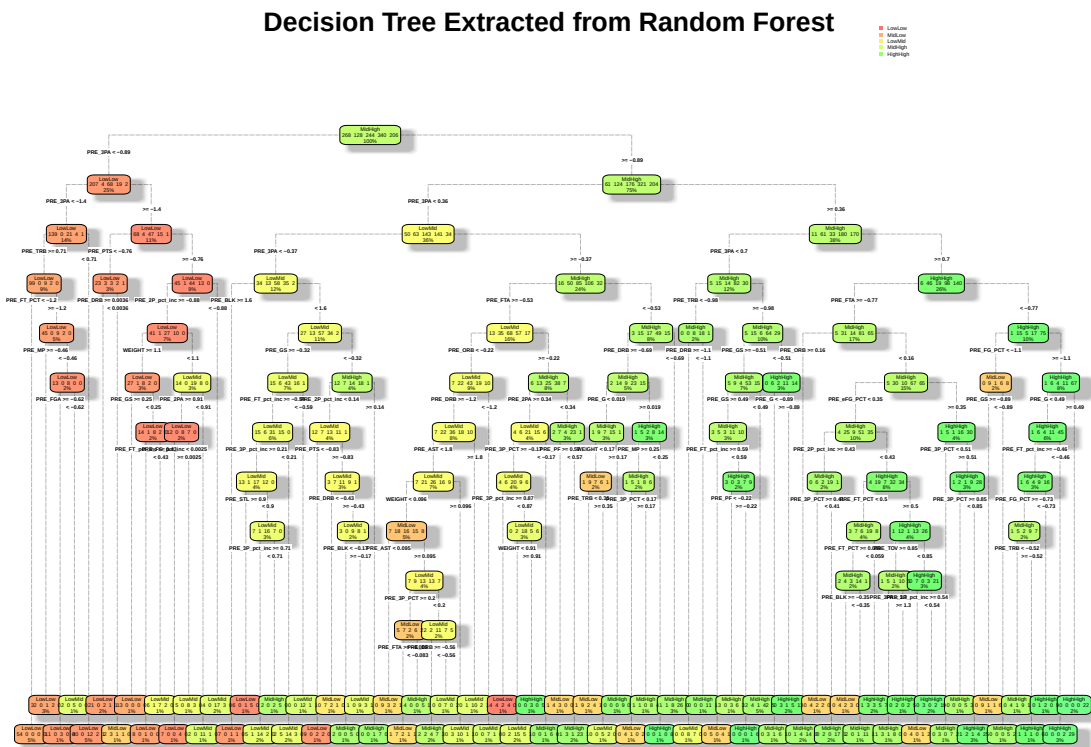
```
## Warning: package 'rpart.plot' was built under R version 4.4.3
```

```
to_rpart <- function(tree, data) {
  response <- all.vars(formula(rf_weighted))[1]
  form <- as.formula(paste(response, "~ ."))
  rpart(form, data = data, control = rpart.control(cp = 0.0001))
}
```

```
rpart_tree <- to_rpart(single_tree, train_data)
rpart.plot(
  rpart_tree,
  type = 4,
  extra = 101,
  fallen.leaves = TRUE,
  box.palette = "RdYlGn",
  branch.lty = 3,
  shadow.col = "gray",
  main = "Decision Tree Extracted from Random Forest"
)
```

```
## Warning: Cannot retrieve the data used to build the model (model.frame: variable lengths differ (four
## To silence this warning:
##   Call rpart.plot with roundint=FALSE,
##   or rebuild the rpart model with model=TRUE.
```

```
## Warning: labs do not fit even at cex 0.15, there may be some overplotting
```



XGBoost

```
set.seed(9999)

# Cross Validation for XGBoost
train_control <- trainControl(method = "cv", number = 10)
tune_grid <- expand.grid(
  nrounds = c(100, 250, 500),
  max_depth = c(3, 5, 7),
  eta = c(0.01, 0.1, 0.3),
  gamma = 0,
  colsample_bytree = 0.8,
  min_child_weight = 1,
  subsample = 0.8
)
```

```
xgb_model <- train(
  response ~ ., data = train_data,
  method = "xgbTree",
  trControl = train_control,
  tuneGrid = tune_grid,
  verbose = FALSE
)
```

[illegible]


```

## [11:14:52] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:14:52] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:15:03] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:15:03] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:15:09] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:15:09] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:15:15] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:15:15] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:15:23] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:15:23] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:15:29] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:15:29] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:15:41] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:15:41] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:15:59] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:15:59] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:16:05] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:16:05] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:16:14] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:16:14] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:16:25] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:16:25] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:16:31] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:16:31] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:16:38] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:16:38] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:16:45] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:16:45] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead

```

```
print(xgb_model)
```

```

## eXtreme Gradient Boosting
##
## 1186 samples
## 27 predictor
## 5 classes: 'LowLow', 'MidLow', 'LowMid', 'MidHigh', 'HighHigh'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1069, 1066, 1069, 1069, 1067, 1066, ...
## Resampling results across tuning parameters:
##
##  eta    max_depth  nrounds  Accuracy    Kappa
##  0.01    3           100      0.5032973  0.3444251
##  0.01    3           250      0.4922859  0.3316637
##  0.01    3           500      0.4998424  0.3443383
##  0.01    5           100      0.5083041  0.3529285
##  0.01    5           250      0.4981335  0.3417283
##  0.01    5           500      0.4922509  0.3359405
##  0.01    7           100      0.4990750  0.3436164
##  0.01    7           250      0.4990605  0.3446290
##  0.01    7           500      0.4931062  0.3374402
##  0.10    3           100      0.4922236  0.3376971
##  0.10    3           250      0.4845593  0.3303759

```



```
## 0.10 3 500 0.4754296 0.3210519
## 0.10 5 100 0.4855992 0.3292741
## 0.10 5 250 0.4822311 0.3274073
## 0.10 5 500 0.4704292 0.3144016
## 0.10 7 100 0.4863758 0.3310957
## 0.10 7 250 0.4796384 0.3241383
## 0.10 7 500 0.4787196 0.3254445
## 0.30 3 100 0.4762979 0.3215116
## 0.30 3 250 0.4637418 0.3053172
## 0.30 3 500 0.4569690 0.2984239
## 0.30 5 100 0.4586430 0.2983826
## 0.30 5 250 0.4595188 0.3013384
## 0.30 5 500 0.4595190 0.3022702
## 0.30 7 100 0.4863684 0.3324019
## 0.30 7 250 0.4838683 0.3316960
## 0.30 7 500 0.4829710 0.3309322
##
## Tuning parameter 'gamma' was held constant at a value of 0
## Tuning
##
## Tuning parameter 'min_child_weight' was held constant at a value of 1
##
## Tuning parameter 'subsample' was held constant at a value of 0.8
## Accuracy was used to select the optimal model using the largest value.
## The final values used for the model were nrounds = 100, max_depth = 5, eta
## = 0.01, gamma = 0, colsample_bytree = 0.8, min_child_weight = 1 and
## subsample = 0.8.
```

```
cat("Best parameters:\n")
```

```
## Best parameters:
```

```
print(xgb_model$bestTune)
```

```
## nrounds max_depth eta gamma colsample_bytree min_child_weight subsample
## 4 100 5 0.01 0 0.8 1 0.8
```

```
cat("Cross-validated accuracy:", max(xgb_model$results$Accuracy), "\n")
```

```
## Cross-validated accuracy: 0.5083041
```

```
preds_xgb <- predict(xgb_model, newdata = test_data)
preds_xgb <- factor(preds_xgb, levels = levels(test_data$response))

cm_xgb <- confusionMatrix(preds_xgb, test_data$response)
print(cm_xgb)
```

```
## Confusion Matrix and Statistics
##
## Reference
## Prediction LowLow MidLow LowMid MidHigh HighHigh
```

```

##      LowLow      48      3      12      7      2
##      MidLow      0      0       1      0      0
##      LowMid     10      0      18      8      0
##      MidHigh      8     23     29     53     25
##      HighHigh     0      5       0     17     24
##
## Overall Statistics
##
##              Accuracy : 0.4881
##              95% CI : (0.4295, 0.5469)
##      No Information Rate : 0.2901
##      P-Value [Acc > NIR] : 7.865e-13
##
##              Kappa : 0.3221
##
## Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##              Class: LowLow Class: MidLow Class: LowMid Class: MidHigh
## Sensitivity              0.7273      0.000000      0.30000      0.6235
## Specificity              0.8943      0.996183      0.92275      0.5913
## Pos Pred Value           0.6667      0.000000      0.50000      0.3841
## Neg Pred Value           0.9186      0.893836      0.83658      0.7935
## Prevalence               0.2253      0.105802      0.20478      0.2901
## Detection Rate           0.1638      0.000000      0.06143      0.1809
## Detection Prevalence     0.2457      0.003413      0.12287      0.4710
## Balanced Accuracy        0.8108      0.498092      0.61137      0.6074
##
##              Class: HighHigh
## Sensitivity              0.47059
## Specificity              0.90909
## Pos Pred Value           0.52174
## Neg Pred Value           0.89069
## Prevalence               0.17406
## Detection Rate           0.08191
## Detection Prevalence     0.15700
## Balanced Accuracy        0.68984

```

```

set.seed(9999)
# Another cross validation
trainControl(method = "cv", number = 10, summaryFunction = multiClassSummary)

```

```

## $method
## [1] "cv"
##
## $number
## [1] 10
##
## $repeats
## [1] NA
##
## $search
## [1] "grid"
##

```

```

## $p
## [1] 0.75
##
## $initialWindow
## NULL
##
## $horizon
## [1] 1
##
## $fixedWindow
## [1] TRUE
##
## $skip
## [1] 0
##
## $verboseIter
## [1] FALSE
##
## $returnData
## [1] TRUE
##
## $returnResamp
## [1] "final"
##
## $savePredictions
## [1] FALSE
##
## $classProbs
## [1] FALSE
##
## $summaryFunction
## function (data, lev = NULL, model = NULL)
## {
##     if (!all(levels(data[, "pred"]) == levels(data[, "obs"])))
##         stop("levels of observed and predicted data do not match")
##     has_class_probs <- all(lev %in% colnames(data))
##     if (has_class_probs) {
##         lloss <- mnLogLoss(data = data, lev = lev, model = model)
##         requireNamespaceQuietStop("pROC")
##         requireNamespaceQuietStop("MLmetrics")
##         prob_stats <- lapply(levels(data[, "pred"]), function(x) {
##             obs <- ifelse(data[, "obs"] == x, 1, 0)
##             prob <- data[, x]
##             roc_auc <- try(pROC::roc(obs, data[, x], direction = "<",
##                 quiet = TRUE), silent = TRUE)
##             roc_auc <- if (inherits(roc_auc, "try-error"))
##                 NA
##             else roc_auc$auc
##             pr_auc <- try(MLmetrics::PRAUC(y_pred = data[, x],
##                 y_true = obs), silent = TRUE)
##             if (inherits(pr_auc, "try-error"))
##                 pr_auc <- NA
##             res <- c(ROC = roc_auc, AUC = pr_auc)
##             return(res)

```

```

##      })
##      prob_stats <- do.call("rbind", prob_stats)
##      prob_stats <- colMeans(prob_stats, na.rm = TRUE)
##    }
##    CM <- confusionMatrix(data[, "pred"], data[, "obs"], mode = "everything")
##    if (length(levels(data[, "pred"])) == 2) {
##      class_stats <- CM$byClass
##    }
##    else {
##      class_stats <- colMeans(CM$byClass)
##      names(class_stats) <- paste("Mean", names(class_stats))
##    }
##    overall_stats <- if (has_class_probs)
##      c(CM$overall, logLoss = as.numeric(lloss), AUC = unname(prob_stats["ROC"]),
##        prAUC = unname(prob_stats["AUC"]))
##    else CM$overall
##    stats <- c(overall_stats, class_stats)
##    stats <- stats[!names(stats) %in% c("AccuracyNull", "AccuracyLower",
##      "AccuracyUpper", "AccuracyPValue", "McnemarPValue", "Mean Prevalence",
##      "Mean Detection Prevalence")]
##    names(stats) <- gsub("[:blank:]]+", "_", names(stats))
##    stat_list <- c("Accuracy", "Kappa", "Mean_F1", "Mean_Sensitivity",
##      "Mean_Specificity", "Mean_Pos_Pred_Value", "Mean_Neg_Pred_Value",
##      "Mean_Precision", "Mean_Recall", "Mean_Detection_Rate",
##      "Mean_Balanced_Accuracy")
##    if (has_class_probs)
##      stat_list <- c("logLoss", "AUC", "prAUC", stat_list)
##    if (length(levels(data[, "pred"])) == 2)
##      stat_list <- gsub("^Mean_", "", stat_list)
##    stats <- stats[c(stat_list)]
##    return(stats)
##  }
## <bytecode: 0x000001952f17cd58>
## <environment: namespace:caret>
##
## $selectionFunction
## [1] "best"
##
## $preProcOptions
## $preProcOptions$thresh
## [1] 0.95
##
## $preProcOptions$ICAcomp
## [1] 3
##
## $preProcOptions$k
## [1] 5
##
## $preProcOptions$freqCut
## [1] 19
##
## $preProcOptions$uniqueCut
## [1] 10
##

```

```

## $preProcOptions$cutoff
## [1] 0.9
##
##
## $sampling
## NULL
##
## $index
## NULL
##
## $indexOut
## NULL
##
## $indexFinal
## NULL
##
## $timingSamps
## [1] 0
##
## $predictionBounds
## [1] FALSE FALSE
##
## $seeds
## [1] NA
##
## $adaptive
## $adaptive$min
## [1] 5
##
## $adaptive$alpha
## [1] 0.05
##
## $adaptive$method
## [1] "gls"
##
## $adaptive$complete
## [1] TRUE
##
##
## $trim
## [1] FALSE
##
## $allowParallel
## [1] TRUE

```

```

tune_grid <- expand_grid(
  nrounds = 100,
  max_depth = 3,
  eta = 0.01,
  gamma = 0,
  colsample_bytree = 0.8,
  min_child_weight = 1,
  subsample = 0.8
)

```

```

xgb_model <- train(response ~ ., data = train_data,
  method = "xgbTree",
  trControl = trainControl(method = "cv", number = 10),
  tuneGrid = tune_grid,
  verbose = FALSE)

cat("Cross-validated accuracy:", max(xgb_model$results$Accuracy), "\n")

## Cross-validated accuracy: 0.4939389

preds_xgb <- predict(xgb_model, newdata = test_data)
preds_xgb <- factor(preds_xgb, levels = levels(test_data$response))

cm_xgb <- confusionMatrix(preds_xgb, test_data$response, mode = "prec_recall")
print(cm_xgb)

## Confusion Matrix and Statistics
##
##           Reference
## Prediction LowLow MidLow LowMid MidHigh HighHigh
## LowLow      50      2      15      10      2
## MidLow       0      0       0       0      0
## LowMid       5      1      12       4      0
## MidHigh     11     25      33      53     25
## HighHigh     0      3       0      18     24
##
## Overall Statistics
##
##           Accuracy : 0.4744
##           95% CI : (0.416, 0.5333)
## No Information Rate : 0.2901
## P-Value [Acc > NIR] : 2.178e-11
##
##           Kappa : 0.3007
##
## McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##           Class: LowLow Class: MidLow Class: LowMid Class: MidHigh
## Precision           0.6329           NA           0.54545           0.3605
## Recall              0.7576           0.0000           0.20000           0.6235
## F1                  0.6897           NA           0.29268           0.4569
## Prevalence          0.2253           0.1058           0.20478           0.2901
## Detection Rate      0.1706           0.0000           0.04096           0.1809
## Detection Prevalence 0.2696           0.0000           0.07509           0.5017
## Balanced Accuracy    0.8149           0.5000           0.57854           0.5858
##
##           Class: HighHigh
## Precision           0.53333
## Recall              0.47059
## F1                  0.50000

```

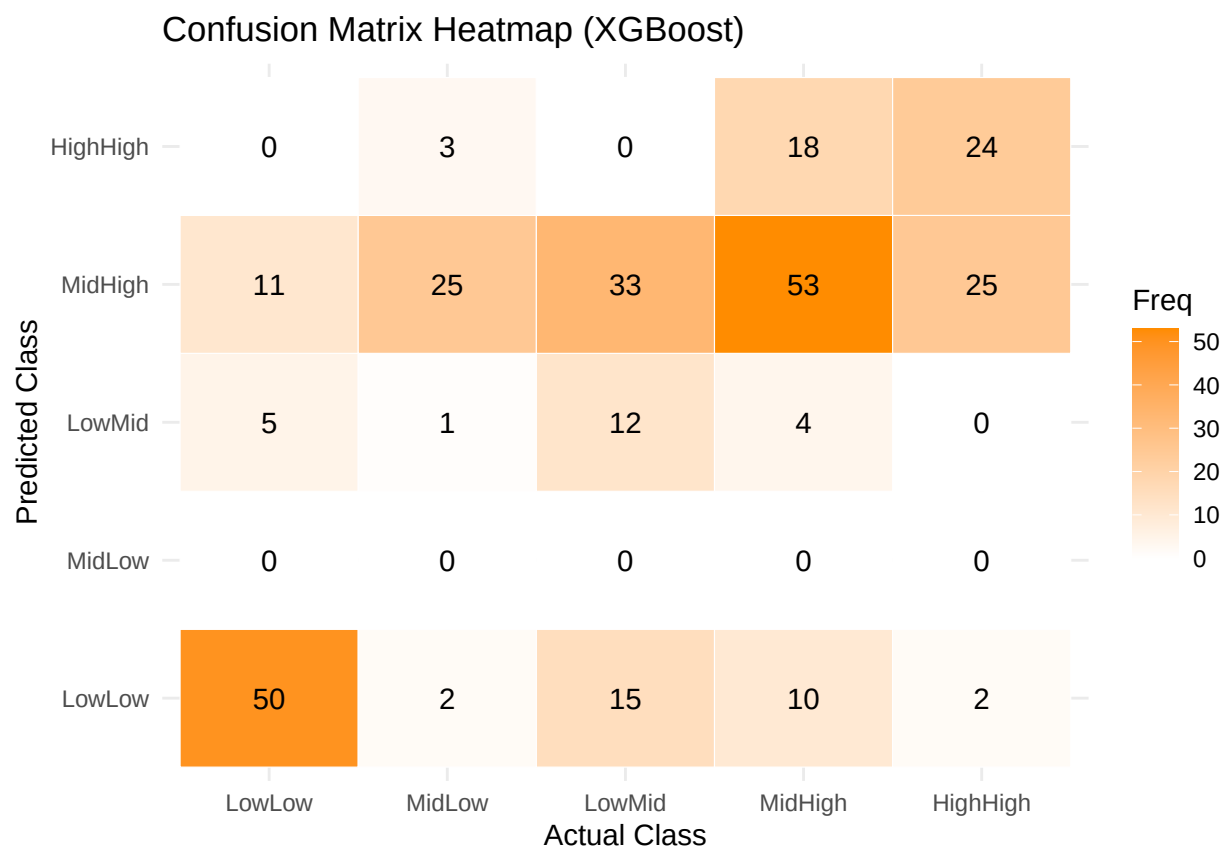
```
## Prevalence          0.17406
## Detection Rate      0.08191
## Detection Prevalence 0.15358
## Balanced Accuracy   0.69191
```

```
probs_xgb <- predict(xgb_model, newdata = test_data, type = "prob")
probs_xgb <- as.matrix(probs_xgb)

colnames(probs_xgb) <- colnames(test_data)
probs_xgb <- probs_xgb[, levels(test_data$response), drop = FALSE]

model_results <- rbind(model_results, data.frame(
  Model = "XGBoost",
  Accuracy = max(xgb_model$results$Accuracy),
  Precision = mean(cm_xgb$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_xgb$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_xgb$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data$response, probs_xgb)
))
```

```
cm_table <- as.data.frame(cm_xgb$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "darkorange") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (XGBoost)",
       x = "Actual Class", y = "Predicted Class")
```



Model Results

```
print(model_results)
```

```
##           Model Accuracy Precision   Recall      F1      AUC
## 1  LASSO Min Lambda 0.4778157 0.4885430 0.4201604 0.4912253 0.7795353
## 2  LASSO 1se Lambda 0.4812287 0.5031451 0.4153298 0.4844480 0.7578509
## 3           LDA 0.4607509 0.4776979 0.4135294 0.4934686 0.7552616
## 4           QDA 0.4744027 0.4357303 0.4197849 0.4001403 0.7359076
## 5           PCA 0.4539249 0.4730732 0.4027273 0.4783739 0.7765284
## 6   SVM Linear 0.5221843 0.5508501 0.4663993 0.5593007 0.7861685
## 7 SVM Radial Kernel 0.5085324 0.6048168 0.4642413 0.4412243 0.7861773
## 8 Base Random Forest 0.4846416 0.5063826 0.4256684 0.5118718 0.7781371
## 9 Upsampling Random Forest 0.4948805 0.4880780 0.4536507 0.4521040 0.7734690
## 10 Weighted Random Forest 0.4709898 0.4261608 0.4142948 0.4083034 0.7851856
## 11          XGBoost 0.4939389 0.5180609 0.4103387 0.4848087 0.7633494
```

```
model_results_combined <- model_results
```


class 3PA

Initialize Train / Test Split

```
# Specific set of predictors mentioned in report
X <- per_min_data[, c(8,13,38:40,42,43,45,46,48:50, 52:55, 56:66)]
X[is.na(X)] <- 0
y <- per_min_data[, 68]

y <- factor(y, levels = 1:5, labels = c("Bad", "BelowAvg", "Average", "AboveAvg", "Great"))

df <- data.frame(X, response = y)
df_scaled <- data.frame(scale(df[, -ncol(df)]), response = df$response)

# Split train and test
set.seed(67)
train_index <- createDataPartition(df_scaled$response, p = 0.8, list = FALSE)
train_data <- df_scaled[train_index, ]
test_data <- df_scaled[-train_index, ]
class_levels <- c("Bad", "BelowAvg", "Average", "AboveAvg", "Great")
```

Backwards Stepwise Selection

Check for collinearity

```
vif(step_model)
```

```
## y>=BelowAvg y>=Average y>=AboveAvg y>=Great PRE_G PRE_MP
## 1.496031 1.411801 1.375525 1.600139 11.171487 11.757821
## PRE_3PA PRE_FT_PCT PRE_DRB PRE_AST PRE_BLK PRE_TOV
## 2.327251 1.479582 1.490329 1.422147 1.855144 1.223393
```

```
predictors <- df_scaled[, c("PRE_G", "PRE_MP", "PRE_3PA", "PRE_FT_PCT", "PRE_DRB", "PRE_AST", "PRE_BLK")]
cor_matrix <- cor(predictors)
print(round(cor_matrix, 3))
```

```
## PRE_G PRE_MP PRE_3PA PRE_FT_PCT PRE_DRB PRE_AST PRE_BLK PRE_TOV
## PRE_G 1.000 0.948 0.028 0.107 -0.058 0.059 -0.104 -0.087
## PRE_MP 0.948 1.000 0.104 0.188 -0.131 0.142 -0.189 -0.067
## PRE_3PA 0.028 0.104 1.000 0.602 -0.304 0.286 -0.579 -0.083
## PRE_FT_PCT 0.107 0.188 0.602 1.000 -0.226 0.277 -0.475 -0.082
## PRE_DRB -0.058 -0.131 -0.304 -0.226 1.000 -0.228 0.373 0.005
## PRE_AST 0.059 0.142 0.286 0.277 -0.228 1.000 -0.434 0.263
## PRE_BLK -0.104 -0.189 -0.579 -0.475 0.373 -0.434 1.000 -0.016
## PRE_TOV -0.087 -0.067 -0.083 -0.082 0.005 0.263 -0.016 1.000
```

Our main issue is between PPG and FGA, check both.

```
noMP <- polr(response ~ PRE_G+PRE_3PA+PRE_FT_PCT+PRE_DRB+PRE_AST+PRE_BLK+PRE_TOV, data = train_data, Hess = TRUE)
noG <- polr(response ~ PRE_MP+PRE_3PA+PRE_FT_PCT+PRE_DRB+PRE_AST+PRE_BLK+PRE_TOV, data = train_data, Hess = TRUE)

AIC(noMP);AIC(noG)
```

```
## [1] 2615.11
```

```
## [1] 2604.255
```

Because the no G model has the lower AIC, it is the better model because we chose AIC as our criterion.

```
summary(noG)
```

```
## Call:
## polr(formula = response ~ PRE_MP + PRE_3PA + PRE_FT_PCT + PRE_DRB +
##       PRE_AST + PRE_BLK + PRE_TOV, data = train_data, Hess = TRUE)
##
## Coefficients:
##              Value Std. Error t value
## PRE_MP        -0.2776    0.06422  -4.322
## PRE_3PA         1.8298    0.09653  18.955
## PRE_FT_PCT     0.4058    0.07998   5.074
## PRE_DRB         0.3912    0.07226   5.414
## PRE_AST         0.3077    0.06608   4.656
## PRE_BLK        -0.4237    0.09006  -4.705
## PRE_TOV        -0.5523    0.06286  -8.787
##
## Intercepts:
##              Value      Std. Error t value
## Bad|BelowAvg    -2.1743     0.1048  -20.7545
## BelowAvg|Average -0.7632     0.0830   -9.1947
## Average|AboveAvg  1.1127     0.0838   13.2780
## AboveAvg|Great    3.8979     0.1458   26.7412
##
## Residual Deviance: 2582.255
## AIC: 2604.255
```

```
vif(noG)
```

```
##              PRE_MP      PRE_3PA      PRE_FT_PCT      PRE_DRB
##          1.101153      2.367892      1.485792      1.475470
##              PRE_AST      PRE_BLK      PRE_TOV      Bad|BelowAvg
##          1.406297      1.865505      1.219604      1.598734
## BelowAvg|Average Average|AboveAvg AboveAvg|Great
##          1.484005      1.358854      1.569415
```

```
predictors <- df_scaled[, c("PRE_MP", "PRE_3PA", "PRE_FT_PCT", "PRE_DRB", "PRE_AST", "PRE_BLK", "PRE_TOV")]
cor_matrix <- cor(predictors)
print(round(cor_matrix, 3))
```

##	PRE_MP	PRE_3PA	PRE_FT_PCT	PRE_DRB	PRE_AST	PRE_BLK	PRE_TOV
## PRE_MP	1.000	0.104	0.188	-0.131	0.142	-0.189	-0.067
## PRE_3PA	0.104	1.000	0.602	-0.304	0.286	-0.579	-0.083
## PRE_FT_PCT	0.188	0.602	1.000	-0.226	0.277	-0.475	-0.082
## PRE_DRB	-0.131	-0.304	-0.226	1.000	-0.228	0.373	0.005
## PRE_AST	0.142	0.286	0.277	-0.228	1.000	-0.434	0.263
## PRE_BLK	-0.189	-0.579	-0.475	0.373	-0.434	1.000	-0.016
## PRE_TOV	-0.067	-0.083	-0.082	0.005	0.263	-0.016	1.000

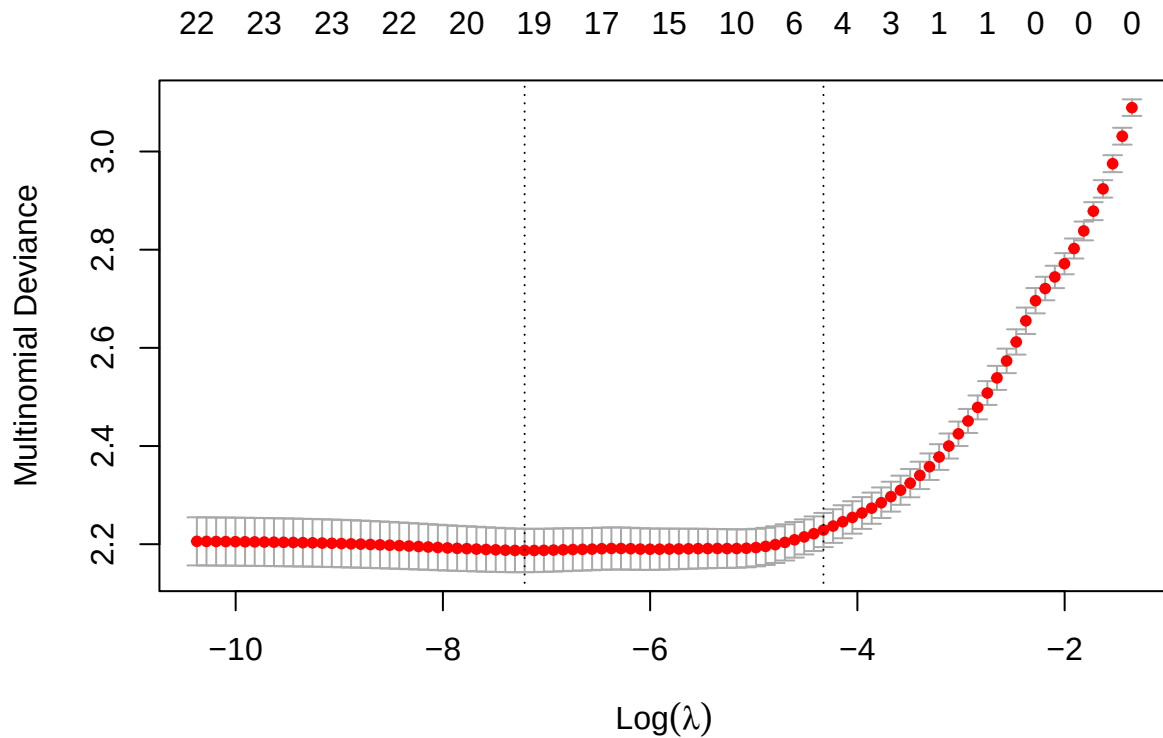
Initializing Storing Results

```
model_results <- data.frame(
  Model = character(),
  Accuracy = numeric(),
  Precision = numeric(),
  Recall = numeric(),
  F1 = numeric(),
  AUC = numeric(),
  stringsAsFactors = FALSE
)
```

LASSO with CV for best lambda

```
set.seed(67)
#initialize data in matrices
x_train <- as.matrix(train_data[, -ncol(train_data)])
y_train <- train_data$response
x_test <- as.matrix(test_data[, -ncol(test_data)])
y_test <- test_data$response

#run CV model
cv_lasso <- cv.glmnet(x_train, y_train,
  family = "multinomial",
  alpha = 1,
  nfolds = 10)
plot(cv_lasso)
```



```
# Check optimal values of lambda
cat("Optimal lambda (min):", cv_lasso$lambda.min, "\n")
```

```
## Optimal lambda (min): 0.000738116
```

```
cat("Optimal lambda (1se):", cv_lasso$lambda.1se, "\n")
```

```
## Optimal lambda (1se): 0.01320231
```

```
coef(cv_lasso, s = "lambda.min")
```

```
## $Bad
## 28 x 1 sparse Matrix of class "dgCMatrix"
##           1
## (Intercept)  -0.2760846
## WEIGHT      .
## HEIGHT_INCHES 0.2130461
## PRE_G        -0.3262398
## PRE_GS       -0.3275792
## PRE_MP       0.9532604
## PRE_FGA      .
## PRE_FG_PCT   0.4787705
## PRE_3PA     -0.9963682
## PRE_3P_PCT  -0.3128918
```

```

## PRE_2PA      .
## PRE_2P_PCT   -0.7375469
## PRE_eFG_PCT  .
## PRE_FTA      0.3951929
## PRE_FT_PCT   -0.4667667
## PRE_ORB      .
## PRE_DRB      -0.4400695
## PRE_TRB      .
## PRE_AST      -0.8281487
## PRE_STL      0.2219281
## PRE_BLK      0.4089347
## PRE_TOV      0.6348130
## PRE_PF       -0.3150089
## PRE_PTS      -0.1882028
## PRE_FG_pct_inc .
## PRE_3P_pct_inc -0.2308326
## PRE_2P_pct_inc 0.4672368
## PRE_FT_pct_inc .
##
## $BelowAvg
## 28 x 1 sparse Matrix of class "dgCMatrix"
##              1
## (Intercept)   0.62584261
## WEIGHT        -0.06849438
## HEIGHT_INCHES .
## PRE_G         -0.75360909
## PRE_GS        -0.40262371
## PRE_MP        0.95276372
## PRE_FGA       0.14595044
## PRE_FG_PCT    .
## PRE_3PA       -0.45989147
## PRE_3P_PCT    -0.15373879
## PRE_2PA       0.17620998
## PRE_2P_PCT    -0.70375914
## PRE_eFG_PCT   .
## PRE_FTA       .
## PRE_FT_PCT    -0.17215551
## PRE_ORB       0.16657488
## PRE_DRB       .
## PRE_TRB       -0.30218595
## PRE_AST       -0.30001157
## PRE_STL       0.26568872
## PRE_BLK       .
## PRE_TOV       0.26366250
## PRE_PF        -0.14375517
## PRE_PTS       .
## PRE_FG_pct_inc 0.28242244
## PRE_3P_pct_inc 0.10463070
## PRE_2P_pct_inc .
## PRE_FT_pct_inc -0.03470597
##
## $Average
## 28 x 1 sparse Matrix of class "dgCMatrix"
##              1

```

```

## (Intercept)      1.309752391
## WEIGHT            -0.114185421
## HEIGHT_INCHES    -0.037334906
## PRE_G             .
## PRE_GS            .
## PRE_MP            .
## PRE_FGA           -0.061547852
## PRE_FG_PCT        -0.421249626
## PRE_3PA           .
## PRE_3P_PCT        .
## PRE_2PA           -0.484110577
## PRE_2P_PCT         0.052828538
## PRE_eFG_PCT        0.001357215
## PRE_FTA           .
## PRE_FT_PCT        .
## PRE_ORB           0.142721385
## PRE_DRB           -0.148529478
## PRE_TRB           -0.179120996
## PRE_AST           0.018939813
## PRE_STL           .
## PRE_BLK           0.055325374
## PRE_TOV           .
## PRE_PF            .
## PRE_PTS           0.506493740
## PRE_FG_pct_inc     0.195496678
## PRE_3P_pct_inc     .
## PRE_2P_pct_inc     -0.029081933
## PRE_FT_pct_inc     -0.064770204
##
## $AboveAvg
## 28 x 1 sparse Matrix of class "dgCMatrix"
##              1
## (Intercept)      1.067557305
## WEIGHT            0.030286142
## HEIGHT_INCHES    -0.260315677
## PRE_G             0.157740756
## PRE_GS            0.174336225
## PRE_MP           -0.435008474
## PRE_FGA           .
## PRE_FG_PCT        0.135941856
## PRE_3PA           1.653649031
## PRE_3P_PCT        .
## PRE_2PA           .
## PRE_2P_PCT        0.117660914
## PRE_eFG_PCT       -0.502874318
## PRE_FTA           .
## PRE_FT_PCT        0.238322339
## PRE_ORB           -0.016509275
## PRE_DRB           0.006854364
## PRE_TRB           0.198817061
## PRE_AST           .
## PRE_STL           -0.127936295
## PRE_BLK           -0.099180786
## PRE_TOV           -0.244023121

```

```
## PRE_PF          0.215736487
## PRE_PTS         -0.031931562
## PRE_FG_pct_inc -0.073019107
## PRE_3P_pct_inc -0.013111302
## PRE_2P_pct_inc  0.107785963
## PRE_FT_pct_inc  0.054782572
##
## $Great
## 28 x 1 sparse Matrix of class "dgCMatrix"
##              1
## (Intercept)  -2.7270677059
## WEIGHT       .
## HEIGHT_INCHES .
## PRE_G         0.7624006910
## PRE_GS        0.3064873434
## PRE_MP       -1.2197452298
## PRE_FGA       .
## PRE_FG_PCT    .
## PRE_3PA       2.9432530159
## PRE_3P_PCT    0.2171844766
## PRE_2PA       .
## PRE_2P_PCT    .
## PRE_eFG_PCT   .
## PRE_FTA      -0.3866801231
## PRE_FT_PCT    0.7484237232
## PRE_ORB      -1.1984223886
## PRE_DRB       0.5483275447
## PRE_TRB       0.3165463938
## PRE_AST       0.0767570357
## PRE_STL      -0.1706721471
## PRE_BLK      -0.6784478054
## PRE_TOV      -0.6319161080
## PRE_PF        0.4233285722
## PRE_PTS       0.0537354612
## PRE_FG_pct_inc -0.1575332440
## PRE_3P_pct_inc  0.2311497139
## PRE_2P_pct_inc -0.0827944063
## PRE_FT_pct_inc  0.0004421279
```

```
# Calculate Test Accuracy
```

```
preds_min <- predict(cv_lasso, newx = x_test, s = "lambda.min", type = "class")
accuracy_min <- mean(as.character(preds_min) == as.character(y_test))
cat("Test accuracy (lambda.min):", accuracy_min, "\n")
```

```
## Test accuracy (lambda.min): 0.5426621
```

```
# Calculate CM
```

```
preds_min <- factor(preds_min, levels = levels(y_test))
conf_mat_min <- confusionMatrix(preds_min, y_test, mode = "prec_recall")
print(conf_mat_min)
```

```
## Confusion Matrix and Statistics
##
```

```
##           Reference
## Prediction Bad BelowAvg Average AboveAvg Great
##   Bad      58      17      12       2       0
##   BelowAvg  2       8       6       2       0
##   Average   7      14      29      15       1
##   AboveAvg  1       4      26      56      18
##   Great     0       0       1       6       8
##
## Overall Statistics
##
##           Accuracy : 0.5427
##           95% CI : (0.4837, 0.6007)
##   No Information Rate : 0.2765
##   P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.3981
##
## McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##           Class: Bad Class: BelowAvg Class: Average Class: AboveAvg
## Precision      0.6517      0.44444      0.43939      0.5333
## Recall         0.8529      0.18605      0.39189      0.6914
## F1             0.7389      0.26230      0.41429      0.6022
## Prevalence     0.2321      0.14676      0.25256      0.2765
## Detection Rate 0.1980      0.02730      0.09898      0.1911
## Detection Prevalence 0.3038      0.06143      0.22526      0.3584
## Balanced Accuracy 0.8576      0.57302      0.61147      0.7301
##
##           Class: Great
## Precision      0.53333
## Recall         0.29630
## F1             0.38095
## Prevalence     0.09215
## Detection Rate 0.02730
## Detection Prevalence 0.05119
## Balanced Accuracy 0.63499
```

```
# Calculate Class Error
class_error_min <- 1 - diag(conf_mat_min$table) / colSums(conf_mat_min$table)
print(class_error_min)
```

```
##           Bad BelowAvg Average AboveAvg Great
## 0.1470588 0.8139535 0.6081081 0.3086420 0.7037037
```

```
# AUC Vectors
probs_min <- predict(cv_lasso, newx = x_test, s = "lambda.min", type = "response")
probs_min <- probs_min[,1]
colnames(probs_min) <- levels(y_test)

# Repeat
preds_1se <- predict(cv_lasso, newx = x_test, s = "lambda.1se", type = "class")
accuracy_1se <- mean(as.character(preds_1se) == as.character(y_test))
cat("Test accuracy (lambda.1se):", accuracy_1se, "\n")
```



```
## Test accuracy (lambda.1se): 0.5324232
```

```
preds_1se <- factor(preds_1se, levels = levels(y_test))
conf_mat_1se <- confusionMatrix(preds_1se, y_test, mode = "prec_recall")
print(conf_mat_1se)
```

```
## Confusion Matrix and Statistics
```

```
##
##           Reference
## Prediction Bad BelowAvg Average AboveAvg Great
##   Bad      60      16      11       4      0
##   BelowAvg  0       0       2       0      0
##   Average   7      22      30      14      1
##   AboveAvg  1       5      29      61     21
##   Great     0       0       2       2      5
```

```
## Overall Statistics
```

```
##
##           Accuracy : 0.5324
##           95% CI   : (0.4735, 0.5907)
##   No Information Rate : 0.2765
##   P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.3765
##
##   McNemar's Test P-Value : NA
```

```
## Statistics by Class:
```

```
##
##           Class: Bad Class: BelowAvg Class: Average Class: AboveAvg
## Precision      0.6593      0.000000      0.4054      0.5214
## Recall         0.8824      0.000000      0.4054      0.7531
## F1             0.7547      NaN          0.4054      0.6162
## Prevalence     0.2321      0.146758      0.2526      0.2765
## Detection Rate 0.2048      0.000000      0.1024      0.2082
## Detection Prevalence 0.3106      0.006826      0.2526      0.3993
## Balanced Accuracy 0.8723      0.496000      0.6022      0.7445
##
##           Class: Great
## Precision      0.55556
## Recall         0.18519
## F1             0.27778
## Prevalence     0.09215
## Detection Rate 0.01706
## Detection Prevalence 0.03072
## Balanced Accuracy 0.58507
```

```
class_error_1se <- 1 - diag(conf_mat_1se$table) / colSums(conf_mat_1se$table)
print(class_error_1se)
```

```
##           Bad BelowAvg Average AboveAvg Great
## 0.1176471 1.0000000 0.5945946 0.2469136 0.8148148
```

```

probs_1se <- predict(cv_lasso, newx = x_test, s = "lambda.1se", type = "response")
probs_1se <- probs_1se[, , 1]
colnames(probs_1se) <- levels(y_test)

```

```

# Load model results (done for every model)
model_results <- rbind(model_results, data.frame(
  Model = "LASSO Min Lambda",
  Accuracy = accuracy_min,
  Precision = mean(conf_mat_min$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(conf_mat_min$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(conf_mat_min$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(y_test, probs_min)
))

model_results <- rbind(model_results, data.frame(
  Model = "LASSO 1se Lambda",
  Accuracy = accuracy_1se,
  Precision = mean(conf_mat_1se$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(conf_mat_1se$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(conf_mat_1se$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(y_test, probs_1se)
))

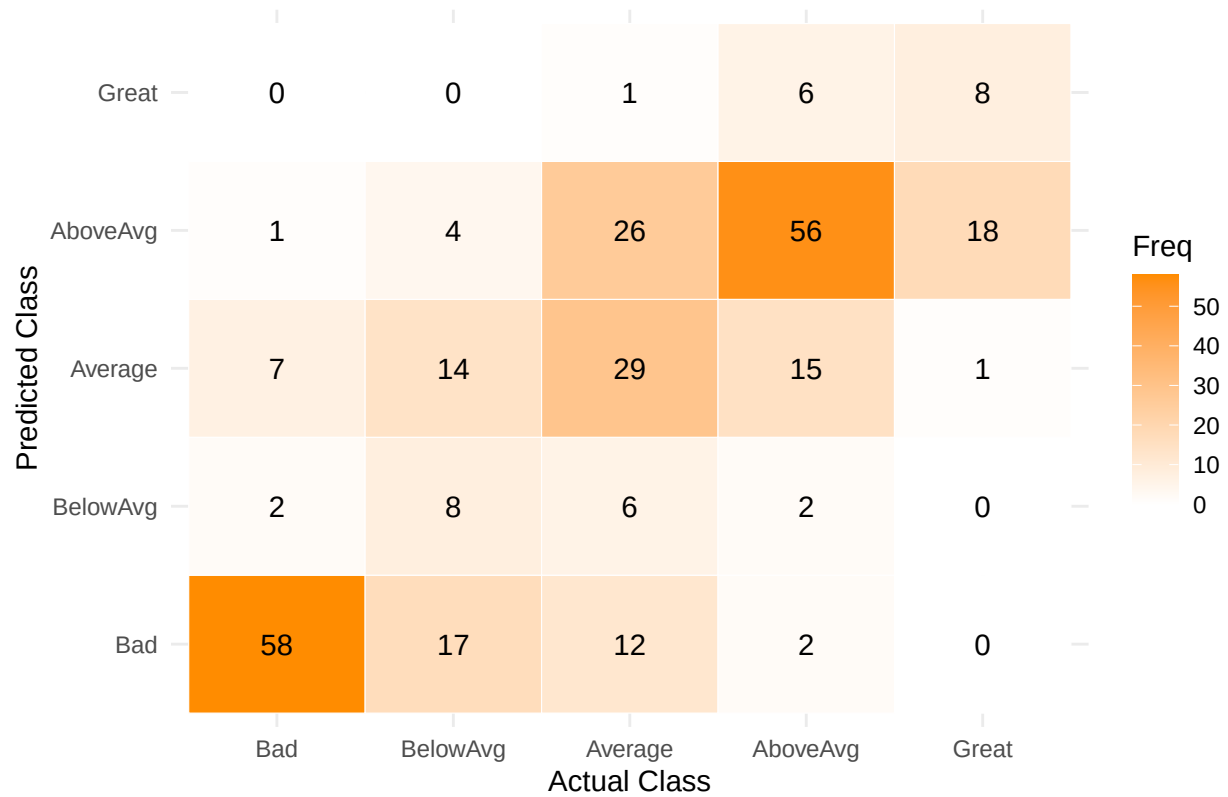
```

```

# Create confusion matrix (done for every model)
cm_table <- as.data.frame(conf_mat_min$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "darkorange") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (LASSO Min Lambda)",
       x = "Actual Class", y = "Predicted Class")

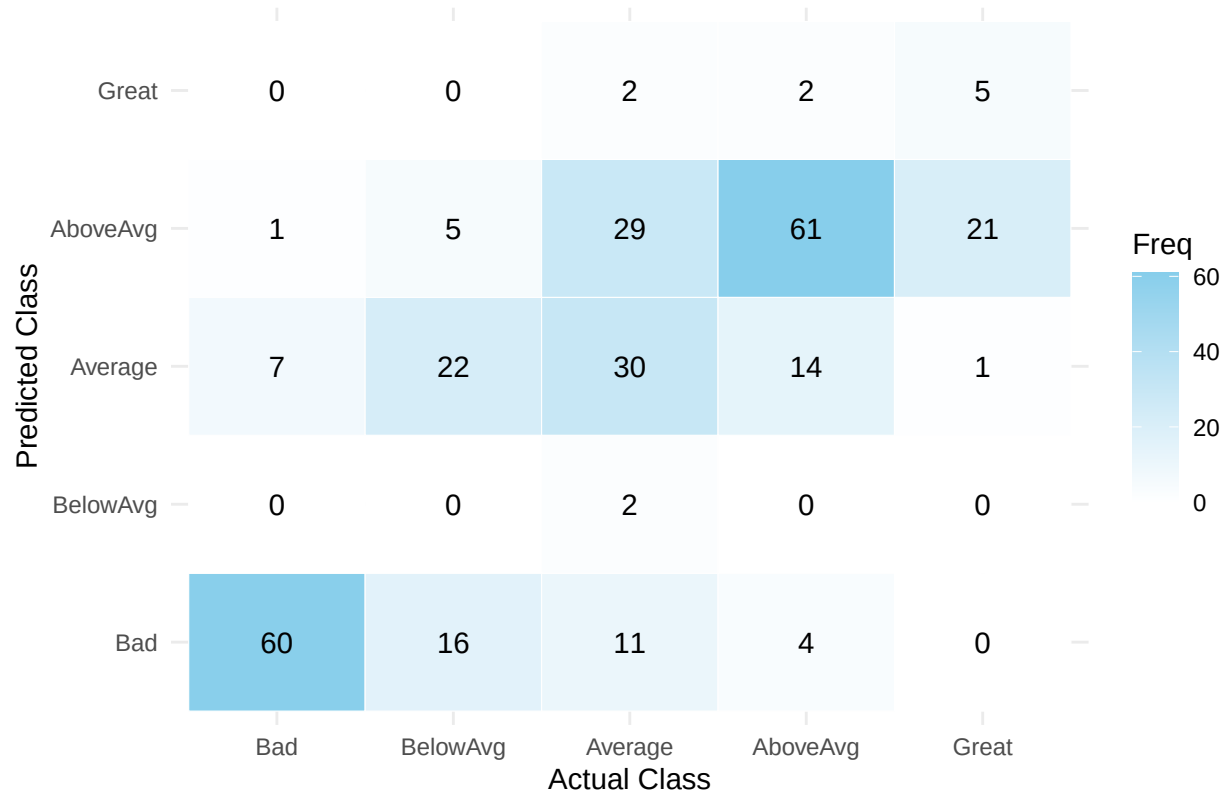
```

Confusion Matrix Heatmap (LASSO Min Lambda)



```
cm_table <- as.data.frame(conf_mat_1se$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "skyblue") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (LASSO Lambda 1se)",
       x = "Actual Class", y = "Predicted Class")
```

Confusion Matrix Heatmap (LASSO Lambda 1se)



LDA

```
set.seed(22222)
# Run cross validation
train_control <- trainControl(method = "cv", number = 10,
                              summaryFunction = multiClassSummary)

lda_cv <- train(response ~ PRE_FG_PCT + PRE_3PA + PRE_FTA + PRE_DRB +
                PRE_BLK + PRE_TOV + PRE_PTS, data = train_data,
                method = "lda",
                trControl = train_control)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.
```

```
print(lda_cv)
```

```
## Linear Discriminant Analysis
##
## 1186 samples
## 7 predictor
## 5 classes: 'Bad', 'BelowAvg', 'Average', 'AboveAvg', 'Great'
```

```
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1067, 1067, 1068, 1066, 1068, 1066, ...
## Resampling results:
##
##   Accuracy   Kappa     Mean_F1     Mean_Sensitivity Mean_Specificity
##   0.5270154  0.3797266  0.4893783  0.4811997        0.8754868
##   Mean_Pos_Pred_Value Mean_Neg_Pred_Value Mean_Precision Mean_Recall
##   0.5065004          0.8788007          0.5065004      0.4811997
##   Mean_Detection_Rate Mean_Balanced_Accuracy
##   0.1054031          0.6783432

cat("Cross-validated Accuracy (training set):", max(lda_cv$results$Accuracy), "\n")
```

```
## Cross-validated Accuracy (training set): 0.5270154
```

```
# Set fit model
lda_fit <- lda(response ~ PRE_FG_PCT + PRE_3PA + PRE_FTA + PRE_DRB +
               PRE_BLK + PRE_TOV + PRE_PTS,
               data = train_data)

pred_obj <- predict(lda_fit, test_data)

all_pred  <- factor(pred_obj$class, levels = class_levels)
all_actual <- factor(test_data$response, levels = class_levels)
all_probs <- pred_obj$posterior

# Find CM
conf_mat_lda <- confusionMatrix(all_pred, all_actual, mode = "prec_recall")
print(conf_mat_lda)
```

```
## Confusion Matrix and Statistics
##
##               Reference
## Prediction Bad BelowAvg Average AboveAvg Great
##   Bad        54         10         8         3         0
##   BelowAvg    7          9         8         2         0
##   Average     6         20        34        13         2
##   AboveAvg    1          4        21        57        15
##   Great       0          0         3         6        10
##
## Overall Statistics
##
##               Accuracy : 0.5597
##               95% CI : (0.5008, 0.6174)
##   No Information Rate : 0.2765
##   P-Value [Acc > NIR] : < 2.2e-16
##
##               Kappa : 0.4241
##
##   Mcnemar's Test P-Value : NA
##
```

```
## Statistics by Class:
##
##           Class: Bad Class: BelowAvg Class: Average Class: AboveAvg
## Precision           0.7200           0.34615           0.4533           0.5816
## Recall              0.7941           0.20930           0.4595           0.7037
## F1                  0.7552           0.26087           0.4564           0.6369
## Prevalence          0.2321           0.14676           0.2526           0.2765
## Detection Rate      0.1843           0.03072           0.1160           0.1945
## Detection Prevalence 0.2560           0.08874           0.2560           0.3345
## Balanced Accuracy   0.8504           0.57065           0.6361           0.7552
##
##           Class: Great
## Precision           0.52632
## Recall              0.37037
## F1                  0.43478
## Prevalence          0.09215
## Detection Rate      0.03413
## Detection Prevalence 0.06485
## Balanced Accuracy   0.66827
```

```
# Calculate Accuracy and AUC
```

```
LDA_acc <- mean(as.character(all_pred) == as.character(all_actual))
cat("Test Accuracy (LDA):", LDA_acc, "\n")
```

```
## Test Accuracy (LDA): 0.559727
```

```
overall_multiclass <- multiclass.roc(all_actual, all_probs)
cat("Overall multiclass AUC (test set):", overall_multiclass$auc, "\n")
```

```
## Overall multiclass AUC (test set): 0.8134318
```

```
# Graph AUC / Calculate Macro AUC
```

```
fpr_grid <- seq(0, 1, length.out = 200)
tpr_matrix <- matrix(NA, nrow = length(fpr_grid), ncol = length(class_levels))
```

```
for (i in seq_along(class_levels)) {
  class <- class_levels[i]
  binary_actual <- factor(ifelse(all_actual == class, class, "Other"),
                           levels = c("Other", class))
  class_probs <- all_probs[, class]
```

```
roc_obj <- roc(response = binary_actual,
               predictor = class_probs,
               levels = c("Other", class),
               positive = class,
               quiet = TRUE)
```

```
cat("AUC of", class, ":", auc(roc_obj), "\n")
```

```
fpr <- 1 - roc_obj$specificities
tpr <- roc_obj$sensitivities
```

```
interp_tpr <- approx(fpr, tpr, xout = fpr_grid, ties = mean, rule = 2)$y
```

```
tpr_matrix[, i] <- interp_tpr
}
```

```
## AUC of Bad : 0.9217647
## AUC of BelowAvg : 0.7412093
## AUC of Average : 0.7062816
## AUC of AboveAvg : 0.8126601
## AUC of Great : 0.8845725
```

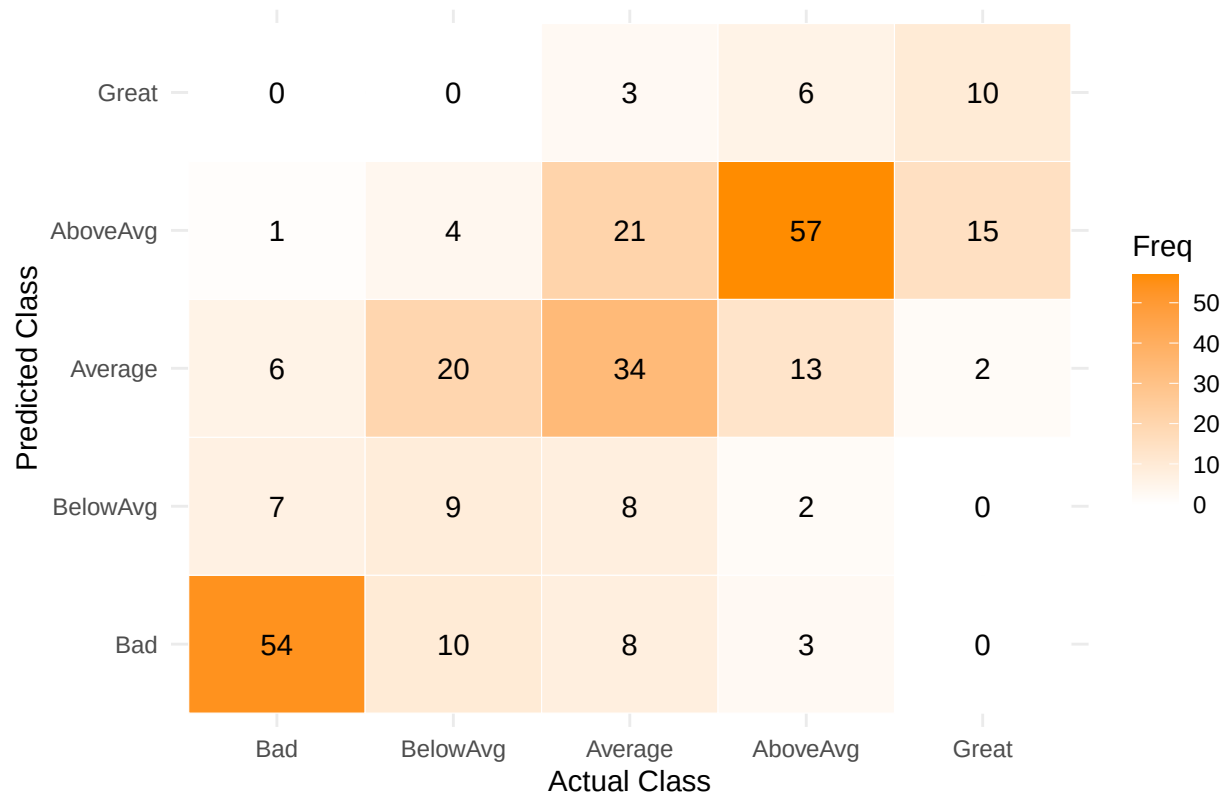
```
macro_tpr_lda <- rowMeans(tpr_matrix, na.rm = TRUE)
macro_auc_lda <- trapz(fpr_grid, macro_tpr_lda)
cat("Macro AUC (LDA test set):", macro_auc_lda, "\n")
```

```
## Macro AUC (LDA test set): 0.8132451
```

```
model_results <- rbind(model_results, data.frame(
  Model = "LDA",
  Accuracy = LDA_acc,
  Precision = mean(conf_mat_lda$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(conf_mat_lda$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(conf_mat_lda$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc_lda
))
```

```
cm_table <- as.data.frame(conf_mat_lda$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "darkorange") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (LDA)",
       x = "Actual Class", y = "Predicted Class")
```

Confusion Matrix Heatmap (LDA)



QDA

```
# Same steps as LDA
set.seed(22222)
train_control <- trainControl(method = "cv", number = 10,
                              summaryFunction = multiClassSummary)

qda_cv <- train(response ~ PRE_FG_PCT + PRE_3PA + PRE_FTA + PRE_DRB +
                 PRE_BLK + PRE_TOV + PRE_PTS,
                 data = train_data,
                 method = "qda",
                 trControl = train_control)

## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.

print(qda_cv)

## Quadratic Discriminant Analysis
##
## 1186 samples
## 7 predictor
```



```
## 5 classes: 'Bad', 'BelowAvg', 'Average', 'AboveAvg', 'Great'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1067, 1067, 1068, 1066, 1068, 1066, ...
## Resampling results:
##
## Accuracy Kappa Mean_F1 Mean_Sensitivity Mean_Specificity
## 0.4867123 0.3315791 0.4564835 0.4636145 0.8657222
## Mean_Pos_Pred_Value Mean_Neg_Pred_Value Mean_Precision Mean_Recall
## 0.4605643 0.8685816 0.4605643 0.4636145
## Mean_Detection_Rate Mean_Balanced_Accuracy
## 0.09734245 0.6646683

cat("Cross-validated Accuracy (training set):", max(qda_cv$results$Accuracy), "\n")
```

```
## Cross-validated Accuracy (training set): 0.4867123
```

```
qda_fit <- qda(response ~ PRE_FG_PCT + PRE_3PA + PRE_FTA + PRE_DRB +
               PRE_BLK + PRE_TOV + PRE_PTS,
               data = train_data)

pred_obj <- predict(qda_fit, test_data)

all_pred <- factor(pred_obj$class, levels = class_levels)
all_actual <- factor(test_data$response, levels = class_levels)
all_probs <- pred_obj$posterior

conf_mat_qda <- confusionMatrix(all_pred, all_actual, mode = "prec_recall")
print(conf_mat_qda)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Bad BelowAvg Average AboveAvg Great
## Bad      57      12      9       2      0
## BelowAvg  5       8      9       0      1
## Average   4      16     25      17      3
## AboveAvg  1       6     28      46     11
## Great    1       1      3      16     12
##
## Overall Statistics
##
##           Accuracy : 0.5051
##           95% CI : (0.4464, 0.5638)
##           No Information Rate : 0.2765
##           P-Value [Acc > NIR] : < 2e-16
##
##           Kappa : 0.3589
##
##           Mcnemar's Test P-Value : 0.05999
##
```

```
## Statistics by Class:
##
##           Class: Bad Class: BelowAvg Class: Average Class: AboveAvg
## Precision           0.7125           0.3478           0.38462           0.5000
## Recall              0.8382           0.1860           0.33784           0.5679
## F1                  0.7703           0.2424           0.35971           0.5318
## Prevalence          0.2321           0.1468           0.25256           0.2765
## Detection Rate      0.1945           0.0273           0.08532           0.1570
## Detection Prevalence 0.2730           0.0785           0.22184           0.3140
## Balanced Accuracy   0.8680           0.5630           0.57759           0.6755
##
##           Class: Great
## Precision           0.36364
## Recall              0.44444
## F1                  0.40000
## Prevalence          0.09215
## Detection Rate      0.04096
## Detection Prevalence 0.11263
## Balanced Accuracy   0.68275
```

```
QDA_acc <- mean(as.character(all_pred) == as.character(all_actual))
cat("Test Accuracy (QDA):", QDA_acc, "\n")
```

```
## Test Accuracy (QDA): 0.5051195
```

```
overall_multiclass <- multiclass.roc(all_actual, all_probs)
cat("Overall multiclass AUC (QDA test set):", overall_multiclass$auc, "\n")
```

```
## Overall multiclass AUC (QDA test set): 0.7817208
```

```
fpr_grid <- seq(0, 1, length.out = 200)
tpr_matrix <- matrix(NA, nrow = length(fpr_grid), ncol = length(class_levels))

for (i in seq_along(class_levels)) {
  class <- class_levels[i]
  binary_actual <- factor(ifelse(all_actual == class, class, "Other"),
                           levels = c("Other", class))
  class_probs <- all_probs[, class]

  roc_obj <- roc(response = binary_actual,
                 predictor = class_probs,
                 levels = c("Other", class),
                 positive = class,
                 quiet = TRUE)

  cat("AUC of", class, ":", auc(roc_obj), "\n")

  fpr <- 1 - roc_obj$specificities
  tpr <- roc_obj$sensitivities

  interp_tpr <- approx(fpr, tpr, xout = fpr_grid, ties = mean, rule = 2)$y
  tpr_matrix[, i] <- interp_tpr
}
```

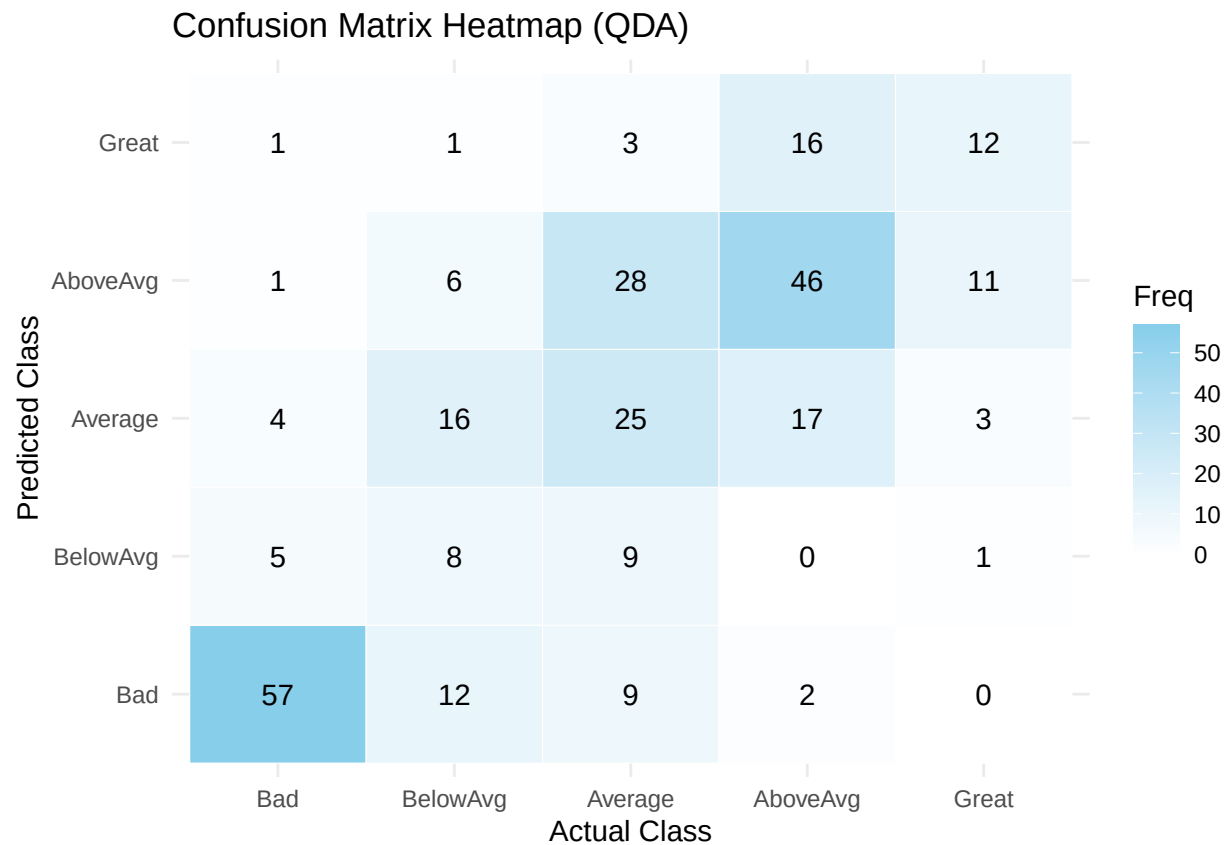
```
## AUC of Bad : 0.9306536
## AUC of BelowAvg : 0.7173953
## AUC of Average : 0.6796865
## AUC of AboveAvg : 0.7911717
## AUC of Great : 0.7844612
```

```
macro_tpr_qda <- rowMeans(tpr_matrix, na.rm = TRUE)
macro_auc_qda <- trapz(fpr_grid, macro_tpr_qda)
cat("Macro AUC (QDA test set):", macro_auc_qda, "\n")
```

```
## Macro AUC (QDA test set): 0.7806059
```

```
model_results <- rbind(model_results, data.frame(
  Model = "QDA",
  Accuracy = QDA_acc,
  Precision = mean(conf_mat_qda$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(conf_mat_qda$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(conf_mat_qda$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc_qda
))
```

```
cm_table <- as.data.frame(conf_mat_qda$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "skyblue") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (QDA)",
       x = "Actual Class", y = "Predicted Class")
```



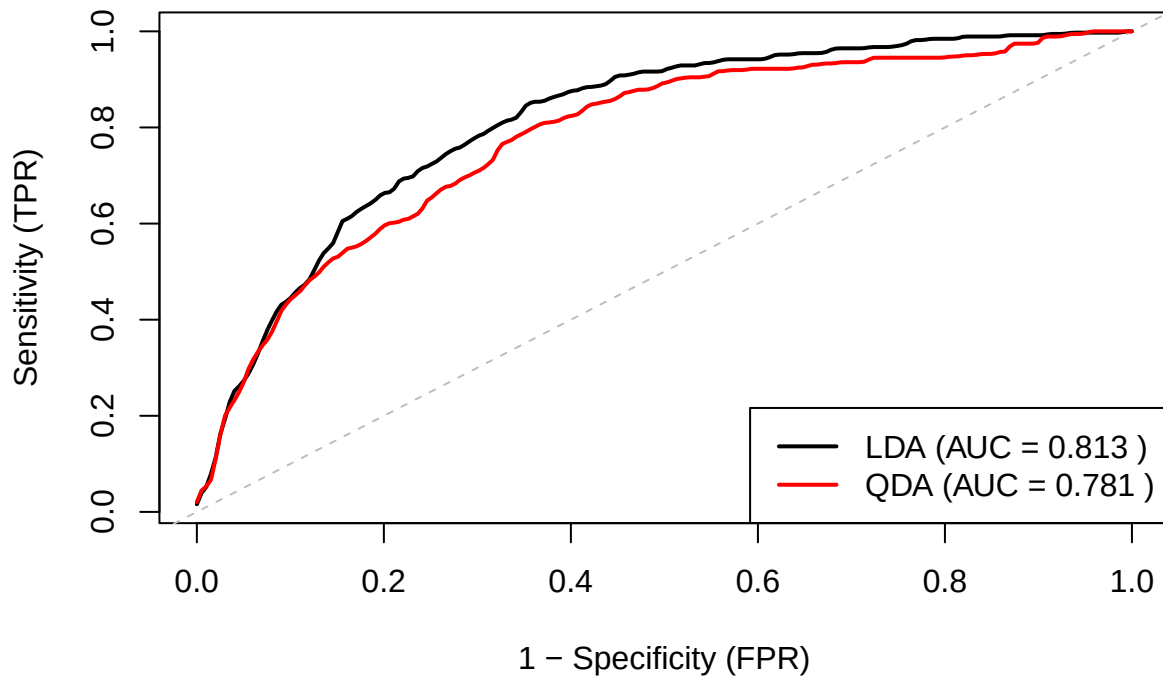
Comparison of LDA vs QDA

```
LDA_acc - QDA_acc
```

```
## [1] 0.05460751
```

```
# Graph AUC curves
plot(fpr_grid, macro_tpr_lda, type = "l", lwd = 2, col = "black",
     xlab = "1 - Specificity (FPR)", ylab = "Sensitivity (TPR)",
     main = paste("Macro-average ROC Curves of LDA and QDA"))
abline(a = 0, b = 1, lty = 2, col = "gray")
lines(fpr_grid, macro_tpr_qda, lwd = 2, col = "red")
legend("bottomright",
      legend = c(paste("LDA (AUC =", round(macro_auc_lda, 3), ")"),
                paste("QDA (AUC =", round(macro_auc_qda, 3), ")")),
      col = c("black", "red"), lwd = 2)
```

Macro-average ROC Curves of LDA and QDA



PCA

```
# Set PCA statistics
pca_result <- prcomp(df_scaled[, -ncol(df_scaled)], center = TRUE, scale. = TRUE)
explained_var <- summary(pca_result)$importance[2, ]
num_pcs <- which(cumsum(explained_var) >= 0.9)[1]

# Set PCA dataframe
df_pca <- data.frame(pca_result$x[, 1:num_pcs], response = df_scaled$response)
train_data_pca <- df_pca[train_index, ]
test_data_pca <- df_pca[-train_index, ]

# PCA with CV
train_control <- trainControl(method = "cv", number = 10,
                             summaryFunction = multiClassSummary)

model_pca_cv <- train(response ~ ., data = train_data_pca,
                      method = "multinom",
                      trControl = train_control,
                      trace = FALSE)

print(model_pca_cv)
```

```
## Penalized Multinomial Regression
##
## 1186 samples
## 13 predictor
```

```
## 5 classes: 'Bad', 'BelowAvg', 'Average', 'AboveAvg', 'Great'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1067, 1067, 1066, 1068, 1069, 1066, ...
## Resampling results across tuning parameters:
##
## decay Accuracy Kappa Mean_F1 Mean_Sensitivity Mean_Specificity
## 0e+00 0.5150567 0.3624974 0.4611786 0.4686500 0.8718006
## 1e-04 0.5150567 0.3624974 0.4611786 0.4686500 0.8718006
## 1e-01 0.5159114 0.3635044 0.4619973 0.4699821 0.8719779
## Mean_Pos_Pred_Value Mean_Neg_Pred_Value Mean_Precision Mean_Recall
## 0.4969275 0.8761858 0.4969275 0.4686500
## 0.4969275 0.8761858 0.4969275 0.4686500
## 0.5033992 0.8764706 0.5033992 0.4699821
## Mean_Detection_Rate Mean_Balanced_Accuracy
## 0.1030113 0.6702253
## 0.1030113 0.6702253
## 0.1031823 0.6709800
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was decay = 0.1.
```

```
cat("Cross-validated Accuracy (training set):", max(model_pca_cv$results$Accuracy), "\n")
```

```
## Cross-validated Accuracy (training set): 0.5159114
```

```
# Set our fit model
model_pca <- multinom(response ~ ., data = train_data_pca, trace = FALSE)

preds_pca <- predict(model_pca, newdata = test_data_pca)
preds_pca <- factor(preds_pca, levels = levels(test_data_pca$response))

# Calc CM
cm_pca <- confusionMatrix(preds_pca, test_data_pca$response, mode = "prec_recall")
print(cm_pca)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Bad BelowAvg Average AboveAvg Great
## Bad          59         13         11          2          0
## BelowAvg      1          6          7          3          0
## Average       7         19         29         12          1
## AboveAvg      1          5         25         52         19
## Great         0          0          2         12          7
##
## Overall Statistics
##
##           Accuracy : 0.5222
##           95% CI : (0.4633, 0.5806)
##           No Information Rate : 0.2765
##           P-Value [Acc > NIR] : < 2.2e-16
```

```
##
##           Kappa : 0.3735
##
## McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##           Class: Bad Class: BelowAvg Class: Average Class: AboveAvg
## Precision           0.6941           0.35294           0.42647           0.5098
## Recall              0.8676           0.13953           0.39189           0.6420
## F1                  0.7712           0.20000           0.40845           0.5683
## Prevalence          0.2321           0.14676           0.25256           0.2765
## Detection Rate      0.2014           0.02048           0.09898           0.1775
## Detection Prevalence 0.2901           0.05802           0.23208           0.3481
## Balanced Accuracy    0.8760           0.54777           0.60690           0.7031
##
##           Class: Great
## Precision           0.33333
## Recall              0.25926
## F1                  0.29167
## Prevalence          0.09215
## Detection Rate      0.02389
## Detection Prevalence 0.07167
## Balanced Accuracy    0.60331
```

```
test_acc <- mean(as.character(preds_pca) == as.character(test_data_pca$response))
cat("Test Accuracy (PCA-based model):", test_acc, "\n")
```

```
## Test Accuracy (PCA-based model): 0.5221843
```

```
loadings <- pca_result$rotation[, 1:num_pcs]
print("PCA Loadings (contribution of original variables):")
```

```
## [1] "PCA Loadings (contribution of original variables):"
```

```
print(loadings)
```

```
##           PC1           PC2           PC3           PC4           PC5
## WEIGHT      0.310576735 -0.11600745  0.022405694 -0.090180825  0.01486769
## HEIGHT_INCHES 0.306018471 -0.12869752 -0.071442881 -0.070894339  0.04853623
## PRE_G       -0.091069205 -0.39616697  0.179522649 -0.150512294 -0.04018886
## PRE_GS      -0.108814951 -0.33153667  0.243856055 -0.150788708 -0.11745689
## PRE_MP      -0.131801927 -0.34671668  0.252903108 -0.147007736 -0.06628751
## PRE_FGA     -0.060335081  0.20465217  0.167377182 -0.448716734  0.07429732
## PRE_FG_PCT   0.297628706 -0.08839393  0.066116020 -0.009784959 -0.09895303
## PRE_3PA     -0.304268051  0.10539397 -0.006859503 -0.100076805  0.11678585
## PRE_3P_PCT  -0.209899910  0.05912609  0.069077397 -0.027605119 -0.03071896
## PRE_2PA      0.163609176  0.25793448  0.356691231 -0.006040583 -0.13122173
## PRE_2P_PCT   0.092818662  0.24657497  0.352240719  0.283858437 -0.18429705
## PRE_eFG_PCT  0.065374880  0.25458067  0.359438727  0.286058254 -0.16684636
## PRE_FTA      0.170662156  0.11379092  0.164509523 -0.343449549 -0.08557226
## PRE_FT_PCT  -0.247252477  0.07428605  0.092619611 -0.120717187  0.02573812
## PRE_ORB      0.252306293 -0.01153781  0.173584896 -0.046970034  0.35464067
```

## PRE_DRB	0.193106037	-0.03361403	-0.099911399	-0.107603026	-0.37179356
## PRE_TRB	0.236534587	-0.06157029	-0.102453639	-0.088923308	-0.45323070
## PRE_AST	-0.235526532	0.03941034	-0.091674798	-0.001542908	-0.37897486
## PRE_STL	-0.150644666	0.00788266	-0.186349012	-0.042236917	-0.47896250
## PRE_BLK	0.306173178	-0.04844280	-0.014637065	0.030408065	-0.02030777
## PRE_TOV	0.016687285	0.09544624	0.009094554	-0.275116011	-0.08817865
## PRE_PF	0.217643408	-0.16667332	-0.183079463	-0.046787190	0.04078897
## PRE_PTS	0.038980952	0.20570127	0.223216572	-0.460695696	0.01450406
## PRE_FG_pct_inc	0.004719971	-0.32262204	0.195555697	0.068318283	0.01528496
## PRE_3P_pct_inc	-0.185918557	-0.15406746	0.155208206	-0.022473690	-0.07223330
## PRE_2P_pct_inc	-0.029798898	-0.10309029	0.331152399	0.300150507	-0.04396151
## PRE_FT_pct_inc	-0.012801073	-0.28517770	0.189787389	0.053510073	-0.02871863
##	PC6	PC7	PC8	PC9	PC10
## WEIGHT	-0.102440463	0.01945161	-0.10761587	0.0441645755	0.071696264
## HEIGHT_INCHES	-0.197861654	0.03426717	-0.07151755	0.0003343841	0.104053219
## PRE_G	-0.003096375	-0.22717250	-0.03676991	-0.0148742508	-0.046949635
## PRE_GS	0.016534577	-0.33539249	-0.03253507	0.0190600895	-0.111168430
## PRE_MP	0.015534053	-0.28855828	-0.01783077	-0.0040057105	-0.074098536
## PRE_FGA	-0.100342569	0.14375073	0.12366593	0.3050992422	0.010059538
## PRE_FG_PCT	-0.004409603	-0.13322309	-0.23111057	-0.0781484906	0.275050775
## PRE_3PA	-0.254619740	0.10951615	0.04972839	0.1597272981	-0.063145956
## PRE_3P_PCT	-0.296272720	0.12555504	-0.25505483	-0.6921348003	0.238149213
## PRE_2PA	0.100909258	-0.04779157	0.12626039	0.0630074925	0.022293237
## PRE_2P_PCT	-0.025008678	-0.09948989	0.03965274	-0.0408271040	0.035244571
## PRE_eFG_PCT	-0.049976061	-0.09752382	0.01237576	-0.0373396158	0.032764705
## PRE_FTA	0.205143540	0.04317515	-0.18480518	-0.1013142886	0.019922747
## PRE_FT_PCT	-0.261178994	0.02336108	-0.33828357	-0.1089984569	-0.003053189
## PRE_ORB	0.124199677	-0.04014109	0.07343547	-0.1625242423	0.025815599
## PRE_DRB	-0.345438973	0.08949877	0.25515900	-0.1880550171	-0.431033802
## PRE_TRB	-0.277619833	0.07917485	0.06957106	0.0551909921	-0.111570151
## PRE_AST	0.310881381	-0.06803405	-0.13439517	-0.0227785634	0.035482104
## PRE_STL	0.158545817	0.08979451	-0.15790259	0.2435688666	0.381844051
## PRE_BLK	-0.036114530	-0.04979654	-0.06310273	0.0026893567	0.123028939
## PRE_TOV	0.531394851	0.18724542	0.12860582	-0.3867088692	-0.291267960
## PRE_PF	0.152259628	0.28034979	-0.09726541	-0.0614783636	0.162613277
## PRE_PTS	-0.109213441	0.11347422	-0.11912295	0.1513782274	0.160113006
## PRE_FG_pct_inc	0.044722428	0.45395118	-0.24968559	0.1021009545	-0.201073762
## PRE_3P_pct_inc	-0.075108836	0.27114166	0.49431413	-0.1656599680	0.269992950
## PRE_2P_pct_inc	0.037758904	0.43277509	-0.26247403	0.1596732804	-0.230500277
## PRE_FT_pct_inc	0.045372377	0.19061518	0.37416558	0.0154122996	0.399782204
##	PC11	PC12	PC13		
## WEIGHT	0.058519996	0.09333076	-0.028184110		
## HEIGHT_INCHES	-0.011694787	0.17386310	0.076222008		
## PRE_G	0.002715547	0.13765013	0.051480749		
## PRE_GS	0.027568844	0.14322979	0.024691520		
## PRE_MP	0.028870044	0.11442966	0.009178278		
## PRE_FGA	-0.076756280	0.25203787	-0.104767828		
## PRE_FG_PCT	0.056708376	-0.27041638	-0.010116838		
## PRE_3PA	-0.118367878	0.23071748	0.168132304		
## PRE_3P_PCT	-0.132992750	0.28816877	-0.346557121		
## PRE_2PA	0.028751024	0.15501850	-0.076811133		
## PRE_2P_PCT	0.016604797	0.09011226	0.147592764		
## PRE_eFG_PCT	-0.009846565	0.09309186	0.167533579		
## PRE_FTA	0.079548232	-0.39511675	-0.123032661		


```
## PRE_FT_PCT      -0.005951696 -0.32729026  0.607281583
## PRE_ORB         0.095369951  0.06374772 -0.145015816
## PRE_DRB        -0.075628303 -0.09555228 -0.006762200
## PRE_TRB        -0.025164286 -0.02048343 -0.052993184
## PRE_AST        -0.058816769 -0.03062988  0.009038546
## PRE_STL         0.101423145  0.20489575 -0.103729875
## PRE_BLK        -0.055082361  0.25445469  0.332816936
## PRE_TOV        -0.140123639  0.15500691  0.244921273
## PRE_PF          0.066511609  0.31256945  0.375978211
## PRE_PTS        -0.013609522 -0.08498533  0.001161631
## PRE_FG_pct_inc  0.078072671 -0.08208679 -0.156071581
## PRE_3P_pct_inc  0.646933312 -0.10277365  0.122327463
## PRE_2P_pct_inc -0.024377950  0.07019345 -0.046466246
## PRE_FT_pct_inc -0.681751633 -0.24304954  0.051015208
```

```
# Run it
train_control <- trainControl(method = "cv", number = 10,
                              summaryFunction = multiClassSummary)

model_pca_cv <- train(response ~ ., data = train_data_pca,
                      method = "multinom",
                      trControl = train_control,
                      trace = FALSE)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.
```

```
cat("Cross-validated accuracy (PCA-based model, training set):",
    max(model_pca_cv$results$Accuracy), "\n")
```

```
## Cross-validated accuracy (PCA-based model, training set): 0.5135406
```

```
model_pca <- multinom(response ~ ., data = train_data_pca, trace = FALSE)

preds_pca <- predict(model_pca, newdata = test_data_pca)
preds_pca <- factor(preds_pca, levels = levels(test_data_pca$response))

cm_pca <- confusionMatrix(preds_pca, test_data_pca$response, mode = "prec_recall")
print(cm_pca)
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction Bad BelowAvg Average AboveAvg Great
##   Bad          59          13          11           2           0
##   BelowAvg      1           6           7           3           0
##   Average       7          19          29          12           1
##   AboveAvg      1           5          25          52          19
##   Great         0           0           2          12           7
##
## Overall Statistics
##
```

```
## Accuracy : 0.5222
## 95% CI : (0.4633, 0.5806)
## No Information Rate : 0.2765
## P-Value [Acc > NIR] : < 2.2e-16
##
## Kappa : 0.3735
##
## McNemar's Test P-Value : NA
##
## Statistics by Class:
##
## Class: Bad Class: BelowAvg Class: Average Class: AboveAvg
## Precision 0.6941 0.35294 0.42647 0.5098
## Recall 0.8676 0.13953 0.39189 0.6420
## F1 0.7712 0.20000 0.40845 0.5683
## Prevalence 0.2321 0.14676 0.25256 0.2765
## Detection Rate 0.2014 0.02048 0.09898 0.1775
## Detection Prevalence 0.2901 0.05802 0.23208 0.3481
## Balanced Accuracy 0.8760 0.54777 0.60690 0.7031
##
## Class: Great
## Precision 0.33333
## Recall 0.25926
## F1 0.29167
## Prevalence 0.09215
## Detection Rate 0.02389
## Detection Prevalence 0.07167
## Balanced Accuracy 0.60331
```

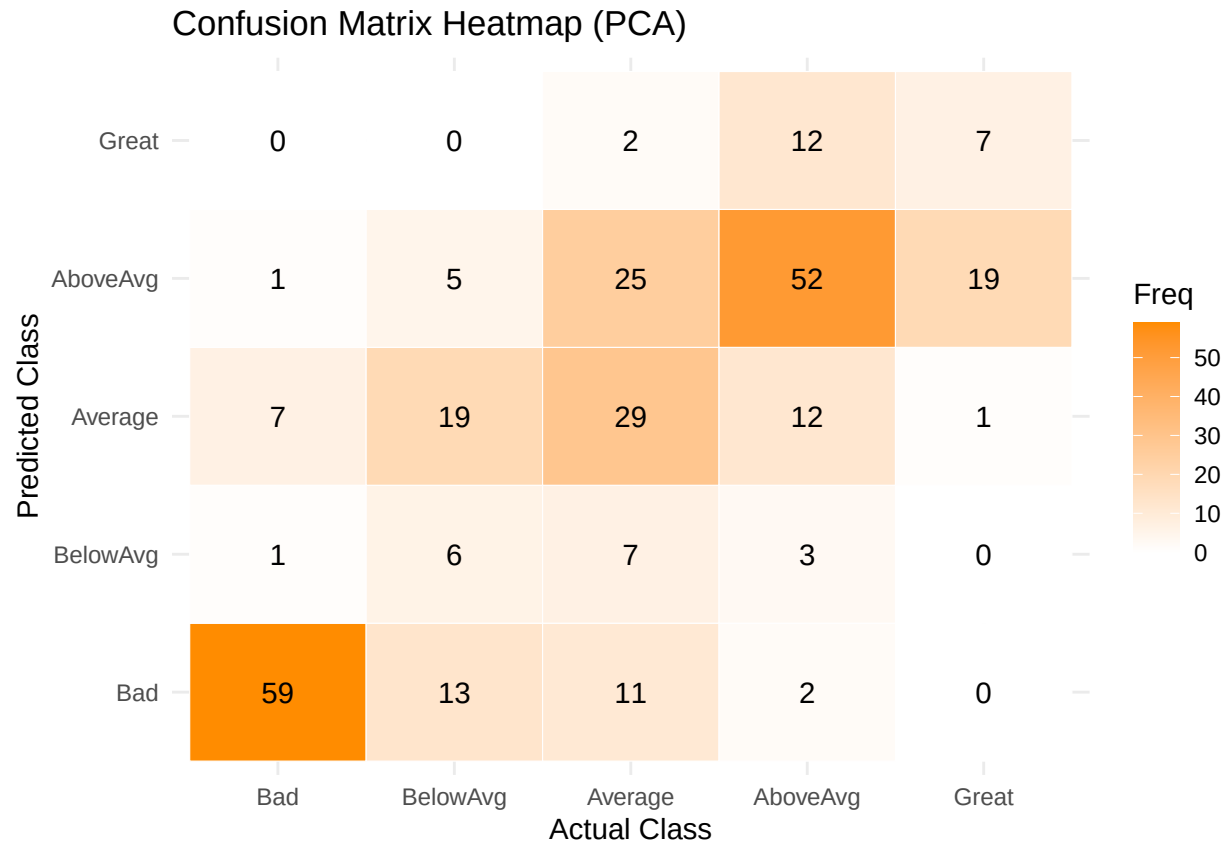
```
test_acc_pca <- mean(as.character(preds_pca) == as.character(test_data_pca$response))
cat("Test Accuracy (PCA-based model):", test_acc_pca, "\n")
```

```
## Test Accuracy (PCA-based model): 0.5221843
```

```
probs_pca <- predict(model_pca, newdata = test_data_pca, type = "probs")
probs_pca <- as.matrix(probs_pca)

colnames(probs_pca) <- colnames(probs_pca)
probs_pca <- probs_pca[, levels(test_data_pca$response), drop = FALSE]
```

```
cm_table <- as.data.frame(cm_pca$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "darkorange") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (PCA)",
       x = "Actual Class", y = "Predicted Class")
```



```
model_results <- rbind(model_results, data.frame(
  Model = "PCA",
  Accuracy = test_acc_pca,
  Precision = mean(cm_pca$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_pca$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_pca$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data_pca$response, probs_pca)
))
```

SVM Radial and Linear

```
set.seed(67)
# CV SVM
svm_linear <- train(
  response ~ .,
  data = train_data,
  method = "svmLinear2",
  trControl = trainControl(
    method = "cv",
    number = 10,
    classProbs = TRUE,
    summaryFunction = multiClassSummary
  ),
```

```
preProcess = c("center", "scale", "pca"),
tuneGrid = expand.grid(cost = c(0.01, 0.1, 1, 10)),
probability = TRUE
)

## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.
```

```
print(svm_linear)
```

```
## Support Vector Machines with Linear Kernel
##
## 1186 samples
## 27 predictor
## 5 classes: 'Bad', 'BelowAvg', 'Average', 'AboveAvg', 'Great'
##
## Pre-processing: centered (27), scaled (27), principal component
## signal extraction (27)
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1066, 1067, 1067, 1067, 1068, 1068, ...
## Resampling results across tuning parameters:
##
## cost logLoss AUC prAUC Accuracy Kappa Mean_F1
## 0.01 1.124024 0.7998709 0.4788145 0.5074987 0.3369517 NaN
## 0.10 1.113398 0.8035649 0.4813545 0.5225609 0.3654929 0.4472837
## 1.00 1.112444 0.8044141 0.4885240 0.5175480 0.3638036 0.4761554
## 10.00 1.114004 0.8046313 0.4895229 0.5141650 0.3604113 0.4648998
## Mean_Sensitivity Mean_Specificity Mean_Pos_Pred_Value Mean_Neg_Pred_Value
## 0.4022080 0.8666948 NaN 0.8757959
## 0.4504322 0.8723032 0.5466037 0.8780289
## 0.4691127 0.8719198 0.5102245 0.8759334
## 0.4668103 0.8713391 0.4953157 0.8750900
## Mean_Precision Mean_Recall Mean_Detection_Rate Mean_Balanced_Accuracy
## NaN 0.4022080 0.1014997 0.6344514
## 0.5466037 0.4504322 0.1045122 0.6613677
## 0.5102245 0.4691127 0.1035096 0.6705163
## 0.4953157 0.4668103 0.1028330 0.6690747
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was cost = 0.1.
```

```
cat("Best cost (C):", svm_linear$bestTune$C, "\n")
```

```
## Best cost (C):
```

```
cat("Cross-validated accuracy (training set):", max(svm_linear$results$Accuracy), "\n")
```

```
## Cross-validated accuracy (training set): 0.5225609
```

```

preds_svm_linear <- predict(svm_linear, newdata = test_data)
preds_svm_linear <- factor(preds_svm_linear, levels = levels(test_data$response))

cm_svm_linear <- confusionMatrix(preds_svm_linear, test_data$response, mode = "prec_recall")
print(cm_svm_linear)

```

```

## Confusion Matrix and Statistics
##
##              Reference
## Prediction Bad BelowAvg Average AboveAvg Great
##   Bad      59      14      11       2       0
##   BelowAvg  1       2       2       0       0
##   Average   7      24      35      15       1
##   AboveAvg  1       3      25      63      23
##   Great     0       0       1       1       3
##
## Overall Statistics
##
##              Accuracy : 0.5529
##              95% CI : (0.494, 0.6107)
##   No Information Rate : 0.2765
##   P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.4028
##
## Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##              Class: Bad Class: BelowAvg Class: Average Class: AboveAvg
## Precision          0.6860          0.400000          0.4268          0.5478
## Recall              0.8676          0.046512          0.4730          0.7778
## F1                  0.7662          0.083333          0.4487          0.6429
## Prevalence          0.2321          0.146758          0.2526          0.2765
## Detection Rate      0.2014          0.006826          0.1195          0.2150
## Detection Prevalence 0.2935          0.017065          0.2799          0.3925
## Balanced Accuracy    0.8738          0.517256          0.6292          0.7662
##
##              Class: Great
## Precision          0.60000
## Recall             0.11111
## F1                  0.18750
## Prevalence          0.09215
## Detection Rate      0.01024
## Detection Prevalence 0.01706
## Balanced Accuracy    0.55180

```

```

test_acc_linear <- mean(as.character(preds_svm_linear) == as.character(test_data$response))
cat("Test Accuracy (SVM Linear):", test_acc_linear, "\n")

```

```

## Test Accuracy (SVM Linear): 0.552901

```

```

probs_svm_linear <- predict(svm_linear, newdata = test_data, type = "prob")
probs_svm_linear <- as.matrix(probs_svm_linear)

colnames(probs_svm_linear) <- colnames(probs_svm_linear)
probs_svm_linear <- probs_svm_linear[, levels(test_data_pca$response), drop = FALSE]

model_results <- rbind(model_results, data.frame(
  Model = "SVM Linear",
  Accuracy = test_acc_linear,
  Precision = mean(cm_svm_linear$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_svm_linear$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_svm_linear$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data$response, probs_svm_linear)
))

```

```

# Repeat
svm_radial <- train(
  response ~ .,
  data = train_data,
  method = "svmRadial",
  trControl = trainControl(
    method = "cv",
    number = 10,
    classProbs = TRUE,
    summaryFunction = multiClassSummary
  ),
  tuneGrid = expand.grid(
    C = c(0.1, 1, 10),
    sigma = c(0.01, 0.05, 0.1)
  ),
  prob.model = TRUE
)

```

```

## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.

```

```
print(svm_radial)
```

```

## Support Vector Machines with Radial Basis Function Kernel
##
## 1186 samples
## 27 predictor
## 5 classes: 'Bad', 'BelowAvg', 'Average', 'AboveAvg', 'Great'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1067, 1068, 1067, 1066, 1070, 1068, ...
## Resampling results across tuning parameters:
##
##  C      sigma  logLoss  AUC      prAUC      Accuracy  Kappa      Mean_F1
##  0.1  0.01   1.167144  0.7863254  0.4801512  0.5159910  0.3601545  0.4657528
##  0.1  0.05   1.198249  0.7790464  0.4719308  0.4924011  0.3308169  0.4124776

```

```
## 0.1 0.10 1.276396 0.7755807 0.4638477 0.4872738 0.3249012 0.4001793
## 1.0 0.01 1.111728 0.8095156 0.4908148 0.5184756 0.3628898 0.4500249
## 1.0 0.05 1.135286 0.8014776 0.4890194 0.5143447 0.3566514 0.4481556
## 1.0 0.10 1.170995 0.7888639 0.4742409 0.5009500 0.3399339 0.4264431
## 10.0 0.01 1.104866 0.8105236 0.4969352 0.5253129 0.3709411 0.4701604
## 10.0 0.05 1.172586 0.7884685 0.4740670 0.4940821 0.3286081 0.4259297
## 10.0 0.10 1.194928 0.7817607 0.4623418 0.4738990 0.3029733 0.3984426
## Mean_Sensitivity Mean_Specificity Mean_Pos_Pred_Value Mean_Neg_Pred_Value
## 0.4572505 0.8712662 0.4739576 0.8768166
## 0.4439869 0.8653471 0.4634804 0.8704416
## 0.4348428 0.8644418 0.4840998 0.8711301
## 0.4591351 0.8717152 0.5193503 0.8764359
## 0.4481352 0.8705154 0.5102327 0.8756882
## 0.4360038 0.8673364 0.5039400 0.8723260
## 0.4558922 0.8734428 0.5381568 0.8778970
## 0.4180697 0.8652626 0.5022029 0.8704274
## 0.4002327 0.8601975 0.4627663 0.8652361
## Mean_Precision Mean_Recall Mean_Detection_Rate Mean_Balanced_Accuracy
## 0.4739576 0.4572505 0.10319820 0.6642583
## 0.4634804 0.4439869 0.09848023 0.6546670
## 0.4840998 0.4348428 0.09745475 0.6496423
## 0.5193503 0.4591351 0.10369511 0.6654251
## 0.5102327 0.4481352 0.10286895 0.6593253
## 0.5039400 0.4360038 0.10018999 0.6516701
## 0.5381568 0.4558922 0.10506259 0.6646675
## 0.5022029 0.4180697 0.09881643 0.6416661
## 0.4627663 0.4002327 0.09477980 0.6302151
##
## Accuracy was used to select the optimal model using the largest value.
## The final values used for the model were sigma = 0.01 and C = 10.
```

```
cat("Best parameters: C =", svm_radial$bestTune$C,
    " sigma =", svm_radial$bestTune$sigma, "\n")
```

```
## Best parameters: C = 10 sigma = 0.01
```

```
cat("Cross-validated accuracy (training set):", max(svm_radial$results$Accuracy), "\n")
```

```
## Cross-validated accuracy (training set): 0.5253129
```

```
preds_svm_radial <- predict(svm_radial, newdata = test_data)
preds_svm_radial <- factor(preds_svm_radial, levels = levels(test_data$response))

cm_svm_radial <- confusionMatrix(preds_svm_radial, test_data$response, mode = "prec_recall")
print(cm_svm_radial)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Bad BelowAvg Average AboveAvg Great
## Bad          59         13         12          2          0
## BelowAvg      2          3          3          1          0
```

```

##      Average      6      25      41      21      3
##      AboveAvg     1       2      16      55     22
##      Great        0       0       2       2      2
##
## Overall Statistics
##
##              Accuracy : 0.5461
##              95% CI : (0.4871, 0.6041)
##      No Information Rate : 0.2765
##      P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.3965
##
## McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##              Class: Bad Class: BelowAvg Class: Average Class: AboveAvg
## Precision              0.6860          0.33333          0.4271          0.5729
## Recall                  0.8676          0.06977          0.5541          0.6790
## F1                      0.7662          0.11538          0.4824          0.6215
## Prevalence              0.2321          0.14676          0.2526          0.2765
## Detection Rate          0.2014          0.01024          0.1399          0.1877
## Detection Prevalence    0.2935          0.03072          0.3276          0.3276
## Balanced Accuracy        0.8738          0.52288          0.6515          0.7428
##
##              Class: Great
## Precision              0.333333
## Recall                  0.074074
## F1                      0.121212
## Prevalence              0.092150
## Detection Rate          0.006826
## Detection Prevalence    0.020478
## Balanced Accuracy        0.529518

```

```

test_acc_radial <- mean(as.character(preds_svm_radial) == as.character(test_data$response))
cat("Test Accuracy (SVM Radial):", test_acc_radial, "\n")

```

```

## Test Accuracy (SVM Radial): 0.5460751

```

```

probs_svm_radial <- predict(svm_radial, newdata = test_data, type = "prob")
probs_svm_radial <- as.matrix(probs_svm_radial)

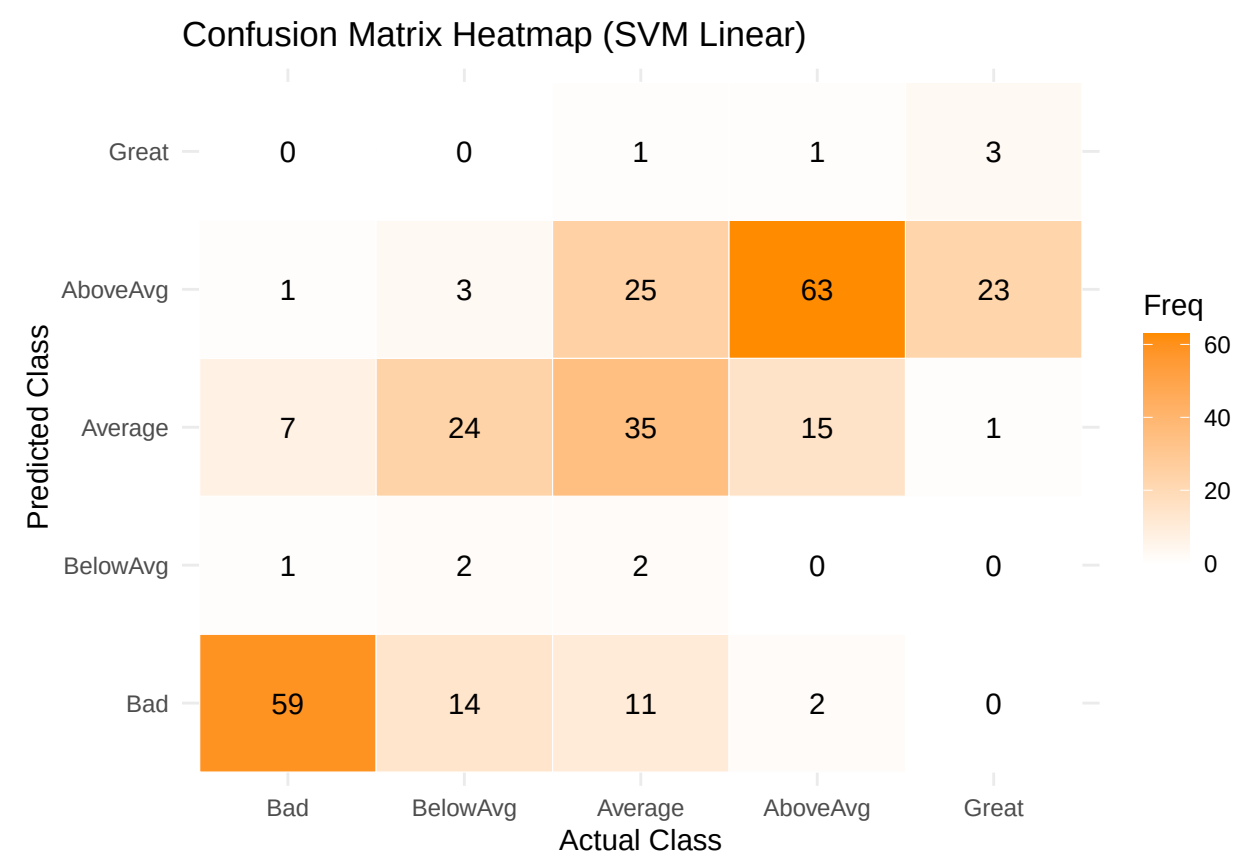
colnames(probs_svm_radial) <- colnames(probs_svm_radial)
probs_svm_radial <- probs_svm_radial[, levels(test_data$response), drop = FALSE]

model_results <- rbind(model_results, data.frame(
  Model = "SVM Radial Kernel",
  Accuracy = test_acc_radial,
  Precision = mean(cm_svm_radial$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_svm_radial$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_svm_radial$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data$response, probs_svm_radial)
))

```

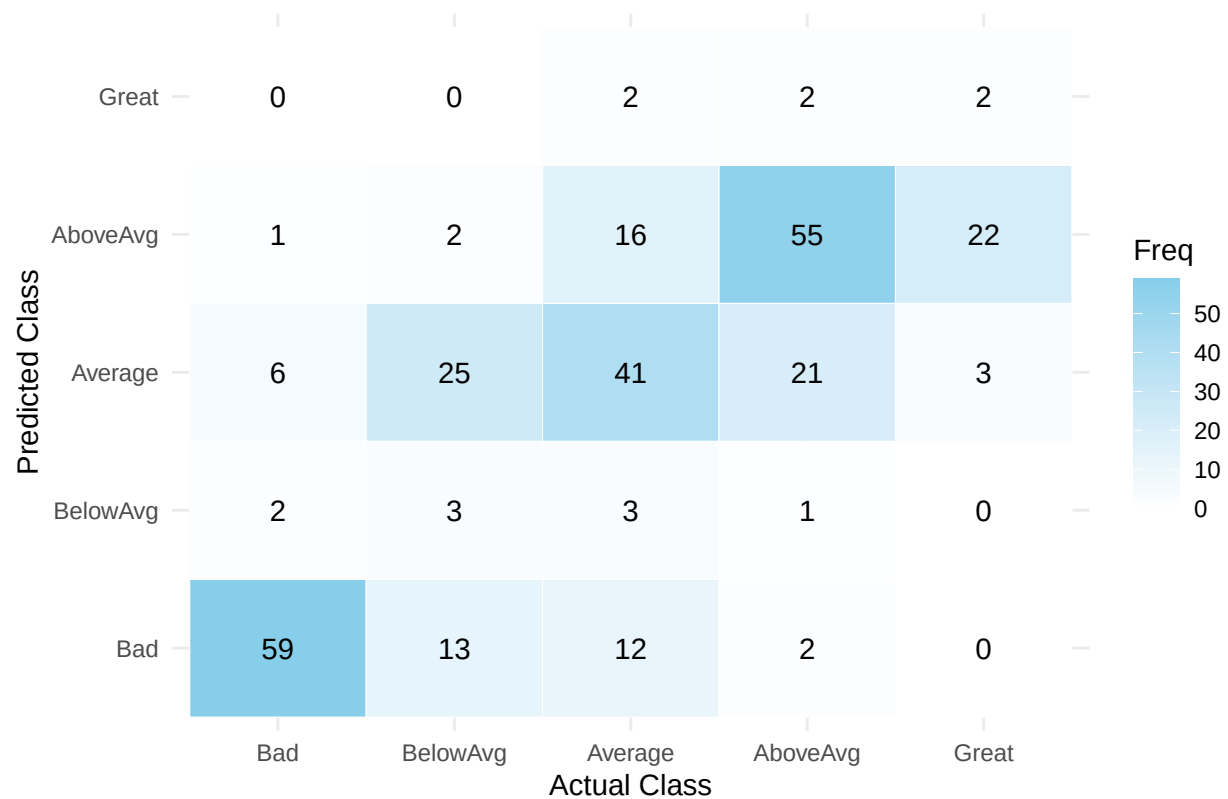


```
cm_table <- as.data.frame(cm_svm_linear$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "darkorange") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (SVM Linear)",
       x = "Actual Class", y = "Predicted Class")
```



```
cm_table <- as.data.frame(cm_svm_radial$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "skyblue") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (SVM Radial)",
       x = "Actual Class", y = "Predicted Class")
```

Confusion Matrix Heatmap (SVM Radial)



Random Forest

```
set.seed(6767)
# CV for random forest
train_control <- trainControl(method = "cv", number = 10)

tune_grid <- expand.grid(mtry = c(2, 5, 7, 10, 15, 20))

results <- list()
ntree_values <- c(100, 250, 500)

for (nt in ntree_values) {
  rf_model <- train(response ~ ., data = train_data,
                    method = "rf",
                    trControl = train_control,
                    tuneGrid = tune_grid,
                    ntree = nt)

  results[[paste0("ntree_", nt)]] <- rf_model

  preds <- predict(rf_model, newdata = test_data)
  cm <- confusionMatrix(preds, test_data$response, mode = "prec_recall")
}
```

```

# Print results
cat("\nResults for ntree =", nt, "\n")
print(rf_model$results)
cat("Best mtry:", rf_model$bestTune$mtry, "\n")
cat("CV Accuracy:", max(rf_model$results$Accuracy), "\n")
}

```

```

##
## Results for ntree = 100 :
##   mtry Accuracy      Kappa AccuracySD      KappaSD
## 1    2 0.4838635 0.3163657 0.05092116 0.06633289
## 2    5 0.4897818 0.3279637 0.06059472 0.07924112
## 3    7 0.5107914 0.3570135 0.05926104 0.07782314
## 4   10 0.5031918 0.3477795 0.04892986 0.06394813
## 5   15 0.5006360 0.3454020 0.06721536 0.08777725
## 6   20 0.4930439 0.3363333 0.05669833 0.07343749
## Best mtry: 7
## CV Accuracy: 0.5107914
##
## Results for ntree = 250 :
##   mtry Accuracy      Kappa AccuracySD      KappaSD
## 1    2 0.4854567 0.3178839 0.05264051 0.07057745
## 2    5 0.5023220 0.3435591 0.05212332 0.06835747
## 3    7 0.5023796 0.3434212 0.05104127 0.06766602
## 4   10 0.5124996 0.3587477 0.03713508 0.04806740
## 5   15 0.5015383 0.3453817 0.03735117 0.04886237
## 6   20 0.4990526 0.3437634 0.04255764 0.05517079
## Best mtry: 10
## CV Accuracy: 0.5124996
##
## Results for ntree = 500 :
##   mtry Accuracy      Kappa AccuracySD      KappaSD
## 1    2 0.5050993 0.3429022 0.03453879 0.04639971
## 2    5 0.5077410 0.3506378 0.03296194 0.04122687
## 3    7 0.5136165 0.3602488 0.04238288 0.05495044
## 4   10 0.5102195 0.3565536 0.03852071 0.05094399
## 5   15 0.5169706 0.3665470 0.03224874 0.04227612
## 6   20 0.5094079 0.3565948 0.03027647 0.04007906
## Best mtry: 15
## CV Accuracy: 0.5169706

```

```

# Use best model as fit
rf_model <- train(response ~ ., data = train_data,
                  method = "rf",
                  trControl = trainControl(method = "cv", number = 10,
                                           summaryFunction = multiClassSummary),
                  tuneGrid = data.frame(mtry = results[["ntree_500"]]$bestTune$mtry),
                  ntree = 500)

```

```

## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.

```

```

preds_rf <- predict(rf_model, newdata = test_data)
cm_rf <- confusionMatrix(preds_rf, test_data$response, mode = "prec_recall")

probs_rf <- predict(rf_model, newdata = test_data, type = "prob")
probs_rf <- as.matrix(probs_rf)

colnames(probs_rf) <- colnames(probs_rf)
probs_rf <- probs_rf[, levels(test_data$response), drop = FALSE]

model_results <- rbind(model_results, data.frame(
  Model = "Base Random Forest",
  Accuracy = mean(preds_rf == test_data$response),
  Precision = mean(cm_rf$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_rf$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_rf$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data$response, probs_rf)
))

print(rf_model)

```

```

## Random Forest
##
## 1186 samples
## 27 predictor
## 5 classes: 'Bad', 'BelowAvg', 'Average', 'AboveAvg', 'Great'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1069, 1066, 1069, 1068, 1066, 1068, ...
## Resampling results:
##
## Accuracy Kappa Mean_F1 Mean_Sensitivity Mean_Specificity
## 0.5126713 0.3614842 0.4742939 0.4726548 0.8716769
## Mean_Pos_Pred_Value Mean_Neg_Pred_Value Mean_Precision Mean_Recall
## 0.492129 0.8745837 0.492129 0.4726548
## Mean_Detection_Rate Mean_Balanced_Accuracy
## 0.1025343 0.6721659
##
## Tuning parameter 'mtry' was held constant at a value of 15

```

```
print(cm_rf)
```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction Bad BelowAvg Average AboveAvg Great
## Bad      57      12      8       1      0
## BelowAvg  4       9      7       1      0
## Average   4      18     27      18      2
## AboveAvg  3       4     31      52     17
## Great     0       0      1       9      8
##

```

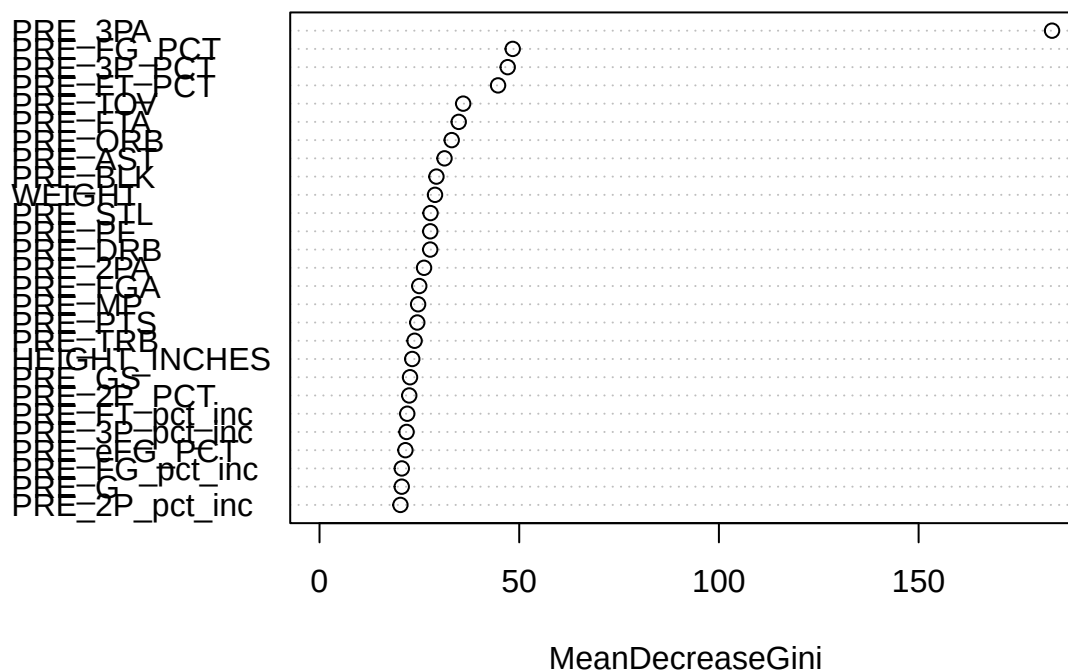
```

## Overall Statistics
##
##           Accuracy : 0.5222
##           95% CI   : (0.4633, 0.5806)
##    No Information Rate : 0.2765
##    P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.3726
##
##    McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##           Class: Bad Class: BelowAvg Class: Average Class: AboveAvg
## Precision           0.7308           0.42857           0.39130           0.4860
## Recall              0.8382           0.20930           0.36486           0.6420
## F1                  0.7808           0.28125           0.37762           0.5532
## Prevalence          0.2321           0.14676           0.25256           0.2765
## Detection Rate      0.1945           0.03072           0.09215           0.1775
## Detection Prevalence 0.2662           0.07167           0.23549           0.3652
## Balanced Accuracy    0.8725           0.58065           0.58654           0.6913
##
##           Class: Great
## Precision           0.44444
## Recall              0.29630
## F1                  0.35556
## Prevalence          0.09215
## Detection Rate      0.02730
## Detection Prevalence 0.06143
## Balanced Accuracy    0.62935

# Print Variable Importance Plot
varImpPlot(rf_model$finalModel)

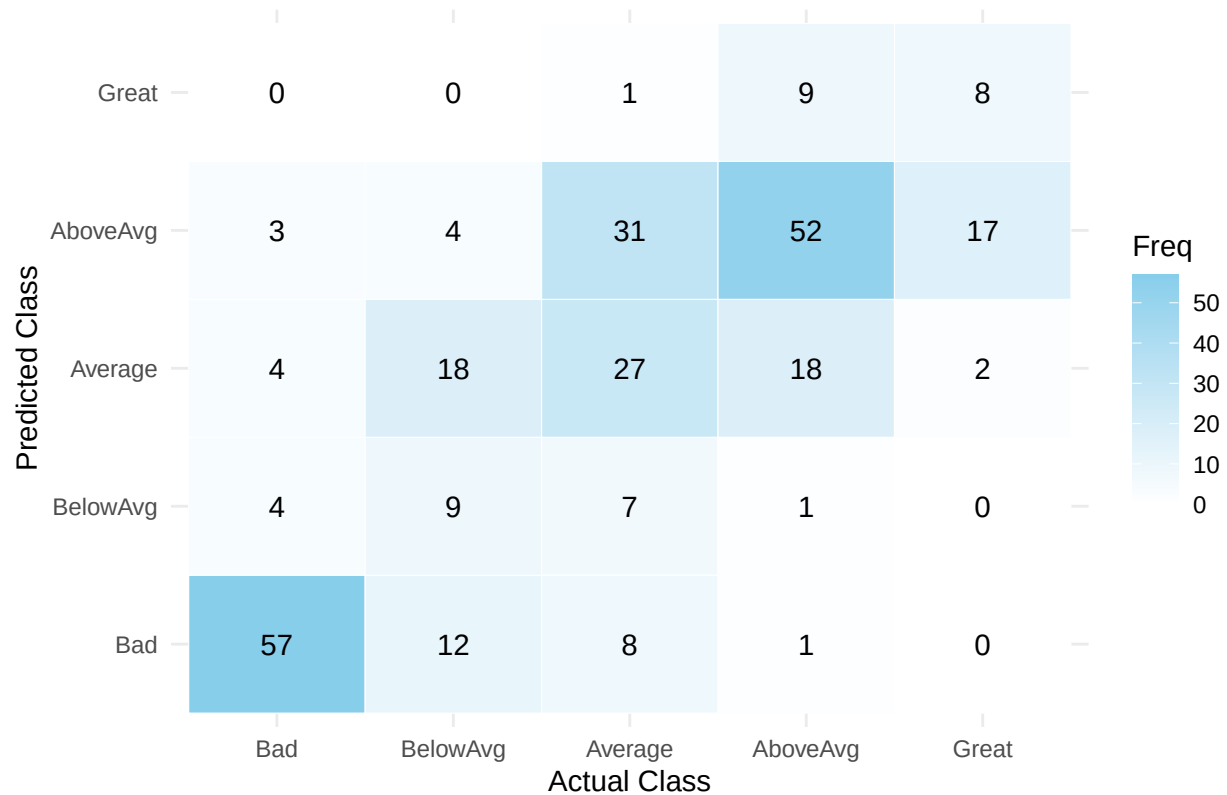
```

rf_model\$finalModel



```
cm_table <- as.data.frame(cm_rf$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "skyblue") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (Random Forest)",
       x = "Actual Class", y = "Predicted Class")
```

Confusion Matrix Heatmap (Random Forest)



Upsampling

```
set.seed(67)

# Cross validation for upsampling
train_control <- trainControl(method = "cv", number = 10,
                              sampling = "up",
                              summaryFunction = multiClassSummary)

rf_model_balanced <- train(response ~ ., data = train_data,
                           method = "rf",
                           trControl = train_control,
                           tuneGrid = data.frame(mtry = c(2, 5, 10, 12, 15, 20)),
                           ntree = 500)

## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.

print(rf_model_balanced)

## Random Forest
##
```

```
## 1186 samples
## 27 predictor
## 5 classes: 'Bad', 'BelowAvg', 'Average', 'AboveAvg', 'Great'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1066, 1067, 1067, 1067, 1068, 1068, ...
## Additional sampling using up-sampling
##
## Resampling results across tuning parameters:
##
## mtry Accuracy Kappa Mean_F1 Mean_Sensitivity Mean_Specificity
## 2 0.4888683 0.3348561 0.4570644 0.4608614 0.8666622
## 5 0.4955775 0.3468527 0.4693075 0.4756494 0.8690802
## 10 0.4998067 0.3525813 0.4739545 0.4784944 0.8703024
## 12 0.4972363 0.3511004 0.4727685 0.4769359 0.8703124
## 15 0.5066008 0.3629083 0.4882125 0.4945112 0.8723737
## 20 0.5022497 0.3595383 0.4885387 0.4952718 0.8718049
## Mean_Pos_Pred_Value Mean_Neg_Pred_Value Mean_Precision Mean_Recall
## 0.4532704 0.8693222 0.4532704 0.4608614
## 0.4794802 0.8710875 0.4794802 0.4756494
## 0.4828297 0.8717675 0.4828297 0.4784944
## 0.4765940 0.8711284 0.4765940 0.4769359
## 0.4915496 0.8734357 0.4915496 0.4945112
## 0.4923994 0.8724463 0.4923994 0.4952718
## Mean_Detection_Rate Mean_Balanced_Accuracy
## 0.09777366 0.6637618
## 0.09911549 0.6723648
## 0.09996134 0.6743984
## 0.09944726 0.6736242
## 0.10132017 0.6834425
## 0.10044994 0.6835384
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was mtry = 15.
```

```
cat("Best mtry:", rf_model_balanced$bestTune$mtry, "\n")
```

```
## Best mtry: 15
```

```
cat("Cross-validated accuracy:", max(rf_model_balanced$results$Accuracy), "\n")
```

```
## Cross-validated accuracy: 0.5066008
```

```
preds_rf_up <- predict(rf_model_balanced, newdata = test_data)
cm_rf_up <- confusionMatrix(preds_rf_up, test_data$response, mode = "prec_recall")

probs_rf_up <- predict(rf_model_balanced, newdata = test_data, type = "prob")
probs_rf_up <- as.matrix(probs_rf_up)

colnames(probs_rf_up) <- colnames(probs_rf_up)
probs_rf_up <- probs_rf_up[, levels(test_data$response), drop = FALSE]
```



```

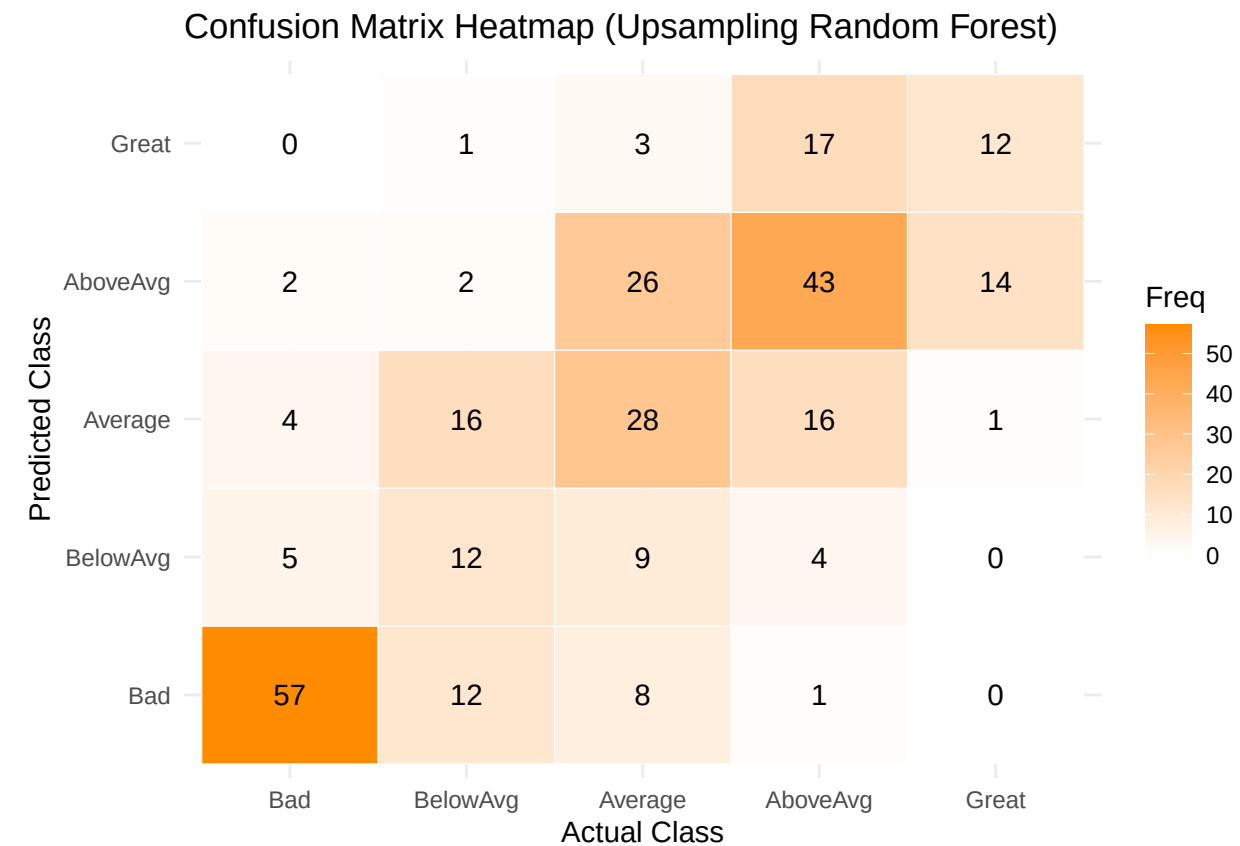
model_results <- rbind(model_results, data.frame(
  Model = "Upsampling Random Forest",
  Accuracy = mean(preds_rf_up == test_data$response),
  Precision = mean(cm_rf_up$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_rf_up$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_rf_up$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data$response, probs_rf_up)
))

```

```

cm_table <- as.data.frame(cm_rf_up$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "darkorange") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (Upsampling Random Forest)",
       x = "Actual Class", y = "Predicted Class")

```



Class Weights

```

set.seed(12)
# Set class weights

```

```

class_counts <- table(train_data$response)
class_weights <- 1 / class_counts
class_weights <- class_weights / sum(class_weights)

# CV for class weights
rf_weighted <- randomForest(response ~ ., data = train_data,
                             ntree = 500,
                             mtry = 7,
                             importance = TRUE,
                             classwt = class_weights)

preds_rf_weighted <- predict(rf_weighted, newdata = test_data)

cm_rf_weighted <- confusionMatrix(preds_rf_weighted, test_data$response, mode = "prec_recall")
accuracy_rf_weighted <- mean(as.character(preds_rf_weighted) == as.character(test_data$response))
cat("Test Accuracy:", accuracy_rf_weighted, "\n")

```

```
## Test Accuracy: 0.5119454
```

```
print(cm_rf_weighted)
```

```

## Confusion Matrix and Statistics
##
##              Reference
## Prediction Bad BelowAvg Average AboveAvg Great
##   Bad       58       12        8         2        0
##   BelowAvg   2        8        3         2        0
##   Average    7       16       27        15        2
##   AboveAvg   1        6       35        51       19
##   Great      0        1        1        11        6
##
## Overall Statistics
##
##              Accuracy : 0.5119
##              95% CI : (0.4531, 0.5705)
##   No Information Rate : 0.2765
##   P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.3576
##
##   McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##              Class: Bad Class: BelowAvg Class: Average Class: AboveAvg
## Precision          0.7250          0.53333          0.40299          0.4554
## Recall              0.8529          0.18605          0.36486          0.6296
## F1                  0.7838          0.27586          0.38298          0.5285
## Prevalence          0.2321          0.14676          0.25256          0.2765
## Detection Rate      0.1980          0.02730          0.09215          0.1741
## Detection Prevalence 0.2730          0.05119          0.22867          0.3823
## Balanced Accuracy    0.8776          0.57902          0.59111          0.6709

```

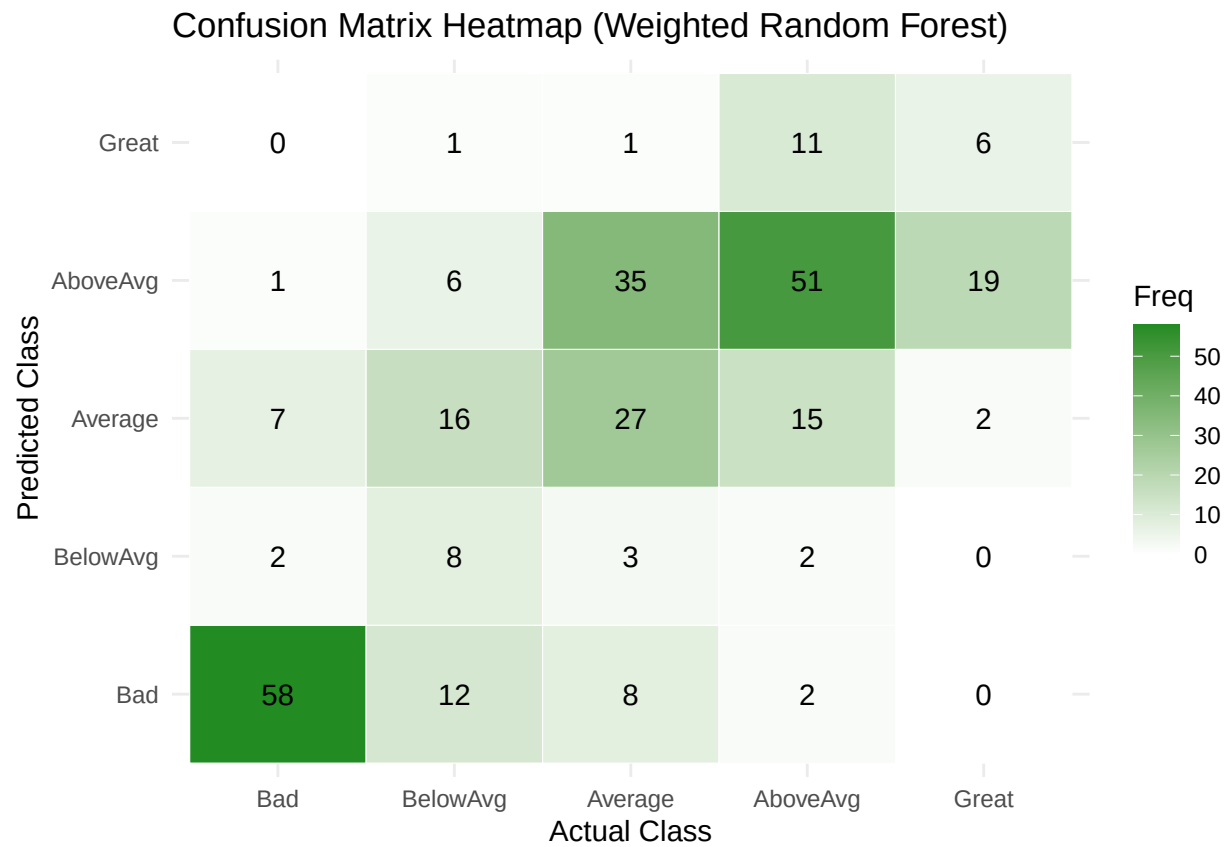
```
##                               Class: Great
## Precision                     0.31579
## Recall                       0.22222
## F1                           0.26087
## Prevalence                   0.09215
## Detection Rate               0.02048
## Detection Prevalence        0.06485
## Balanced Accuracy            0.58668
```

```
probs_rf_weighted <- predict(rf_weighted, newdata = test_data, type = "prob")
probs_rf_weighted <- as.matrix(probs_rf_weighted)

colnames(probs_rf_weighted) <- colnames(probs_rf_weighted)
probs_rf_weighted <- probs_rf_weighted[, levels(test_data$response), drop = FALSE]

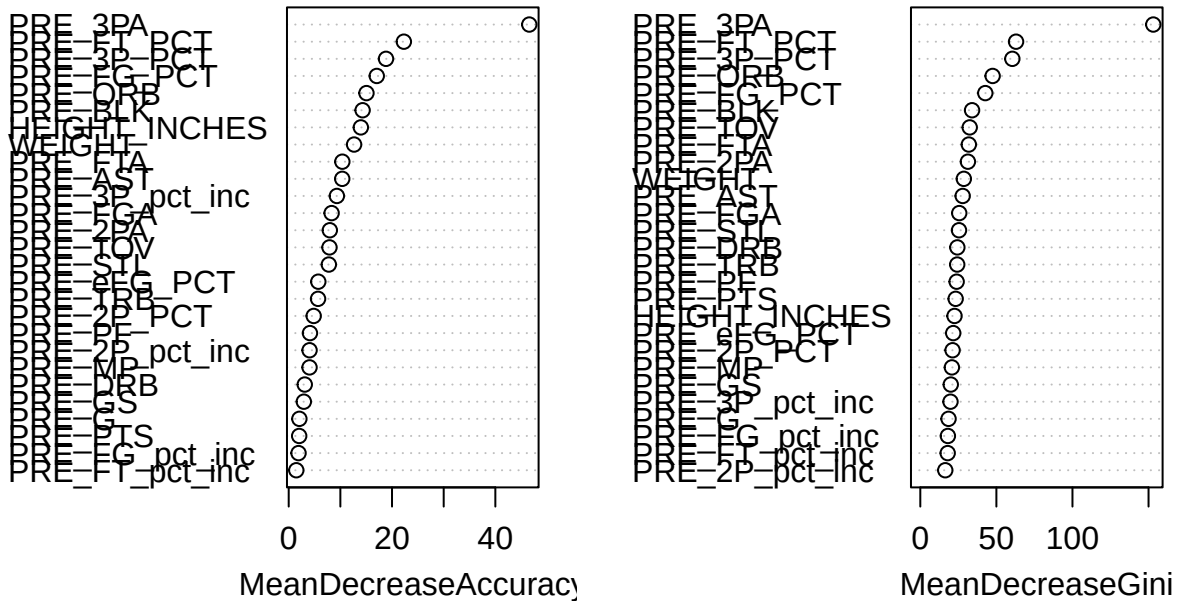
model_results <- rbind(model_results, data.frame(
  Model = "Weighted Random Forest",
  Accuracy = accuracy_rf_weighted,
  Precision = mean(cm_rf_weighted$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_rf_weighted$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_rf_weighted$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data$response, probs_rf_weighted)
))
```

```
cm_table <- as.data.frame(cm_rf_weighted$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "forestgreen") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (Weighted Random Forest)",
       x = "Actual Class", y = "Predicted Class")
```



```
# Original best model, so we check the VarImpPlot  
varImpPlot(rf_weighted, main="RF Weighted Variable Importance")
```

RF Weighted Variable Importance



Original best model, so we print a pretty tree `single_tree <- randomForest::getTree(rf_weighted, k = 1, labelVar = TRUE)`

`library(rpart) library(rpart.plot)`

`to_rpart <- function(tree, data) { response <- all.vars(formula(rf_weighted))[1] form <- as.formula(paste(response, "~.")) rpart(form, data = data, control = rpart.control(cp = 0.0001)) }`

`rpart_tree <- to_rpart(single_tree, train_data) rpart.plot( rpart_tree, type = 4, extra = 101, fallen.leaves = TRUE, box.palette = "RdYlGn", branch.lty = 3, shadow.col = "gray", main = "Decision Tree Extracted from Random Forest" )`

XGBoost

```
set.seed(9999)

# Cross Validation for XGBoost
train_control <- trainControl(method = "cv", number = 10)
tune_grid <- expand.grid(
  nrounds = c(100, 250, 500),
  max_depth = c(3, 5, 7),
  eta = c(0.01, 0.1, 0.3),
  gamma = 0,
  colsample_bytree = 0.8,
  min_child_weight = 1,
```

```
)
  subsample = 0.8
)

xgb_model <- train(
  response ~ ., data = train_data,
  method = "xgbTree",
  trControl = train_control,
  tuneGrid = tune_grid,
  verbose = FALSE
)
```

[illegible]

[illegible]

[illegible]


```

## [11:32:53] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:32:57] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:32:57] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:33:04] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:33:04] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:33:13] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:33:13] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:33:17] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:33:17] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:33:23] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:33:23] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:33:29] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:33:29] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:33:33] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:33:33] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:33:43] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:33:43] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:33:57] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:33:57] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:34:01] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:34:01] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:34:09] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:34:09] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:34:17] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:34:17] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:34:21] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:34:21] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:34:27] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:34:27] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:34:32] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:34:32] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead

```

```
print(xgb_model)
```

```

## eXtreme Gradient Boosting
##
## 1186 samples
## 27 predictor
## 5 classes: 'Bad', 'BelowAvg', 'Average', 'AboveAvg', 'Great'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1066, 1067, 1067, 1067, 1068, 1067, ...
## Resampling results across tuning parameters:
##
##  eta    max_depth  nrounds  Accuracy    Kappa
##  0.01    3          100      0.5151406  0.3559921
##  0.01    3          250      0.5041521  0.3435051
##  0.01    3          500      0.5050277  0.3477027
##  0.01    5          100      0.5109034  0.3535679
##  0.01    5          250      0.5058751  0.3489374
##  0.01    5          500      0.4906707  0.3307142
##  0.01    7          100      0.4948795  0.3338344
##  0.01    7          250      0.4923584  0.3312743

```

```
## 0.01 7 500 0.4983046 0.3409279
## 0.10 3 100 0.4914755 0.3325633
## 0.10 3 250 0.4703957 0.3066552
## 0.10 3 500 0.4695625 0.3072692
## 0.10 5 100 0.4940753 0.3381268
## 0.10 5 250 0.4746335 0.3121293
## 0.10 5 500 0.4619856 0.2960361
## 0.10 7 100 0.4830720 0.3230745
## 0.10 7 250 0.4822534 0.3232069
## 0.10 7 500 0.4704318 0.3083653
## 0.30 3 100 0.4578544 0.2914710
## 0.30 3 250 0.4578472 0.2932566
## 0.30 3 500 0.4586169 0.2932525
## 0.30 5 100 0.4653822 0.3012627
## 0.30 5 250 0.4645348 0.3009230
## 0.30 5 500 0.4687015 0.3071358
## 0.30 7 100 0.4805869 0.3213055
## 0.30 7 250 0.4696056 0.3073779
## 0.30 7 500 0.4654037 0.3026067
##
## Tuning parameter 'gamma' was held constant at a value of 0
## Tuning
##
## Tuning parameter 'min_child_weight' was held constant at a value of 1
##
## Tuning parameter 'subsample' was held constant at a value of 0.8
## Accuracy was used to select the optimal model using the largest value.
## The final values used for the model were nrounds = 100, max_depth = 3, eta
## = 0.01, gamma = 0, colsample_bytree = 0.8, min_child_weight = 1 and
## subsample = 0.8.
```

```
cat("Best parameters:\n")
```

```
## Best parameters:
```

```
print(xgb_model$bestTune)
```

```
## nrounds max_depth eta gamma colsample_bytree min_child_weight subsample
## 1 100 3 0.01 0 0.8 1 0.8
```

```
cat("Cross-validated accuracy:", max(xgb_model$results$Accuracy), "\n")
```

```
## Cross-validated accuracy: 0.5151406
```

```
preds_xgb <- predict(xgb_model, newdata = test_data)
preds_xgb <- factor(preds_xgb, levels = levels(test_data$response))
```

```
cm_xgb <- confusionMatrix(preds_xgb, test_data$response)
print(cm_xgb)
```

```
## Confusion Matrix and Statistics
```

```
##
##           Reference
## Prediction Bad BelowAvg Average AboveAvg Great
##   Bad      61      13      8       3       0
##   BelowAvg  0       0      1       0       0
##   Average   6      23     36      12       2
##   AboveAvg  1       7     27      60      16
##   Great     0       0      2       6       9
##
## Overall Statistics
##
##           Accuracy : 0.5666
##           95% CI : (0.5077, 0.6241)
##   No Information Rate : 0.2765
##   P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.4251
##
## Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##           Class: Bad Class: BelowAvg Class: Average Class: AboveAvg
## Sensitivity      0.8971      0.000000      0.4865      0.7407
## Specificity      0.8933      0.996000      0.8037      0.7594
## Pos Pred Value   0.7176      0.000000      0.4557      0.5405
## Neg Pred Value   0.9663      0.852740      0.8224      0.8846
## Prevalence       0.2321      0.146758      0.2526      0.2765
## Detection Rate   0.2082      0.000000      0.1229      0.2048
## Detection Prevalence 0.2901      0.003413      0.2696      0.3788
## Balanced Accuracy 0.8952      0.498000      0.6451      0.7501
##
##           Class: Great
## Sensitivity      0.33333
## Specificity      0.96992
## Pos Pred Value   0.52941
## Neg Pred Value   0.93478
## Prevalence       0.09215
## Detection Rate   0.03072
## Detection Prevalence 0.05802
## Balanced Accuracy 0.65163
```

```
set.seed(9999)
# Another cross validation
trainControl(method = "cv", number = 10, summaryFunction = multiClassSummary)
```

```
## $method
## [1] "cv"
##
## $number
## [1] 10
##
## $repeats
## [1] NA
##
```

```

## $search
## [1] "grid"
##
## $p
## [1] 0.75
##
## $initialWindow
## NULL
##
## $horizon
## [1] 1
##
## $fixedWindow
## [1] TRUE
##
## $skip
## [1] 0
##
## $verboseIter
## [1] FALSE
##
## $returnData
## [1] TRUE
##
## $returnResamp
## [1] "final"
##
## $savePredictions
## [1] FALSE
##
## $classProbs
## [1] FALSE
##
## $summaryFunction
## function (data, lev = NULL, model = NULL)
## {
##   if (!all(levels(data[, "pred"]) == levels(data[, "obs"])))
##     stop("levels of observed and predicted data do not match")
##   has_class_probs <- all(lev %in% colnames(data))
##   if (has_class_probs) {
##     lloss <- mnLogLoss(data = data, lev = lev, model = model)
##     requireNamespaceQuietStop("pROC")
##     requireNamespaceQuietStop("MLmetrics")
##     prob_stats <- lapply(levels(data[, "pred"]), function(x) {
##       obs <- ifelse(data[, "obs"] == x, 1, 0)
##       prob <- data[, x]
##       roc_auc <- try(pROC::roc(obs, data[, x], direction = "<",
##         quiet = TRUE), silent = TRUE)
##       roc_auc <- if (inherits(roc_auc, "try-error"))
##         NA
##       else roc_auc$auc
##       pr_auc <- try(MLmetrics::PRAUC(y_pred = data[, x],
##         y_true = obs), silent = TRUE)
##       if (inherits(pr_auc, "try-error"))

```

```

##           pr_auc <- NA
##           res <- c(ROC = roc_auc, AUC = pr_auc)
##           return(res)
##       })
##       prob_stats <- do.call("rbind", prob_stats)
##       prob_stats <- colMeans(prob_stats, na.rm = TRUE)
##   }
##   CM <- confusionMatrix(data[, "pred"], data[, "obs"], mode = "everything")
##   if (length(levels(data[, "pred"])) == 2) {
##       class_stats <- CM$byClass
##   }
##   else {
##       class_stats <- colMeans(CM$byClass)
##       names(class_stats) <- paste("Mean", names(class_stats))
##   }
##   overall_stats <- if (has_class_probs)
##       c(CM$overall, logLoss = as.numeric(lloss), AUC = unname(prob_stats["ROC"]),
##         prAUC = unname(prob_stats["AUC"]))
##   else CM$overall
##   stats <- c(overall_stats, class_stats)
##   stats <- stats[!names(stats) %in% c("AccuracyNull", "AccuracyLower",
##     "AccuracyUpper", "AccuracyPValue", "McNemarPValue", "Mean Prevalence",
##     "Mean Detection Prevalence")]
##   names(stats) <- gsub("[:blank:]]+", "_", names(stats))
##   stat_list <- c("Accuracy", "Kappa", "Mean_F1", "Mean_Sensitivity",
##     "Mean_Specificity", "Mean_Pos_Pred_Value", "Mean_Neg_Pred_Value",
##     "Mean_Precision", "Mean_Recall", "Mean_Detection_Rate",
##     "Mean_Balanced_Accuracy")
##   if (has_class_probs)
##       stat_list <- c("logLoss", "AUC", "prAUC", stat_list)
##   if (length(levels(data[, "pred"])) == 2)
##       stat_list <- gsub("^Mean_", "", stat_list)
##   stats <- stats[c(stat_list)]
##   return(stats)
## }
## <bytecode: 0x000001952f17cd58>
## <environment: namespace:caret>
##
## $selectionFunction
## [1] "best"
##
## $preProcOptions
## $preProcOptions$thresh
## [1] 0.95
##
## $preProcOptions$ICAcomp
## [1] 3
##
## $preProcOptions$k
## [1] 5
##
## $preProcOptions$freqCut
## [1] 19
##

```

```

## $preProcOptions$uniqueCut
## [1] 10
##
## $preProcOptions$cutoff
## [1] 0.9
##
##
## $sampling
## NULL
##
## $index
## NULL
##
## $indexOut
## NULL
##
## $indexFinal
## NULL
##
## $timingSamps
## [1] 0
##
## $predictionBounds
## [1] FALSE FALSE
##
## $seeds
## [1] NA
##
## $adaptive
## $adaptive$min
## [1] 5
##
## $adaptive$alpha
## [1] 0.05
##
## $adaptive$method
## [1] "gls"
##
## $adaptive$complete
## [1] TRUE
##
##
## $trim
## [1] FALSE
##
## $allowParallel
## [1] TRUE

```

```

tune_grid <- expand_grid(
  nrounds = 100,
  max_depth = 3,
  eta = 0.01,
  gamma = 0,
  colsample_bytree = 0.8,

```

```

    min_child_weight = 1,
    subsample = 0.8
)

xgb_model <- train(response ~ ., data = train_data,
  method = "xgbTree",
  trControl = trainControl(method = "cv", number = 10),
  tuneGrid = tune_grid,
  verbose = FALSE)

cat("Cross-validated accuracy:", max(xgb_model$results$Accuracy), "\n")

```

```
## Cross-validated accuracy: 0.5176759
```

```

preds_xgb <- predict(xgb_model, newdata = test_data)
preds_xgb <- factor(preds_xgb, levels = levels(test_data$response))

cm_xgb <- confusionMatrix(preds_xgb, test_data$response, mode = "prec_recall")
print(cm_xgb)

```

```
## Confusion Matrix and Statistics
```

```
##
##           Reference
## Prediction Bad BelowAvg Average AboveAvg Great
##   Bad       60      14      8       3      0
##   BelowAvg   0       0      1       0      0
##   Average    7      21     37      14      2
##   AboveAvg   1       8     27      57     17
##   Great      0       0      1       7      8
```

```
## Overall Statistics
```

```
##
##           Accuracy : 0.5529
##           95% CI   : (0.494, 0.6107)
##   No Information Rate : 0.2765
##   P-Value [Acc > NIR] : < 2.2e-16
```

```
##
##           Kappa : 0.4067
```

```
## McNemar's Test P-Value : NA
```

```
## Statistics by Class:
```

```
##
##           Class: Bad Class: BelowAvg Class: Average Class: AboveAvg
## Precision          0.7059          0.000000          0.4568          0.5182
## Recall              0.8824          0.000000          0.5000          0.7037
## F1                  0.7843              NaN          0.4774          0.5969
## Prevalence          0.2321          0.146758          0.2526          0.2765
## Detection Rate      0.2048          0.000000          0.1263          0.1945
## Detection Prevalence 0.2901          0.003413          0.2765          0.3754
## Balanced Accuracy    0.8856          0.498000          0.6495          0.7269
```

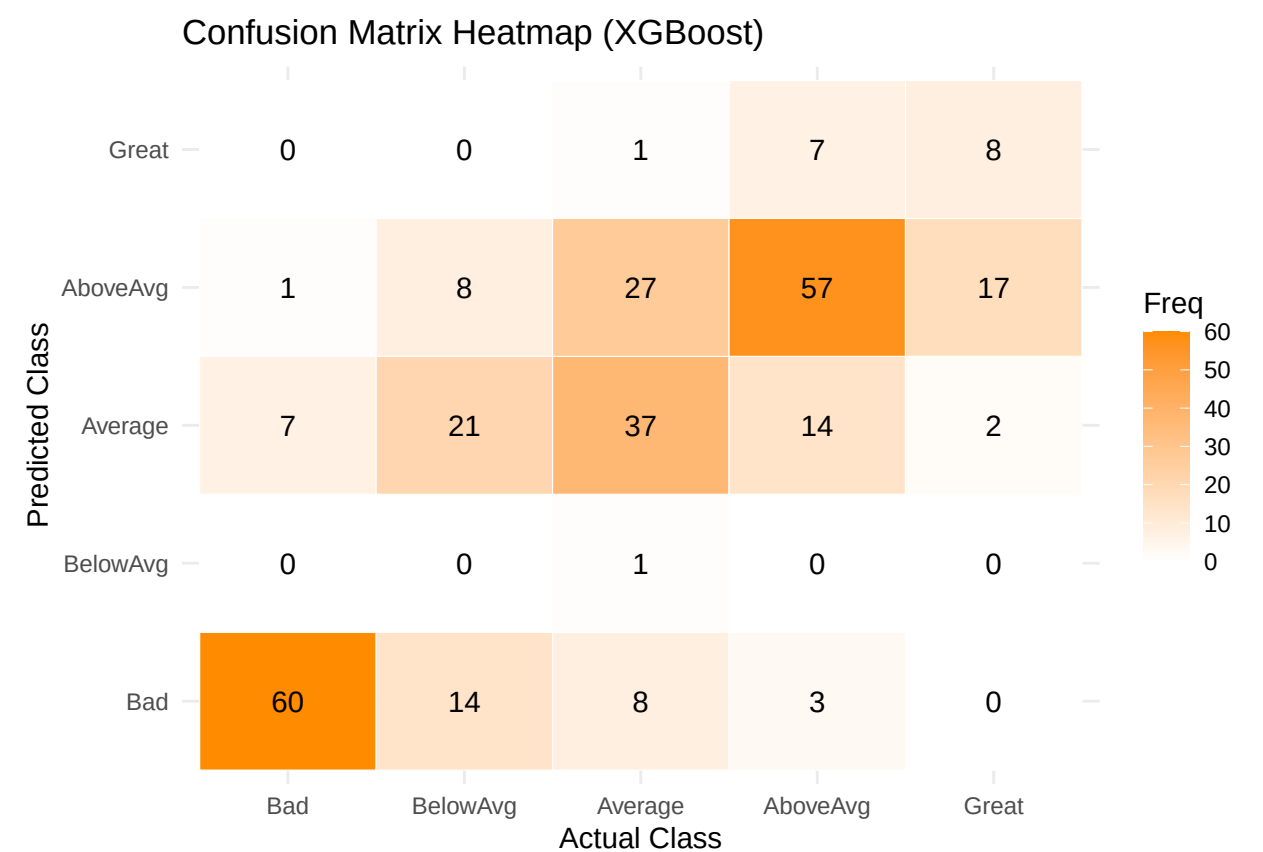
```
##                               Class: Great
## Precision                     0.50000
## Recall                       0.29630
## F1                           0.37209
## Prevalence                   0.09215
## Detection Rate               0.02730
## Detection Prevalence        0.05461
## Balanced Accuracy            0.63311
```

```
probs_xgb <- predict(xgb_model, newdata = test_data, type = "prob")
probs_xgb <- as.matrix(probs_xgb)
```

```
colnames(probs_xgb) <- colnames(test_data)
probs_xgb <- probs_xgb[, levels(test_data$response), drop = FALSE]
```

```
model_results <- rbind(model_results, data.frame(
  Model = "XGBoost",
  Accuracy = max(xgb_model$results$Accuracy),
  Precision = mean(cm_xgb$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_xgb$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_xgb$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data$response, probs_xgb)
))
```

```
cm_table <- as.data.frame(cm_xgb$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "darkorange") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (XGBoost)",
       x = "Actual Class", y = "Predicted Class")
```

Model Results

```
print(model_results)
```

```
##           Model Accuracy Precision   Recall      F1      AUC
## 1  LASSO Min Lambda 0.5426621 0.5204381 0.4837068 0.4797074 0.8197688
## 2  LASSO 1se Lambda 0.5324232 0.4283338 0.4452060 0.5135154 0.8178302
## 3           LDA 0.5597270 0.5254871 0.5073907 0.5088289 0.8132451
## 4           QDA 0.5051195 0.4617156 0.4748931 0.4608397 0.7806059
## 5           PCA 0.5221843 0.4633333 0.4600617 0.4479330 0.8055374
## 6  SVM Linear 0.5529010 0.5321404 0.4552041 0.4257284 0.8127122
## 7  SVM Radial Kernel 0.5460751 0.4705426 0.4489110 0.4213305 0.8125289
## 8  Base Random Forest 0.5221843 0.4962142 0.4701348 0.4696883 0.8064159
## 9  Upsampling Random Forest 0.5187713 0.4838855 0.4941984 0.4848743 0.8028763
## 10 Weighted Random Forest 0.5119454 0.4864930 0.4511409 0.4463983 0.7899169
## 11           XGBoost 0.5176759 0.4361709 0.4764706 0.5576712 0.8118248
```

```
model_results_3PA <- model_results
```

Class 3PCT

Initialize Train / Test Split

```
# Specific set of predictors mentioned in report
X <- per_min_data[, c(8,13,38:40,42,43,45,46,48:50, 52:55, 56:66)]
X[is.na(X)] <- 0
y <- per_min_data[, 69]

y <- factor(y, levels = 1:6, labels = c("NoVol", "LowVol", "BelowAvgVol", "AverageVol", "AboveAvgVol", "HighVol"))

df <- data.frame(X, response = y)
df_scaled <- data.frame(scale(df[, -ncol(df)]), response = df$response)

# Split train and test
set.seed(67)
train_index <- createDataPartition(df_scaled$response, p = 0.8, list = FALSE)
train_data <- df_scaled[train_index, ]
test_data <- df_scaled[-train_index, ]
class_levels <- c("NoVol", "LowVol", "BelowAvgVol", "AverageVol", "AboveAvgVol", "HighVol")
```

Backwards Stepwise Selection

Check for collinearity

```
vif(step_model)
```

```
##          y>=LowVol y>=BelowAvgVol y>=AverageVol y>=AboveAvgVol y>=HighVol
##      2.074938      2.543645      2.198426      1.927310      1.440174
##      PRE_G      PRE_3PA      PRE_3P_PCT      PRE_FTA      PRE_FT_PCT
##      1.045153      1.910806      1.452762      1.154423      1.625338
```

```
predictors <- df_scaled[, c("PRE_G", "PRE_3PA", "PRE_3P_PCT", "PRE_FTA", "PRE_FT_PCT")]
cor_matrix <- cor(predictors)
print(round(cor_matrix, 3))
```

```
##          PRE_G PRE_3PA PRE_3P_PCT PRE_FTA PRE_FT_PCT
## PRE_G      1.000  0.028   0.063  -0.105   0.107
## PRE_3PA     0.028  1.000   0.495  -0.350   0.602
## PRE_3P_PCT  0.063  0.495   1.000  -0.189   0.466
## PRE_FTA    -0.105 -0.350  -0.189   1.000  -0.123
## PRE_FT_PCT  0.107  0.602   0.466  -0.123   1.000
```

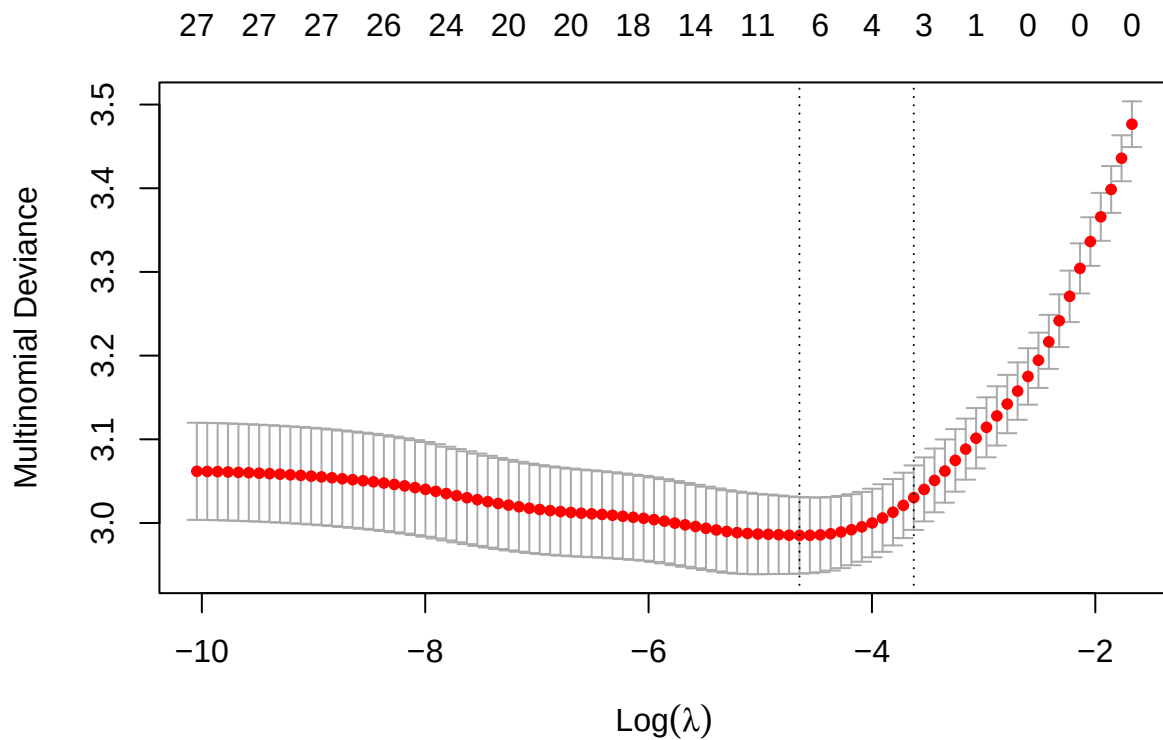
No issues

Initializing Storing Results

```
model_results <- data.frame(  
  Model = character(),  
  Accuracy = numeric(),  
  Precision = numeric(),  
  Recall = numeric(),  
  F1 = numeric(),  
  AUC = numeric(),  
  stringsAsFactors = FALSE  
)
```

LASSO with CV for best lambda

```
set.seed(67)  
#initialize data in matrices  
x_train <- as.matrix(train_data[, -ncol(train_data)])  
y_train <- train_data$response  
x_test  <- as.matrix(test_data[, -ncol(test_data)])  
y_test  <- test_data$response  
  
#run CV model  
cv_lasso <- cv.glmnet(x_train, y_train,  
                      family = "multinomial",  
                      alpha = 1,  
                      nfolds = 10)  
plot(cv_lasso)
```



```
# Check optimal values of lambda
cat("Optimal lambda (min):", cv_lasso$lambda.min, "\n")
```

```
## Optimal lambda (min): 0.009572293
```

```
cat("Optimal lambda (1se):", cv_lasso$lambda.1se, "\n")
```

```
## Optimal lambda (1se): 0.02663547
```

```
coef(cv_lasso, s = "lambda.min")
```

```
## $NoVol
## 28 x 1 sparse Matrix of class "dgCMatrix"
##           1
## (Intercept)  -0.77574210
## WEIGHT       0.15848583
## HEIGHT_INCHES 0.06039274
## PRE_G        0.15885411
## PRE_GS       .
## PRE_MP       .
## PRE_FGA      .
## PRE_FG_PCT   .
## PRE_3PA     -1.30175533
## PRE_3P_PCT  -0.21438060
```

```

## PRE_2PA      .
## PRE_2P_PCT   .
## PRE_eFG_PCT  .
## PRE_FTA      .
## PRE_FT_PCT   -0.18706968
## PRE_ORB      .
## PRE_DRB      .
## PRE_TRB      .
## PRE_AST      .
## PRE_STL      .
## PRE_BLK      0.10139792
## PRE_TOV      .
## PRE_PF       0.03158579
## PRE_PTS      .
## PRE_FG_pct_inc .
## PRE_3P_pct_inc -0.02650347
## PRE_2P_pct_inc 0.02077538
## PRE_FT_pct_inc 0.11128908
##
## $LowVol
## 28 x 1 sparse Matrix of class "dgCMatrix"
##              1
## (Intercept)  -0.56887742
## WEIGHT       .
## HEIGHT_INCHES .
## PRE_G        .
## PRE_GS       .
## PRE_MP       .
## PRE_FGA      .
## PRE_FG_PCT   .
## PRE_3PA      -0.34453508
## PRE_3P_PCT   -0.02697938
## PRE_2PA      .
## PRE_2P_PCT   .
## PRE_eFG_PCT  .
## PRE_FTA      .
## PRE_FT_PCT   .
## PRE_ORB      0.15973887
## PRE_DRB      .
## PRE_TRB      .
## PRE_AST      .
## PRE_STL      .
## PRE_BLK      0.08264267
## PRE_TOV      .
## PRE_PF       .
## PRE_PTS      .
## PRE_FG_pct_inc 0.05345887
## PRE_3P_pct_inc .
## PRE_2P_pct_inc .
## PRE_FT_pct_inc .
##
## $BelowAvgVol
## 28 x 1 sparse Matrix of class "dgCMatrix"
##              1

```

```

## (Intercept)      0.08279245
## WEIGHT            .
## HEIGHT_INCHES    .
## PRE_G            .
## PRE_GS           .
## PRE_MP           .
## PRE_FGA          .
## PRE_FG_PCT       -0.01912739
## PRE_3PA          .
## PRE_3P_PCT       .
## PRE_2PA          0.07147520
## PRE_2P_PCT       .
## PRE_eFG_PCT      .
## PRE_FTA          0.01849419
## PRE_FT_PCT       .
## PRE_ORB          .
## PRE_DRB          .
## PRE_TRB          .
## PRE_AST          0.02682075
## PRE_STL          0.07870468
## PRE_BLK          .
## PRE_TOV          .
## PRE_PF           0.00885165
## PRE_PTS          .
## PRE_FG_pct_inc   .
## PRE_3P_pct_inc   0.02152424
## PRE_2P_pct_inc   -0.18103699
## PRE_FT_pct_inc   .
##
## $AverageVol
## 28 x 1 sparse Matrix of class "dgCMatrix"
##              1
## (Intercept)    0.336197566
## WEIGHT         -0.040597775
## HEIGHT_INCHES  .
## PRE_G          0.008972836
## PRE_GS         .
## PRE_MP         .
## PRE_FGA        .
## PRE_FG_PCT     -0.184849010
## PRE_3PA        .
## PRE_3P_PCT     .
## PRE_2PA        .
## PRE_2P_PCT     .
## PRE_eFG_PCT    .
## PRE_FTA        .
## PRE_FT_PCT     .
## PRE_ORB        .
## PRE_DRB        0.084525783
## PRE_TRB        .
## PRE_AST        .
## PRE_STL        .
## PRE_BLK        .
## PRE_TOV        .

```

```

## PRE_PF .
## PRE_PTS .
## PRE_FG_pct_inc -0.010227672
## PRE_3P_pct_inc .
## PRE_2P_pct_inc .
## PRE_FT_pct_inc -0.023785715
##
## $AboveAvgVol
## 28 x 1 sparse Matrix of class "dgCMatrix"
## 1
## (Intercept) 0.695400777
## WEIGHT .
## HEIGHT_INCHES -0.041423506
## PRE_G .
## PRE_GS .
## PRE_MP .
## PRE_FGA 0.062570984
## PRE_FG_PCT .
## PRE_3PA 0.067501209
## PRE_3P_PCT .
## PRE_2PA .
## PRE_2P_PCT .
## PRE_eFG_PCT .
## PRE_FTA -0.172155186
## PRE_FT_PCT 0.158780956
## PRE_ORB .
## PRE_DRB .
## PRE_TRB .
## PRE_AST .
## PRE_STL .
## PRE_BLK -0.023425251
## PRE_TOV .
## PRE_PF .
## PRE_PTS .
## PRE_FG_pct_inc .
## PRE_3P_pct_inc .
## PRE_2P_pct_inc .
## PRE_FT_pct_inc -0.007085801
##
## $HighVol
## 28 x 1 sparse Matrix of class "dgCMatrix"
## 1
## (Intercept) 0.23022873
## WEIGHT .
## HEIGHT_INCHES .
## PRE_G -0.06557832
## PRE_GS .
## PRE_MP .
## PRE_FGA .
## PRE_FG_PCT .
## PRE_3PA 0.22870102
## PRE_3P_PCT 0.04440333
## PRE_2PA -0.02708378
## PRE_2P_PCT .

```

```
## PRE_eFG_PCT      .
## PRE_FTA          -0.10537613
## PRE_FT_PCT       0.76639616
## PRE_ORB          .
## PRE_DRB          .
## PRE_TRB          .
## PRE_AST          .
## PRE_STL          .
## PRE_BLK          .
## PRE_TOV          -0.24940043
## PRE_PF           .
## PRE_PTS          .
## PRE_FG_pct_inc   .
## PRE_3P_pct_inc   .
## PRE_2P_pct_inc   .
## PRE_FT_pct_inc   .
```

```
# Calculate Test Accuracy
```

```
preds_min <- predict(cv_lasso, newx = x_test, s = "lambda.min", type = "class")
accuracy_min <- mean(as.character(preds_min) == as.character(y_test))
cat("Test accuracy (lambda.min):", accuracy_min, "\n")
```

```
## Test accuracy (lambda.min): 0.4095563
```

```
# Calculate CM
```

```
preds_min <- factor(preds_min, levels = levels(y_test))
conf_mat_min <- confusionMatrix(preds_min, y_test, mode = "prec_recall")
print(conf_mat_min)
```

```
## Confusion Matrix and Statistics
```

```
##
##              Reference
## Prediction   NoVol LowVol BelowAvgVol AverageVol AboveAvgVol HighVol
## NoVol        44      6         12          3         3         2
## LowVol        0      0          0          0         0         0
## BelowAvgVol   0      2          1          1         1         0
## AverageVol    1      0          0          2         1         0
## AboveAvgVol   4      9         19         34         43        30
## HighVol       1      4          6         10         24        30
```

```
## Overall Statistics
```

```
##
##              Accuracy : 0.4096
##              95% CI : (0.3527, 0.4682)
##      No Information Rate : 0.2457
##      P-Value [Acc > NIR] : 5.24e-10
```

```
##
##              Kappa : 0.2468
```

```
##
##      McNemar's Test P-Value : NA
```

```
##
## Statistics by Class:
```



```
##          Class: NoVol Class: LowVol Class: BelowAvgVol
## Precision          0.6286          NA          0.200000
## Recall             0.8800          0.00000          0.026316
## F1                 0.7333          NA          0.046512
## Prevalence         0.1706          0.07167          0.129693
## Detection Rate     0.1502          0.00000          0.003413
## Detection Prevalence 0.2389          0.00000          0.017065
## Balanced Accuracy   0.8865          0.50000          0.505315
##          Class: AverageVol Class: AboveAvgVol Class: HighVol
## Precision          0.500000          0.3094          0.4000
## Recall             0.040000          0.5972          0.4839
## F1                 0.074074          0.4076          0.4380
## Prevalence         0.170648          0.2457          0.2116
## Detection Rate     0.006826          0.1468          0.1024
## Detection Prevalence 0.013652          0.4744          0.2560
## Balanced Accuracy   0.515885          0.5814          0.6445
```

```
# Calculate Class Error
class_error_min <- 1 - diag(conf_mat_min$stable) / colSums(conf_mat_min$stable)
print(class_error_min)
```

```
##          NoVol      LowVol BelowAvgVol AverageVol AboveAvgVol      HighVol
## 0.1200000  1.0000000  0.9736842  0.9600000  0.4027778  0.5161290
```

```
# AUC Vectors
probs_min <- predict(cv_lasso, newx = x_test, s = "lambda.min", type = "response")
probs_min <- probs_min[,1]
colnames(probs_min) <- levels(y_test)
```

```
# Repeat
preds_1se <- predict(cv_lasso, newx = x_test, s = "lambda.1se", type = "class")
accuracy_1se <- mean(as.character(preds_1se) == as.character(y_test))
cat("Test accuracy (lambda.1se):", accuracy_1se, "\n")
```

```
## Test accuracy (lambda.1se): 0.3959044
```

```
preds_1se <- factor(preds_1se, levels = levels(y_test))
conf_mat_1se <- confusionMatrix(preds_1se, y_test, mode = "prec_recall")
print(conf_mat_1se)
```

```
## Confusion Matrix and Statistics
```

```
##
##          Reference
## Prediction  NoVol LowVol BelowAvgVol AverageVol AboveAvgVol HighVol
## NoVol      44      7      11          3          3          2
## LowVol      0      0      0          0          0          0
## BelowAvgVol 0      0      0          0          0          0
## AverageVol  0      0      0          0          0          0
## AboveAvgVol 5     11     19         36         42         30
## HighVol     1      3      8          11         27         30
##
```

```
## Overall Statistics
```

```
##
##          Accuracy : 0.3959
##          95% CI   : (0.3395, 0.4544)
##    No Information Rate : 0.2457
##    P-Value [Acc > NIR] : 1.016e-08
##
##          Kappa : 0.227
##
##    McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##          Class: NoVol Class: LowVol Class: BelowAvgVol
## Precision          0.6286          NA          NA
## Recall              0.8800          0.00000          0.0000
## F1                  0.7333          NA          NA
## Prevalence          0.1706          0.07167          0.1297
## Detection Rate      0.1502          0.00000          0.0000
## Detection Prevalence 0.2389          0.00000          0.0000
## Balanced Accuracy    0.8865          0.50000          0.5000
##
##          Class: AverageVol Class: AboveAvgVol Class: HighVol
## Precision          NA          0.2937          0.3750
## Recall              0.0000          0.5833          0.4839
## F1                  NA          0.3907          0.4225
## Prevalence          0.1706          0.2457          0.2116
## Detection Rate      0.0000          0.1433          0.1024
## Detection Prevalence 0.0000          0.4881          0.2730
## Balanced Accuracy    0.5000          0.5632          0.6337
```

```
class_error_1se <- 1 - diag(conf_mat_1se$table) / colSums(conf_mat_1se$table)
print(class_error_1se)
```

```
##          NoVol          LowVol BelowAvgVol AverageVol AboveAvgVol          HighVol
##    0.1200000    1.0000000    1.0000000    1.0000000    0.4166667    0.5161290
```

```
probs_1se <- predict(cv_lasso, newx = x_test, s = "lambda.1se", type = "response")
probs_1se <- probs_1se[,1]
colnames(probs_1se) <- levels(y_test)
```

```
# Load model results (done for every model)
model_results <- rbind(model_results, data.frame(
  Model = "LASSO Min Lambda",
  Accuracy = accuracy_min,
  Precision = mean(conf_mat_min$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(conf_mat_min$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(conf_mat_min$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(y_test, probs_min)
))

model_results <- rbind(model_results, data.frame(
  Model = "LASSO 1se Lambda",
  Accuracy = accuracy_1se,
  Precision = mean(conf_mat_1se$byClass[, "Precision"], na.rm = TRUE),
```

```

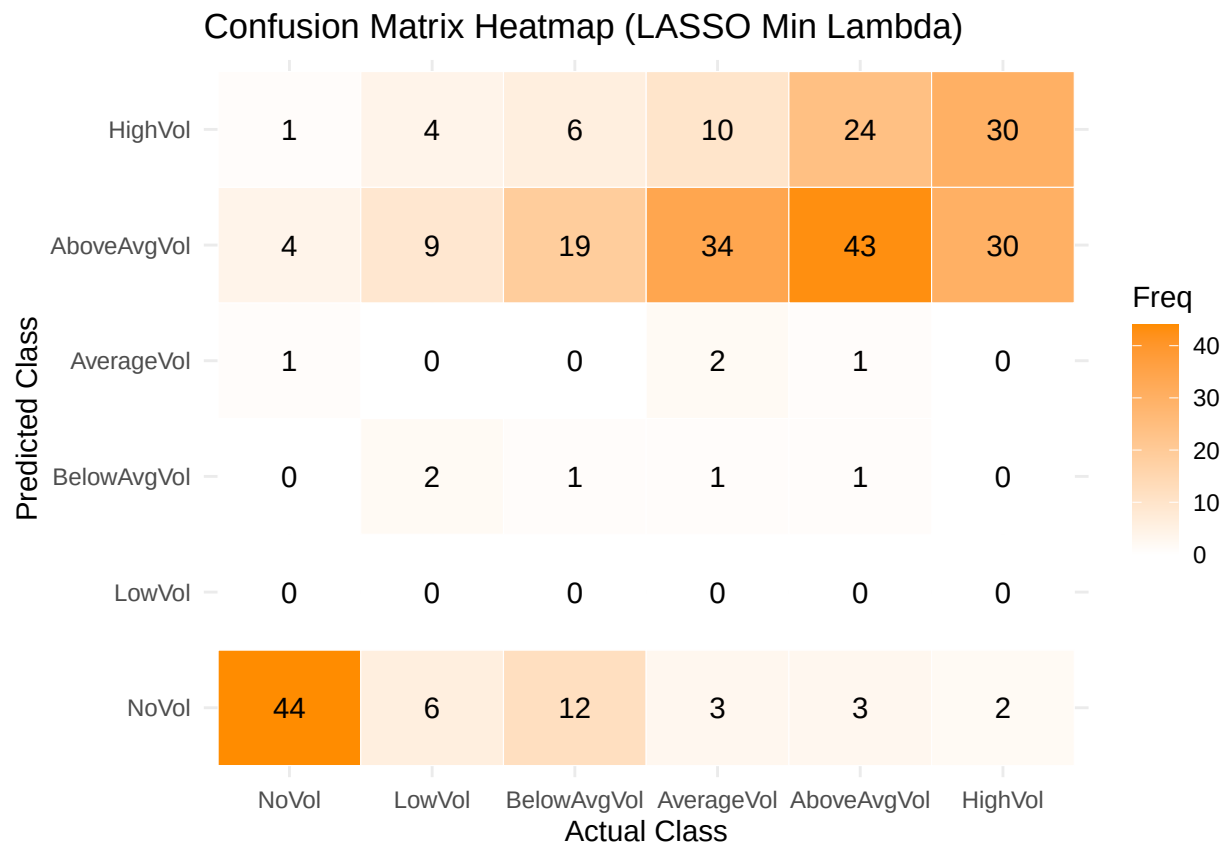
Recall = mean(conf_mat_1se$byClass[, "Recall"], na.rm = TRUE),
F1 = mean(conf_mat_1se$byClass[, "F1"], na.rm = TRUE),
AUC = macro_auc(y_test, probs_1se)
))

```

```

# Create confusion matrix (done for every model)
cm_table <- as.data.frame(conf_mat_min$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "darkorange") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (LASSO Min Lambda)",
       x = "Actual Class", y = "Predicted Class")

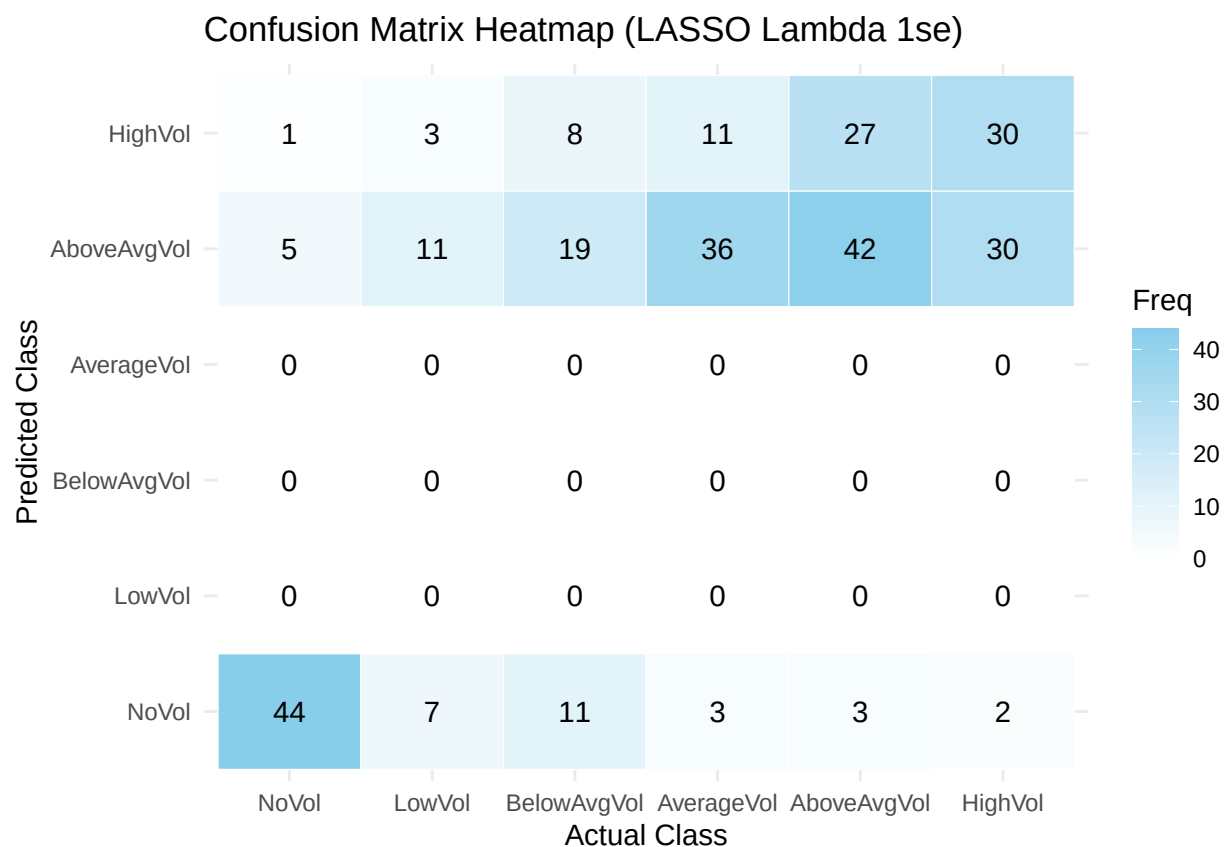
```



```

cm_table <- as.data.frame(conf_mat_1se$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "skyblue") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (LASSO Lambda 1se)",
       x = "Actual Class", y = "Predicted Class")

```



LDA

```
set.seed(22222)
# Run cross validation
train_control <- trainControl(method = "cv", number = 10,
                              summaryFunction = multiClassSummary)

lda_cv <- train(response ~ PRE_FG_PCT + PRE_3PA + PRE_FTA + PRE_DRB +
                 PRE_BLK + PRE_TOV + PRE_PTS, data = train_data,
                 method = "lda",
                 trControl = train_control)

## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.

print(lda_cv)

## Linear Discriminant Analysis
##
## 1186 samples
## 7 predictor
## 6 classes: 'NoVol', 'LowVol', 'BelowAvgVol', 'AverageVol', 'AboveAvgVol', 'HighVol'
```

```
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1068, 1068, 1066, 1066, 1068, 1067, ...
## Resampling results:
##
##   Accuracy   Kappa     Mean_F1     Mean_Sensitivity Mean_Specificity
##   0.3693455  0.2047093  0.3233881  0.3143355        0.8666899
##   Mean_Pos_Pred_Value Mean_Neg_Pred_Value Mean_Precision Mean_Recall
##   0.3340006          0.8698257          0.3340006      0.3143355
##   Mean_Detection_Rate Mean_Balanced_Accuracy
##   0.06155759          0.5905127

cat("Cross-validated Accuracy (training set):", max(lda_cv$results$Accuracy), "\n")
```

```
## Cross-validated Accuracy (training set): 0.3693455
```

```
# Set fit model
lda_fit <- lda(response ~ PRE_FG_PCT + PRE_3PA + PRE_FTA + PRE_DRB +
               PRE_BLK + PRE_TOV + PRE_PTS,
               data = train_data)

pred_obj <- predict(lda_fit, test_data)

all_pred  <- factor(pred_obj$class, levels = class_levels)
all_actual <- factor(test_data$response, levels = class_levels)
all_probs <- pred_obj$posterior

# Find CM
conf_mat_lda <- confusionMatrix(all_pred, all_actual, mode = "prec_recall")
print(conf_mat_lda)
```

```
## Confusion Matrix and Statistics
```

```
##
##               Reference
## Prediction    NoVol LowVol BelowAvgVol AverageVol AboveAvgVol HighVol
## NoVol         44      7          9          4          4          3
## LowVol         0      1          0          0          1          0
## BelowAvgVol    0      2          3          1          1          1
## AverageVol     2      0          3          7          8          3
## AboveAvgVol    3      7         16         27         33         24
## HighVol        1      4          7         11         25         31
##
```

```
## Overall Statistics
```

```
##
##               Accuracy : 0.4061
##               95% CI : (0.3494, 0.4648)
##   No Information Rate : 0.2457
##   P-Value [Acc > NIR] : 1.123e-09
##
##               Kappa : 0.2501
##
##   Mcnemar's Test P-Value : NA
```

```
##
## Statistics by Class:
##
##           Class: NoVol Class: LowVol Class: BelowAvgVol
## Precision           0.6197      0.500000      0.37500
## Recall              0.8800      0.047619      0.07895
## F1                  0.7273      0.086957      0.13043
## Prevalence          0.1706      0.071672      0.12969
## Detection Rate      0.1502      0.003413      0.01024
## Detection Prevalence 0.2423      0.006826      0.02730
## Balanced Accuracy    0.8844      0.521971      0.52967
##
##           Class: AverageVol Class: AboveAvgVol Class: HighVol
## Precision           0.30435      0.3000      0.3924
## Recall              0.14000      0.4583      0.5000
## F1                  0.19178      0.3626      0.4397
## Prevalence          0.17065      0.2457      0.2116
## Detection Rate      0.02389      0.1126      0.1058
## Detection Prevalence 0.07850      0.3754      0.2696
## Balanced Accuracy    0.53708      0.5550      0.6461
```

```
# Calculate Accuracy and AUC
```

```
LDA_acc <- mean(as.character(all_pred) == as.character(all_actual))
cat("Test Accuracy (LDA):", LDA_acc, "\n")
```

```
## Test Accuracy (LDA): 0.4061433
```

```
overall_multiclass <- multiclass.roc(all_actual, all_probs)
cat("Overall multiclass AUC (test set):", overall_multiclass$auc, "\n")
```

```
## Overall multiclass AUC (test set): 0.6991904
```

```
# Graph AUC / Calculate Macro AUC
```

```
fpr_grid <- seq(0, 1, length.out = 200)
tpr_matrix <- matrix(NA, nrow = length(fpr_grid), ncol = length(class_levels))
```

```
for (i in seq_along(class_levels)) {
  class <- class_levels[i]
  binary_actual <- factor(ifelse(all_actual == class, class, "Other"),
                           levels = c("Other", class))
  class_probs <- all_probs[, class]
```

```
roc_obj <- roc(response = binary_actual,
               predictor = class_probs,
               levels = c("Other", class),
               positive = class,
               quiet = TRUE)
```

```
cat("AUC of", class, ":", auc(roc_obj), "\n")
```

```
fpr <- 1 - roc_obj$specificities
tpr <- roc_obj$sensitivities
```

```

interp_tpr <- approx(fpr, tpr, xout = fpr_grid, ties = mean, rule = 2)$y
tpr_matrix[, i] <- interp_tpr
}

```

```

## AUC of NoVol : 0.9100412
## AUC of LowVol : 0.6878501
## AUC of BelowAvgVol : 0.60516
## AUC of AverageVol : 0.6201646
## AUC of AboveAvgVol : 0.6126194
## AUC of HighVol : 0.760229

```

```

macro_tpr_lda <- rowMeans(tpr_matrix, na.rm = TRUE)
macro_auc_lda <- trapz(fpr_grid, macro_tpr_lda)
cat("Macro AUC (LDA test set):", macro_auc_lda, "\n")

```

```

## Macro AUC (LDA test set): 0.6993283

```

```

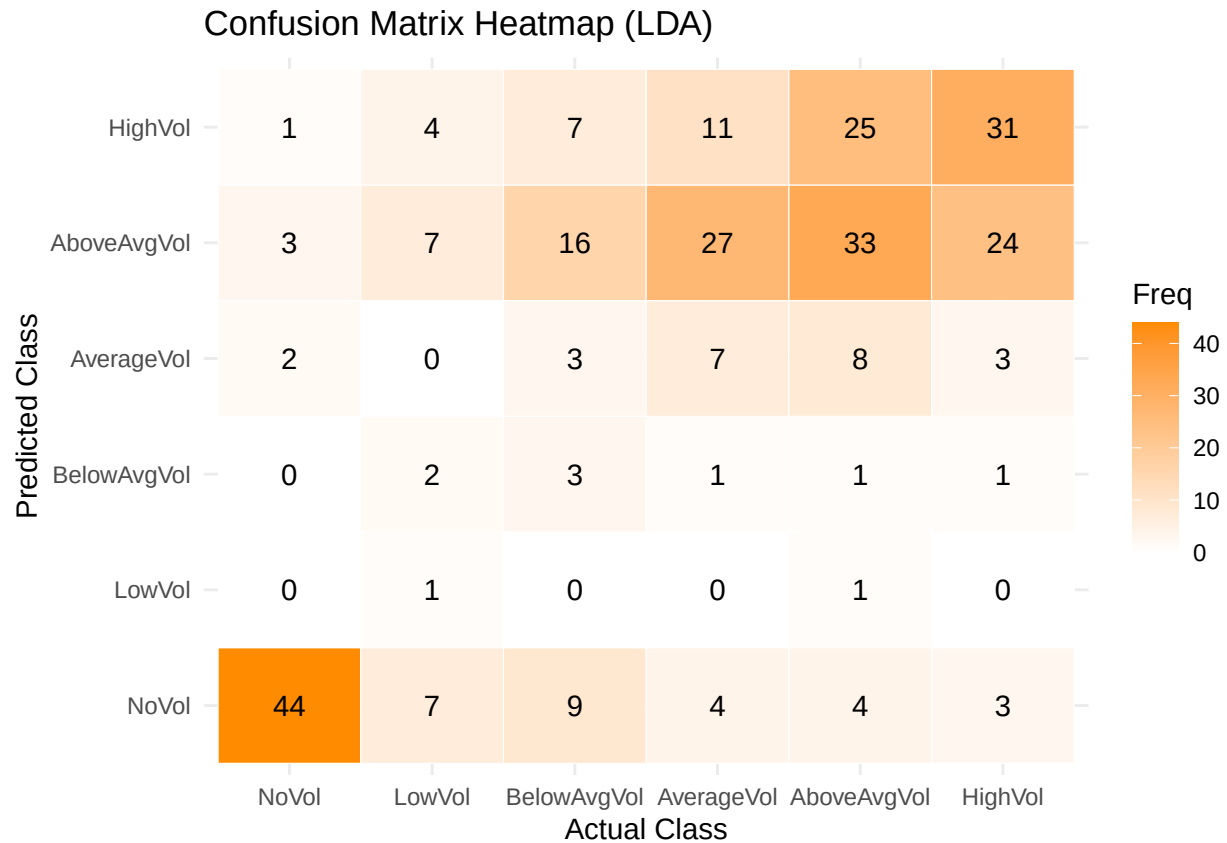
model_results <- rbind(model_results, data.frame(
  Model = "LDA",
  Accuracy = LDA_acc,
  Precision = mean(conf_mat_lda$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(conf_mat_lda$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(conf_mat_lda$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc_lda
))

```

```

cm_table <- as.data.frame(conf_mat_lda$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "darkorange") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (LDA)",
       x = "Actual Class", y = "Predicted Class")

```



QDA

```
# Same steps as LDA
set.seed(22222)
train_control <- trainControl(method = "cv", number = 10,
                              summaryFunction = multiClassSummary)

qda_cv <- train(response ~ PRE_FG_PCT + PRE_3PA + PRE_FTA + PRE_DRB +
                 PRE_BLK + PRE_TOV + PRE_PTS,
                 data = train_data,
                 method = "qda",
                 trControl = train_control)

## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.

print(qda_cv)

## Quadratic Discriminant Analysis
##
## 1186 samples
##    7 predictor
```



```
## 6 classes: 'NoVol', 'LowVol', 'BelowAvgVol', 'AverageVol', 'AboveAvgVol', 'HighVol'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1068, 1068, 1066, 1066, 1068, 1067, ...
## Resampling results:
##
## Accuracy Kappa Mean_F1 Mean_Sensitivity Mean_Specificity
## 0.3641758 0.2059095 0.3089344 0.3182294 0.8671104
## Mean_Pos_Pred_Value Mean_Neg_Pred_Value Mean_Precision Mean_Recall
## 0.311606 0.8717144 0.311606 0.3182294
## Mean_Detection_Rate Mean_Balanced_Accuracy
## 0.06069597 0.5926699
```

```
cat("Cross-validated Accuracy (training set):", max(qda_cv$results$Accuracy), "\n")
```

```
## Cross-validated Accuracy (training set): 0.3641758
```

```
qda_fit <- qda(response ~ PRE_FG_PCT + PRE_3PA + PRE_FTA + PRE_DRB +
               PRE_BLK + PRE_TOV + PRE_PTS,
               data = train_data)

pred_obj <- predict(qda_fit, test_data)

all_pred <- factor(pred_obj$class, levels = class_levels)
all_actual <- factor(test_data$response, levels = class_levels)
all_probs <- pred_obj$posterior

conf_mat_qda <- confusionMatrix(all_pred, all_actual, mode = "prec_recall")
print(conf_mat_qda)
```

```
## Confusion Matrix and Statistics
```

```
##
##              Reference
## Prediction  NoVol LowVol BelowAvgVol AverageVol AboveAvgVol HighVol
## NoVol      38     9      9           3         2         2
## LowVol      1     1      0           0         4         2
## BelowAvgVol 4     2      4           7         1         2
## AverageVol  1     1      2           5        11         1
## AboveAvgVol 4     3     10          17        20        15
## HighVol     2     5     13          18        34        40
##
```

```
## Overall Statistics
```

```
##
##              Accuracy : 0.3686
##              95% CI : (0.3132, 0.4267)
## No Information Rate : 0.2457
## P-Value [Acc > NIR] : 1.961e-06
##
##              Kappa : 0.2122
##
## Mcnemar's Test P-Value : 1.001e-06
```

```
##
## Statistics by Class:
##
##           Class: NoVol Class: LowVol Class: BelowAvgVol
## Precision           0.6032      0.125000      0.20000
## Recall              0.7600      0.047619      0.10526
## F1                  0.6726      0.068966      0.13793
## Prevalence          0.1706      0.071672      0.12969
## Detection Rate      0.1297      0.003413      0.01365
## Detection Prevalence 0.2150      0.027304      0.06826
## Balanced Accuracy    0.8286      0.510942      0.52126
##
##           Class: AverageVol Class: AboveAvgVol Class: HighVol
## Precision           0.23810      0.28986      0.3571
## Recall              0.10000      0.27778      0.6452
## F1                  0.14085      0.28369      0.4598
## Prevalence          0.17065      0.24573      0.2116
## Detection Rate      0.01706      0.06826      0.1365
## Detection Prevalence 0.07167      0.23549      0.3823
## Balanced Accuracy    0.51708      0.52803      0.6667
```

```
QDA_acc <- mean(as.character(all_pred) == as.character(all_actual))
cat("Test Accuracy (QDA):", QDA_acc, "\n")
```

```
## Test Accuracy (QDA): 0.3686007
```

```
overall_multiclass <- multiclass.roc(all_actual, all_probs)
cat("Overall multiclass AUC (QDA test set):", overall_multiclass$auc, "\n")
```

```
## Overall multiclass AUC (QDA test set): 0.6774986
```

```
fpr_grid <- seq(0, 1, length.out = 200)
tpr_matrix <- matrix(NA, nrow = length(fpr_grid), ncol = length(class_levels))

for (i in seq_along(class_levels)) {
  class <- class_levels[i]
  binary_actual <- factor(ifelse(all_actual == class, class, "Other"),
                           levels = c("Other", class))
  class_probs <- all_probs[, class]

  roc_obj <- roc(response = binary_actual,
                 predictor = class_probs,
                 levels = c("Other", class),
                 positive = class,
                 quiet = TRUE)

  cat("AUC of", class, ":", auc(roc_obj), "\n")

  fpr <- 1 - roc_obj$specificities
  tpr <- roc_obj$sensitivities

  interp_tpr <- approx(fpr, tpr, xout = fpr_grid, ties = mean, rule = 2)$y
  tpr_matrix[, i] <- interp_tpr
}
```

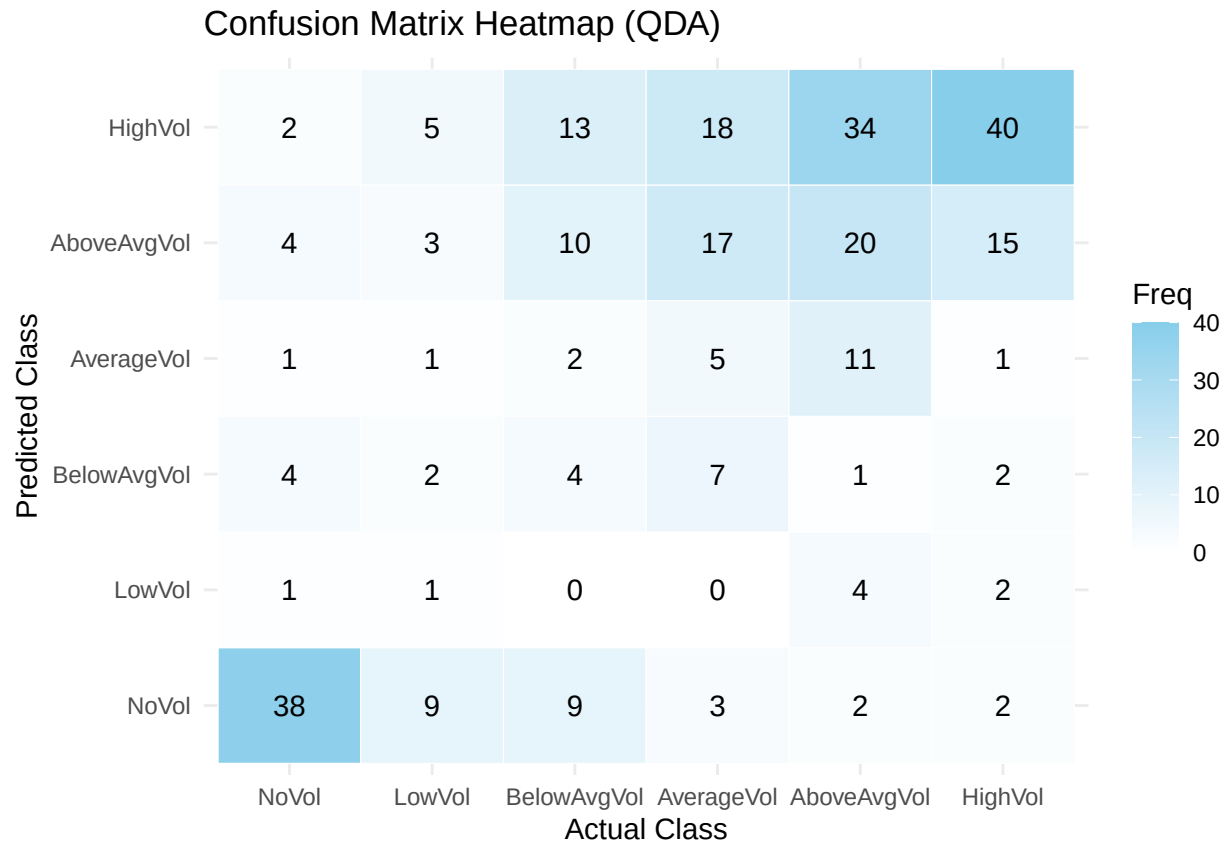
```
## AUC of NoVol : 0.9121811
## AUC of LowVol : 0.6491597
## AUC of BelowAvgVol : 0.576161
## AUC of AverageVol : 0.6260905
## AUC of AboveAvgVol : 0.5824535
## AUC of HighVol : 0.7204301
```

```
macro_tpr_qda <- rowMeans(tpr_matrix, na.rm = TRUE)
macro_auc_qda <- trapz(fpr_grid, macro_tpr_qda)
cat("Macro AUC (QDA test set):", macro_auc_qda, "\n")
```

```
## Macro AUC (QDA test set): 0.6777023
```

```
model_results <- rbind(model_results, data.frame(
  Model = "QDA",
  Accuracy = QDA_acc,
  Precision = mean(conf_mat_qda$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(conf_mat_qda$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(conf_mat_qda$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc_qda
))
```

```
cm_table <- as.data.frame(conf_mat_qda$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "skyblue") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (QDA)",
       x = "Actual Class", y = "Predicted Class")
```



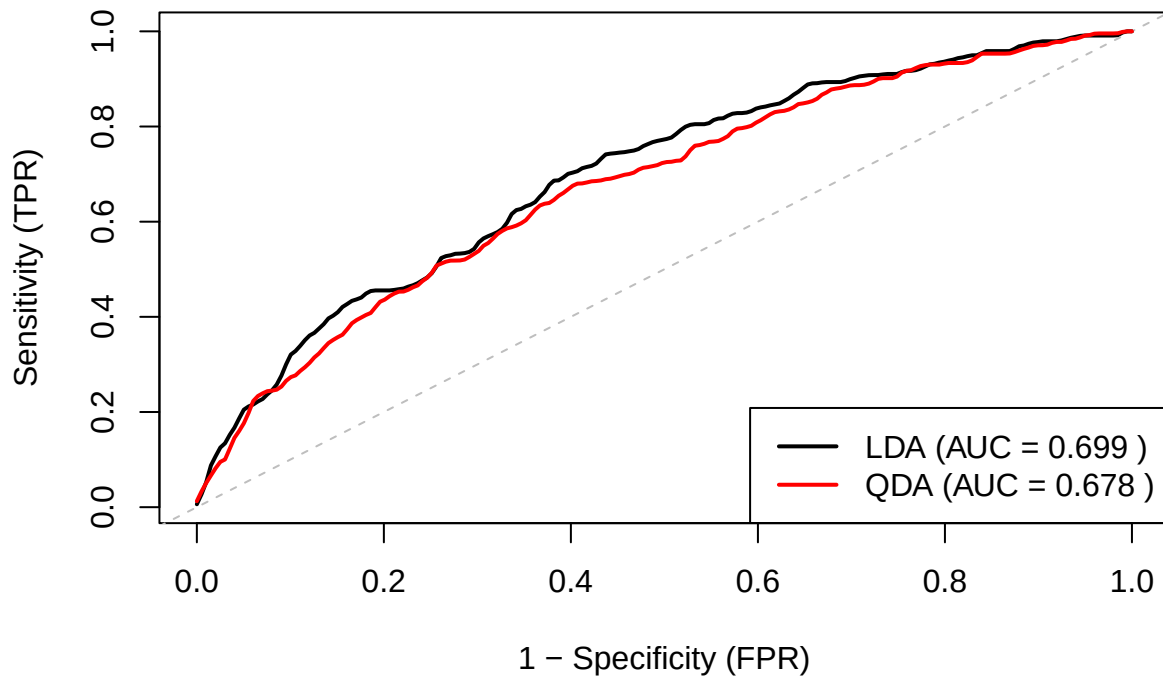
Comparison of LDA vs QDA

```
LDA_acc - QDA_acc
```

```
## [1] 0.03754266
```

```
# Graph AUC curves
plot(fpr_grid, macro_tpr_lda, type = "l", lwd = 2, col = "black",
     xlab = "1 - Specificity (FPR)", ylab = "Sensitivity (TPR)",
     main = paste("Macro-average ROC Curves of LDA and QDA"))
abline(a = 0, b = 1, lty = 2, col = "gray")
lines(fpr_grid, macro_tpr_qda, lwd = 2, col = "red")
legend("bottomright",
      legend = c(paste("LDA (AUC =", round(macro_auc_lda, 3), ")"),
                 paste("QDA (AUC =", round(macro_auc_qda, 3), ")")),
      col = c("black", "red"), lwd = 2)
```

Macro-average ROC Curves of LDA and QDA



PCA

```
# Set PCA statistics
pca_result <- prcomp(df_scaled[, -ncol(df_scaled)], center = TRUE, scale. = TRUE)
explained_var <- summary(pca_result)$importance[2, ]
num_pcs <- which(cumsum(explained_var) >= 0.9)[1]

# Set PCA dataframe
df_pca <- data.frame(pca_result$x[, 1:num_pcs], response = df_scaled$response)
train_data_pca <- df_pca[train_index, ]
test_data_pca <- df_pca[-train_index, ]

# PCA with CV
train_control <- trainControl(method = "cv", number = 10,
                             summaryFunction = multiClassSummary)

model_pca_cv <- train(response ~ ., data = train_data_pca,
                     method = "multinom",
                     trControl = train_control,
                     trace = FALSE)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.
```

```
print(model_pca_cv)
```

```
## Penalized Multinomial Regression
```

```
##
## 1186 samples
## 13 predictor
## 6 classes: 'NoVol', 'LowVol', 'BelowAvgVol', 'AverageVol', 'AboveAvgVol', 'HighVol'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1068, 1067, 1066, 1068, 1068, 1068, ...
## Resampling results across tuning parameters:
##
## decay Accuracy Kappa Mean_F1 Mean_Sensitivity Mean_Specificity
## 0e+00 0.3811264 0.2176120 0.3187728 0.3243613 0.8686862
## 1e-04 0.3811264 0.2176120 0.3187728 0.3243613 0.8686862
## 1e-01 0.3802719 0.2164675 0.3187728 0.3235280 0.8684969
## Mean_Pos_Pred_Value Mean_Neg_Pred_Value Mean_Precision Mean_Recall
## 0.4798056 0.8721760 0.4798056 0.3243613
## 0.4798056 0.8721760 0.4798056 0.3243613
## 0.4798056 0.8720045 0.4798056 0.3235280
## Mean_Detection_Rate Mean_Balanced_Accuracy
## 0.06352106 0.5965238
## 0.06352106 0.5965238
## 0.06337865 0.5960124
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was decay = 1e-04.
```

```
cat("Cross-validated Accuracy (training set):", max(model_pca_cv$results$Accuracy), "\n")
```

```
## Cross-validated Accuracy (training set): 0.3811264
```

```
# Set our fit model
model_pca <- multinom(response ~ ., data = train_data_pca, trace = FALSE)

preds_pca <- predict(model_pca, newdata = test_data_pca)
preds_pca <- factor(preds_pca, levels = levels(test_data_pca$response))

# Calc CM
cm_pca <- confusionMatrix(preds_pca, test_data_pca$response, mode = "prec_recall")
print(cm_pca)
```

```
## Confusion Matrix and Statistics
```

```
##
##               Reference
## Prediction   NoVol LowVol BelowAvgVol AverageVol AboveAvgVol HighVol
## NoVol        43     6         10          4          3         2
## LowVol         0     0          0          0          0         0
## BelowAvgVol    0     3          3          2          3         0
## AverageVol     2     3          3          7          5         0
## AboveAvgVol    3     5         17         27         37        29
## HighVol        2     4          5         10         24        31
```

```
##
## Overall Statistics
##
```

```

##              Accuracy : 0.413
##              95% CI : (0.356, 0.4717)
##      No Information Rate : 0.2457
##      P-Value [Acc > NIR] : 2.413e-10
##
##              Kappa : 0.2569
##
##      McNemar's Test P-Value : 1.760e-09
##
## Statistics by Class:
##
##              Class: NoVol Class: LowVol Class: BelowAvgVol
## Precision              0.6324              NA              0.27273
## Recall                  0.8600              0.00000              0.07895
## F1                      0.7288              NA              0.12245
## Prevalence              0.1706              0.07167              0.12969
## Detection Rate          0.1468              0.00000              0.01024
## Detection Prevalence    0.2321              0.00000              0.03754
## Balanced Accuracy        0.8786              0.50000              0.52379
##
##              Class: AverageVol Class: AboveAvgVol Class: HighVol
## Precision              0.35000              0.3136              0.4079
## Recall                  0.14000              0.5139              0.5000
## F1                      0.20000              0.3895              0.4493
## Prevalence              0.17065              0.2457              0.2116
## Detection Rate          0.02389              0.1263              0.1058
## Detection Prevalence    0.06826              0.4027              0.2594
## Balanced Accuracy        0.54325              0.5737              0.6526

test_acc <- mean(as.character(preds_pca) == as.character(test_data_pca$response))
cat("Test Accuracy (PCA-based model):", test_acc, "\n")

## Test Accuracy (PCA-based model): 0.4129693

loadings <- pca_result$rotation[, 1:num_pcs]
print("PCA Loadings (contribution of original variables):")

## [1] "PCA Loadings (contribution of original variables):"

print(loadings)

##              PC1              PC2              PC3              PC4              PC5
## WEIGHT          0.310576735 -0.11600745  0.022405694 -0.090180825  0.01486769
## HEIGHT_INCHES   0.306018471 -0.12869752 -0.071442881 -0.070894339  0.04853623
## PRE_G           -0.091069205 -0.39616697  0.179522649 -0.150512294 -0.04018886
## PRE_GS          -0.108814951 -0.33153667  0.243856055 -0.150788708 -0.11745689
## PRE_MP          -0.131801927 -0.34671668  0.252903108 -0.147007736 -0.06628751
## PRE_FGA         -0.060335081  0.20465217  0.167377182 -0.448716734  0.07429732
## PRE_FG_PCT       0.297628706 -0.08839393  0.066116020 -0.009784959 -0.09895303
## PRE_3PA         -0.304268051  0.10539397 -0.006859503 -0.100076805  0.11678585
## PRE_3P_PCT      -0.209899910  0.05912609  0.069077397 -0.027605119 -0.03071896
## PRE_2PA          0.163609176  0.25793448  0.356691231 -0.006040583 -0.13122173
## PRE_2P_PCT       0.092818662  0.24657497  0.352240719  0.283858437 -0.18429705

```

## PRE_eFG_PCT	0.065374880	0.25458067	0.359438727	0.286058254	-0.16684636
## PRE_FTA	0.170662156	0.11379092	0.164509523	-0.343449549	-0.08557226
## PRE_FT_PCT	-0.247252477	0.07428605	0.092619611	-0.120717187	0.02573812
## PRE_ORB	0.252306293	-0.01153781	0.173584896	-0.046970034	0.35464067
## PRE_DRB	0.193106037	-0.03361403	-0.099911399	-0.107603026	-0.37179356
## PRE_TRB	0.236534587	-0.06157029	-0.102453639	-0.088923308	-0.45323070
## PRE_AST	-0.235526532	0.03941034	-0.091674798	-0.001542908	-0.37897486
## PRE_STL	-0.150644666	0.00788266	-0.186349012	-0.042236917	-0.47896250
## PRE_BLK	0.306173178	-0.04844280	-0.014637065	0.030408065	-0.02030777
## PRE_TOV	0.016687285	0.09544624	0.009094554	-0.275116011	-0.08817865
## PRE_PF	0.217643408	-0.16667332	-0.183079463	-0.046787190	0.04078897
## PRE_PTS	0.038980952	0.20570127	0.223216572	-0.460695696	0.01450406
## PRE_FG_pct_inc	0.004719971	-0.32262204	0.195555697	0.068318283	0.01528496
## PRE_3P_pct_inc	-0.185918557	-0.15406746	0.155208206	-0.022473690	-0.07223330
## PRE_2P_pct_inc	-0.029798898	-0.10309029	0.331152399	0.300150507	-0.04396151
## PRE_FT_pct_inc	-0.012801073	-0.28517770	0.189787389	0.053510073	-0.02871863
##	PC6	PC7	PC8	PC9	PC10
## WEIGHT	-0.102440463	0.01945161	-0.10761587	0.0441645755	0.071696264
## HEIGHT_INCHES	-0.197861654	0.03426717	-0.07151755	0.0003343841	0.104053219
## PRE_G	-0.003096375	-0.22717250	-0.03676991	-0.0148742508	-0.046949635
## PRE_GS	0.016534577	-0.33539249	-0.03253507	0.0190600895	-0.111168430
## PRE_MP	0.015534053	-0.28855828	-0.01783077	-0.0040057105	-0.074098536
## PRE_FGA	-0.100342569	0.14375073	0.12366593	0.3050992422	0.010059538
## PRE_FG_PCT	-0.004409603	-0.13322309	-0.23111057	-0.0781484906	0.275050775
## PRE_3PA	-0.254619740	0.10951615	0.04972839	0.1597272981	-0.063145956
## PRE_3P_PCT	-0.296272720	0.12555504	-0.25505483	-0.6921348003	0.238149213
## PRE_2PA	0.100909258	-0.04779157	0.12626039	0.0630074925	0.022293237
## PRE_2P_PCT	-0.025008678	-0.09948989	0.03965274	-0.0408271040	0.035244571
## PRE_eFG_PCT	-0.049976061	-0.09752382	0.01237576	-0.0373396158	0.032764705
## PRE_FTA	0.205143540	0.04317515	-0.18480518	-0.1013142886	0.019922747
## PRE_FT_PCT	-0.261178994	0.02336108	-0.33828357	-0.1089984569	-0.003053189
## PRE_ORB	0.124199677	-0.04014109	0.07343547	-0.1625242423	0.025815599
## PRE_DRB	-0.345438973	0.08949877	0.25515900	-0.1880550171	-0.431033802
## PRE_TRB	-0.277619833	0.07917485	0.06957106	0.0551909921	-0.111570151
## PRE_AST	0.310881381	-0.06803405	-0.13439517	-0.0227785634	0.035482104
## PRE_STL	0.158545817	0.08979451	-0.15790259	0.2435688666	0.381844051
## PRE_BLK	-0.036114530	-0.04979654	-0.06310273	0.0026893567	0.123028939
## PRE_TOV	0.531394851	0.18724542	0.12860582	-0.3867088692	-0.291267960
## PRE_PF	0.152259628	0.28034979	-0.09726541	-0.0614783636	0.162613277
## PRE_PTS	-0.109213441	0.11347422	-0.11912295	0.1513782274	0.160113006
## PRE_FG_pct_inc	0.044722428	0.45395118	-0.24968559	0.1021009545	-0.201073762
## PRE_3P_pct_inc	-0.075108836	0.27114166	0.49431413	-0.1656599680	0.269992950
## PRE_2P_pct_inc	0.037758904	0.43277509	-0.26247403	0.1596732804	-0.230500277
## PRE_FT_pct_inc	0.045372377	0.19061518	0.37416558	0.0154122996	0.399782204
##	PC11	PC12	PC13		
## WEIGHT	0.058519996	0.09333076	-0.028184110		
## HEIGHT_INCHES	-0.011694787	0.17386310	0.076222008		
## PRE_G	0.002715547	0.13765013	0.051480749		
## PRE_GS	0.027568844	0.14322979	0.024691520		
## PRE_MP	0.028870044	0.11442966	0.009178278		
## PRE_FGA	-0.076756280	0.25203787	-0.104767828		
## PRE_FG_PCT	0.056708376	-0.27041638	-0.010116838		
## PRE_3PA	-0.118367878	0.23071748	0.168132304		
## PRE_3P_PCT	-0.132992750	0.28816877	-0.346557121		


```
## PRE_2PA      0.028751024  0.15501850 -0.076811133
## PRE_2P_PCT   0.016604797  0.09011226  0.147592764
## PRE_eFG_PCT -0.009846565  0.09309186  0.167533579
## PRE_FTA      0.079548232 -0.39511675 -0.123032661
## PRE_FT_PCT  -0.005951696 -0.32729026  0.607281583
## PRE_ORB      0.095369951  0.06374772 -0.145015816
## PRE_DRB     -0.075628303 -0.09555228 -0.006762200
## PRE_TRB     -0.025164286 -0.02048343 -0.052993184
## PRE_AST     -0.058816769 -0.03062988  0.009038546
## PRE_STL      0.101423145  0.20489575 -0.103729875
## PRE_BLK     -0.055082361  0.25445469  0.332816936
## PRE_TOV     -0.140123639  0.15500691  0.244921273
## PRE_PF       0.066511609  0.31256945  0.375978211
## PRE_PTS     -0.013609522 -0.08498533  0.001161631
## PRE_FG_pct_inc 0.078072671 -0.08208679 -0.156071581
## PRE_3P_pct_inc 0.646933312 -0.10277365  0.122327463
## PRE_2P_pct_inc -0.024377950  0.07019345 -0.046466246
## PRE_FT_pct_inc -0.681751633 -0.24304954  0.051015208
```

```
# Run it
train_control <- trainControl(method = "cv", number = 10,
                             summaryFunction = multiClassSummary)

model_pca_cv <- train(response ~ ., data = train_data_pca,
                     method = "multinom",
                     trControl = train_control,
                     trace = FALSE)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.
```

```
cat("Cross-validated accuracy (PCA-based model, training set):",
    max(model_pca_cv$results$Accuracy), "\n")
```

```
## Cross-validated accuracy (PCA-based model, training set): 0.3734422
```

```
model_pca <- multinom(response ~ ., data = train_data_pca, trace = FALSE)

preds_pca <- predict(model_pca, newdata = test_data_pca)
preds_pca <- factor(preds_pca, levels = levels(test_data_pca$response))

cm_pca <- confusionMatrix(preds_pca, test_data_pca$response, mode = "prec_recall")
print(cm_pca)
```

```
## Confusion Matrix and Statistics
```

```
##
##              Reference
## Prediction   NoVol LowVol BelowAvgVol AverageVol AboveAvgVol HighVol
## NoVol        43     6         10         4         3         2
## LowVol        0     0          0          0          0         0
## BelowAvgVol   0     3          3          2          3         0
## AverageVol    2     3          3          7          5         0
```

```
## AboveAvgVol      3      5      17      27      37      29
## HighVol          2      4       5      10      24      31
##
## Overall Statistics
##
##           Accuracy : 0.413
##           95% CI : (0.356, 0.4717)
##       No Information Rate : 0.2457
##       P-Value [Acc > NIR] : 2.413e-10
##
##           Kappa : 0.2569
##
## McNemar's Test P-Value : 1.760e-09
##
## Statistics by Class:
##
##           Class: NoVol Class: LowVol Class: BelowAvgVol
## Precision          0.6324          NA          0.27273
## Recall             0.8600          0.00000          0.07895
## F1                 0.7288          NA          0.12245
## Prevalence         0.1706          0.07167          0.12969
## Detection Rate     0.1468          0.00000          0.01024
## Detection Prevalence 0.2321          0.00000          0.03754
## Balanced Accuracy   0.8786          0.50000          0.52379
##
##           Class: AverageVol Class: AboveAvgVol Class: HighVol
## Precision          0.35000          0.3136          0.4079
## Recall             0.14000          0.5139          0.5000
## F1                 0.20000          0.3895          0.4493
## Prevalence         0.17065          0.2457          0.2116
## Detection Rate     0.02389          0.1263          0.1058
## Detection Prevalence 0.06826          0.4027          0.2594
## Balanced Accuracy   0.54325          0.5737          0.6526
```

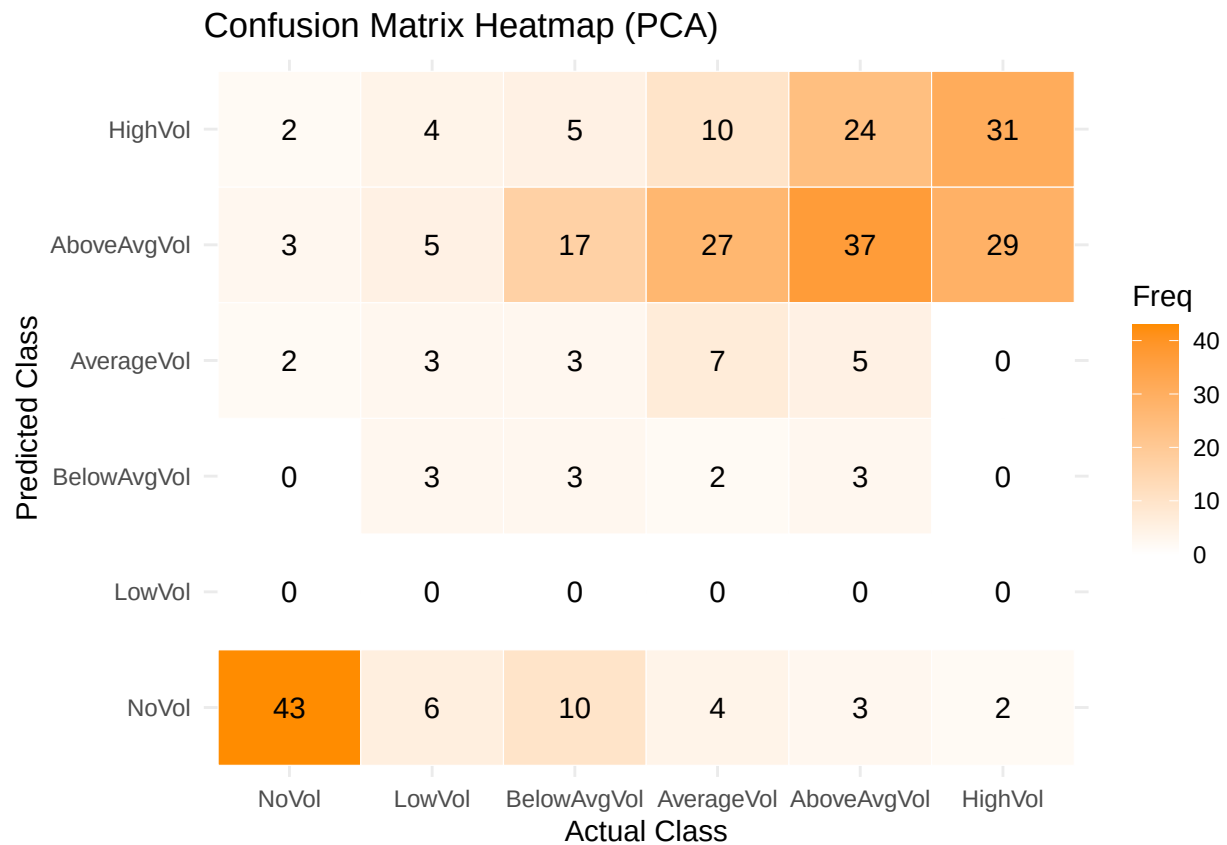
```
test_acc_pca <- mean(as.character(preds_pca) == as.character(test_data_pca$response))
cat("Test Accuracy (PCA-based model):", test_acc_pca, "\n")
```

```
## Test Accuracy (PCA-based model): 0.4129693
```

```
probs_pca <- predict(model_pca, newdata = test_data_pca, type = "probs")
probs_pca <- as.matrix(probs_pca)

colnames(probs_pca) <- colnames(probs_pca)
probs_pca <- probs_pca[, levels(test_data_pca$response), drop = FALSE]
```

```
cm_table <- as.data.frame(cm_pca$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "darkorange") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (PCA)",
       x = "Actual Class", y = "Predicted Class")
```



```
model_results <- rbind(model_results, data.frame(
  Model = "PCA",
  Accuracy = test_acc_pca,
  Precision = mean(cm_pca$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_pca$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_pca$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data_pca$response, probs_pca)
))
```

SVM Radial and Linear

```
set.seed(67)
# CV SVM
svm_linear <- train(
  response ~ .,
  data = train_data,
  method = "svmLinear2",
  trControl = trainControl(
    method = "cv",
    number = 10,
    classProbs = TRUE,
    summaryFunction = multiClassSummary
  ),
```

```

preProcess = c("center", "scale", "pca"),
tuneGrid = expand.grid(cost = c(0.01, 0.1, 1, 10)),
probability = TRUE
)

## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.

print(svm_linear)

## Support Vector Machines with Linear Kernel
##
## 1186 samples
## 27 predictor
## 6 classes: 'NoVol', 'LowVol', 'BelowAvgVol', 'AverageVol', 'AboveAvgVol', 'HighVol'
##
## Pre-processing: centered (27), scaled (27), principal component
## signal extraction (27)
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1067, 1067, 1067, 1068, 1067, 1067, ...
## Resampling results across tuning parameters:
##
## cost logLoss AUC prAUC Accuracy Kappa Mean_F1
## 0.01 1.509100 0.6792844 0.3141959 0.3846124 0.2095981 NaN
## 0.10 1.503194 0.6840327 0.3216547 0.3820772 0.2117968 NaN
## 1.00 1.505209 0.6831011 0.3194446 0.3803320 0.2114761 NaN
## 10.00 1.508640 0.6819518 0.3209769 0.3820484 0.2133861 NaN
## Mean_Sensitivity Mean_Specificity Mean_Pos_Pred_Value Mean_Neg_Pred_Value
## 0.3105115 0.8670944 NaN 0.8735801
## 0.3142701 0.8675695 NaN 0.8721886
## 0.3161824 0.8675378 NaN 0.8713327
## 0.3178491 0.8678122 NaN 0.8713699
## Mean_Precision Mean_Recall Mean_Detection_Rate Mean_Balanced_Accuracy
## NaN 0.3105115 0.06410207 0.5888030
## NaN 0.3142701 0.06367953 0.5909198
## NaN 0.3161824 0.06338867 0.5918601
## NaN 0.3178491 0.06367474 0.5928306
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was cost = 0.01.

cat("Best cost (C):", svm_linear$bestTune$C, "\n")

## Best cost (C):

cat("Cross-validated accuracy (training set):", max(svm_linear$results$Accuracy), "\n")

## Cross-validated accuracy (training set): 0.3846124

```

```

preds_svm_linear <- predict(svm_linear, newdata = test_data)
preds_svm_linear <- factor(preds_svm_linear, levels = levels(test_data$response))

cm_svm_linear <- confusionMatrix(preds_svm_linear, test_data$response, mode = "prec_recall")
print(cm_svm_linear)

```

```

## Confusion Matrix and Statistics
##
##
##           Reference
## Prediction   NoVol LowVol BelowAvgVol AverageVol AboveAvgVol HighVol
## NoVol        41     5         7         4         3         2
## LowVol        0     0         0         0         0         0
## BelowAvgVol   0     0         0         0         0         0
## AverageVol    0     1         0         0         0         0
## AboveAvgVol   9    11        27        40        53        42
## HighVol       0     4         4         6        16        18
##
## Overall Statistics
##
##           Accuracy : 0.3823
##           95% CI : (0.3263, 0.4406)
## No Information Rate : 0.2457
## P-Value [Acc > NIR] : 1.578e-07
##
##           Kappa : 0.2039
##
## Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##           Class: NoVol Class: LowVol Class: BelowAvgVol
## Precision           0.6613           NA           NA
## Recall              0.8200           0.00000           0.0000
## F1                  0.7321           NA           NA
## Prevalence          0.1706           0.07167           0.1297
## Detection Rate      0.1399           0.00000           0.0000
## Detection Prevalence 0.2116           0.00000           0.0000
## Balanced Accuracy    0.8668           0.50000           0.5000
##
##           Class: AverageVol Class: AboveAvgVol Class: HighVol
## Precision           0.000000           0.2912           0.37500
## Recall              0.000000           0.7361           0.29032
## F1                  NaN              0.4173           0.32727
## Prevalence          0.170648           0.2457           0.21160
## Detection Rate      0.000000           0.1809           0.06143
## Detection Prevalence 0.003413           0.6212           0.16382
## Balanced Accuracy    0.497942           0.5762           0.58023

```

```

test_acc_linear <- mean(as.character(preds_svm_linear) == as.character(test_data$response))
cat("Test Accuracy (SVM Linear):", test_acc_linear, "\n")

```

```

## Test Accuracy (SVM Linear): 0.3822526

```

```

probs_svm_linear <- predict(svm_linear, newdata = test_data, type = "prob")
probs_svm_linear <- as.matrix(probs_svm_linear)

colnames(probs_svm_linear) <- colnames(probs_svm_linear)
probs_svm_linear <- probs_svm_linear[, levels(test_data_pca$response), drop = FALSE]

model_results <- rbind(model_results, data.frame(
  Model = "SVM Linear",
  Accuracy = test_acc_linear,
  Precision = mean(cm_svm_linear$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_svm_linear$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_svm_linear$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data$response, probs_svm_linear)
))

```

```

# Repeat
svm_radial <- train(
  response ~ .,
  data = train_data,
  method = "svmRadial",
  trControl = trainControl(
    method = "cv",
    number = 10,
    classProbs = TRUE,
    summaryFunction = multiClassSummary
  ),
  tuneGrid = expand.grid(
    C = c(0.1, 1, 10),
    sigma = c(0.01, 0.05, 0.1)
  ),
  prob.model = TRUE
)

```

```

## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.

```

```
print(svm_radial)
```

```

## Support Vector Machines with Radial Basis Function Kernel
##
## 1186 samples
## 27 predictor
## 6 classes: 'NoVol', 'LowVol', 'BelowAvgVol', 'AverageVol', 'AboveAvgVol', 'HighVol'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1067, 1067, 1069, 1067, 1066, 1067, ...
## Resampling results across tuning parameters:
##
##  C      sigma  logLoss  AUC      prAUC      Accuracy  Kappa      Mean_F1
##  0.1  0.01   1.523221  0.6778607  0.3130310  0.3726001  0.1984290  NaN
##  0.1  0.05   1.540457  0.6740704  0.3066921  0.3776640  0.2038411  NaN

```

```
##      0.1  0.10  1.566786  0.6671746  0.2953044  0.3658637  0.1898579  NaN
##      1.0  0.01  1.504907  0.6893255  0.3241451  0.3894998  0.2159268  NaN
##      1.0  0.05  1.522568  0.6821665  0.3147043  0.3810679  0.2056882  NaN
##      1.0  0.10  1.551974  0.6675783  0.2968688  0.3675655  0.1889347  NaN
##     10.0  0.01  1.509057  0.6963007  0.3289518  0.3811175  0.2064056  NaN
##     10.0  0.05  1.582647  0.6532462  0.2855711  0.3616761  0.1797472  NaN
##     10.0  0.10  1.589084  0.6481721  0.2800583  0.3457085  0.1586046  NaN
## Mean_Sensitivity Mean_Specificity Mean_Pos_Pred_Value Mean_Neg_Pred_Value
##      0.3057122      0.8653656              NaN      0.8718037
##      0.3069392      0.8663387      0.2814631      0.8748142
##      0.2978454      0.8640994      0.2099280      0.8728062
##      0.3148333      0.8681611              NaN      0.8772496
##      0.3094682      0.8664702      0.3730154      0.8764327
##      0.2979489      0.8637757      0.3213590      0.8744241
##      0.3098696      0.8666253      0.3201850      0.8759536
##      0.2916855      0.8622553      0.3747724      0.8749317
##      0.2778655      0.8587577      0.2245549      0.8700378
## Mean_Precision Mean_Recall Mean_Detection_Rate Mean_Balanced_Accuracy
##              NaN      0.3057122      0.06210001      0.5855389
##      0.2814631      0.3069392      0.06294401      0.5866390
##      0.2099280      0.2978454      0.06097729      0.5809724
##              NaN      0.3148333      0.06491664      0.5914972
##      0.3730154      0.3094682      0.06351131      0.5879692
##      0.3213590      0.2979489      0.06126092      0.5808623
##      0.3201850      0.3098696      0.06351958      0.5882474
##      0.3747724      0.2916855      0.06027934      0.5769704
##      0.2245549      0.2778655      0.05761808      0.5683116
##
## Accuracy was used to select the optimal model using the largest value.
## The final values used for the model were sigma = 0.01 and C = 1.
```

```
cat("Best parameters: C =", svm_radial$bestTune$C,
    " sigma =", svm_radial$bestTune$sigma, "\n")
```

```
## Best parameters: C = 1  sigma = 0.01
```

```
cat("Cross-validated accuracy (training set):", max(svm_radial$results$Accuracy), "\n")
```

```
## Cross-validated accuracy (training set): 0.3894998
```

```
preds_svm_radial <- predict(svm_radial, newdata = test_data)
preds_svm_radial <- factor(preds_svm_radial, levels = levels(test_data$response))

cm_svm_radial <- confusionMatrix(preds_svm_radial, test_data$response, mode = "prec_recall")
print(cm_svm_radial)
```

```
## Confusion Matrix and Statistics
```

```
##
##              Reference
## Prediction  NoVol LowVol BelowAvgVol AverageVol AboveAvgVol HighVol
##   NoVol      43      6          10           3           4          2
##   LowVol       0      0           0           0           0          0
```

```
## BelowAvgVol      0      0      0      0      0      0
## AverageVol       0      0      1      2      1      0
## AboveAvgVol      7     13     24     41     58     46
## HighVol          0      2      3      4      9     14
```

```
##
## Overall Statistics
##
##           Accuracy : 0.3993
##           95% CI   : (0.3428, 0.4579)
##       No Information Rate : 0.2457
##       P-Value [Acc > NIR] : 4.943e-09
##
##           Kappa : 0.2264
##
## McNemar's Test P-Value : NA
```

```
##
## Statistics by Class:
##
##           Class: NoVol Class: LowVol Class: BelowAvgVol
## Precision          0.6324          NA          NA
## Recall              0.8600          0.00000          0.0000
## F1                  0.7288          NA          NA
## Prevalence          0.1706          0.07167          0.1297
## Detection Rate      0.1468          0.00000          0.0000
## Detection Prevalence 0.2321          0.00000          0.0000
## Balanced Accuracy    0.8786          0.50000          0.5000
##
##           Class: AverageVol Class: AboveAvgVol Class: HighVol
## Precision          0.500000          0.3069          0.43750
## Recall              0.040000          0.8056          0.22581
## F1                  0.074074          0.4444          0.29787
## Prevalence          0.170648          0.2457          0.21160
## Detection Rate      0.006826          0.1980          0.04778
## Detection Prevalence 0.013652          0.6451          0.10922
## Balanced Accuracy    0.515885          0.6064          0.57394
```

```
test_acc_radial <- mean(as.character(preds_svm_radial) == as.character(test_data$response))
cat("Test Accuracy (SVM Radial):", test_acc_radial, "\n")
```

```
## Test Accuracy (SVM Radial): 0.3993174
```

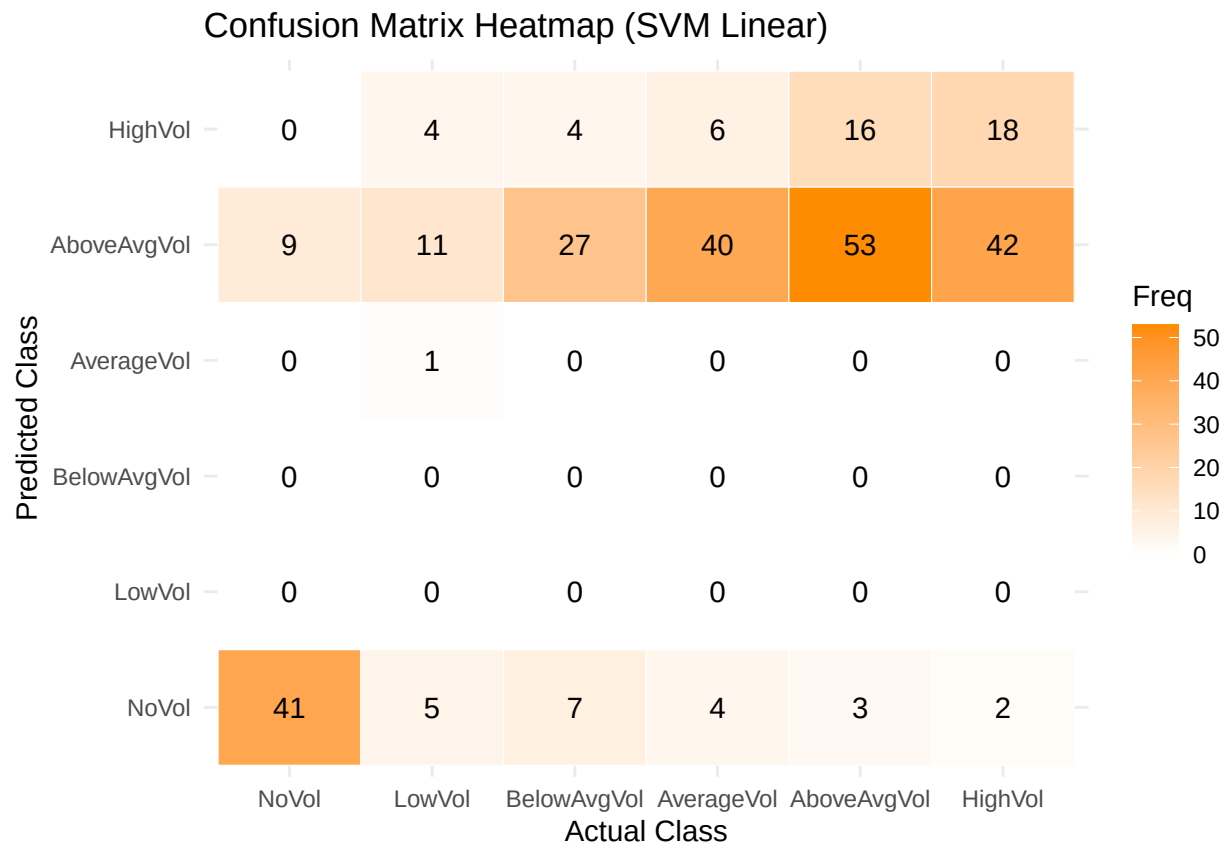
```
probs_svm_radial <- predict(svm_radial, newdata = test_data, type = "prob")
probs_svm_radial <- as.matrix(probs_svm_radial)

colnames(probs_svm_radial) <- colnames(probs_svm_radial)
probs_svm_radial <- probs_svm_radial[, levels(test_data$response), drop = FALSE]

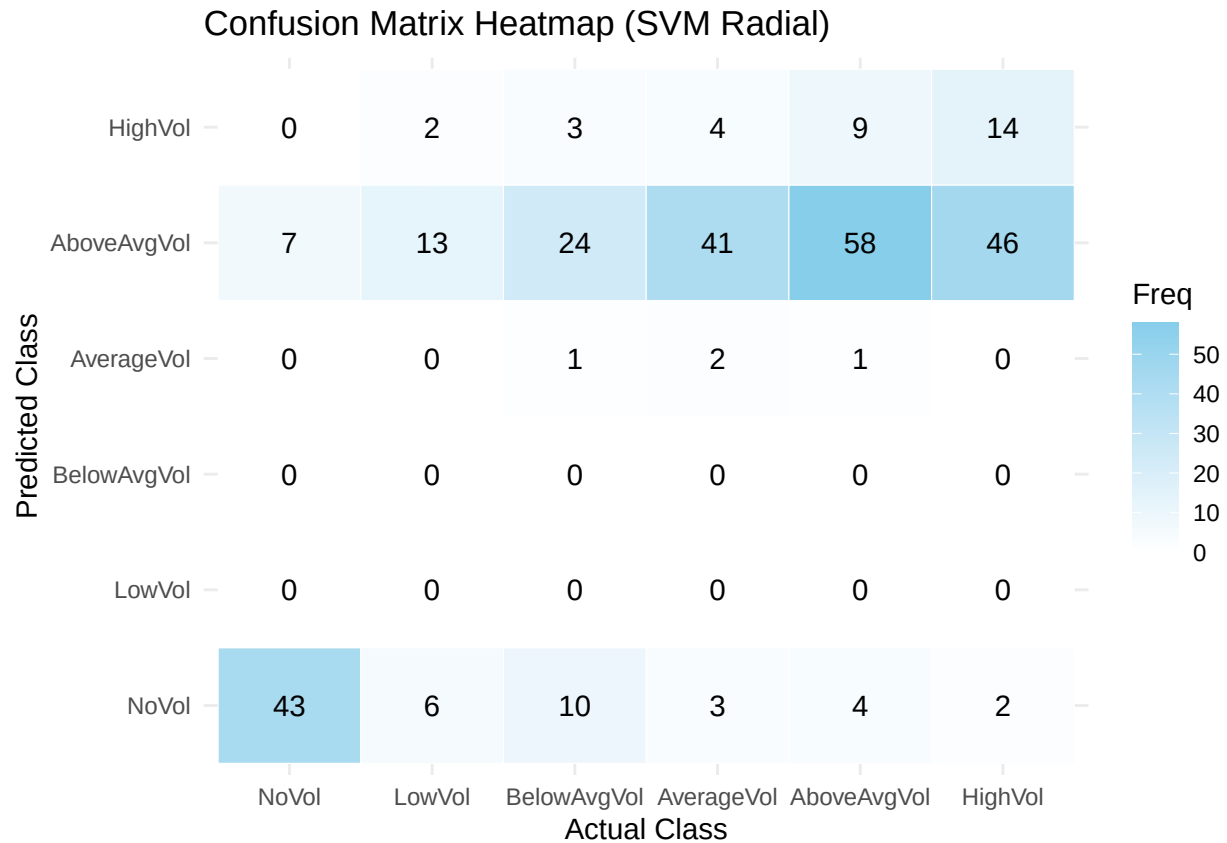
model_results <- rbind(model_results, data.frame(
  Model = "SVM Radial Kernel",
  Accuracy = test_acc_radial,
  Precision = mean(cm_svm_radial$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_svm_radial$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_svm_radial$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data$response, probs_svm_radial)
))
```



```
cm_table <- as.data.frame(cm_svm_linear$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "darkorange") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (SVM Linear)",
       x = "Actual Class", y = "Predicted Class")
```



```
cm_table <- as.data.frame(cm_svm_radial$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "skyblue") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (SVM Radial)",
       x = "Actual Class", y = "Predicted Class")
```



Random Forest

```

set.seed(6767)
# CV for random forest
train_control <- trainControl(method = "cv", number = 10)

tune_grid <- expand.grid(mtry = c(2, 5, 7, 10, 15, 20))

results <- list()
ntree_values <- c(100, 250, 500)

for (nt in ntree_values) {
  rf_model <- train(response ~ ., data = train_data,
                    method = "rf",
                    trControl = train_control,
                    tuneGrid = tune_grid,
                    ntree = nt)

  results[[paste0("ntree_", nt)]] <- rf_model

  preds <- predict(rf_model, newdata = test_data)
  cm <- confusionMatrix(preds, test_data$response, mode = "prec_recall")

```

```

# Print results
cat("\nResults for ntree =", nt, "\n")
print(rf_model$results)
cat("Best mtry:", rf_model$bestTune$mtry, "\n")
cat("CV Accuracy:", max(rf_model$results$Accuracy), "\n")
}

```

```

##
## Results for ntree = 100 :
##   mtry Accuracy      Kappa AccuracySD      KappaSD
## 1    2 0.3524503 0.1864433 0.02768472 0.03423824
## 2    5 0.3540747 0.1903051 0.02441391 0.02945340
## 3    7 0.3609108 0.2007208 0.02577532 0.03234038
## 4   10 0.3516243 0.1883746 0.03087478 0.03710596
## 5   15 0.3466037 0.1827515 0.02632486 0.03254602
## 6   20 0.3657900 0.2081381 0.03330479 0.03823543
## Best mtry: 20
## CV Accuracy: 0.36579
##
## Results for ntree = 250 :
##   mtry Accuracy      Kappa AccuracySD      KappaSD
## 1    2 0.3659291 0.2023273 0.02561935 0.03215853
## 2    5 0.3650536 0.2012531 0.02762828 0.03467515
## 3    7 0.3582379 0.1944616 0.02937631 0.03614214
## 4   10 0.3684006 0.2075065 0.02704106 0.03137079
## 5   15 0.3675532 0.2073282 0.02551926 0.03270529
## 6   20 0.3583160 0.1960604 0.01510551 0.02026674
## Best mtry: 10
## CV Accuracy: 0.3684006
##
## Results for ntree = 500 :
##   mtry Accuracy      Kappa AccuracySD      KappaSD
## 1    2 0.3559075 0.1884530 0.03343548 0.04320946
## 2    5 0.3786899 0.2183361 0.04289489 0.05327537
## 3    7 0.3677722 0.2051522 0.04201090 0.05251766
## 4   10 0.3601454 0.1964154 0.04568556 0.05798442
## 5   15 0.3686123 0.2082473 0.04870355 0.06242953
## 6   20 0.3651009 0.2045421 0.03176198 0.03960543
## Best mtry: 5
## CV Accuracy: 0.3786899

```

```

# Use best model as fit
rf_model <- train(response ~ ., data = train_data,
                  method = "rf",
                  trControl = trainControl(method = "cv", number = 10,
                                           summaryFunction = multiClassSummary),
                  tuneGrid = data.frame(mtry = results[["ntree_500"]]$bestTune$mtry),
                  ntree = 500)

```

```

## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.

```

```

preds_rf <- predict(rf_model, newdata = test_data)
cm_rf <- confusionMatrix(preds_rf, test_data$response, mode = "prec_recall")

probs_rf <- predict(rf_model, newdata = test_data, type = "prob")
probs_rf <- as.matrix(probs_rf)

colnames(probs_rf) <- colnames(probs_rf)
probs_rf <- probs_rf[, levels(test_data$response), drop = FALSE]

model_results <- rbind(model_results, data.frame(
  Model = "Base Random Forest",
  Accuracy = mean(preds_rf == test_data$response),
  Precision = mean(cm_rf$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_rf$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_rf$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data$response, probs_rf)
))

print(rf_model)

```

```

## Random Forest
##
## 1186 samples
## 27 predictor
## 6 classes: 'NoVol', 'LowVol', 'BelowAvgVol', 'AverageVol', 'AboveAvgVol', 'HighVol'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1068, 1068, 1069, 1067, 1067, 1069, ...
## Resampling results:
##
## Accuracy Kappa Mean_F1 Mean_Sensitivity Mean_Specificity
## 0.3516662 0.1854508 0.3280593 0.3071768 0.8632893
## Mean_Pos_Pred_Value Mean_Neg_Pred_Value Mean_Precision Mean_Recall
## 0.2949431 0.8650158 0.2949431 0.3071768
## Mean_Detection_Rate Mean_Balanced_Accuracy
## 0.05861103 0.585233
##
## Tuning parameter 'mtry' was held constant at a value of 5

```

```
print(cm_rf)
```

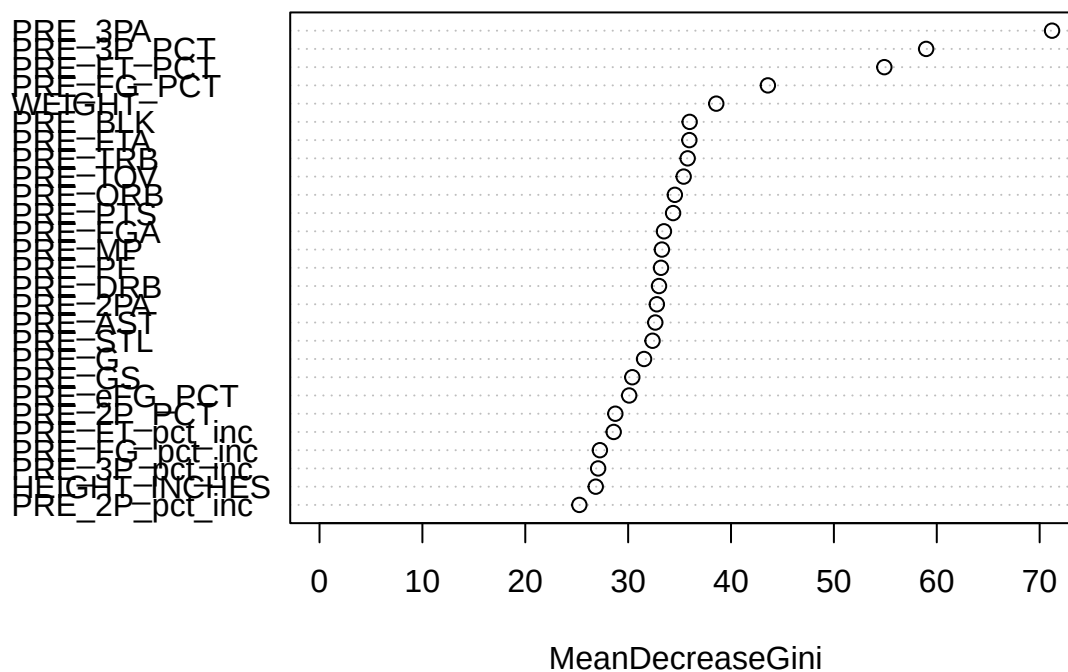
```

## Confusion Matrix and Statistics
##
##
## Prediction Reference
## Prediction NoVol LowVol BelowAvgVol AverageVol AboveAvgVol HighVol
## NoVol 39 6 9 1 2 1
## LowVol 2 0 1 0 1 0
## BelowAvgVol 2 1 2 5 2 0
## AverageVol 3 2 9 13 10 5
## AboveAvgVol 3 9 12 24 33 27
## HighVol 1 3 5 7 24 29

```

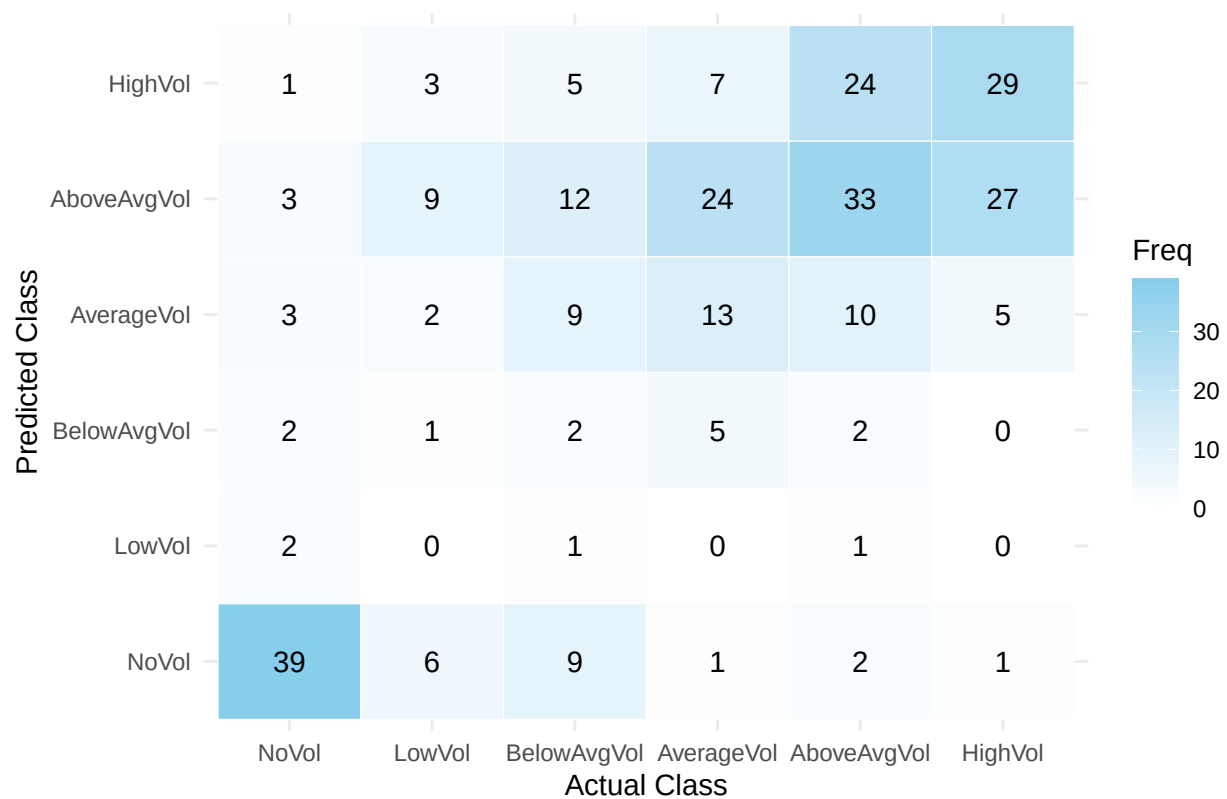
```
##
## Overall Statistics
##
##           Accuracy : 0.3959
##           95% CI : (0.3395, 0.4544)
##           No Information Rate : 0.2457
##           P-Value [Acc > NIR] : 1.016e-08
##
##           Kappa : 0.2402
##
## Mcnemar's Test P-Value : 0.0007315
##
## Statistics by Class:
##
##           Class: NoVol Class: LowVol Class: BelowAvgVol
## Precision           0.6724           0.00000           0.166667
## Recall               0.7800           0.00000           0.052632
## F1                   0.7222           NaN             0.080000
## Prevalence           0.1706           0.07167           0.129693
## Detection Rate        0.1331           0.00000           0.006826
## Detection Prevalence  0.1980           0.01365           0.040956
## Balanced Accuracy     0.8509           0.49265           0.506708
##
##           Class: AverageVol Class: AboveAvgVol Class: HighVol
## Precision           0.30952           0.3056           0.42029
## Recall               0.26000           0.4583           0.46774
## F1                   0.28261           0.3667           0.44275
## Prevalence           0.17065           0.2457           0.21160
## Detection Rate        0.04437           0.1126           0.09898
## Detection Prevalence  0.14334           0.3686           0.23549
## Balanced Accuracy     0.57033           0.5595           0.64729

# Print Variable Importance Plot
varImpPlot(rf_model$finalModel)
```

rf_model\$finalModel

```
cm_table <- as.data.frame(cm_rf$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "skyblue") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (Random Forest)",
       x = "Actual Class", y = "Predicted Class")
```

Confusion Matrix Heatmap (Random Forest)



Upsampling

```
set.seed(67)

# Cross validation for upsampling
train_control <- trainControl(method = "cv", number = 10,
                              sampling = "up",
                              summaryFunction = multiClassSummary)

rf_model_balanced <- train(response ~ ., data = train_data,
                           method = "rf",
                           trControl = train_control,
                           tuneGrid = data.frame(mtry = c(2, 5, 10, 12, 15, 20)),
                           ntree = 500)

## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.

print(rf_model_balanced)

## Random Forest
##
```

```
## 1186 samples
## 27 predictor
## 6 classes: 'NoVol', 'LowVol', 'BelowAvgVol', 'AverageVol', 'AboveAvgVol', 'HighVol'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1067, 1067, 1067, 1068, 1067, 1067, ...
## Additional sampling using up-sampling
##
## Resampling results across tuning parameters:
##
## mtry Accuracy Kappa Mean_F1 Mean_Sensitivity Mean_Specificity
## 2 0.3491459 0.1908180 0.3557709 0.3129477 0.8644594
## 5 0.3465533 0.1886136 0.3172293 0.3152955 0.8640807
## 10 0.3635253 0.2120695 0.3441696 0.3342385 0.8681345
## 12 0.3542671 0.2015712 0.3212874 0.3254409 0.8663318
## 15 0.3508120 0.1990182 0.3210947 0.3240434 0.8661028
## 20 0.3415689 0.1880403 0.3314672 0.3158737 0.8642310
## Mean_Pos_Pred_Value Mean_Neg_Pred_Value Mean_Precision Mean_Recall
## 0.2900154 0.8670320 0.2900154 0.3129477
## 0.3033359 0.8656065 0.3033359 0.3152955
## 0.3332752 0.8694577 0.3332752 0.3342385
## 0.3163397 0.8676956 0.3163397 0.3254409
## 0.3115623 0.8672455 0.3115623 0.3240434
## 0.3033351 0.8653037 0.3033351 0.3158737
## Mean_Detection_Rate Mean_Balanced_Accuracy
## 0.05819099 0.5887036
## 0.05775889 0.5896881
## 0.06058755 0.6011865
## 0.05904452 0.5958864
## 0.05846866 0.5950731
## 0.05692815 0.5900523
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was mtry = 10.
```

```
cat("Best mtry:", rf_model_balanced$bestTune$mtry, "\n")
```

```
## Best mtry: 10
```

```
cat("Cross-validated accuracy:", max(rf_model_balanced$results$Accuracy), "\n")
```

```
## Cross-validated accuracy: 0.3635253
```

```
preds_rf_up <- predict(rf_model_balanced, newdata = test_data)
cm_rf_up <- confusionMatrix(preds_rf_up, test_data$response, mode = "prec_recall")

probs_rf_up <- predict(rf_model_balanced, newdata = test_data, type = "prob")
probs_rf_up <- as.matrix(probs_rf_up)

colnames(probs_rf_up) <- colnames(probs_rf_up)
probs_rf_up <- probs_rf_up[, levels(test_data$response), drop = FALSE]
```



```

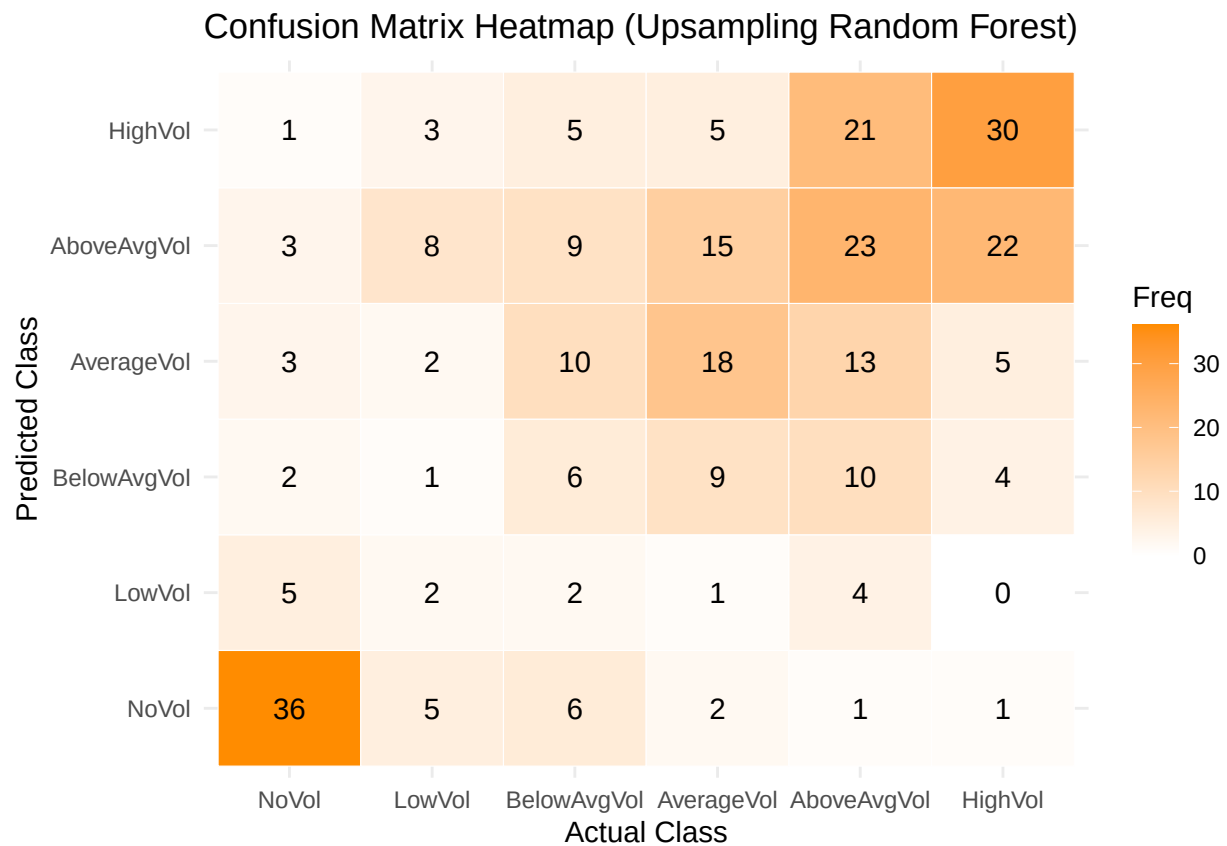
model_results <- rbind(model_results, data.frame(
  Model = "Upsampling Random Forest",
  Accuracy = mean(preds_rf_up == test_data$response),
  Precision = mean(cm_rf_up$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_rf_up$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_rf_up$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data$response, probs_rf_up)
))

```

```

cm_table <- as.data.frame(cm_rf_up$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "darkorange") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (Upsampling Random Forest)",
       x = "Actual Class", y = "Predicted Class")

```



Class Weights

```

set.seed(12)
# Set class weights

```

```

class_counts <- table(train_data$response)
class_weights <- 1 / class_counts
class_weights <- class_weights / sum(class_weights)

# CV for class weights
rf_weighted <- randomForest(response ~ ., data = train_data,
                             ntree = 500,
                             mtry = 7,
                             importance = TRUE,
                             classwt = class_weights)

preds_rf_weighted <- predict(rf_weighted, newdata = test_data)

cm_rf_weighted <- confusionMatrix(preds_rf_weighted, test_data$response, mode = "prec_recall")
accuracy_rf_weighted <- mean(as.character(preds_rf_weighted) == as.character(test_data$response))
cat("Test Accuracy:", accuracy_rf_weighted, "\n")

```

```
## Test Accuracy: 0.4232082
```

```
print(cm_rf_weighted)
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction   NoVol LowVol BelowAvgVol AverageVol AboveAvgVol HighVol
## NoVol        40     6         9         3         2         1
## LowVol        1     0         0         0         1         0
## BelowAvgVol    1     2         2         2         2         2
## AverageVol     2     1        10        14         8         3
## AboveAvgVol    5     9        11        22        40        28
## HighVol       1     3         6         9        19        28
##
## Overall Statistics
##
##              Accuracy : 0.4232
##              95% CI : (0.366, 0.482)
##      No Information Rate : 0.2457
##      P-Value [Acc > NIR] : 2.169e-11
##
##              Kappa : 0.2725
##
## Mcnemar's Test P-Value : 1.976e-05
##
## Statistics by Class:
##
##              Class: NoVol Class: LowVol Class: BelowAvgVol
## Precision          0.6557      0.000000      0.181818
## Recall              0.8000      0.000000      0.052632
## F1                  0.7207      NaN          0.081633
## Prevalence          0.1706      0.071672      0.129693
## Detection Rate      0.1365      0.000000      0.006826
## Detection Prevalence 0.2082      0.006826      0.037543

```

## Balanced Accuracy	0.8568	0.496324	0.508669
##	Class: AverageVol	Class: AboveAvgVol	Class: HighVol
## Precision	0.36842	0.3478	0.42424
## Recall	0.28000	0.5556	0.45161
## F1	0.31818	0.4278	0.43750
## Prevalence	0.17065	0.2457	0.21160
## Detection Rate	0.04778	0.1365	0.09556
## Detection Prevalence	0.12969	0.3925	0.22526
## Balanced Accuracy	0.59062	0.6081	0.64356

```

probs_rf_weighted <- predict(rf_weighted, newdata = test_data, type = "prob")
probs_rf_weighted <- as.matrix(probs_rf_weighted)

colnames(probs_rf_weighted) <- colnames(probs_rf_weighted)
probs_rf_weighted <- probs_rf_weighted[, levels(test_data$response), drop = FALSE]

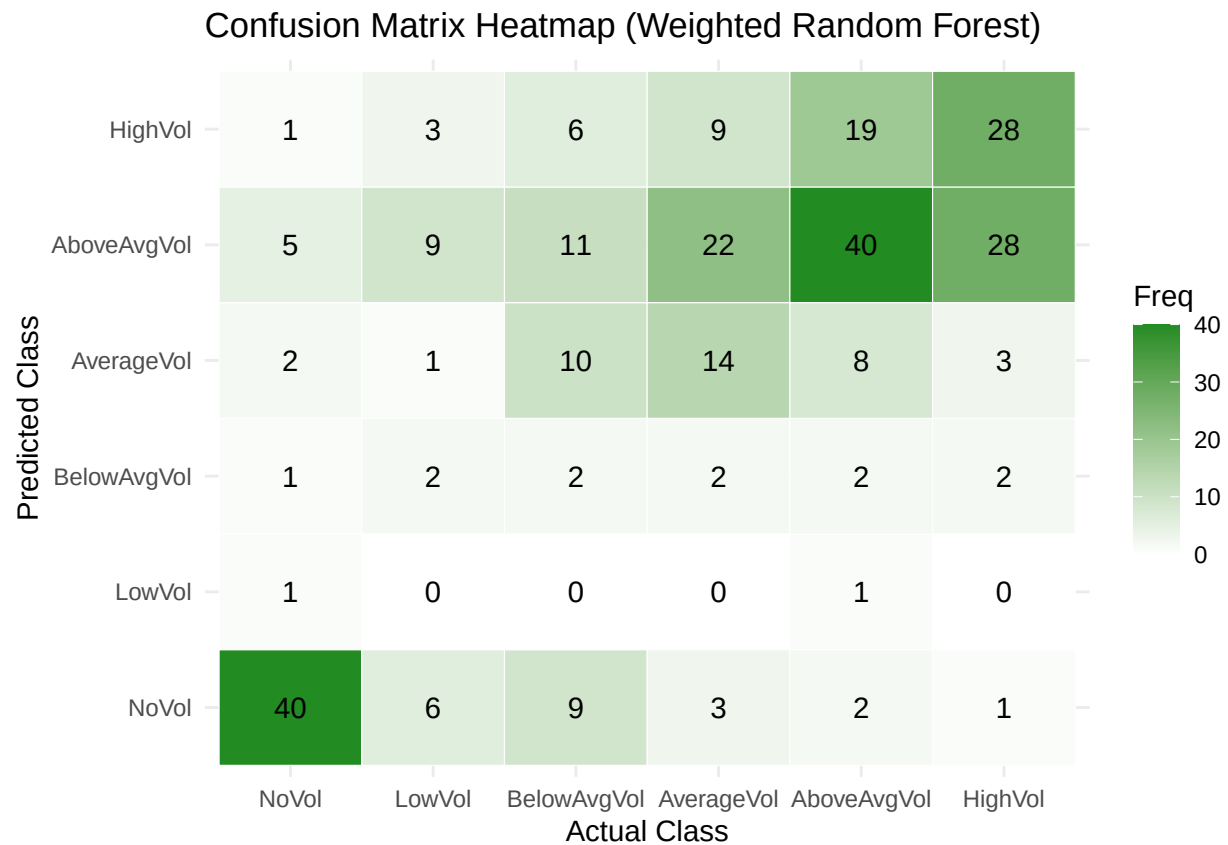
model_results <- rbind(model_results, data.frame(
  Model = "Weighted Random Forest",
  Accuracy = accuracy_rf_weighted,
  Precision = mean(cm_rf_weighted$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_rf_weighted$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_rf_weighted$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data$response, probs_rf_weighted)
))

```

```

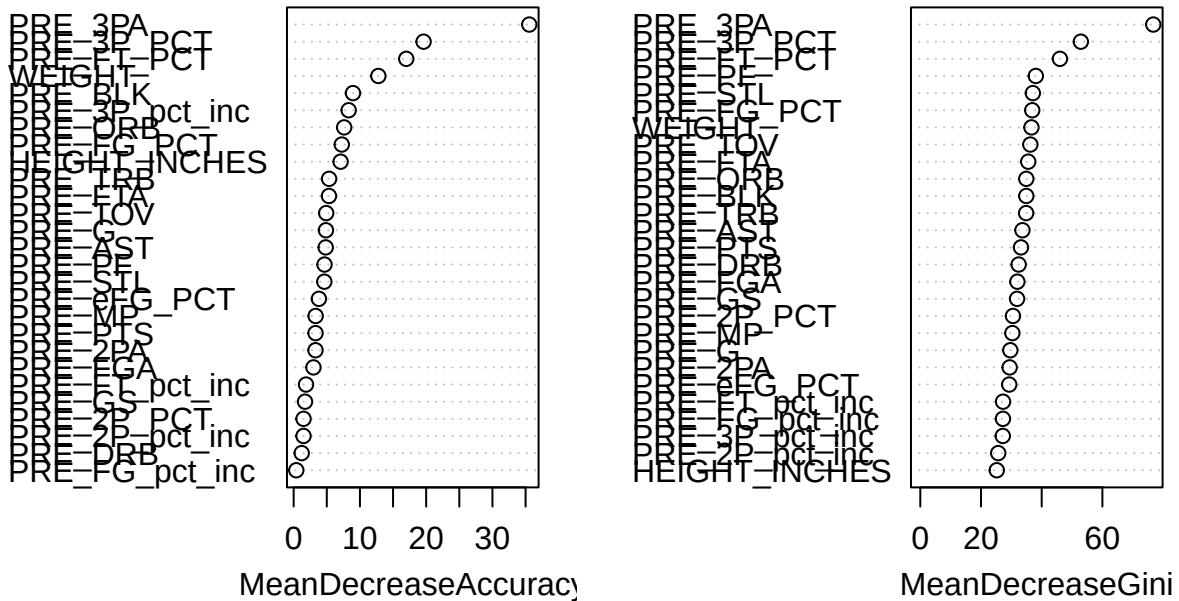
cm_table <- as.data.frame(cm_rf_weighted$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "forestgreen") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (Weighted Random Forest)",
       x = "Actual Class", y = "Predicted Class")

```



```
# Original best model, so we check the VarImpPlot
varImpPlot(rf_weighted, main="RF Weighted Variable Importance")
```

RF Weighted Variable Importance



```
Original best model, so we print a pretty tree single_tree <- randomForest::getTree(rf_weighted, k = 1,
labelVar = TRUE)
```

```
library(rpart) library(rpart.plot)
```

```
to_rpart <- function(tree, data) { response <- all.vars(formula(rf_weighted))[1] form <- as.formula(paste(response,
"~ .")) rpart(form, data = data, control = rpart.control(cp = 0.0001)) }
```

```
rpart_tree <- to_rpart(single_tree, train_data) rpart.plot( rpart_tree, type = 4, extra = 101, fallen.leaves
= TRUE, box.palette = "RdYlGn", branch.lty = 3, shadow.col = "gray", main = "Decision Tree Extracted
from Random Forest" )
```

XGBoost

```
set.seed(9999)

# Cross Validation for XGBoost
train_control <- trainControl(method = "cv", number = 10)
tune_grid <- expand.grid(
  nrounds = c(100, 250, 500),
  max_depth = c(3, 5, 7),
  eta = c(0.01, 0.1, 0.3),
  gamma = 0,
  colsample_bytree = 0.8,
  min_child_weight = 1,
```


[illegible]

[illegible]


```

## [11:51:52] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:51:58] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:51:58] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:52:07] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:52:07] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:52:18] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:52:18] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:52:23] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:52:23] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:52:29] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:52:29] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:52:36] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:52:36] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:52:42] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:52:42] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:52:53] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:52:53] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:53:10] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:53:10] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:53:16] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:53:16] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:53:25] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:53:25] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:53:35] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:53:35] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:53:41] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:53:41] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:53:47] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:53:47] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:53:54] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [11:53:54] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead

```

```
print(xgb_model)
```

```

## eXtreme Gradient Boosting
##
## 1186 samples
## 27 predictor
## 6 classes: 'NoVol', 'LowVol', 'BelowAvgVol', 'AverageVol', 'AboveAvgVol', 'HighVol'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1067, 1068, 1069, 1068, 1068, 1067, ...
## Resampling results across tuning parameters:
##
##  eta    max_depth  nrounds  Accuracy  Kappa
##  0.01    3          100      0.3843504  0.2199239
##  0.01    3          250      0.3724504  0.2066783
##  0.01    3          500      0.3665895  0.2033059
##  0.01    5          100      0.3673647  0.2027578
##  0.01    5          250      0.3556781  0.1903617
##  0.01    5          500      0.3489334  0.1839881
##  0.01    7          100      0.3530857  0.1870333
##  0.01    7          250      0.3565181  0.1930132

```

```
## 0.01 7 500 0.3455500 0.1799582
## 0.10 3 100 0.3396330 0.1733299
## 0.10 3 250 0.3345905 0.1693221
## 0.10 3 500 0.3202332 0.1528697
## 0.10 5 100 0.3337082 0.1674823
## 0.10 5 250 0.3302542 0.1640475
## 0.10 5 500 0.3235379 0.1576901
## 0.10 7 100 0.3505297 0.1891518
## 0.10 7 250 0.3438272 0.1819934
## 0.10 7 500 0.3438200 0.1830695
## 0.30 3 100 0.3168425 0.1487565
## 0.30 3 250 0.3076271 0.1385857
## 0.30 3 500 0.3000138 0.1295095
## 0.30 5 100 0.3193639 0.1515280
## 0.30 5 250 0.3311575 0.1680860
## 0.30 5 500 0.3311432 0.1688161
## 0.30 7 100 0.3278252 0.1661643
## 0.30 7 250 0.3236730 0.1614525
## 0.30 7 500 0.3236734 0.1622561
##
## Tuning parameter 'gamma' was held constant at a value of 0
## Tuning
##
## Tuning parameter 'min_child_weight' was held constant at a value of 1
##
## Tuning parameter 'subsample' was held constant at a value of 0.8
## Accuracy was used to select the optimal model using the largest value.
## The final values used for the model were nrounds = 100, max_depth = 3, eta
## = 0.01, gamma = 0, colsample_bytree = 0.8, min_child_weight = 1 and
## subsample = 0.8.
```

```
cat("Best parameters:\n")
```

```
## Best parameters:
```

```
print(xgb_model$bestTune)
```

```
## nrounds max_depth eta gamma colsample_bytree min_child_weight subsample
## 1 100 3 0.01 0 0.8 1 0.8
```

```
cat("Cross-validated accuracy:", max(xgb_model$results$Accuracy), "\n")
```

```
## Cross-validated accuracy: 0.3843504
```

```
preds_xgb <- predict(xgb_model, newdata = test_data)
preds_xgb <- factor(preds_xgb, levels = levels(test_data$response))
```

```
cm_xgb <- confusionMatrix(preds_xgb, test_data$response)
print(cm_xgb)
```

```
## Confusion Matrix and Statistics
```

```

##
##           Reference
## Prediction   NoVol LowVol BelowAvgVol AverageVol AboveAvgVol HighVol
##   NoVol       39    4         8         3         1         1
##   LowVol       1    0         0         1         1         0
##   BelowAvgVol  1    1         0         1         1         1
##   AverageVol   3    3         5        11        11         2
##   AboveAvgVol  5   11        20        25        38        31
##   HighVol      1    2         5         9        20        27
##
## Overall Statistics
##
##           Accuracy : 0.3925
##           95% CI : (0.3362, 0.451)
##   No Information Rate : 0.2457
##   P-Value [Acc > NIR] : 2.059e-08
##
##           Kappa : 0.2298
##
##   McNemar's Test P-Value : 8.080e-07
##
## Statistics by Class:
##
##           Class: NoVol Class: LowVol Class: BelowAvgVol
## Sensitivity           0.7800           0.00000           0.00000
## Specificity           0.9300           0.98897           0.98039
## Pos Pred Value        0.6964           0.00000           0.00000
## Neg Pred Value        0.9536           0.92759           0.86806
## Prevalence            0.1706           0.07167           0.12969
## Detection Rate        0.1331           0.00000           0.00000
## Detection Prevalence  0.1911           0.01024           0.01706
## Balanced Accuracy     0.8550           0.49449           0.49020
##
##           Class: AverageVol Class: AboveAvgVol Class: HighVol
## Sensitivity           0.22000           0.5278           0.43548
## Specificity           0.90123           0.5837           0.83983
## Pos Pred Value        0.31429           0.2923           0.42188
## Neg Pred Value        0.84884           0.7914           0.84716
## Prevalence            0.17065           0.2457           0.21160
## Detection Rate        0.03754           0.1297           0.09215
## Detection Prevalence  0.11945           0.4437           0.21843
## Balanced Accuracy     0.56062           0.5557           0.63766

```

```

set.seed(9999)
# Another cross validation
trainControl(method = "cv", number = 10, summaryFunction = multiClassSummary)

```

```

## $method
## [1] "cv"
##
## $number
## [1] 10
##
## $repeats
## [1] NA

```

```

##
## $search
## [1] "grid"
##
## $p
## [1] 0.75
##
## $initialWindow
## NULL
##
## $horizon
## [1] 1
##
## $fixedWindow
## [1] TRUE
##
## $skip
## [1] 0
##
## $verboseIter
## [1] FALSE
##
## $returnData
## [1] TRUE
##
## $returnResamp
## [1] "final"
##
## $savePredictions
## [1] FALSE
##
## $classProbs
## [1] FALSE
##
## $summaryFunction
## function (data, lev = NULL, model = NULL)
## {
##   if (!all(levels(data[, "pred"]) == levels(data[, "obs"])))
##     stop("levels of observed and predicted data do not match")
##   has_class_probs <- all(lev %in% colnames(data))
##   if (has_class_probs) {
##     lloss <- mnLogLoss(data = data, lev = lev, model = model)
##     requireNamespaceQuietStop("pROC")
##     requireNamespaceQuietStop("MLmetrics")
##     prob_stats <- lapply(levels(data[, "pred"]), function(x) {
##       obs <- ifelse(data[, "obs"] == x, 1, 0)
##       prob <- data[, x]
##       roc_auc <- try(pROC::roc(obs, data[, x], direction = "<",
##         quiet = TRUE), silent = TRUE)
##       roc_auc <- if (inherits(roc_auc, "try-error"))
##         NA
##       else roc_auc$auc
##     })
##     pr_auc <- try(MLmetrics::PRAUC(y_pred = data[, x],
##       y_true = obs), silent = TRUE)

```

```

##           if (inherits(pr_auc, "try-error"))
##             pr_auc <- NA
##           res <- c(ROC = roc_auc, AUC = pr_auc)
##           return(res)
##         })
##       prob_stats <- do.call("rbind", prob_stats)
##       prob_stats <- colMeans(prob_stats, na.rm = TRUE)
##     }
##     CM <- confusionMatrix(data[, "pred"], data[, "obs"], mode = "everything")
##     if (length(levels(data[, "pred"])) == 2) {
##       class_stats <- CM$byClass
##     }
##     else {
##       class_stats <- colMeans(CM$byClass)
##       names(class_stats) <- paste("Mean", names(class_stats))
##     }
##     overall_stats <- if (has_class_probs)
##       c(CM$overall, logLoss = as.numeric(lloss), AUC = unname(prob_stats["ROC"]),
##         prAUC = unname(prob_stats["AUC"]))
##     else CM$overall
##     stats <- c(overall_stats, class_stats)
##     stats <- stats[!names(stats) %in% c("AccuracyNull", "AccuracyLower",
##       "AccuracyUpper", "AccuracyPValue", "McnemarPValue", "Mean Prevalence",
##       "Mean Detection Prevalence")]
##     names(stats) <- gsub("[:blank:]]+", "_", names(stats))
##     stat_list <- c("Accuracy", "Kappa", "Mean_F1", "Mean_Sensitivity",
##       "Mean_Specificity", "Mean_Pos_Pred_Value", "Mean_Neg_Pred_Value",
##       "Mean_Precision", "Mean_Recall", "Mean_Detection_Rate",
##       "Mean_Balanced_Accuracy")
##     if (has_class_probs)
##       stat_list <- c("logLoss", "AUC", "prAUC", stat_list)
##     if (length(levels(data[, "pred"])) == 2)
##       stat_list <- gsub("^Mean_", "", stat_list)
##     stats <- stats[c(stat_list)]
##     return(stats)
##   }
## <bytecode: 0x000001952f17cd58>
## <environment: namespace:caret>
##
## $selectionFunction
## [1] "best"
##
## $preProcOptions
## $preProcOptions$thresh
## [1] 0.95
##
## $preProcOptions$ICAcomp
## [1] 3
##
## $preProcOptions$k
## [1] 5
##
## $preProcOptions$freqCut
## [1] 19

```

```

##
## $preProcOptions$uniqueCut
## [1] 10
##
## $preProcOptions$cutoff
## [1] 0.9
##
##
## $sampling
## NULL
##
## $index
## NULL
##
## $indexOut
## NULL
##
## $indexFinal
## NULL
##
## $timingSamps
## [1] 0
##
## $predictionBounds
## [1] FALSE FALSE
##
## $seeds
## [1] NA
##
## $adaptive
## $adaptive$min
## [1] 5
##
## $adaptive$alpha
## [1] 0.05
##
## $adaptive$method
## [1] "gls"
##
## $adaptive$complete
## [1] TRUE
##
##
## $trim
## [1] FALSE
##
## $allowParallel
## [1] TRUE

```

```

tune_grid <- expand_grid(
  nrounds = 100,
  max_depth = 3,
  eta = 0.01,
  gamma = 0,

```

```

    colsample_bytree = 0.8,
    min_child_weight = 1,
    subsample = 0.8
)

xgb_model <- train(response ~ ., data = train_data,
  method = "xgbTree",
  trControl = trainControl(method = "cv", number = 10),
  tuneGrid = tune_grid,
  verbose = FALSE)

cat("Cross-validated accuracy:", max(xgb_model$results$Accuracy), "\n")

## Cross-validated accuracy: 0.3826981

preds_xgb <- predict(xgb_model, newdata = test_data)
preds_xgb <- factor(preds_xgb, levels = levels(test_data$response))

cm_xgb <- confusionMatrix(preds_xgb, test_data$response, mode = "prec_recall")
print(cm_xgb)

```

```

## Confusion Matrix and Statistics
##
##               Reference
## Prediction   NoVol LowVol BelowAvgVol AverageVol AboveAvgVol HighVol
## NoVol         39     5         8         2         1         1
## LowVol         1     0         1         0         1         0
## BelowAvgVol    1     1         1         2         1         1
## AverageVol     3     2         4        11         9         2
## AboveAvgVol    5    11        18        26        39        27
## HighVol        1     2         6         9        21        31
##
## Overall Statistics
##
##               Accuracy : 0.413
##               95% CI   : (0.356, 0.4717)
##   No Information Rate : 0.2457
##   P-Value [Acc > NIR] : 2.413e-10
##
##               Kappa   : 0.2562
##
## Mcnemar's Test P-Value : 1.112e-06
##
## Statistics by Class:
##
##               Class: NoVol Class: LowVol Class: BelowAvgVol
## Precision           0.6964         0.00000         0.142857
## Recall              0.7800         0.00000         0.026316
## F1                  0.7358           NaN         0.044444
## Prevalence          0.1706         0.07167         0.129693
## Detection Rate      0.1331         0.00000         0.003413

```

## Detection Prevalence	0.1911	0.01024	0.023891
## Balanced Accuracy	0.8550	0.49449	0.501393
##	Class: AverageVol	Class: AboveAvgVol	Class: HighVol
## Precision	0.35484	0.3095	0.4429
## Recall	0.22000	0.5417	0.5000
## F1	0.27160	0.3939	0.4697
## Prevalence	0.17065	0.2457	0.2116
## Detection Rate	0.03754	0.1331	0.1058
## Detection Prevalence	0.10580	0.4300	0.2389
## Balanced Accuracy	0.56885	0.5740	0.6656

```

probs_xgb <- predict(xgb_model, newdata = test_data, type = "prob")
probs_xgb <- as.matrix(probs_xgb)

colnames(probs_xgb) <- colnames(probs_xgb)
probs_xgb <- probs_xgb[, levels(test_data$response), drop = FALSE]

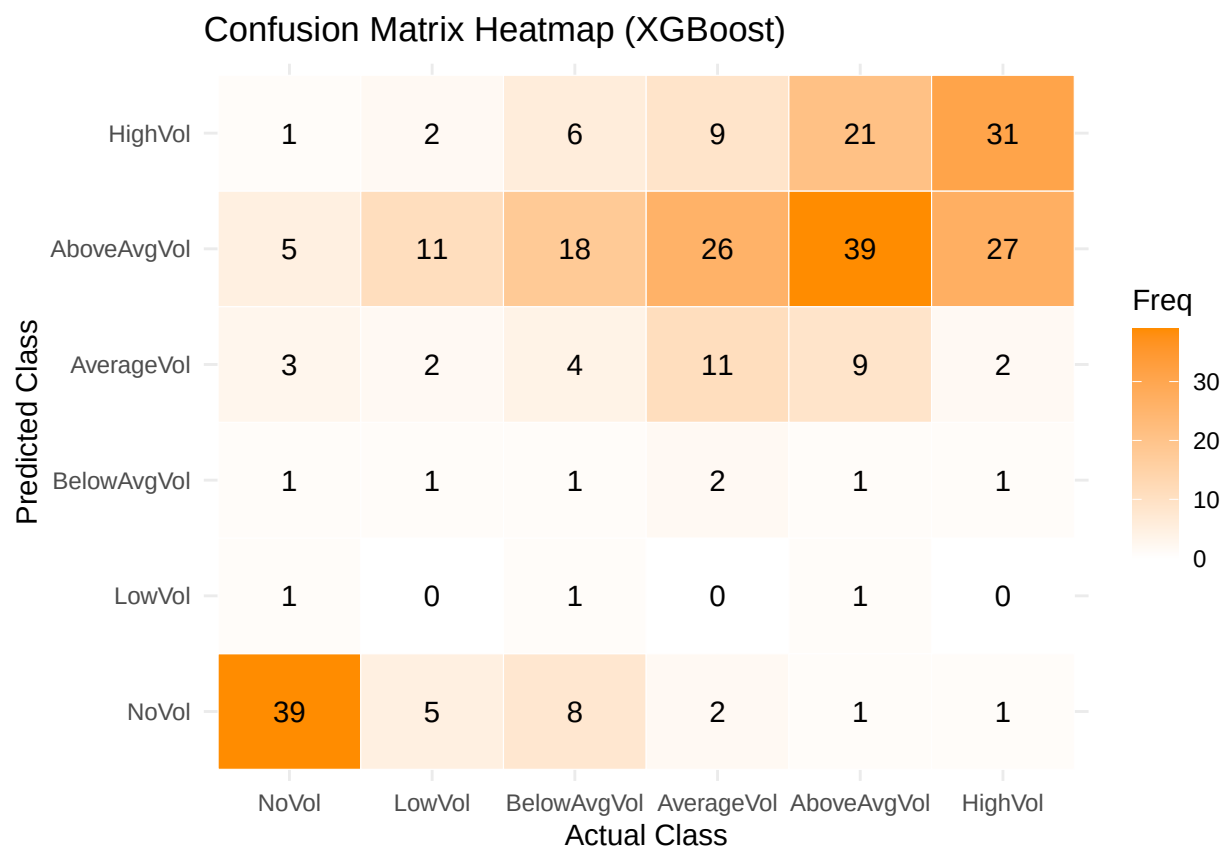
model_results <- rbind(model_results, data.frame(
  Model = "XGBoost",
  Accuracy = max(xgb_model$results$Accuracy),
  Precision = mean(cm_xgb$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_xgb$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_xgb$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data$response, probs_xgb)
))

```

```

cm_table <- as.data.frame(cm_xgb$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "darkorange") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (XGBoost)",
       x = "Actual Class", y = "Predicted Class")

```

Model Results

```
print(model_results)
```

```
##           Model  Accuracy Precision   Recall      F1      AUC
## 1  LASSO Min Lambda 0.4095563 0.4075848 0.3379015 0.3398916 0.7104183
## 2  LASSO 1se Lambda 0.3959044 0.4324259 0.3245341 0.5155221 0.7116951
## 3           LDA 0.4061433 0.4152452 0.3508166 0.3231331 0.6993283
## 4           QDA 0.3686007 0.3022113 0.3226369 0.2939610 0.6777023
## 5           PCA 0.4129693 0.3953069 0.3488060 0.3780023 0.7237638
## 6  SVM Linear 0.3822526 0.3318748 0.3077389 0.4922461 0.7048862
## 7  SVM Radial Kernel 0.3993174 0.4691828 0.3218937 0.3863011 0.7033990
## 8  Base Random Forest 0.3959044 0.3124083 0.3364511 0.3788491 0.7099696
## 9  Upsampling Random Forest 0.3924915 0.3563699 0.3560747 0.3550156 0.7249062
## 10 Weighted Random Forest 0.4232082 0.3296742 0.3566333 0.3971685 0.7323013
## 11           XGBoost 0.3826981 0.3244176 0.3446637 0.3831070 0.7266969
```

```
model_results_3PCT <- model_results
```